PLACEMENT EDITION · 2027

Network Engineer Placements 2027

From first principles to the offer: networks, routing, systems, code and interviews for Cisco, Qualcomm, Google, Microsoft, Amazon, Juniper, Arista, Nokia and Broadcom.

Working edition · built 29 October 2026 · information correct as of this date

CONTENTS

Contents

0 How to Use This Book
1 Data Structures and Algorithms
2 Programming: C, Python, C++
3 Computer Networks Fundamentals
4 Routing and Switching, CCNA to CCNP ENCOR
7 Operating Systems and Socket Programming

PART 0 · THE PLANS

0

How to Use This Book

A single path from first principles to the offer, with one home for every idea.

0.1 What this book is —

0.2 Tracks and tags —

0.3 The 8-week plan —

0.1 What this book is

This book prepares a beginner for network-engineer and networking-software roles at Cisco, Qualcomm, Google, Microsoft, Amazon, Juniper, Arista, Nokia and Broadcom. It is designed to be the only resource one needs. Every concept is explained from first principles, every interview question is solved in full, and every lab prints the output one should expect, so that a lab can also be followed on paper.

Nothing is printed twice. Each concept, figure and solved question has exactly one home, and every other mention is a cross-reference that prints its page number. Thus, the book stays comprehensive without growing long, and a reader who meets a term out of order can always find its single, complete treatment.

> BIG IDEA
>
> Read the Must sections in order, solve the Problem Set at the end of each pillar, and revise from the two-page Revision Spread; the rest of the book is depth you add when time allows.

Each pillar follows the same order: (i) teaching sections, each opening with its Big Idea; (ii) a Problem Set, in which every reported interview question for the pillar is solved; and (iii) a Revision Spread, whose boxed top half is the version to re-read in the fifteen minutes before an interview.

0.2 Tracks, tags and the six elements

Track — the family of roles one is preparing for. Most students prepare for two.

Roles fall into three tracks, which differ in what the interviews emphasise. The **Software** track covers networking software engineers at Cisco, Juniper, Arista, Nokia and Broadcom, and software engineers on networking teams at Google and Microsoft. The **Infrastructure** track covers network engineers, Amazon network development engineers and Cisco consulting engineers. The **Wireless** track covers WLAN and modem software roles, principally at Qualcomm.

Pillar	Software	Infrastructure	Wireless
1 DSA & coding	●●●	●●	●●
2 C, Python, C++	●●●	●●	●●●
3 Computer Networks	●●●	●●●	●●●
4 Routing & Switching	●	●●●	●
5 Data Centre & Cloud	●●	●●●	●
6 Wireless & Cellular	—	●	●●●
7 OS & Sockets	●●●	●	●●●
8 Linux & Tools	●●	●●●	●●
9 Security	●	●●	●
10 Automation	●●	●●	—
11 Troubleshooting	●	●●●	●
12 Supporting subjects	●●●	●●●	●●●

Every section heading carries a tag such as `MUST · ■□□`. The word states priority: **Must** sections form the 8-week plan, **Should** sections are added by the 3-month plan, and **Optional** sections complete the 6-month plan. The squares state difficulty, from foundation (■□□) to the level of the hardest interviews (■■■). Note that the tags guide the order of reading; they do not change the content, which is written to the same standard throughout.

The text uses only six recurring elements. A **Big Idea** states what a section exists to teach. A **Worked Example** solves a problem step by step. A **Trap** names a common misconception and why it is wrong. An **Interview Lens** states how the topic is examined and which companies report asking it. A **Lab** gives commands and the output to expect. **Check Yourself** questions close a section, with answers in the footnotes. Definitions, analogies and mnemonics sit in the outer margin.

0.3 The 8-week plan

The plan assumes about 4 hours on weekdays and 6 hours on Saturdays. Each weekday has the same shape: 60 minutes of coding practice, 120 minutes of the day's Must sections, 40 minutes on a lab or a worked example redone without the text, and 20 minutes of recall. Sunday holds the weekly checkpoint and a buffer, because a plan with no slack fails at the first missed day.

Week	Main topic	Daily coding strand	Sunday
1	Layers, Ethernet, ARP, subnetting; C pointers and structs	Complexity, arrays, strings, hashing, bits	Checkpoint 1
2	TCP and UDP, congestion control, DNS, DHCP, "type google.com"; C traps	Linked lists, stacks, queues, sliding window	Checkpoint 2
3	VLANs, STP, the routing table, OSPF, BGP, ACLs and NAT	Trees, tries, heaps, BFS and DFS	Checkpoint 3, mock OA
4	Processes, threads, synchronisation, memory; sockets and epoll; Linux tools	Shortest paths, union-find, backtracking	Checkpoint 4
5	Spine-leaf, ECMP, VXLAN, cloud networks; TLS, IPsec, firewalls	Dynamic programming, greedy	Checkpoint 5
6	Track variant: troubleshooting and automation, or design coding, or Wi-Fi	LRU cache, rate limiter	Checkpoint 6, mock OA
7	Troubleshooting rounds; aptitude; OOP and DBMS	Timed mixed sets	Checkpoint 7, mock OA
8	Company playbook; behavioural stories; Revision Spreads	The top-30 list	Checkpoint 8, mock OA

> **TRAP**
>
> *Claim:* one should finish all the theory before starting coding practice. *Why it is wrong:* coding fluency is built by short, repeated practice spread over weeks, and it cannot be compressed

> into the final fortnight. *Correct statement:* code every day from the first day, in parallel with the networking chapters.

PART I · PILLAR 1

1

Data Structures and Algorithms

Coding rounds for network engineers: from a blank screen to a longest-prefix-match table, an LRU cache and a token bucket.

1.0 How network-company coding rounds work	MUST
1.1 Complexity and the problem-solving routine	MUST
1.2 Arrays: two pointers, windows, prefix sums	MUST
1.3 Hashing: maps and sets	MUST
1.4 Strings and parsing network data	MUST
1.5 Bit manipulation and subnet code	MUST
1.6 Linked lists in C	MUST
1.7 Stacks, queues, ring buffers, deques	MUST
1.8 Trees and binary search trees	MUST
1.9 Tries and longest-prefix match	MUST
1.10 Heaps and priority queues	SHOULD
1.11 Graphs: networks are graphs	MUST
1.12 Recursion and backtracking	SHOULD
1.13 Sorting, binary search, intervals	MUST
1.14 Dynamic programming essentials	SHOULD
1.15 Design-style coding: caches and rate limiters	MUST
1.16 Finite-state machines	SHOULD
1.17 Testing your own code	MUST

1.0 How network-company coding rounds work MUST · ■□□

Candidate reports describe the coding rounds of networking companies as narrower than those of general software firms. The problems are mostly easy or medium, they favour linked lists, bits, strings and trees over contest techniques, and several companies expect the solution in C. Hence, breadth and clean code earn more than obscure algorithms. The table summarises what candidates report; it is indicative, not a guarantee, and every drive should be checked against its own invitation.

Company	Reported shape of the coding part	Topics most often reported
Cisco	online test, about 90 min: 1–2 coding problems and 30–40 MCQs; C, Python or Java	graphs (snake and ladder), strings, longest substring, bits, pointer output
Juniper	online test, about 90 min: 1–3 coding problems and about 35 MCQs	merge sorted arrays, multiply without `*`, list cycles, bits, state machines
Qualcomm	written test then 2–3 technical rounds	bit counting and flipping, struct padding, `volatile`
Arista	online test in C or C++, then a shared-editor onsite	linked lists from scratch, BST operations, program memory layout
Google	phone screen with 1–2 problems	tries and prefix match on IPs, log parsing, frequency sorting
Microsoft	standard software loop	LRU cache with TTL and thread safety, rate limiters
Amazon (network development)	online test of one easy and one medium problem, plus Python in the loop	strings, trees, graphs, LRU variants
Nokia	aptitude test and two simple programs	list loops and intersection, depth-first search with a stack
Broadcom	phone and onsite rounds	bits, list reversal, stacks, queues, state machines

Three consequences follow. First, a test with partial marks per test case rewards a correct brute force submitted early; one should read all problems, bank the easiest, then return. Secondly, an onsite round is judged on the visible process as much as on the code, which is the reason for the routine of §1.1 (p. 9). Thirdly, the language is a decision to be made before the clock starts.

If …	Write in …	Because
the role mentions embedded, data plane, drivers or the company is Qualcomm, Broadcom or Arista	C	pointer questions are the point; the STL is unavailable or disapproved
the role is network development or automation	Python	brevity; built-in dictionaries, `heapq`, `deque`
the test allows it and you know the STL	C++	brevity close to Python, with no hidden costs

This chapter uses C++ for general algorithms and C for linked lists, bit tricks and networking problems, as the interviews do. One language is used for each solution; where Python differs materially, one sentence says so.

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA
>
> Interviewers ask for a complete, compilable function, not pseudocode. A strong answer states the signature and the edge cases first, writes the code in small named steps, and ends by tracing one example aloud. Expect “now do it without extra memory” or “now in C” as the follow-up.

1.1 Complexity and the problem-solving routine MUST · ■□□

> BIG IDEA
>
> The input size in the problem statement tells us which algorithms can finish in time, because a judge performs about 10^8 simple operations per second.

We count growth, not seconds. An algorithm taking $c \cdot n$ steps and one taking $c' \cdot n + 50$ steps are both O(n), since for large n the constants do not change the shape of the curve. The three loops below show the three cases that cover most code.

Big-O — an upper bound on how the work grows with the input size n, ignoring constants and lower-order terms.

```
for (i = 0; i < n; i++) sum += a[i];  /* O(n): one pass          */
for (i = 0; i < n; i++)
    for (j = i + 1; j < n; j++) pairs++;  /* O(n^2): n(n-1)/2 steps  */
while (n) n >>= 1;  /* O(log n): halves n      */
```

Consider a switch. Flooding a frame to every port costs O(p) for p ports, whereas finding the egress port in the MAC table costs O(1) on average because the table is hashed. Thus, the data structure, not the hardware speed, sets the cost per frame.

Reading the constraints. A test limits n, and the limit selects the algorithm. At n = 10^5 an O(n^2) method needs 10^10 steps and fails, whereas O(n log n) needs about 1.7 × 10^6 and passes. The full table from $n \le 12$ up to $n \le$ 10^9 is in the Revision Spread; the two rules to remember are that 10^5 allows O(n log n) and $10^{3–10}4$ allows O(n^2). Python is slower by a factor of 10 to 50, so one steps down a row.

Space and the call stack. A recursive call stores a frame holding its arguments, locals and return address. Hence, a recursion of depth d uses O(d) memory even when it allocates nothing, and a depth of 10^6 overflows a typical 8 MB stack. Space counts the extra memory only; the input and the required output are excluded, so “O(1) extra” permits a few variables.

Amortised cost. A dynamic array doubles its capacity when full. A push is normally one write, but a push into a full array copies every element. Sixteen pushes starting from capacity one copy 1 + 2 + 4 + 8 = 15 elements in total, fewer than the number of pushes. In general the copies sum to less than 2n, so the cost per push averages to O(1); we call this an *amortised* bound. It holds for the whole sequence, not for each push.

Push number	1	2	3	5	9	17
Capacity after	1	2	4	8	16	32
Elements copied	0	1	2	4	8	16

The six-step routine

A visible process earns marks even when the code is unfinished. The steps are: (i) *understand*, by restating the problem and asking about size, duplicates, negatives and empty input; (ii) *examples*, by working one by hand; (iii) *brute force*, stated with its complexity; (iv) *optimise*, by finding the repeated work; (v) *code*; and (vi) *test*, by tracing the example and the edge cases of §1.17 (p. 48).

WORKED EXAMPLE

Two Sum, through all six steps. Given an array of integers and a target, return the indices of two elements that add to the target. Exactly one answer exists and an element may not be used twice.

1. *Understand.* Ask whether the array is sorted (no), whether it may hold negatives (yes), and what to return when no pair exists (an empty result).
2. *Example.* `[2, 7, 11, 15]`, target 9 gives indices `{0, 1}`.
3. *Brute force.* Test every pair: O(n^2) time, O(1) space.
4. *Optimise.* The inner loop searches for `target - a[i]`. A hash map from value to index answers that search in O(1), so one pass suffices.
5. *Code.* Below.
6. *Test.* Trace the example, then `[3, 3]` with target 6 (a duplicate value) and a single element.

```
vector<int> twoSum(const vector<int>& a, int target) {
    unordered_map<int, int> seen;                 // value -> index
    for (int i = 0; i < (int)a.size(); ++i) {
        auto it = seen.find(target - a[i]);
        if (it != seen.end()) return {it->second, i};
        seen[a[i]] = i;  // insert after the lookup
    }
    return {};
}
```

i	a[i]	need	in map?	map after
0	2	7	no	{2→0}
1	7	2	yes, index 0	return {0, 1}

Result: O(n) time and O(n) space. Inserting after the lookup is what prevents an element pairing with itself.

CHECK YOURSELF

1. What is the complexity of `for (i = 1; i < n; i *= 2) work();`? [1]
2. A function recurses to depth n and allocates nothing. What is its space complexity? [2]
3. Why is appending to a dynamic array O(1) amortised although one push can copy n elements? [3]

1.2 Arrays: two pointers, windows and prefix sums MUST · ■□□

BIG IDEA

> Most array problems become linear when we maintain two indices or one running total, because each element is then inspected a constant number of times.

An array stores its elements contiguously, so the address of `a[i]` is `base + i * sizeof(T)` and indexing costs O(1). For the same reason, inserting or deleting in the middle costs O(*n*), since the later elements must move. A two-dimensional array in C is stored row by row, so `m[r][c]` sits at offset `r * cols + c`; scanning along a row is therefore faster than scanning down a column, because it follows memory order.

Two pointers, opposite ends

Idea. On a sorted array, one pointer at each end moves inward, and each step discards an element that cannot be part of any answer. *Recognise:* “sorted”, “pair with sum”, “palindrome”, “container”. *Complexity:* O(*n*) time, O(1) space.

@

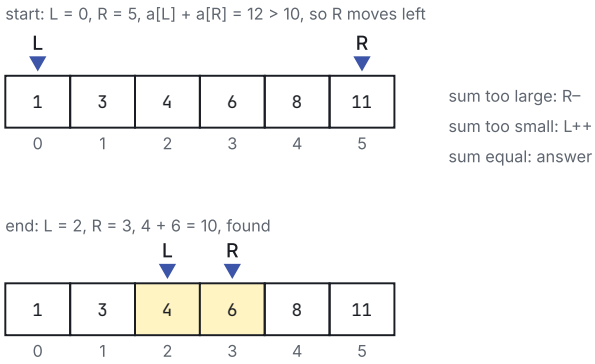


Figure 1.1 Opposite-end pointers on a sorted array with target 10. Each step discards one element that cannot belong to any answer, so at most n steps occur.

```
pair<int,int> pairSum(const vector<int>& a, long long t) {   // a sorted
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        long long s = (long long)a[l] + a[r];
        if (s == t) return {l, r};
        if (s < t) ++l; else --r;  // too small: need a larger left value
    }
    return {-1, -1};
}
```

The argument for correctness is that if `a[l] + a[r]` exceeds *t*, then `a[r]` plus any element at or right of `l` also exceeds *t*, so `r` can move without losing an answer. The same argument for `s < t` justifies moving `l`. For `[1, 3, 4, 6, 8, 11]` and *t* = 10 the pointers visit (0,5) → (0,4) → (1,4) → (1,3) → (2,3), and the last pair sums to 10.

Two pointers, read and write

Idea. A fast index `r` reads every element; a slow index `w` marks the end of the answer built so far. *Recognise:* “in place”, “remove duplicates”, “move zeroes”, “partition”. *Complexity:* O(*n*) time, O(1) space.

```cpp
int dedupe(vector<int>& a) {  // a sorted; returns the new length
    int w = 0;                          // a[0..w) holds the answer so far
    for (int r = 0; r < (int)a.size(); ++r)
        if (w == 0 || a[r] != a[w - 1]) a[w++] = a[r];
    return w;
}
```

r	a[r]	compared with	action	answer so far
0	1	none	write	1
1	1	a[w-1] = 1	skip	1
2	2	1	write	1 2
3	2	2	skip	1 2
4	3	2	write	1 2 3

> **TRAP**
>
> *Claim:* compare `a[r]` with `a[r-1]` to detect a duplicate. *Why it is wrong:* once the writer has overwritten earlier cells, `a[r-1]` may no longer be the previous kept value. *Correct statement:* compare with `a[w-1]`, the last element of the answer, and never write before the cell has been read.

Sliding window, fixed size

Idea. Instead of re-summing k elements for each position, add the entering element and subtract the leaving one. *Recognise:* "subarray of size k", "every window of length k". *Complexity:* O(n) time, O(1) space.

```cpp
long long maxSumK(const vector<int>& a, int k) {  // requires 1 <= k <= n
    long long cur = 0, best = LLONG_MIN;
    for (int i = 0; i < (int)a.size(); ++i) {
        cur += a[i];
        if (i >= k) cur -= a[i - k];                // element leaving
        if (i >= k - 1) best = max(best, cur);      // window is full
    }
    return best;
}
```

The classic bug is the off-by-one at eviction: the element leaving when index `i` enters is `a[i-k]`, not `a[i-k+1]`. For `[2, 1, 5, 1, 3, 2]` with $k = 3$ the window sums are 8, 7, 9, 6, so the answer is 9.

Sliding window, variable size

Idea. The right edge advances every step; the left edge advances only while the window violates a condition. Each index enters and leaves once, so the work is O(n) although two loops are nested. *Recognise:* "longest or shortest contiguous subarray or substring such that …". *Complexity:* O(n) time, O(*alphabet*) space.

@

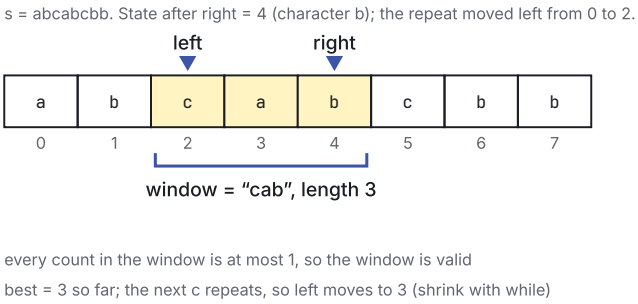


Figure 1.2 Variable sliding window. The right edge always advances; the left edge advances only to restore the invariant.

```
int lengthOfLongest(const string& s) {  // longest run with no repeats
    int cnt[256] = {0}, best = 0, left = 0;
    for (int right = 0; right < (int)s.size(); ++right) {
        ++cnt[(unsigned char)s[right]];
        while (cnt[(unsigned char)s[right]] > 1)  // shrink: while, not if
            --cnt[(unsigned char)s[left++]];
        best = max(best, right - left + 1);
    }
    return best;
}
```

right	char	window after shrinking	length	best
0	a	a	1	1
1	b	ab	2	2
2	c	abc	3	3
3	a	bca	3	3
4	b	cab	3	3
5	c	abc	3	3
6	b	cb	2	3
7	b	b	1	3

At `right = 6` the window `abc` meets a second `b`; the loop removes `a`, then `b`, leaving `cb`. A single `if` would remove only `a` and leave the invalid window `bcb`. This is the standard bug, and it is the reason the template uses `while`.

Prefix sums and the hash map

Idea. Let `pre[i]` be the sum of the first i elements. The sum of `a[j..i)` is `pre[i] - pre[j]`, so “a subarray with sum k ends here” means “an earlier prefix equals `pre[i] - k`”. A map from prefix value to occurrence count answers this in O(1). *Recognise:* “subarray sum equals k”, negatives present (which defeats a sliding window), “divisible by k”. *Complexity:* O(n) time and space.

```cpp
int subarraySum(const vector<int>& a, int k) {
    unordered_map<long long, int> seen{{0, 1}};  // the empty prefix, seeded
    long long pre = 0; int count = 0;
    for (int x : a) {
        pre += x;
        auto it = seen.find(pre - k);
        if (it != seen.end()) count += it->second;
        ++seen[pre];
    }
    return count;
}
```

For `[1, 2, 3, -2, 2]` and $k = 3$, the prefixes are 1, 3, 6, 4, 6. At prefix 3 we find the seed 0 (subarray `[1,2]`); at 6 we find 3 (`[3]`); at 4 we find 1 (`[2,3,-2]`); at the second 6 we find 3 again (`[3,-2,2]`). The count is 4. Omitting the seed `{0, 1}` loses every subarray that starts at index 0, which is the classic bug.

> CHECK YOURSELF
>
> 1. In `maxSumK`, why is `best` initialised to `LLONG_MIN` rather than 0? [4]
> 2. Why does a sliding window fail on `[1, -1, 5]` with the condition "sum at most 5"? [5]

1.3 Hashing: maps and sets MUST · ■□□

> BIG IDEA
>
> A hash table turns "have I seen this?" from a scan into a single computed lookup, which is why so many O(n^2) solutions collapse to O(n).

Hash function — a function mapping a key to an integer; equal keys must give equal values, and different keys should rarely collide.

A hash table stores each key in a bucket chosen by `hash(key) mod m`. Two keys sharing a bucket collide, and the usual remedy, *chaining*, keeps a short list in each bucket. With a good hash and a load factor (entries per bucket) kept below about one, a lookup inspects a constant expected number of entries. Thus, insert, find and erase are O(1) on average, but O(n) in the worst case when many keys collide. The ordered `map` in C++ is a balanced tree instead: O(log n) per operation, but it keeps keys sorted.

Need	Structure	Cost
membership, no order	`unordered_set`	O(1) average
key → value, no order	`unordered_map`	O(1) average
sorted iteration, nearest key	`set`, `map`	O(log n)
counts of small integers or letters	array indexed by value	O(1), smallest constant

A switch's MAC table and a router's flow table are the networking forms of this structure: the key is the destination MAC or the 5-tuple, and the value is the egress port or the action. Further, when values are bounded by a small range, a plain array replaces the hash and is faster.

Recognise: "have we seen", "count occurrences", "pair that …", "group by", "first non-repeating", "duplicate". *Typical templates:* a set for membership, a map for counts, and a sorted or canonical form of an item as the key (for anagrams, the sorted letters).

WORKED EXAMPLE

Longest consecutive sequence. Given an unsorted array, find the length of the longest run of consecutive integers (for example 1, 2, 3, 4), in O(*n*) time.

1. Sorting gives O(*n* log *n*); the target forbids it.
2. Put all values in a set. A value *x* starts a run exactly when *x* − 1 is absent.
3. For each start, extend with `x+1, x+2, …` while present. Each element is visited once as a start-check and at most once inside a run, so the total is O(*n*).

```
int longestConsecutive(const vector<int>& a) {
    unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;          // not the start of a run
        int y = x;
        while (y < INT_MAX && s.count(y + 1)) ++y;
        best = max(best, y - x + 1);               // run length
    }
    return best;
}
```

x	x − 1 present?	run extended	length
100	no	100	1
4	yes	skipped	–
200	no	200	1
1	no	1, 2, 3, 4	4

Result for `[100, 4, 200, 1, 3, 2]`: 4. Time O(*n*), space O(*n*).

TRAP

Claim: `m[key]` is a harmless way to test whether a key exists. *Why it is wrong:* in C++ the subscript operator inserts a default value when the key is absent, which changes the map and its size during the test. *Correct statement:* use `find` or `count` to test, and `[]` only when inserting or incrementing is intended.

CHECK YOURSELF

1. Which is faster for counting lowercase letters: `unordered_map<char,int>` or `int cnt[26]`? [6]
2. What is the worst-case lookup in a chained hash table, and when does it occur? [7]

1. O(log *n*), because *i* doubles each time.↩
2. O(*n*), from the call stack.↩
3. Copies occur only at doublings and total fewer than 2*n* over *n* pushes.↩
4. Every window sum may be negative, and 0 would then be returned wrongly.↩

1.4 Strings and parsing network data MUST · ■■□

> BIG IDEA
>
> Parsing is validation first and conversion second, because network input is untrusted and every malformed case an interviewer can imagine is a test case.

A C string is an array of `char` ended by a zero byte, so `strlen` must walk the whole string and costs O(*n*). Hence a loop written `for (i = 0; i < strlen(s); i++)` is O(n^2), because the length is recomputed on every iteration. A C++ `string` stores its length and answers `size()` in O(1). A string literal in C is read-only, so writing to `char *p = "abc"` is undefined behaviour; copy it into an array first.

> TRAP
>
> *Claim:* `strncpy(dst, src, n)` is a safe `strcpy`. *Why it is wrong:* when `src` has `n` or more characters, `strncpy` copies exactly `n` characters and writes no terminating zero. *Correct statement:* always set `dst[n-1] = '\0'` after `strncpy`, or use `snprintf(dst, n, "%s", src)`, which always terminates.

Recognise: “valid”, “parse”, “split”, “tokens”, dotted or colon-separated text, “log lines”. *Method:* scan once with an index; at each field check its character class, its length and its numeric range, and then check that nothing remains. Do not use `atoi` or `strtok` for validation: `atoi` accepts leading spaces, signs and trailing garbage and cannot report an error, and `strtok` modifies its input and keeps hidden state.

The function below parses a dotted-quad IPv4 address into a host-order 32-bit integer. It rejects an empty octet, a non-digit, a value above 255, a leading zero (“01”) and trailing characters.

```c
#include <stdint.h>
#include <stdbool.h>

bool parse_ipv4(const char *s, uint32_t *out) {
    uint32_t addr = 0;
    for (int part = 0; part < 4; part++) {
        if (*s < '0' || *s > '9') return false;  /* empty octet or junk */
        const char *start = s;
        int val = 0, digits = 0;
        while (*s >= '0' && *s <= '9') {
            val = val * 10 + (*s - '0');
            if (++digits > 3 || val > 255) return false;
            s++;
        }
        if (digits > 1 && *start == '0') return false;  /* leading zero */
        addr = (addr << 8) | (uint32_t)val;
        if (part < 3) {
            if (*s != '.') return false;
            s++;
        }
    }
    if (*s != '\0') return false;  /* trailing junk */
    *out = addr;
    return true;
}
```

Input	Result	Reason

5. The window sum is not monotone in its length when negatives exist, so shrinking cannot restore validity reliably; prefix sums are used instead.↩
6. The array, since it has no hashing or allocation; both are O(*n*) overall.↩
7. O(*n*), when every key lands in one bucket, for example with a poor hash or adversarial input.↩

`192.168.1.1`	true, 0xC0A80101	four valid octets
`01.2.3.4`	false	leading zero in first octet
`1.2.3`	false	after the third octet the next character is the terminator, not a dot
`1.2.3.4.5`	false	trailing characters remain after the fourth octet
`1..2.3`	false	second octet is empty
`256.1.1.1`	false	value exceeds 255

The reverse conversion extracts each byte with a shift and a mask, using `%u` with an explicit cast because `uint32_t` is not guaranteed to be `unsigned int`:

```
void format_ipv4(uint32_t a, char buf[16]) {  /* 15 characters + NUL */
    snprintf(buf, 16, "%u.%u.%u.%u",
             (unsigned)(a >> 24), (unsigned)((a >> 16) & 255),
             (unsigned)((a >> 8) & 255), (unsigned)(a & 255));
}
```

A MAC address follows the same pattern with hexadecimal digits and colons. Note how the second digit is read only after the first has been validated, so the code never reads past the terminator.

```
static int hexval(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return -1;
}
bool parse_mac(const char *s, uint8_t mac[6]) {
    for (int i = 0; i < 6; i++) {
        int hi = hexval(s[0]), lo = hi < 0 ? -1 : hexval(s[1]);
        if (hi < 0 || lo < 0) return false;
        mac[i] = (uint8_t)(hi << 4 | lo);
        s += 2;
        if (i < 5) { if (*s != ':') return false; s++; }
    }
    return *s == '\0';
}
```

A CIDR string `a.b.c.d/p` is handled by parsing the address up to the slash, then parsing *p* as a decimal number in 0–32 with the same no-leading-zero rule. Validation of IPv6 groups and the reporting problems built on these parsers are in the Problem Set (Problem Set (p. 51)).

> CHECK YOURSELF
>
> 1. Why is `01.2.3.4` rejected even though `atoi("01")` is 1? [1]
> 2. What goes wrong with `char buf[15]` in `format_ipv4`? [2]

1.5 Bit manipulation and subnet code MUST · ■■□

> BIG IDEA
>
> An integer is a row of switches, and a mask with AND, OR, XOR and shifts reads or writes any group of them in one step, which is why addresses, flags and registers are all handled this way.

The theory of subnetting, masks and CIDR is taught in the networking chapter (Pillar 3, §3.4); this section supplies only the bit code that implements it. The operators are `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` and `>>`. We use unsigned types throughout, because left-shifting a negative signed value or shifting a `1` into the sign bit is undefined behaviour in C, and `>>` on a signed negative value is implementation-defined.

> **TRAP**
>
> *Claim:* `if (x & 1 == 0)` tests whether *x* is even. *Why it is wrong:* `==` binds tighter than `&`, so the expression means `x & (1 == 0)`, that is `x & 0`, which is always zero. *Correct statement:* write `(x & 1) == 0`, and parenthesise every bitwise sub-expression.

Task	Expression	Why it works
set bit *n*	`x | (1u << n)`	OR with a one forces the bit to 1
clear bit *n*	`x & ~(1u << n)`	AND with a zero forces the bit to 0
toggle bit *n*	`x ^ (1u << n)`	XOR with a one flips the bit
test bit *n*	`(x >> n) & 1u`	brings the bit to position 0
clear lowest set bit	`x & (x - 1)`	subtracting 1 flips the lowest 1 and the zeros below it
isolate lowest set bit	`x & -x`	two's complement negation keeps only that bit
power of two (x > 0)	`x && !(x & (x - 1))`	exactly one bit is set
count set bits	loop `x &= x - 1`	each pass removes one bit; runs popcount times
round up to multiple of *a* (power of two)	`(x + a - 1) & ~(a - 1)`	adds slack, then clears the low bits

As macros for C, with arguments in parentheses and `1u` to avoid signed overflow, together with a population count and a byte swap (the conversion between big-endian and little-endian forms):

```
#define SET_BIT(x, n)    ((x) |=  (1u << (n)))
#define CLEAR_BIT(x, n)  ((x) &= ~(1u << (n)))
#define TOGGLE_BIT(x, n) ((x) ^=  (1u << (n)))
#define TEST_BIT(x, n)   (((x) >> (n)) & 1u)

static unsigned popcount(uint32_t x) {
    unsigned c = 0;
    while (x) { x &= x - 1; c++; }
    return c;
}
static uint32_t swap32(uint32_t x) {
    return (x >> 24) | ((x >> 8) & 0x0000FF00u) |
           ((x << 8) & 0x00FF0000u) | (x << 24);
}
```

Subnet arithmetic as bit operations

A prefix length *p* means that the top *p* bits are the network part. The mask has *p* ones followed by 32 − *p* zeros.

```c
static uint32_t mask_of(unsigned p) {            /* p in 0..32 */
    return p ? 0xFFFFFFFFu << (32 - p) : 0u;  /* shift by 32 would be UB */
}
bool same_subnet(uint32_t a, uint32_t b, unsigned p) {
    uint32_t m = mask_of(p);
    return (a & m) == (b & m);
}
void describe(uint32_t ip, unsigned p,
              uint32_t *net, uint32_t *bc, uint64_t *hosts) {
    uint32_t m = mask_of(p);  /* p <= 30 for a host range */
    *net = ip & m;
    *bc = *net | ~m;
    *hosts = (1ull << (32 - p)) - 2;  /* usable: net + 1 .. bc - 1 */
}
```

TRAP

Claim: `0xFFFFFFFFu << (32 - p)` is correct for every prefix length. *Why it is wrong:* for *p* = 0 the shift count is 32, which equals the width of the type, and the C standard leaves that undefined (x86 silently reduces the count modulo 32 and returns the unshifted value). *Correct statement:* special-case *p* = 0 as in `mask_of`.

@

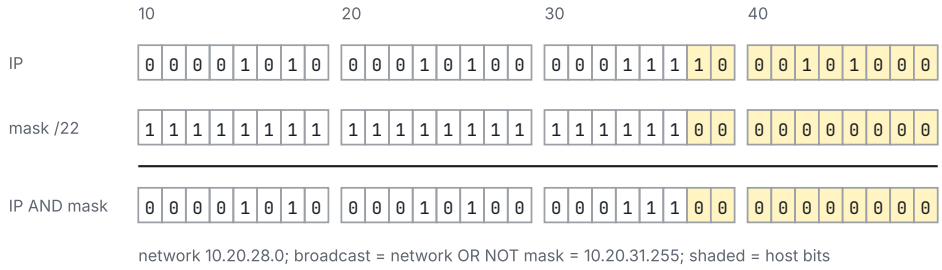


Figure 1.3 The same-subnet test in bits. AND with the mask keeps the 22 network bits and clears the 10 host bits.

WORKED EXAMPLE

Describe 10.20.30.40/22 with bit operations only.

1. The mask has 22 ones: `0xFFFFFC00`, which is 255.255.252.0.
2. The network address is `ip & mask`. The third octet 30 = `00011110` keeps its top six bits, giving `00011100` = 28, and the fourth octet is cleared. Network: 10.20.28.0.
3. The broadcast address is `net | ~mask`, which sets the ten host bits: third octet 28 + 3 = 31, fourth octet 255. Broadcast: 10.20.31.255.
4. Usable hosts run from 10.20.28.1 to 10.20.31.254, which is 2^10 − 2 = 1022 addresses.
5. Test 10.20.29.7 against it: its third octet 29 = `00011101` also clears to 28, so it lies in the subnet.

Result: network 10.20.28.0, broadcast 10.20.31.255, 1022 hosts.

Complexity. Every operation here is O(1); `popcount` by clearing the lowest bit is O(*number of set bits*). GCC and Clang offer `__builtin_popcount` and `__builtin_clz`, but interviewers usually want the loop.

INTERVIEW LENS

QUALCOMM · BROADCOM · JUNIPER · CISCO

Bit questions are asked in C and are examined for care with types: unsigned operands, `1u`, parentheses and the shift-by-width case. A strong answer states the idiom, explains why it works in one sentence, and mentions the undefined-behaviour pitfall unprompted. Typical follow-up: “do it without a loop” or “do it for a 64-bit value”.

CHECK YOURSELF

1. What does `x & (x - 1)` return for $x = 40$ (101000)? [3]
2. How many usable hosts has a /27? [4]

1.6 Linked lists in C MUST · ■■□

BIG IDEA

A linked list is a chain of nodes joined by pointers, and every list problem is a question about which pointer to rewrite, and in which order, so that no node is lost.

A singly linked list node holds a value and the address of the next node; the last node holds `NULL`. Access by position costs O(n), but inserting or deleting after a known node costs O(1) because only pointers change. Routers keep many such lists (the chains of a hash table, a queue of packet buffers), and interviews at Arista, Juniper, Nokia and Broadcom ask for them in C from a blank page.

```
typedef struct Node { int val; struct Node *next; } Node;

Node *push_front(Node *head, int v) {
    Node *n = malloc(sizeof *n);
    if (!n) return head;  /* allocation failed: list unchanged */
    n->val = v; n->next = head;
    return n;                          /* the new head */
}
```

The pointer-to-pointer idiom

To delete a node we must re-point the link that leads to it. For a middle node that link is the previous node’s `next`; for the first node it is the `head` variable itself, which is why a naive version needs a special case for the head. If we hold a pointer to the link (a `Node **`), both cases are the same.

@

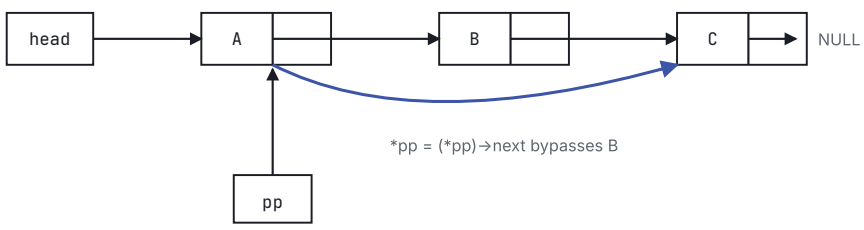


Figure 1.4 The pointer-to-pointer idiom. pp holds the address of the link that points at the current node (head, or the next field of the predecessor), so the head needs no special case.

```c
Node *delete_value(Node *head, int key) {          /* first match only */
    Node **pp = &head;
    while (*pp && (*pp)->val != key) pp = &(*pp)->next;
    if (*pp) { Node *dead = *pp; *pp = dead->next; free(dead); }
    return head;
}
Node *delete_all(Node *head, int key) {            /* every match */
    Node **pp = &head;
    while (*pp) {
        if ((*pp)->val == key) {
            Node *dead = *pp; *pp = dead->next; free(dead);
        } else pp = &(*pp)->next;  /* advance only if nothing was removed */
    }
    return head;
}
```

For `3 → 5 → 3 → 7` and key 3, `delete_all` removes the head (the link `head` now points at the 5), skips the 5, removes the second 3 (the link `5.next` now points at the 7), and returns `5 → 7`. The alternative to `Node **` is a dummy node placed before the head; both remove the special case.

> **TRAP**
> *Claim:* after `free(dead)` the loop may continue with `dead->next`. *Why it is wrong:* the memory has been returned and reading it is undefined behaviour. *Correct statement:* read `dead->next` into a local (or into `*pp`) before calling `free`.

Reversal

Three pointers walk the list: `prev` (the already reversed part), `head` (the current node) and `next`, saved before the current `next` field is overwritten. The classic bug is to overwrite first and so lose the rest of the list.

@

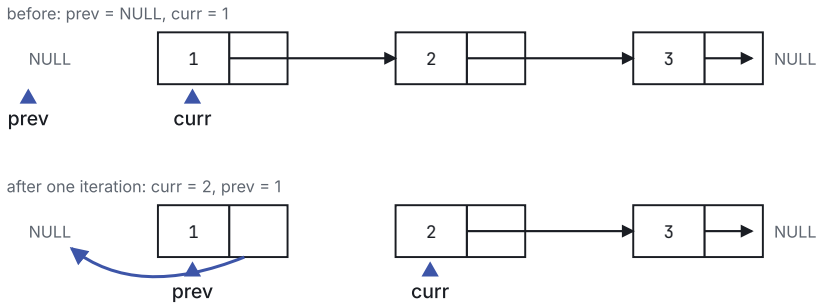


Figure 1.5 One step of in-place reversal. The accent arrow is the single pointer rewritten; next was saved first so that the rest of the list is not lost.

```
Node *reverse(Node *head) {
    Node *prev = NULL;
    while (head) {
        Node *next = head->next;       /* save */
        head->next = prev;             /* rewire */
        prev = head;                   /* advance both */
        head = next;
    }
    return prev;
}
```

Step	head (current)	next saved	After rewire	prev
1	1	2	1 → NULL	1
2	2	3	2 → 1	2
3	3	NULL	3 → 2	3

The loop ends with `head == NULL` and `prev` pointing at the old tail, which is the new head. Time O(n), space O(1).

Fast and slow pointers

Two pointers moving at speeds one and two give the middle node and detect cycles. For the middle, when `fast` reaches the end `slow` is halfway. For a cycle, `fast` closes on `slow` by one node per step, so inside a cycle they must meet.

```
Node *middle(Node *head) {  /* second middle when the length is even */
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
    }
    return slow;
}
Node *cycle_start(Node *head) {            /* NULL if there is no cycle */
    Node *slow = head, *fast = head;
    while (fast && fast->next) {  /* test fast before fast->next */
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            for (slow = head; slow != fast; ) {
                slow = slow->next; fast = fast->next;
            }
            return slow;
        }
    }
    return NULL;
}
```

Why the second phase finds the start. Let a be the number of nodes from the head to the cycle start, b the distance from the start to the meeting point along the cycle, and c the remainder of the cycle, so the cycle length is $L = b + c$. When they meet, `slow` has taken $a + b$ steps and `fast` twice as many, and the difference is a whole number k of laps: $2(a + b) - (a + b) = kL$. Hence $a = kL - b = (k - 1)L + c$. A pointer from the head and one from the meeting point, both moving one step at a time, therefore reach the cycle start together, because the second one needs c steps to reach it and then some whole laps.

TRAP

> *Claim:* `while (fast->next && fast)` is an equivalent loop condition. *Why it is wrong:* the first operand dereferences `fast` before it is known to be non-null, so an empty list crashes. *Correct statement:* test `fast` first, then `fast->next`, and only then use `fast->next->next`.

Merging with a dummy head

A temporary node on the stack gives the output list a fixed first node, so that the first append needs no special case.

```
Node *merge(Node *a, Node *b) {            /* both sorted ascending */
    Node dummy = { 0, NULL }, *tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else                  { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;  /* append the non-empty remainder */
    return dummy.next;  /* the pointer, not the stack object */
}
```

Returning `dummy.next` is legal: it points to nodes that were allocated elsewhere, and only the local `dummy` itself vanishes on return. Returning `&dummy`, or a pointer to any local array, would be the classic dangling-pointer bug.

Doubly linked lists

A doubly linked list adds a `prev` field, which makes deleting a *given* node O(1) with no search for its predecessor. Using a circular list with a sentinel node removes every `NULL` check:

```
typedef struct DNode { int val; struct DNode *prev, *next; } DNode;

void dl_init(DNode *s) { s->prev = s->next = s; }  /* empty list */
void dl_insert_after(DNode *p, DNode *n) {
    n->prev = p; n->next = p->next;
    p->next->prev = n; p->next = n;
}
void dl_unlink(DNode *n) {
    n->prev->next = n->next; n->next->prev = n->prev;
}
```

The order of the four writes in `dl_insert_after` matters: `n` is wired to its neighbours first, and the neighbours are changed last. This structure is the core of the LRU cache in §1.15 (p. 43) and of the intrusive lists inside operating-system kernels.

> INTERVIEW LENS
>
> ARISTA · JUNIPER · NOKIA · BROADCOM
>
> The question is usually phrased as “write it in C on the shared editor”: delete a node (head included), delete all nodes with a value, reverse, merge, find the loop. A strong answer draws the boxes first, names the three pointers, and checks the empty list, the one-node list and the head. Typical follow-ups are “what if the node is the tail” and “reverse it recursively”.

> CHECK YOURSELF
>
> 1. In `delete_value`, what does `pp` point to when the list is empty? [5]
> 2. Why does `reverse` need the variable `next`? [6]

1.7 Stacks, queues, ring buffers and deques MUST · ■■□

> BIG IDEA
>
> A stack remembers the most recent unfinished thing and a queue remembers the oldest, and a ring buffer gives a queue of fixed size with no allocation, which is exactly what a network driver needs.

A *stack* is last in, first out: `push`, `pop` and `top` are O(1) on an array with a top index. *Recognise:* nested structure, “undo”, “matching”, “most recent”, expression evaluation. A *queue* is first in, first out and is the engine of breadth-first search and of packet queues. A *deque* allows insertion and removal at both ends in O(1); `std::deque` provides it, and Python’s `collections.deque` does the same.

Matching brackets is the model stack problem. Each opening bracket is pushed; each closing bracket must match the top.

```
bool balanced(const char *s) {
    size_t top = 0;
    char *st = malloc(strlen(s) + 1);
    if (!st) return false;
    bool ok = true;
    for (; *s && ok; s++) {
        switch (*s) {
        case '(': case '[': case '{': st[top++] = *s; break;
        case ')': ok = top && st[--top] == '('; break;
        case ']': ok = top && st[--top] == '['; break;
        case '}': ok = top && st[--top] == '{'; break;
        }
    }
    free(st);
    return ok && top == 0;  /* leftovers mean unclosed brackets */
}
```

The ring buffer

Packet drivers hand frames from an interrupt handler to a consumer through a fixed array used circularly. The producer writes at `tail` and the consumer reads at `head`; both wrap with a modulus. The difficulty is that `head == tail` describes both an empty and a full buffer. Three remedies exist: keep a `count`, waste one slot, or let the indices run freely and compare their difference.

@

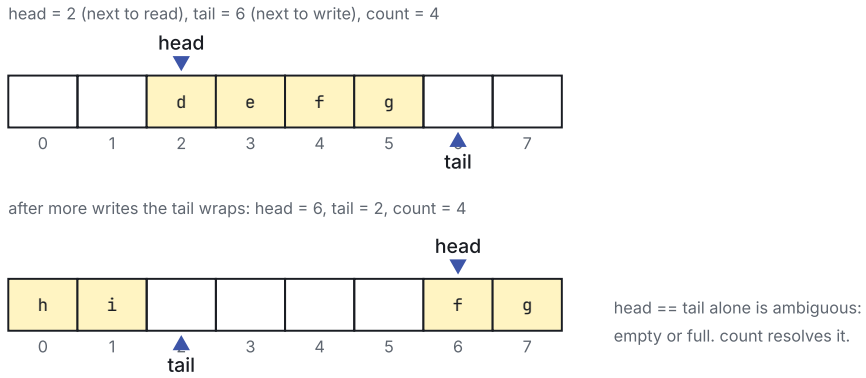


Figure 1.6 A circular buffer of capacity eight, before and after the write index wraps. A separate count removes the full-versus-empty ambiguity.

```
#define N 8
typedef struct { uint8_t buf[N]; size_t head, tail, count; } Ring;

bool ring_push(Ring *r, uint8_t b) {
    if (r->count == N) return false;              /* full */
    r->buf[r->tail] = b;
    r->tail = (r->tail + 1) % N;
    r->count++;
    return true;
}
bool ring_pop(Ring *r, uint8_t *out) {
    if (r->count == 0) return false;              /* empty */
    *out = r->buf[r->head];
    r->head = (r->head + 1) % N;
    r->count--;
    return true;
}
```

When one thread produces and another consumes, the `count` variable is written by both and needs a lock. The lock-free alternative keeps `head` and `tail` as free-running unsigned counters and uses a power-of-two capacity: the slot is `idx & (N - 1)`, the buffer is empty when `head == tail` and full when `tail - head == N`. The subtraction is correct even after the counters wrap, because unsigned arithmetic is modular. The producer writes only `tail` and the consumer only `head`, so no lock is needed, provided each side publishes its index with the memory ordering that the concurrency chapters describe.

The monotonic stack

Idea. To find, for each element, the next greater element, keep a stack of indices whose values are decreasing. A new value pops every smaller entry, and each popped entry has found its answer. Every index is pushed and popped once, so the cost is O(*n*). *Recognise:* “next greater or smaller”, “days until”, “span”, “histogram”.

```cpp
vector<int> dailyTemps(const vector<int>& t) {
    vector<int> ans(t.size(), 0), st;   // st: indices, temps decreasing
    for (int i = 0; i < (int)t.size(); ++i) {
        while (!st.empty() && t[st.back()] < t[i]) {
            ans[st.back()] = i - st.back();   // wait = index distance
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;  // never-warmer days stay 0
}
```

i	t[i]	indices popped (answers set)	stack after
0	73	none	0
1	74	0 → 1	1
2	75	1 → 1	2
3	71	none	2, 3
4	69	none	2, 3, 4
5	72	4 → 1, 3 → 2	2, 5
6	76	5 → 1, 2 → 4	6
7	73	none	6, 7

Result for `[73, 74, 75, 71, 69, 72, 76, 73]`: `[1, 1, 4, 2, 1, 1, 0, 0]`. The classic bug is to store values instead of indices, which makes the distance impossible to compute.

The sliding-window maximum uses the same idea with a deque; it is solved in the Problem Set.

> CHECK YOURSELF
>
> 1. In the lock-free ring, why does `tail - head == N` identify a full buffer after both counters have wrapped past 2^32? [7]
> 2. What is the time complexity of `dailyTemps`, given the inner `while`? [8]

1. Leading zeros are ambiguous (some parsers read them as octal), so strict validators reject them.↩
2. The longest address `255.255.255.255` has 15 characters, and the terminating zero needs a sixteenth byte.↩
3. 32 (100000): the lowest set bit is cleared.↩
4. 2^5 − 2 = 30.↩
5. To `head` itself, which is `NULL`, so the loop does not run and nothing is freed.↩

1.8 Trees and binary search trees MUST · ■■□

BIG IDEA

A binary search tree keeps its keys ordered so that each comparison discards a whole subtree, giving search in time proportional to the height, which is logarithmic only while the tree stays balanced.

A *binary tree* is a node with at most two children; a *binary search tree* (BST) adds the rule that every key in the left subtree is smaller than the node and every key in the right subtree is larger. The *height h* is the number of edges on the longest root-to-leaf path. Search, insert and delete each follow one path, so they cost O(*h*). For *n* random keys *h* is about 2 log *n*, but inserting sorted keys produces a chain with *h* = *n*, and every operation degrades to O(*n*). Self-balancing trees (AVL, red-black) restore O(log *n*); C++ `std::map` and `std::set` are red-black trees.

@

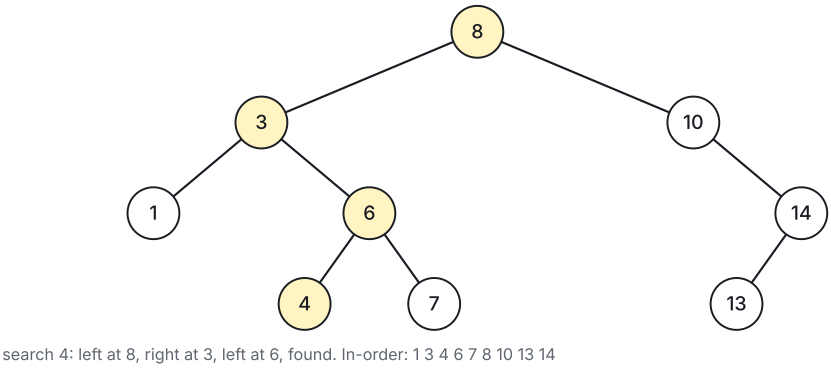


Figure 1.7 A binary search tree. Everything left of a node is smaller and everything right is larger; the highlighted path is the search for 4.

Traversals

Depth-first traversals differ only in when the node is visited: *pre-order* (node, left, right), *in-order* (left, node, right) and *post-order* (left, right, node). The in-order traversal of a BST yields the keys in sorted order, which is the fact behind “k-th smallest” and “validate BST”. Breadth-first traversal visits level by level with a queue. *Recognise:* “depth”, “path”, “subtree” (DFS, usually post-order, returning information to the parent); “level”, “right side view”, “shortest” (BFS); “sorted”, “k-th”, “validate” (in-order).

6. Without it, writing `head->next = prev` destroys the only reference to the rest of the list.↩
7. Unsigned subtraction is modulo 2^32, so the difference is the true occupancy as long as it never exceeds *N*.↩
8. O(*n*): each index is pushed once and popped at most once.↩

```cpp
struct TreeNode { int val; TreeNode *left = nullptr, *right = nullptr; };

int height(TreeNode* t) {                  // nodes on the longest path
    return t ? 1 + max(height(t->left), height(t->right)) : 0;
}

vector<int> inorderIter(TreeNode* root) {  // no recursion: explicit stack
    vector<int> out; vector<TreeNode*> st; TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push_back(cur); cur = cur->left; }  // far left
        cur = st.back(); st.pop_back();
        out.push_back(cur->val);
        cur = cur->right;
    }
    return out;
}

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res; if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size(); res.emplace_back();  // sz = nodes on this level
        while (sz--) {
            TreeNode* t = q.front(); q.pop();
            res.back().push_back(t->val);
            if (t->left) q.push(t->left);
            if (t->right) q.push(t->right);
        }
    }
    return res;
}
```

All three run in O(*n*) time. Their space is O(*h*) for the explicit stack and O(*width*) for the queue. A skewed tree of 10^6 nodes also overflows the call stack of the recursive `height`, which is the reason for asking in an interview whether the tree may be skewed.

BST operations in C

Insert and delete return the root of the (possibly changed) subtree, and the caller stores the result back into its link. Forgetting to return or store it is the standard delete bug.

```c
typedef struct T { int key; struct T *l, *r; } T;

T *bst_insert(T *t, int k) {
    if (!t) { t = calloc(1, sizeof *t); if (t) t->key = k; return t; }
    if (k < t->key)      t->l = bst_insert(t->l, k);
    else if (k > t->key) t->r = bst_insert(t->r, k);
    return t;  /* duplicates are ignored */
}
const T *bst_search(const T *t, int k) {
    while (t && t->key != k) t = k < t->key ? t->l : t->r;
    return t;
}
static T *min_node(T *t) { while (t->l) t = t->l; return t; }

T *bst_delete(T *t, int k) {
    if (!t) return NULL;
    if (k < t->key)      t->l = bst_delete(t->l, k);
    else if (k > t->key) t->r = bst_delete(t->r, k);
    else {
        if (!t->l || !t->r) {  /* zero or one child */
            T *child = t->l ? t->l : t->r;
            free(t);
            return child;  /* parent adopts the child */
        }
        T *s = min_node(t->r);  /* in-order successor */
        t->key = s->key;
        t->r = bst_delete(t->r, s->key);
    }
    return t;
}
```

Deletion has three cases: a leaf is removed; a node with one child is replaced by that child; a node with two children takes the key of its in-order successor (the smallest key of the right subtree), and the successor, which has no left child, is then deleted by one of the first two cases.

> WORKED EXAMPLE
>
> **Delete 3 from the tree of Figure 1.7.**
>
> 1. Search: 3 < 8, so go left and find the node 3.
> 2. It has two children (1 and 6), so find the successor: the left-most node of the right subtree, `6 → 4`. The successor is 4.
> 3. Copy 4 into the node, so the node now holds 4.
> 4. Delete 4 from the right subtree. The node 4 is a leaf, so it is removed and the left link of 6 becomes `NULL`.
> 5. The in-order sequence is now 1 4 6 7 8 10 13 14: still sorted, with 3 removed.
>
> **Result: the node 3 now holds 4, and the old leaf 4 is gone.**

Validating a BST

Checking each node only against its parent is wrong, since a node deep in the right subtree may still be smaller than an ancestor. We pass down the open interval that each key must lie in.

```
bool valid(TreeNode* t, long long lo, long long hi) {  // lo < key < hi
    return !t || (lo < t->val && t->val < hi &&
                  valid(t->left, lo, t->val) &&
                  valid(t->right, t->val, hi));
}
// call: valid(root, LLONG_MIN, LLONG_MAX)
```

TRAP

Claim: a tree is a BST if every node is greater than its left child and smaller than its right child. *Why it is wrong:* the tree 5 with left child 1 and right child 6, where 6 has left child 4, satisfies the local rule but 4 sits in the right subtree of 5. *Correct statement:* every key must lie between the bounds inherited from all its ancestors, or the in-order sequence must be strictly increasing.

CHECK YOURSELF

1. What is the height of the BST built by inserting 1, 2, 3, 4, 5 in order? [1]
2. Which traversal prints a BST in sorted order? [2]

1.9 Tries and longest-prefix match MUST · ■■■

BIG IDEA

A router forwards a packet along the most specific route that contains its destination, and a trie finds that route by reading the address one bit at a time, in a number of steps bounded by the address length and independent of the number of routes.

A *trie* (prefix tree) stores a set of strings along the edges of a tree: the path from the root spells the string, and a flag on a node marks that a stored string ends there. Lookup costs O(*L*) for a key of length *L*, regardless of how many keys are stored. *Recognise:* "words with this prefix", "auto-complete", "longest common prefix", "maximum XOR", and above all "longest prefix match" on IP addresses.

The routing problem. A forwarding table maps *prefixes* to next hops. A prefix `10.1.0.0/16` matches every address whose first 16 bits equal those of 10.1.0.0. Prefixes overlap: an address in 10.1.2.0/24 also lies in 10.1.0.0/16 and in 10.0.0.0/8. The forwarding rule is that the matching prefix with the greatest length wins, termed *longest-prefix match* (LPM), because the longer prefix is the more specific route. (Why routers need this, and how routes are learned, is taught in the routing chapter; here we implement the lookup.)

Binary trie. We treat a 32-bit address as a string over {0, 1}, with one level per bit. Inserting a prefix of length *p* follows (and creates) *p* edges and stores the next hop at the final node. A lookup follows the address bits from the root and records the next hop of every marked node it passes; when the next edge is missing, the last recorded hop is the longest match.

@

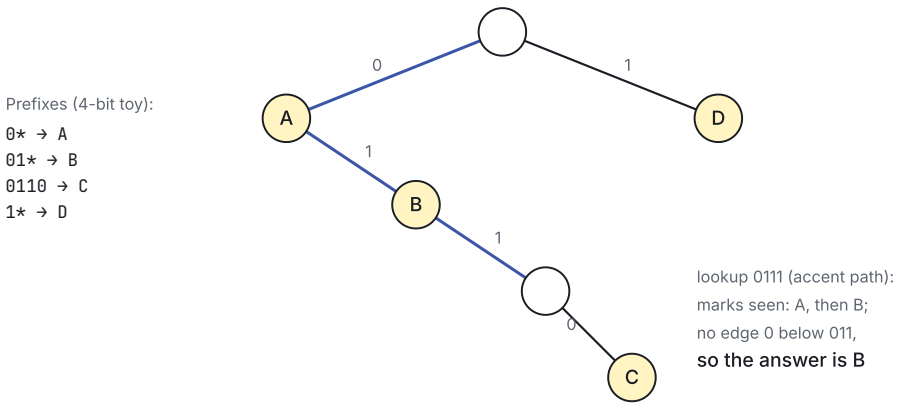


Figure 1.8 A binary trie for four toy prefixes. A shaded node carries a next hop; a lookup walks the address bits and remembers the deepest shaded node it passed.

```c
#include <stdint.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node {
    struct Node *c[2];
    int hop;                        /* hop < 0: no route ends here */
} Node;

static Node *new_node(void) {
    Node *n = malloc(sizeof *n);
    if (n) { n->c[0] = n->c[1] = NULL; n->hop = -1; }
    return n;
}
bool trie_insert(Node *root, uint32_t prefix, unsigned len, int hop) {
    /* len <= 32 */
    Node *t = root;
    for (unsigned i = 0; i < len; i++) {
        unsigned bit = (prefix >> (31 - i)) & 1u;
        if (!t->c[bit] && !(t->c[bit] = new_node())) return false;
        t = t->c[bit];
    }
    t->hop = hop;
    return true;
}
int trie_lookup(const Node *root, uint32_t addr) {
    int best = root->hop;  /* the default route /0, if any */
    const Node *t = root;
    for (unsigned i = 0; i < 32 && t; i++) {
        t = t->c[(addr >> (31 - i)) & 1u];
        if (t && t->hop >= 0) best = t->hop;   /* longer match found */
    }
    return best;  /* -1: no matching prefix */
}
```

Complexity. Insert O(p), lookup O(32), independent of the table size n. The memory is at most 32 nodes per prefix in the worst case, and typically far fewer because prefixes share their upper bits. A linear scan of all prefixes costs O(n) per lookup, which is the baseline this structure replaces.

Multibit tries. Reading eight bits per level (a *stride* of 8) needs only four steps for IPv4, at the cost of nodes with 256 children. Figure 1.9 draws such a trie for four prefixes. Real routers go further: they use compressed tries or TCAM hardware (the device internals are treated in Pillar 5,

§5.2.8). The software alternative to a trie is one hash table per prefix length: probe the /32 table, then /31, and so on, down to /0, and keep the first hit; this costs up to 32 hash lookups but needs no pointer chasing.

@

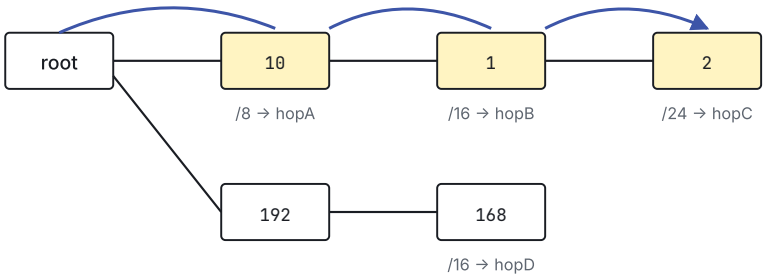


Figure 1.9 A stride-8 trie (one level per octet) for 10.0.0.0/8, 10.1.0.0/16, 10.1.2.0/24 and 192.168.0.0/16. The accent walk is the lookup of 10.1.2.77.

WORKED EXAMPLE

Lookup in a table of four prefixes and a default route. Routes: 10.0.0.0/8 → A, 10.1.0.0/16 → B, 10.1.2.0/24 → C, 192.168.0.0/16 → D, 0.0.0.0/0 → E.

1. Insert each prefix: the /8 path has 8 edges and marks hop A; the /16 path shares those 8 edges and extends by 8 more; the /24 extends by 8 more again.
2. Look up 10.1.2.77. The first 8 bits (10) reach the node marked A, the next 8 (1) reach the node marked B, the next 8 (2) reach the node marked C, and no further route exists. The last mark seen is C.
3. Look up 10.1.9.9. The walk reaches B after 16 bits; the third octet 9 = `00001001` leaves the path of 10.1.2.0/24 (third octet `00000010`) at its fifth bit. The last mark seen is B.
4. Look up 10.200.0.1 → A (leaves the 10.1 path at the ninth bit). Look up 172.16.0.1: the walk follows the first bit (1) shared with 192.168.0.0/16 and leaves that path at the second bit, having passed no mark, so only the default route E matches.

Address	Deepest mark on the walk	Next hop
10.1.2.77	/24	C
10.1.9.9	/16	B
10.200.0.1	/8	A
172.16.0.1	/0	E

Result: lookups touch at most 32 nodes whatever the table size; the answer is the last mark seen.

TRAP

Claim: stop the lookup at the first node that carries a next hop. *Why it is wrong:* a shorter prefix such as 10.0.0.0/8 is met before the longer 10.1.2.0/24, so stopping early returns the least

specific route. *Correct statement:* keep walking while edges exist and remember the deepest marked node; return it when the walk ends.

Deleting a prefix clears the hop at its node and removes any chain of childless, unmarked nodes below the last useful ancestor. For text keys the same code applies with 26 children per node and an `end` flag in place of `hop`; "implement a trie" and "maximum XOR of two numbers" (a binary trie of the numbers' bits, queried greedily for the opposite bit) are solved in the Problem Set.

INTERVIEW LENS

GOOGLE · CISCO · JUNIPER · ARISTA

The reported form is: "given a list of IP prefixes and a target address, find the longest matching prefix; which data structure, and what is the lookup complexity?" A strong answer names the binary trie, states O(32) per lookup, explains why a hash set of prefixes needs up to 32 probes, and mentions that hardware uses TCAM. Follow-ups: deletion, memory use, updating the table while lookups continue, IPv6 (128 levels, hence compression).

CHECK YOURSELF

1. How many nodes does a binary trie contain after inserting only 10.0.0.0/8? [3]
2. Why is lookup time independent of the number of prefixes? [4]

1.10 Heaps and priority queues SHOULD · ■■□

BIG IDEA

A heap keeps the smallest (or largest) element at the front of a partially ordered array, so that the extreme can be read at once and replaced in logarithmic time, which suits "the best k so far".

A *binary min-heap* is a complete binary tree in which every parent is no larger than its children. Because the tree is complete, it is stored in an array with no pointers: the children of index *i* are `2i+1` and `2i+2`, and its parent is `(i-1)/2`.

@

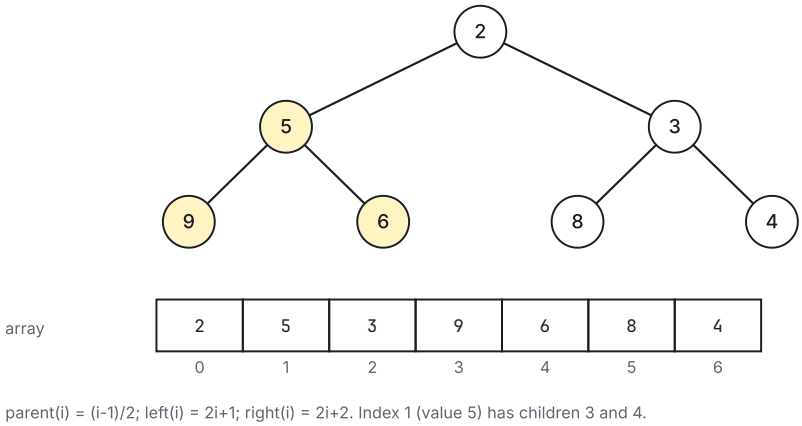


Figure 1.10 A min-heap as a tree and as the array that stores it. No pointers are needed because the shape is always a complete tree.

`push` appends at the end and swaps upward while smaller than the parent (sift-up); `pop` moves the last element to the root and swaps downward with the smaller child (sift-down). Each follows

one root-to-leaf path, so both cost O(log *n*), and reading the minimum costs O(1). Building a heap from *n* elements by sifting down from the middle takes O(*n*), not O(*n* log *n*), because most nodes lie near the leaves.

Recognise: “k largest or smallest”, “k most frequent”, “merge k sorted”, “median of a stream”, “next event by time”, “reorder packets by sequence number”. In C++ `priority_queue<int>` is a max-heap; for a min-heap pass `greater<int>`. Python’s `heapq` is a min-heap; negate values for a max-heap.

```
vector<int> topK(const vector<int>& a, int k) {  // the k largest, ascending
    priority_queue<int, vector<int>, greater<int>> h;  // min-heap of k
    for (int x : a) {
        h.push(x);
        if ((int)h.size() > k) h.pop();  // discard the smallest
    }
    vector<int> out;
    while (!h.empty()) { out.push_back(h.top()); h.pop(); }
    return out;
}
```

The heap holds the *k* best seen so far; its root is the weakest of them, which is the one to evict. For `[3, 1, 5, 12, 2, 11]` and *k* = 3 the heap evolves as {3}, {1,3}, {1,3,5}, then 12 arrives and 1 is evicted giving {3,5,12}, then 2 arrives and is itself evicted, then 11 evicts 3, giving {5,11,12}. The cost is O(*n* log *k*) time and O(*k*) space. For the median of a stream, a max-heap holds the lower half and a min-heap the upper half, rebalanced so that their sizes differ by at most one; the median is at one of the two roots (Problem Set).

> TRAP
>
> *Claim:* use a max-heap of size *k* to keep the *k* largest values. *Why it is wrong:* the root of a max-heap is the largest element, but the element to evict is the smallest of the kept ones. *Correct statement:* keep the *k* largest in a min-heap, and the *k* smallest in a max-heap.

1.11 Graphs: networks are graphs MUST · ■■■

> BIG IDEA
>
> A network is a graph of nodes and links, so routing, loop detection and reachability become traversal, shortest-path and connectivity problems, each solved by one of five short templates.

A *graph* has *V* vertices and *E* edges; an edge may be directed, may carry a weight (a link cost or delay), and the graph may be disconnected. The usual representation is an *adjacency list*: for each vertex, the list of its neighbours (with weights). It takes O(*V* + *E*) space, which suits the sparse graphs of networks. An *adjacency matrix* takes O(V^2) space and answers “is there an edge” in O(1); it suits dense graphs and small *V*. The choice of algorithm follows the question.

Question	Algorithm	Time
reachable? components? loop present?	DFS or BFS	O(*V* + *E*)
fewest hops (unweighted shortest path)	BFS	O(*V* + *E*)
cheapest path, non-negative weights	Dijkstra	O((*V* + *E*) log *V*)

negative weights, or at most k edges	Bellman–Ford	$O(VE)$
all pairs, $V \le 400$	Floyd–Warshall	$O(V^3)$
order tasks with dependencies	topological sort (Kahn)	$O(V + E)$
dynamic connectivity, "does this edge close a loop"	union–find	about $O(\alpha(n))$ per operation

Breadth-first and depth-first search

BFS explores in rings of increasing hop count, using a queue, so the first time it reaches a vertex is by a shortest path. Marking a vertex visited when it is *pushed* (not when it is popped) keeps duplicates out of the queue. DFS follows one path as deep as it goes, using a stack, and is the tool for connectivity, cycles and ordering. The iterative form avoids the recursion limit (about 1000 in Python, and the stack size in C).

```
vector<int> bfs(const vector<vector<int>>& g, int src) {
    // hop distance from src, -1 if unreachable
    vector<int> dist(g.size(), -1);
    queue<int> q; dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u])
            if (dist[v] < 0) {                      // mark on push
                dist[v] = dist[u] + 1; q.push(v);
            }
    }
    return dist;
}

vector<int> dfsOrder(const vector<vector<int>>& g, int s) {   // iterative
    vector<char> seen(g.size(), 0); vector<int> st{s}, order;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        if (seen[u]) continue;
        seen[u] = 1; order.push_back(u);
        for (int i = (int)g[u].size() - 1; i >= 0; --i)  // reverse order
            if (!seen[g[u][i]]) st.push_back(g[u][i]);
    }
    return order;
}
```

> **TRAP**
>
> *Claim:* a traversal needs no visited set if the graph "looks like a tree". *Why it is wrong:* a single cycle, or a second path to the same vertex, makes the traversal revisit vertices, and on a cycle it never ends. This is a broadcast storm in code form: a frame flooded around a switched loop is never removed. *Correct statement:* mark each vertex when first reached and skip marked vertices.

A grid is a graph whose vertices are cells and whose neighbours are the four adjacent cells; "number of islands" is the count of DFS starts, and "rotting oranges" and "distance to the nearest gate" are BFS started from all sources at once (multi-source BFS). These are in the Problem Set.

Dijkstra's algorithm

Idea. Keep a tentative distance to each vertex. Repeatedly settle the unsettled vertex with the smallest tentative distance, and relax its edges: if going through it improves a neighbour's dist-

ance, update it. Because every weight is non-negative, no later path can reach a settled vertex more cheaply, so a settled distance is final. A min-heap supplies the next vertex. Entries are never updated in place; an improved vertex is pushed again, and the older, larger entry is discarded when it surfaces. *Recognise:* “cheapest”, “minimum time”, “network delay”, weights present and non-negative.

@

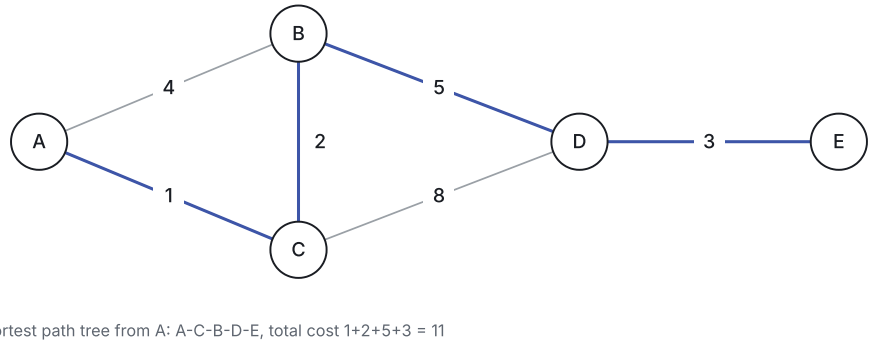


Figure 1.11 A weighted undirected graph (link costs). The accent path is the shortest route from A to E.

```
using Graph = vector<vector<pair<int, int>>>;  // g[u] = {(v, weight), ...}

vector<long long> dijkstra(const Graph& g, int src) {
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(g.size(), INF);
    using P = pair<long long, int>;
    priority_queue<P, vector<P>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;  // stale entry
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    return dist;
}
```

WORKED EXAMPLE

Shortest paths from A in Figure 1.11. Edges: A–B 4, A–C 1, C–B 2, B–D 5, C–D 8, D–E 3.

Pop (d, u)	Action	dist after (A B C D E)	Heap after
(0, A)	relax B = 4, C = 1	0 4 1 ∞ ∞	(1,C) (4,B)
(1, C)	B improves to 3; D = 9	0 3 1 9 ∞	(3,B) (4,B) (9,D)
(3, B)	D improves to 8	0 3 1 8 ∞	(4,B) (8,D) (9,D)
(4, B)	stale: 4 > 3, skipped	unchanged	(8,D) (9,D)
(8, D)	E = 11	0 3 1 8 11	(9,D) (11,E)
(9, D)	stale: 9 > 8, skipped	unchanged	(11,E)
(11, E)	no edges to improve	final	empty

> **Result: distances 0, 3, 1, 8, 11; the route to E is A–C–B–D–E. Without the stale check, the entries (4,B) and (9,D) would be expanded again, repeating work.**

Complexity. Each edge causes at most one push, so the heap holds at most E entries and the total cost is $O((V + E) \log V)$. The algorithm is wrong when an edge weight is negative, because a settled vertex could later be improved.

> TRAP
>
> *Claim:* the line `if (d > dist[u]) continue;` is an optimisation that can be left out. *Why it is wrong:* the result stays correct, but each stale entry re-expands its vertex, and on dense graphs the running time grows from $O(E \log V)$ to $O(VE)$. *Correct statement:* always skip stale entries.

Bellman–Ford

Bellman–Ford relaxes every edge, $V - 1$ times. After round r, `dist[v]` is the cost of the best path using at most r edges, so stopping after k rounds answers "cheapest path with at most k edges", the hop-limited form that appears in "cheapest flights within k stops". Copying the array each round ensures that round r uses only distances from round $r - 1$. A further round that still improves some distance proves a negative cycle.

```
struct Edge { int u, v, w; };
vector<long long> bellmanFord(int n, const vector<Edge>& es,
                              int src, int rounds) {
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(n, INF); dist[src] = 0;
    for (int r = 0; r < rounds; ++r) {  // rounds <= n - 1
        vector<long long> nxt = dist;  // use last round's values only
        for (const Edge& e : es)
            if (dist[e.u] < INF && dist[e.u] + e.w < nxt[e.v])
                nxt[e.v] = dist[e.u] + e.w;
        dist = nxt;
    }
    return dist;
}
```

Dijkstra and Bellman–Ford are also the two shortest-path methods of routing protocols. Link-state protocols such as OSPF and IS-IS give every router the whole graph, and each router runs Dijkstra locally; distance-vector protocols such as RIP run Bellman–Ford in a distributed way, each router relaxing the costs its neighbours advertise. The protocols are taught in the routing chapter; the comparison is a common interview bridge question, and this paragraph is its core.

Topological order and union–find

A *topological order* lists the vertices of a directed acyclic graph so that every edge goes forward; it is the order in which dependent tasks can run. Kahn's algorithm repeatedly removes a vertex of in-degree zero. If fewer than V vertices are output, a cycle exists.

```cpp
vector<int> topo(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n, 0), order;
    for (const auto& adj : g) for (int v : adj) ++indeg[v];
    queue<int> q;
    for (int i = 0; i < n; ++i) if (!indeg[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order;  // size < n: a cycle exists
}
```

Union–find (disjoint-set union, DSU) maintains a partition of vertices into components under "add edge". `find` returns a component's representative, with path compression; `unite` merges two components, attaching the smaller to the larger. An edge whose endpoints already share a representative closes a loop, which is how a network with redundant links, or a spanning-tree protocol, is reasoned about.

```cpp
struct DSU {
    vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;  // already connected: edge closes a loop
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

With both optimisations each operation is O(α(*n*)), where α is the inverse Ackermann function, below 5 for any input that fits in memory. Kruskal's minimum spanning tree sorts the edges by weight and adds each edge whose `unite` returns true. Floyd–Warshall computes all pairs with three nested loops, `for k, for i, for j: d[i][j] = min(d[i][j], d[i][k] + d[k][j])`, and the loop over *k* must be outermost.

> **INTERVIEW LENS**
>
> CISCO · GOOGLE · AMAZON · NOKIA
>
> Reported problems include the minimum dice throws of a snake-and-ladder board (BFS on 100 cells), depth-first search without recursion, the time for a signal to reach all routers (Dijkstra), links whose failure partitions the network (bridges), and the first link that creates a loop (union–find). A strong answer names the graph (what is a vertex, what is an edge), picks the algorithm from the table above, and states its complexity. If asked "why not union–find, why BFS", answering with the trade-off, not defending one tool, is what is scored.

> **CHECK YOURSELF**
>
> 1. Why must BFS mark a vertex when it is pushed rather than popped? [5]
> 2. What does Dijkstra do wrongly on a graph with a negative edge? [6]
> 3. In union–find, what does `unite` returning false mean? [7]

1.12 Recursion and backtracking SHOULD · ■■□

BIG IDEA

Backtracking builds an answer one choice at a time and undoes each choice after exploring it, so a single recursive function enumerates every candidate while the constraints prune the hopeless ones early.

A recursive function needs a *base case* that stops the recursion and a *step* that moves towards it. Backtracking adds a *path* that is extended before the recursive call and restored after it: choose, explore, un-choose. *Recognise:* "all combinations, permutations or subsets", "generate all", "place so that no two conflict", and small limits ($n \le 12$ for permutations, $n \le 20$ for subsets). The cost is the size of the search tree: 2^*n* for subsets, *n*! for permutations.

```
void subsets(const vector<int>& a, int i, vector<int>& path,
             vector<vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(path); return; }   // copy
    subsets(a, i + 1, path, out);  // skip a[i]
    path.push_back(a[i]);                                      // choose
    subsets(a, i + 1, path, out);  // explore
    path.pop_back();  // un-choose
}
```

The two classic bugs are forgetting the un-choose step, which leaks choices into sibling branches, and (in Python) appending `path` itself rather than a copy `path[:]`, so that every stored answer changes as the path changes.

WORKED EXAMPLE

Restore IP addresses. Given a string of digits such as `25525511135`, return every way to insert three dots so that the result is a valid IPv4 address (four parts, each 0–255, no leading zeros).

1. The search has four levels. At each level we choose the length of the next part, 1 to 3 digits.
2. A part is rejected if it has a leading zero and length above one, or exceeds 255.
3. After four parts, the answer is valid only if all digits have been consumed.
4. Pruning by these rules leaves very few branches: at most 3^4 = 81 leaves are ever reached.

```
void restore(const string& s, int pos, int parts, string& cur,
             vector<string>& out) {
    if (parts == 4) {
        if (pos == (int)s.size())                    // all digits used
            out.push_back(cur.substr(0, cur.size() - 1));   // drop last '.'
        return;
    }
    size_t keep = cur.size();
    for (int len = 1; len <= 3 && pos + len <= (int)s.size(); ++len) {
        string seg = s.substr(pos, len);
        if ((len > 1 && seg[0] == '0') || stoi(seg) > 255) continue;
        cur += seg + '.';  // choose
        restore(s, pos + len, parts + 1, cur, out);  // explore
        cur.resize(keep);  // un-choose
    }
}
```

1. 4 edges (5 nodes): each key becomes the right child of the previous one, so operations cost O(*n*).↩
2. In-order (left, node, right).↩
3. Nine: the root and one node per bit of the prefix, eight in all.↩
4. The walk consumes one address bit per step and stops after at most 32 steps, whatever else the trie holds.↩
5. Otherwise a vertex reachable by two paths can be queued twice before either copy is popped, doing the work again.↩
6. It may settle a vertex with a distance that a later negative edge would have lowered, and it never revisits settled vertices.↩
7. The two endpoints were already connected, so the edge would close a loop.↩

Part lengths tried	Parts	Outcome
3, 3, 2, 3	255.255.11.135	all 11 digits used: accepted
3, 3, 3, 2	255.255.111.35	all 11 digits used: accepted
3, 3, 1, ?	255.255.1.?	the remaining 1135 has four digits, no length 1–3 uses them all: dead end
2, 2, 2, 3	25.52.55.111	four parts built but two digits are left over: rejected
2, 5, ...	25.5... then 525	a part such as 511 exceeds 255 and is skipped

Result for `25525511135`: `255.255.11.135` and `255.255.111.35`. Call as `restore(s, 0, 0, cur, out)` with an empty `cur`.

TRAP

Claim: Python's recursion handles a depth-first search of any size. *Why it is wrong:* CPython allows about 1000 nested calls by default, and a path graph of 10^5 vertices raises `RecursionError`; C overflows its stack in the same way. *Correct statement:* use an explicit stack for deep graphs, or raise the limit only when the depth is known and modest.

1.13 Sorting, binary search and intervals MUST · ■■□

BIG IDEA

Sorting costs O(*n* log *n*) once and then makes many questions cheap, because order lets us discard half the data at each comparison.

Comparison sorts cannot beat O(*n* log *n*); the table of algorithms is in the Revision Spread (the Revision Spread). In C, `qsort` takes a comparator that must return negative, zero or positive, and it must never be written as `return a - b`, which overflows for large values.

```c
int cmp_int(const void *pa, const void *pb) {
    int a = *(const int *)pa, b = *(const int *)pb;
    return (a > b) - (a < b);  /* -1, 0 or 1; no overflow */
}
/* qsort(arr, n, sizeof arr[0], cmp_int); */
```

Binary search

Idea. In a sorted range, compare with the middle element and discard the half that cannot contain the target. We use one canonical form with a half-open range `[lo, hi)`, which finds the first index whose value is not less than *x*, termed the *lower bound*. Every variant (first occurrence, insertion point, first true of a monotone predicate) is this loop.

```cpp
int lowerBound(const vector<int>& a, int x) {   // first a[i] >= x, or n
    int lo = 0, hi = (int)a.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;  // not (lo + hi) / 2: avoids overflow
        if (a[mid] < x) lo = mid + 1; else hi = mid;
    }
    return lo;
}
```

The two classic bugs are the midpoint overflow in `(lo + hi) / 2` for ranges beyond 2^30, and an update such as `lo = mid`, which loops forever when `hi = lo + 1`. The loop runs $\lceil \log_2 n \rceil$ times.

Binary search on the answer. When the answer is a number and a yes/no test "is *x* enough?" is monotone (false up to a threshold, true after it), the same loop searches the range of possible answers. *Recognise:* "minimise the maximum", "smallest capacity such that …", "at most *D* days".

WORKED EXAMPLE

Ship packages within D days. Packages of weights `1..10` must be shipped in order; a ship carries at most its capacity per day. Find the smallest capacity that ships everything in 5 days.

1. The capacity is at least the heaviest package (10) and at most the total (55).
2. For a trial capacity, load greedily and count days: a larger capacity never needs more days, so the test is monotone.
3. Search the range for the first capacity needing at most 5 days.

```
int shipWithinDays(const vector<int>& w, int days) {
    int lo = *max_element(w.begin(), w.end());
    int hi = accumulate(w.begin(), w.end(), 0);  // answer lies in [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2, need = 1, load = 0;
        for (int x : w) {
            if (load + x > mid) { ++need; load = 0; }  // start the next day
            load += x;
        }
        if (need <= days) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

lo	hi	mid	days needed	action
10	55	32	2	hi = 32
10	32	21	3	hi = 21
10	21	15	5	hi = 15
10	15	12	6	lo = 13
13	15	14	6	lo = 15

Result: 15. Time O(*n* log(sum)).

Intervals

Idea. Sorting intervals by start brings overlapping ones next to each other, so one pass can merge them: an interval overlaps the last merged interval when its start is at most that interval's end. *Recognise:* "merge", "overlapping", "meeting rooms", "insert interval", IP address ranges (as pairs of 32-bit integers).

```
vector<array<long long, 2>> mergeIntervals(vector<array<long long, 2>> v) {
    sort(v.begin(), v.end());  // by start, then end
    vector<array<long long, 2>> out;
    for (auto& iv : v) {
        if (!out.empty() && iv[0] <= out.back()[1])   // overlaps the last
            out.back()[1] = max(out.back()[1], iv[1]);
        else
            out.push_back(iv);
    }
    return out;
}
```

For `[1,3], [2,6], [8,10], [15,18]`: `[2,6]` starts at $2 \le 3$ and extends the first interval to `[1,6]`; `[8,10]` starts after 6 and is kept; `[15,18]` is kept. Whether `[1,2]` and `[3,4]` should merge (as integer ranges they are adjacent) or `[1,2]` and `[2,3]` should (they touch) is a question to ask; the code above merges touching intervals but not adjacent ones, and `iv[0] <= out.back()[1] + 1` merges adjacent integer ranges as well. Time O(n log n) for the sort.

> CHECK YOURSELF
>
> 1. Why does `lo = mid` in a half-open search risk an infinite loop? [1]
> 2. After sorting by start, why is it enough to compare with only the last merged interval? [2]

1.14 Dynamic programming essentials SHOULD · ■■■

> BIG IDEA
>
> Dynamic programming replaces an exponential recursion by a table, because the same subproblems recur and each need be solved only once.

Recognise: "number of ways", "minimum or maximum cost", "can we reach", and a choice at each step whose consequences depend only on a small *state*. The routine has five steps: (i) define the state (what `dp[i]` means in words); (ii) write the recurrence relating it to smaller states; (iii) set the base cases; (iv) choose an evaluation order in which dependencies come first; (v) read the answer from the right cell. Space often reduces to the last row or two.

Family	State	Recurrence (sketch)
linear	`dp[i]` best for prefix *i*	`dp[i] = f(dp[i-1], dp[i-2])` (stairs, house robber)
knapsack / coins	`dp[c]` for capacity or amount *c*	`dp[c] = min(dp[c - coin] + 1)`
subsequence	`dp[i][j]` for prefixes of two strings	match: `dp[i-1][j-1] + 1`; else max or min of neighbours (LCS, edit distance)
grid	`dp[r][c]`	from the cell above and the cell to the left
increasing subsequence	`dp[i]` ending at *i*	`1 + max dp[j]` over smaller `a[j]`, j < i

> WORKED EXAMPLE
>
> **Coin change.** With coins `{1, 3, 4}`, find the fewest coins that sum to 6, or −1 if impossible.
>
> 1. State: `dp[a]` is the fewest coins for amount *a*.

2. Recurrence: `dp[a] = 1 + min over coins c ≤ a of dp[a - c]`.
3. Base: `dp[0] = 0`; unreachable amounts hold a large sentinel.
4. Fill *a* = 1 to 6 in increasing order, since `dp[a]` needs smaller amounts.

```
int coinChange(const vector<int>& coins, int amount) {
    const int INF = 1e9;
    vector<int> dp(amount + 1, INF); dp[0] = 0;
    for (int a = 1; a <= amount; ++a)
        for (int c : coins)
            if (c <= a && dp[a - c] + 1 < dp[a]) dp[a] = dp[a - c] + 1;
    return dp[amount] >= INF ? -1 : dp[amount];
}
```

a	0	1	2	3	4	5	6
dp[a]	0	1	2	1	1	2	2

Result: 2 (3 + 3). A greedy choice of the largest coin gives 4 + 1 + 1 = 3 coins, which shows why greedy is unsafe here. Time O(amount × coins), space O(amount).

When counting *combinations* (order ignored), the loop over coins is the outer loop; when counting *sequences* (order matters), the loop over amounts is the outer loop. Swapping them changes the answer, and this is the standard knapsack bug. For two-string problems, edit distance and longest common subsequence are solved in the Problem Set with a two-dimensional table.

1.15 Design-style coding: caches and rate limiters MUST · ■■■

BIG IDEA

Design-style questions combine two structures so that each covers the other's weakness: a hash map gives O(1) lookup, a doubly linked list gives O(1) reordering, and a counter driven by a clock gives rate control without storing history.

LRU cache

A *least-recently-used* (LRU) cache holds at most *c* entries and, when full, evicts the entry that was used longest ago. Router software uses the idea for ARP and route caches. The interface is `get(key)`, which returns the value or −1 and marks the entry as most recent, and `put(key, value)`, which inserts or updates and evicts if over capacity. Both must be O(1).

No single structure does both jobs: a hash map finds an entry in O(1) but keeps no order, and a list keeps order but finds in O(*n*). Combining them works because the map stores, for each key, the *address of its list node*; and a doubly linked list can unlink a node it has the address of in O(1), since the node knows both neighbours.

@

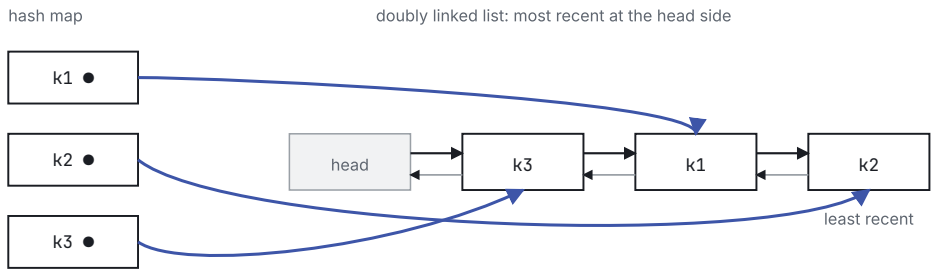


Figure 1.12 LRU cache: the map finds a node in O(1); the list orders nodes by recency and lets us unlink one in O(1) because the node holds both neighbours.

```
class LRU {
    using Item = pair<int, int>;                          // (key, value)
    size_t cap;
    list<Item> items;  // front = most recent
    unordered_map<int, list<Item>::iterator> pos;         // key -> node
public:
    explicit LRU(size_t c) : cap(c) {}
    int get(int k) {
        auto it = pos.find(k);
        if (it == pos.end()) return -1;
        items.splice(items.begin(), items, it->second);   // to front, O(1)
        return it->second->second;
    }
    void put(int k, int v) {
        if (cap == 0) return;
        auto it = pos.find(k);
        if (it != pos.end()) {  // update and refresh
            it->second->second = v;
            items.splice(items.begin(), items, it->second);
            return;
        }
        if (items.size() == cap) {  // evict the least recent
            pos.erase(items.back().first);  // erase from the map as well
            items.pop_back();
        }
        items.emplace_front(k, v);
        pos[k] = items.begin();
    }
};
```

WORKED EXAMPLE

Trace of a cache of capacity 2.

Operation	List after (front = most recent)	Result
put(1,1)	[1]	
put(2,2)	[2, 1]	
get(1)	[1, 2]	1
put(3,3)	[3, 1] (key 2 evicted)	
get(2)	[3, 1]	−1

put(4,4)	[4, 3] (key 1 evicted)	
get(1)	[4, 3]	−1
get(3)	[3, 4]	3
get(4)	[4, 3]	4

Result: every operation is O(1) on average: one hash lookup and at most four pointer writes.

TRAP

Claim: on eviction it is enough to remove the least recent node from the list. *Why it is wrong:* the map still holds a pointer to the freed node, so a later `get` of that key reads freed memory or reports a stale hit. *Correct statement:* every list change must be mirrored in the map: erase the key on eviction, and re-point it on insertion.

Variants. With a time-to-live (an ARP entry expires after a fixed time), store an expiry time in each node and treat an expired entry as a miss on `get`; entries expire lazily, or a sweep removes them from the tail end. For thread safety, a single mutex around `get` and `put` is correct because even `get` modifies the list; sharding the cache by key hash across several mutexes reduces contention. LFU (least frequently used) needs a second ordering and is solved in the Problem Set.

Token-bucket rate limiter

A *token bucket* limits the average rate to *r* tokens per second while allowing bursts of at most *B* tokens. The bucket holds up to *B* tokens, and tokens arrive at *r* per second. A packet of size *s* may pass if the bucket holds at least *s* tokens, which it then removes; otherwise the packet is dropped (policing) or delayed (shaping). In router terms, *r* is the committed information rate and *B* the committed burst size.

@

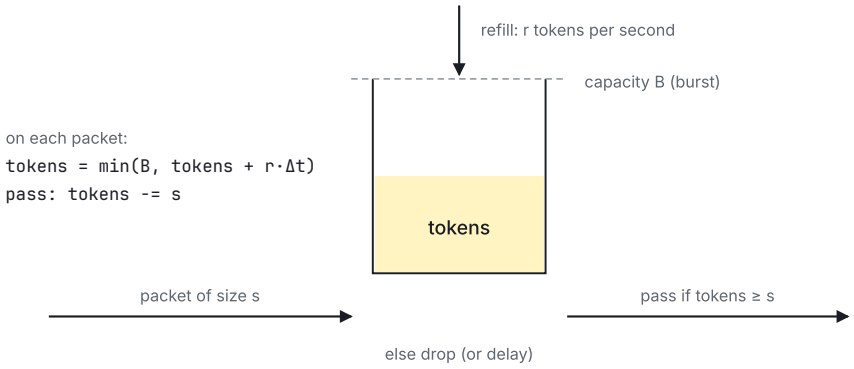


Figure 1.13 A token bucket. The long-run rate is r and the largest burst is B, because the level can never exceed the capacity.

We do not run a timer. Instead, on each packet we compute how many tokens would have arrived since the last update, add them, and cap the total at *B*. The state is only the current level and the last timestamp. Two details matter. First, time must come from a monotonic clock, because wall-clock time can jump backwards. Secondly, floating-point accumulation drifts, so we use integers: the level is stored in *micro-tokens* (tokens × 10^6), and with a clock in microseconds the refill over Δt µs is exactly `r × Δt` micro-tokens.

```c
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    uint64_t rate;       /* tokens (bytes) per second           */
    uint64_t burst;      /* capacity B in tokens                */
    uint64_t level;      /* current level, in micro-tokens      */
    uint64_t last_us;    /* timestamp of the last update (µs)   */
} Bucket;

void tb_init(Bucket *b, uint64_t rate, uint64_t burst, uint64_t now_us) {
    b->rate = rate; b->burst = burst;
    b->level = burst * 1000000ull;                /* start full */
    b->last_us = now_us;
}
bool tb_allow(Bucket *b, uint64_t size, uint64_t now_us) {
    uint64_t cap = b->burst * 1000000ull;
    uint64_t dt = now_us - b->last_us;  /* monotonic: never negative */
    b->last_us = now_us;
    if (b->rate && dt > cap / b->rate)  /* long idle: bucket is full, and */
        b->level = cap;  /* r*dt could overflow otherwise   */
    else {
        b->level += b->rate * dt;
        if (b->level > cap) b->level = cap;
    }
    uint64_t need = size * 1000000ull;
    if (b->level < need) return false;  /* not enough tokens: drop */
    b->level -= need;
    return true;
}
```

WORKED EXAMPLE

Rate 1000 bytes per second, burst 1500 bytes. The bucket starts full (1500 tokens).

Time (s)	Packet (bytes)	Tokens before	Decision	Tokens after
0.0	1500	1500	pass	0
0.5	600	500	drop (500 < 600)	500
0.7	600	700	pass	100
2.7	1500	1500 (capped; 2100 earned)	pass	0

Result: the long-run rate is 1000 bytes per second and no burst exceeds 1500 bytes. Note that the dropped packet still advances the timestamp, so its interval is not credited twice.

Scheme	Memory	Behaviour at the boundary	Use
fixed window counter	one counter	up to 2× the limit across a window edge	simplest; coarse
sliding window log	one timestamp per request	exact	strict limits, small limits
token bucket	two numbers	bursts up to *B*, exact average	traffic policing, API limits
leaky bucket	queue and a drain rate	smooth output, no bursts	traffic shaping

A fixed-window counter of five requests per ten seconds accepts five requests at second 9 and five more at second 11, ten in two seconds. A sliding log and a token bucket reject the excess; both are asked as follow-ups in the Problem Set.

> INTERVIEW LENS
>
> MICROSOFT · AMAZON · CISCO · JUNIPER
>
> For the LRU cache the expected sequence is: hash map plus doubly linked list, the O(1) argument, code, then TTL and thread safety. For the rate limiter, interviewers listen for “token bucket”, the two parameters, refill-on-demand rather than a timer thread, the monotonic clock and the integer arithmetic. A strong answer also names the concurrency model and what happens at the boundary.

1.16 Finite-state machines SHOULD · ■■□

> BIG IDEA
>
> A finite-state machine replaces tangled conditionals by a table that gives the next state for every state and event, so the protocol logic can be read, checked and extended as data.

A *finite-state machine* (FSM) has a set of states, a set of events (or input symbols), a start state and a transition function from (state, event) to the next state; an event with no entry is illegal. Protocols such as TCP, link-state adjacency formation and DHCP are specified this way, and Juniper and Broadcom interviews ask for one in C. *Recognise:* “validate a sequence of events”, “parse a protocol”, “valid number” (a number grammar is an FSM over characters).

@

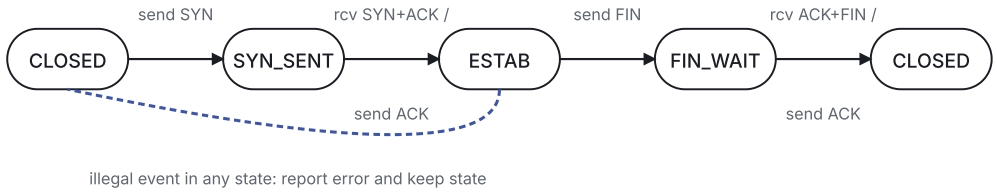


Figure 1.14 A reduced connection state machine (handshake and teardown only). A table indexed by state and event replaces nested conditionals.

A two-dimensional array indexed by state and event is the compact form. Entries of −1 mark illegal transitions; the driver loop reads the table and stops at the first illegal event.

```c
#include <stddef.h>
#include <stdint.h>

typedef enum { CLOSED, SYN_SENT, ESTAB, FIN_WAIT, NSTATES } State;
typedef enum { E_SEND_SYN, E_RCV_SYNACK, E_SEND_FIN, E_RCV_FINACK,
               NEVENTS } Event;

static const int8_t next_state[NSTATES][NEVENTS] = {
    /* CLOSED   */ { SYN_SENT, -1,    -1,       -1     },
    /* SYN_SENT */ { -1,       ESTAB, -1,       -1     },
    /* ESTAB    */ { -1,       -1,    FIN_WAIT, -1     },
    /* FIN_WAIT */ { -1,       -1,    -1,       CLOSED },
};

/* Final state, or -1 if event *bad is illegal. */
int run_fsm(const Event *ev, size_t n, size_t *bad) {
    State s = CLOSED;
    for (size_t i = 0; i < n; i++) {
        int t = (ev[i] < NEVENTS) ? next_state[s][ev[i]] : -1;
        if (t < 0) { *bad = i; return -1; }
        s = (State)t;
    }
    return s;
}
```

For the events (send SYN, receive SYN+ACK, send FIN) the machine moves CLOSED → SYN_SENT → ESTAB → FIN_WAIT and returns FIN_WAIT. For (send SYN, send FIN) the second event is illegal in SYN_SENT, so the function returns −1 with `*bad = 1`. When actions must accompany transitions, the table cells hold function pointers instead of states, which is how production protocol code is written. Table size is *states* × *events*, so the transition lookup is O(1).

> CHECK YOURSELF
>
> 1. Why is a table preferable to nested `switch` statements for a protocol with 11 states? [3]
> 2. How does an FSM relate to “valid number”? [4]

1.17 Testing your own code and interview performance

MUST · ■□□

> BIG IDEA
>
> Code that has been traced by hand on one normal case and three edge cases is usually correct, and showing that trace is itself what the interviewer is assessing.

Before declaring a solution finished, one performs four checks in order. (i) *Dry-run*: execute the code on the worked example, line by line, keeping a small table of variable values, exactly as this chapter’s tables do. (ii) *Edge cases*: run the list in the Revision Spread (the Revision Spread): empty input, one element, duplicates, extremes of the value range, and for networking input the “/0”, “/32”, leading-zero and 256 cases. (iii) *Complexity*: state time and space, and justify them by pointing to the loop or the recursion. (iv) *Cross-check*: if time remains, compare against the brute force on small inputs.

In C, a tiny harness costs a minute and catches most mistakes:

```c
#include <stdio.h>
static int fails;
#define CHECK(c) do { if (!(c)) { fails++; \
    printf("FAIL line %d: %s\n", __LINE__, #c); } } while (0)

int main(void) {
    uint32_t a;
    CHECK(parse_ipv4("10.0.0.1", &a) && a == 0x0A000001u);
    CHECK(!parse_ipv4("01.0.0.1", &a));
    CHECK(mask_of(0) == 0 && mask_of(32) == 0xFFFFFFFFu);
    printf(fails ? "%d failures\n" : "all passed\n", fails);
    return fails != 0;
}
```

Symptom	Likely cause	Where to look
crash on empty input	missing null or length guard	first lines of the function
off by one in the last element	< against <=, or a window not evicted	loop bounds, eviction index
works on small input, times out on large	hidden O(*n*) inside a loop (`strlen`, list `size`, `erase` at front)	the loop body
wrong answer only with duplicates	strict versus non-strict comparison	comparisons and the map update
garbage or a crash after `free`	use after free, or a stale pointer in a second structure	the deletion path
sign or overflow surprises	`int` where `long long` or unsigned is needed	sums, shifts, products

Communication. A round is scored on four things: the correctness of the code, the choice of algorithm, the testing, and the communication. The following habits cost nothing. State the brute force first and its cost, then the improvement and why it is valid. Name the invariant of each loop in a comment. When stuck, say what repeated work you are trying to remove. When given a hint, adopt it openly. Never code silently for more than two minutes, and never claim a complexity without pointing at the loop that produces it.

Minute (45-min round)	Step
0–5	restate, ask about size, duplicates, negatives, empty input; work one example
5–15	brute force, optimisation, and agreement on the plan
15–32	write code in small named pieces, narrating intent
32–38	dry-run and edge cases
38–45	complexity, trade-offs and follow-ups

> CHECK YOURSELF
>
> 1. What are the first two edge cases to try on a linked-list function? [5]
> 2. Why state the brute force before the optimised method? [6]

1. With `hi = lo + 1`, `mid` equals `lo`, so the range never shrinks.↩
2. Every earlier merged interval ends before the last one starts, so a later interval cannot reach them.↩
3. The table is data, so the full set of transitions can be reviewed at a glance and tested exhaustively; nested switches hide missing cases.↩
4. States record how much of a number has been read (sign, digits, point, exponent), and each character class is an event.↩
5. The empty list and the one-node list, followed by operating on the head and the tail.↩
6. It shows the problem is understood, gives a correct fallback, and makes the source of the improvement explicit.↩

Problem Set

Every question of this pillar is solved once, here. Difficulty is ■□□ (foundation), ■■□ (typical test) or ■■■ (hardest interviews). The tag **S** means that candidates report this specific question; **T** means that the topic is reported and the wording is ours. Round names are OA (online assessment), TECH-1 and TECH-2 (technical interviews) and HM (hiring manager). Code is C++ for general algorithms and C for networking and systems problems, and each solution is a complete function. The first table lists the problems whose full solutions are already in the teaching sections, so that nothing is solved twice.

Q	Problem	Tags	Solved in
Q1.1	Two Sum: indices of two values adding to a target	CISCO · AMAZON · OA · ■□□ · S	§1.1 (p. 9)
Q1.2	Longest substring without repeating characters	CISCO · OA · ■■□ · S	§1.2 (p. 10)
Q1.3	Number of subarrays with sum *k* (negatives allowed)	GOOGLE · MICROSOFT · TECH-1 · ■■□ · T	§1.2 (p. 10)
Q1.4	Longest consecutive sequence in O(*n*)	GOOGLE · TECH-1 · ■■□ · T	§1.3 (p. 14)
Q1.5	Validate a dotted-quad IPv4 address; parse a MAC address	CISCO · AMAZON · OA · ■■□ · T	§1.4 (p. 16)
Q1.6	Are two IPs in the same subnet (mask or prefix)?	CISCO · AMAZON · OA · ■□□ · T	§1.5 (p. 17)
Q1.7	Network, broadcast, first, last and host count of 10.20.30.40/22	CISCO · TECH-1 · ■■□ · T	§1.5 (p. 17)
Q1.8	Set, clear and toggle the *k*-th bit as macros	QUALCOMM · TECH-1 · ■□□ · S	§1.5 (p. 17)
Q1.9	Delete a node by value (head included); delete all matches	ARISTA · OA · ■■□ · S	§1.6 (p. 20)
Q1.10	Reverse a singly linked list	BROADCOM · ARISTA · TECH-1 · ■□□ · S	§1.6 (p. 20)
Q1.11	Merge two sorted linked lists	ARISTA · TECH-1 · ■□□ · S	§1.6 (p. 20)
Q1.12	Detect a cycle; find where it starts; prove it	JUNIPER · NOKIA · TECH-1 · ■■□ · S	§1.6 (p. 20)
Q1.13	Middle node of a linked list	NOKIA · TECH-1 · ■□□ · T	§1.6 (p. 20)
Q1.14	Circular buffer for a producer and a consumer: full versus empty	BROADCOM · QUALCOMM · ARISTA · TECH-1 · ■■□ · T	§1.7 (p. 24)
Q1.15	Balanced brackets	NOKIA · OA · ■□□ · S	§1.7 (p. 24)
Q1.16	Days until a warmer temperature (next greater element)	MICROSOFT · TECH-1 · ■■□ · T	§1.7 (p. 24)

Q1.17	BST insert, search, delete; the effect of sorted input	ARISTA · TECH-1 · ■■□ · S	§1.8 (p. 27)
Q1.18	Validate a BST; level-order traversal	ARISTA · GOOGLE · TECH-1 · ■■□ · S	§1.8 (p. 27)
Q1.19	Longest-prefix match of an IP against a list of prefixes	GOOGLE · TECH-1 · ■■■ · S	§1.9 (p. 30)
Q1.20	Shortest paths in a weighted network; link-state versus distance-vector	CISCO · GOOGLE · TECH-1 · ■■□ · T	§1.11 (p. 34)
Q1.21	Depth-first search without recursion	NOKIA · OA · ■■□ · S	§1.11 (p. 34)
Q1.22	Restore IP addresses from a digit string	AMAZON · MICROSOFT · TECH-1 · ■■□ · T	§1.12 (p. 39)
Q1.23	Binary search: lower bound; minimum capacity to ship in *D* days	GOOGLE · MICROSOFT · TECH-1 · ■■□ · T	§1.13 (p. 40)
Q1.24	Merge overlapping intervals	GOOGLE · AMAZON · TECH-1 · ■■□ · T	§1.13 (p. 40)
Q1.25	Coin change: fewest coins for an amount	MICROSOFT · GOOGLE · TECH-1 · ■■□ · T	§1.14 (p. 42)
Q1.26	LRU cache with O(1) `get` and `put`	MICROSOFT · AMAZON · TECH-1 · ■■□ · S	§1.15 (p. 43)
Q1.27	Token-bucket rate limiter and its router parameters	CISCO · JUNIPER · ARISTA · MICROSOFT · TECH-1 · ■■□ · T	§1.15 (p. 43)
Q1.28	Validate a sequence of TCP-style events against a state machine	JUNIPER · BROADCOM · TECH-1 · ■■□ · S	§1.16 (p. 47)

Arrays, hashing, strings and bits

Q1.29 Merge two sorted arrays in place

JUNIPER · OA · ■□□ · S

Array `a` holds `m` sorted values followed by `n` unused cells; array `b` holds `n` sorted values. Merge `b` into `a` with no extra array.

30-second answer: fill `a` from the back, taking the larger of the two current tails, so no unread value is overwritten.

Approach. Writing from the front would overwrite elements of `a` not yet read; writing from the back cannot, because the free cells are at the back. When `a` runs out, the rest of `b` is copied; when `b` runs out, the rest of `a` is already in place.

```
void mergeInto(vector<int>& a, int m,
               vector<int>& b, int n) {
  int i = m - 1, j = n - 1, k = m + n -
1;
  while (j >= 0)
    a[k--] = (i >= 0 && a[i] > b[j])
             ? a[i--] : b[j--];
}
```

Complexity. O($m + n$) time, O(1) space. **Follow-ups.** Stability (use `>` so equal values keep the order of `a` first). **Mistakes.** Looping while `i >= 0 && j >= 0` and forgetting to copy the remainder of `b`.

Q1.30 Rotate an array right by one, then by k

JUNIPER · TECH-1 · ■□□ · S

Rotate an array right by `k` positions in O(1) extra space.

30-second answer: reverse the whole array, then reverse the first `k` and the last `n-k` elements.

Approach. For one step, save the last element, shift the rest right, and store it at the front: O(n). For k steps, rotating one at a time costs O(nk). Three reversals do it in O(n): reversing everything puts the last k elements first but backwards, and reversing each part restores their order. Reduce k modulo n first.

```
void rotateRight(vector<int>& a, int k)
{
  int n = a.size();
  if (!n) return;
  k %= n;
  reverse(a.begin(), a.end());
  reverse(a.begin(), a.begin() + k);
  reverse(a.begin() + k, a.end());
}
```

[1,2,3,4,5], $k = 2$: reversed 5,4,3,2,1; first two reversed 4,5,3,2,1; rest reversed 4,5,1,2,3, which is the right rotation. **Complexity.** O(n) time, O(1) space. **Mistakes.** Forgetting `k %= n` (and `n == 0`).

Q1.31 Move zeroes and remove an element in place

JUNIPER · AMAZON · OA · ■□□ · T

Move all zeroes of an array to the end, keeping the order of the other elements.

30-second answer: read/write pointers: copy each non-zero forward, then fill the tail with zeroes.

```
void moveZeroes(vector<int>& a) {
  size_t w = 0;
  for (size_t r = 0; r < a.size(); ++r)
    if (a[r]) a[w++] = a[r];
  while (w < a.size()) a[w++] = 0;
}
```

Removing a given value is the same loop with the test `a[r] != val`, returning `w`. **Complexity.** O(n) time, O(1) space, one write per kept element. **Mistakes.** Starting `w` at 1, or swapping elements needlessly (correct, but it does more writes).

Q1.32 Three Sum

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

Return all unique triples of an array whose sum is zero.

30-second answer: sort, fix the first value, and run opposite-end two pointers on the rest, skipping duplicates.

Approach. Brute force is O(n^3). After sorting, for each `i` the pair sum `-a[i]` is searched with two pointers in O(n), giving O(n^2). Uniqueness comes from skipping equal values at `i`, and, after a hit, at both pointers.

```
vector<vector<int>>
threeSum(vector<int> a) {
  sort(a.begin(), a.end());
  vector<vector<int>> r;
  int n = a.size();
  for (int i = 0; i + 2 < n; ++i) {
    if (i && a[i] == a[i-1]) continue;
    int l = i + 1, h = n - 1;
    while (l < h) {
      long long s = (long long)a[i] +
a[l] + a[h];
      if (s < 0) ++l;
      else if (s > 0) --h;
      else {
        r.push_back({a[i], a[l], a[h]});
        ++l; --h;
        while (l < h && a[l] == a[l-1])
++l;
      }
    }
  }
  return r;
}
```

Complexity. O(n^2) time, O(1) extra. **Mistakes.** Not skipping duplicates (repeated triples); using `int` for the sum.

Q1.33 Container with most water

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Given heights `h[i]`*, choose two lines maximising* `min(h[i], h[j]) * (j - i)`.

30-second answer: start with the widest pair and always move the pointer at the shorter line, because only a taller line can improve the area.

Approach. Moving the taller pointer can only reduce the width and cannot raise the minimum height, so it never helps; moving the shorter one might.

```
int maxArea(const vector<int>& h) {
  int l = 0, r = h.size() - 1, best = 0;
  while (l < r) {
    best = max(best,
               min(h[l], h[r]) * (r -
l));
    if (h[l] < h[r]) ++l; else --r;
  }
  return best;
}
```

Complexity. O(n) time, O(1) space.

Q1.34 Product of array except self

AMAZON · MICROSOFT · TECH-1 · ■■□ · T

Return an array whose `i`*-th element is the product of all others, without division and in O(*n*).*

30-second answer: the answer at `i` is (product of the elements before `i`) times (product of the elements after `i`); build the first in a forward pass and multiply in the second in a backward pass.

```
vector<int> productExceptSelf(
    const vector<int>& a) {
  int n = a.size();
  vector<int> r(n, 1);
  for (int i = 1; i < n; ++i)
    r[i] = r[i-1] * a[i-1];
  for (int i = n - 1, s = 1; i >= 0; --
i) {
    r[i] *= s;
    s *= a[i];
  }
  return r;
}
```

Complexity. O(n) time, O(1) extra beyond the output. **Mistakes.** Using division fails when a value is zero. **Follow-ups.** The same prefix-and-suffix idea gives “trapping rain water”.

Q1.35 Maximum subarray (Kadane)

MICROSOFT · GOOGLE · OA · ■■□ · T

Find the largest sum of a non-empty contiguous subarray.

30-second answer: at each element, either extend the best subarray ending before it or start afresh; keep the maximum seen.

```
int maxSubarray(const vector<int>& a) {
  int cur = a[0], best = a[0];
  for (size_t i = 1; i < a.size(); ++i)
{
    cur = max(a[i], cur + a[i]);
    best = max(best, cur);
  }
  return best;
}
```

For `[-2,1,-3,4,-1,2,1,-5,4]`: `cur` runs −2, 1, −2, 4, 3, 5, 6, 1, 5 and `best` is 6 (the subarray `4,-1,2,1`). **Complexity.** O(*n*) time, O(1) space. **Mistakes.** Initialising `best` to 0, which is wrong when every value is negative.

Q1.36 Group anagrams

AMAZON · GOOGLE · OA · ■■□ · T

Group the strings of a list that are anagrams of each other.

30-second answer: two strings are anagrams exactly when their sorted letters agree, so use the sorted string as the map key.

```
vector<vector<string>> groupAnagrams(
    const vector<string>& v) {
  unordered_map<string, vector<string>>
m;
  for (auto& s : v) {
    string k = s;
    sort(k.begin(), k.end());
    m[k].push_back(s);
  }
  vector<vector<string>> r;
  for (auto& p : m)
r.push_back(p.second);
  return r;
}
```

Complexity. O($N\,L \log L$) for N strings of length at most L. **Follow-ups.** For lowercase letters only, a 26-count signature as the key gives O(NL).

Q1.37 Extract IPs from a log and sort by frequency

GOOGLE · TECH-1 · ■□□ · S

Read a log, find every IPv4 address, and print the addresses ordered by how often they occur (most frequent first, ties by address).

30-second answer: split the input into tokens, keep those that parse as IPv4, count them in a hash map, then sort the (address, count) pairs.

Approach. Validation reuses `parse_ipv4` of §1.4 (p. 16). A single pass counts; the sort costs O(d log d) for d distinct addresses. A heap of size k replaces the sort when only the top k are wanted.

```
extern "C" bool parse_ipv4(const char*,
                           uint32_t*);
vector<pair<string,int>> topIPs(is-
tream& in) {
  unordered_map<string,int> cnt;
  string t; uint32_t x;
  while (in >> t)
    if (parse_ipv4(t.c_str(), &x))
++cnt[t];
  vector<pair<string,int>> v(cnt.be-
gin(),
                             cnt.end());
  sort(v.begin(), v.end(),
       [](auto& a, auto& b) {
    return a.second != b.second
        ? a.second > b.second
        : a.first < b.first; });
  return v;
}
```

Complexity. O($N + d \log d$) time, O(d) space. **Mistakes.** Counting `10.0.0.1:80` or `10.0.0.1,` as a different token; real logs need a scan for the dotted pattern rather than whitespace tokens.

Q1.38 Design a hash map in C

ARISTA · JUNIPER · TECH-1 · ■■□ · T

Implement `put`, `get` *and* `remove` *for integer keys without a library.*

30-second answer: an array of bucket heads, each a linked list; the bucket is `key % size`, and collisions share a list.

```
#define B 1009
typedef struct E { int k, v;
                   struct E *next; } E;
static E *tab[B];

static E **slot(int k) {   /* link to
entry */
  E **pp = &tab[(unsigned)k % B];
  while (*pp && (*pp)->k != k)
    pp = &(*pp)->next;
  return pp;
}
void put(int k, int v) {
  E **pp = slot(k);
  if (*pp) { (*pp)->v = v; return; }
  E *e = malloc(sizeof *e);
  if (!e) return;
  e->k = k; e->v = v; e->next = NULL;
  *pp = e;
}
int get(int k, int *out) {
  E *e = *slot(k);
  if (!e) return 0;
  *out = e->v; return 1;
}
void remove_key(int k) {
  E **pp = slot(k);
  if (*pp) {
    E *d = *pp; *pp = d->next; free(d);
  }
}
```

Complexity. O(1) average, O(*n*) worst. **Follow-ups.** Resize when the load factor exceeds one; a prime size spreads keys evenly for simple moduli. The pointer-to-pointer idiom of §1.6 (p. 20) removes the head special case.

Q1.39 Classify a string as IPv4, IPv6 or neither

CISCO · AMAZON · MICROSOFT · TECH-1 · ■■□ · T

Return which kind of address a string is. IPv6 here means eight groups of one to four hexadecimal digits separated by colons (no `::` *compression).*

30-second answer: choose the parser by the separator, then validate group by group, rejecting empty groups, too many digits and trailing text.

Approach. IPv4 uses `parse_ipv4` of §1.4 (p. 16). IPv6 reads, for each of eight groups, one to four hex digits and then a colon (not after the last group), and requires the end of the string.

```
#include <ctype.h>
bool valid_ipv6(const char *s) {
  for (int g = 0; g < 8; g++) {
    int n = 0;
    while (isxdigit((unsigned char)*s))
{
      s++; n++;
    }
    if (n < 1 || n > 4) return false;
    if (g < 7 && *s++ != ':') return
false;
  }
  return *s == '\0';
}
```

Classification: contains `.` and no `:` → test IPv4; contains `:` → test IPv6; otherwise neither. **Complexity.** O(*n*). **Mistakes.** Accepting `01.1.1.1` or `1.1.1.1.`; letters beyond `f`; groups of five digits. Real parsers such as `inet_pton` also accept `::`, which needs a count of groups on each side.

Q1.40 String to integer (atoi) with clamping

QUALCOMM · BROADCOM · OA · ■■□ · T

Skip leading spaces, read an optional sign and digits, stop at the first other character, and clamp to the `int` range.

30-second answer: accumulate in a 64-bit value, stop growing once it passes the limit, apply the sign and clamp.

```
#include <limits.h>
int my_atoi(const char *s) {
  while (*s == ' ') s++;
  int neg = 0;
  if (*s == '+' || *s == '-')
    neg = (*s++ == '-');
  long long v = 0;
  while (*s >= '0' && *s <= '9') {
    v = v * 10 + (*s++ - '0');
    if (v > 2147483648LL) break;
  }
  if (neg) v = -v;
  if (v > INT_MAX) return INT_MAX;
  if (v < INT_MIN) return INT_MIN;
  return (int)v;
}
```

Complexity. O(*n*). **Mistakes.** Overflowing the accumulator before checking; treating `-` followed by a space as a number; mishandling `"-2147483648"`, which is exactly `INT_MIN`.

Q1.41 Compare version numbers

CISCO · AMAZON · OA · ■■□ · T

Compare `"1.01"` with `"1.001"`: dot-separated revisions are compared numerically from the left; missing revisions count as zero.

30-second answer: walk both strings at once, parse one number from each up to the next dot, and compare.

```
int compareVersion(const string& a,
                   const string& b) {
  size_t i = 0, j = 0;
  while (i < a.size() || j < b.size()) {
    long x = 0, y = 0;
    while (i < a.size() && a[i] != '.')
      x = x * 10 + (a[i++] - '0');
    while (j < b.size() && b[j] != '.')
      y = y * 10 + (b[j++] - '0');
    if (x != y) return x < y ? -1 : 1;
    ++i; ++j;
  }
  return 0;
}
```

Complexity. O(*n* + *m*) time, O(1) space. **Mistakes.** Comparing revisions as strings (so `"9"` > `"10"`); splitting into arrays when a scan suffices.

Q1.42 Expressive words (stretchy strings)

CISCO · OA · ■■□ · S

A word is stretchy to `S` if repeated letter groups in the word can be extended to length at least three to give `S`. Count the stretchy words in a list.

30-second answer: compare run-length groups: each pair of groups has the same letter, and the group in `S` either equals the word's group or is longer and has length at least three.

```
bool stretchy(const string& s,
              const string& w) {
  size_t i = 0, j = 0;
  while (i < s.size() && j < w.size()) {
    if (s[i] != w[j]) return false;
    size_t i2 = i, j2 = j;
    while (i2 < s.size() && s[i2] ==
s[i])
      ++i2;
    while (j2 < w.size() && w[j2] ==
w[j])
      ++j2;
    size_t a = i2 - i, b = j2 - j;
    if (a < b || (a != b && a < 3))
      return false;
    i = i2; j = j2;
  }
  return i == s.size() && j == w.size();
}
```

For `S = "heeellooo"`, `"hello"` is stretchy (e: 3 ≥ 3, l: 2 = 2, o: 3 ≥ 3) but `"hi"` is not. **Complexity.** O(| S | + |word|) per word. **Mistakes.** Allowing a group of size two to stretch (it must already match or reach three).

Q1.43 Longest common prefix of strings

GOOGLE · AMAZON · OA · ■□□ · T

Return the longest string that begins every string of a list.

30-second answer: compare column by column against the first string and stop at the first mismatch.

```
string lcp(const vector<string>& v) {
  if (v.empty()) return "";
  for (size_t i = 0; i < v[0].size();
++i)
    for (auto& s : v)
      if (i == s.size() || s[i] != v[0]
[i])
        return v[0].substr(0, i);
  return v[0];
}
```

Complexity. O(total characters). **Follow-ups.** For two IPv4 addresses the common prefix in *bits* is `clz(a ^ b)` (with 32 when equal), the shape of the CIDR block that covers both. A trie handles many strings with repeated queries.

Q1.44 Count the bits to flip to turn A into B

QUALCOMM · TECH-1 · ■□□ · S

How many bit positions differ between two integers?

30-second answer: XOR marks the differing positions; count the set bits of the result.

```
unsigned flips(uint32_t a, uint32_t b) {
  uint32_t x = a ^ b;
  unsigned c = 0;
  while (x) { x &= x - 1; c++; }
  return c;
}
```

`flips(10, 20)`: 01010 ^ 10100 = 11110, four set bits, so four flips. **Complexity.** O(number of differing bits). **Follow-ups.** The Hamming distance of two addresses; `__builtin_popcount` as the one-line form.

Q1.45 Multiply without the * operator

JUNIPER · TECH-1 · ■■□ · S

Multiply two integers using only shifts, additions and tests.

30-second answer: shift-and-add: for each set bit of the multiplier, add the shifted multiplicand.

```
unsigned mul(unsigned a, unsigned b) {
  unsigned r = 0;
  while (b) {
    if (b & 1) r += a;
    a <<= 1;
    b >>= 1;
  }
  return r;
}
```

`mul(6, 5)`: b = 101 adds 6 (b bit 0) and 6 << 2 = 24 (bit 2): 30. **Complexity.** O(bit-width) = O(32). **Follow-ups.** For signed operands, multiply the absolute values and apply the sign (the sign is the XOR of the signs). Also ask: what if the result overflows?

Q1.46 Power of two, and counting bits for 0 to n

QUALCOMM · BROADCOM · OA · ■□□ · T

Test whether x* is a power of two in one expression. Then return the popcount of every number from 0 to *n.**

30-second answer: a power of two has exactly one set bit, so `x && !(x & (x-1))`; for the table, `bits[i] = bits[i >> 1] + (i & 1)`.

```
bool isPow2(unsigned x) {
  return x && !(x & (x - 1));
}
vector<int> countBits(int n) {
  vector<int> b(n + 1, 0);
  for (int i = 1; i <= n; ++i)
    b[i] = b[i >> 1] + (i & 1);
  return b;
}
```

Complexity. O(1) and O(*n*). **Mistakes.** Omitting `x &&`, which wrongly accepts zero.

Q1.47 Reverse the bits of a 32-bit integer

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

Reverse the bit order of a `uint32_t`. *Make it fast for a million calls.*

30-second answer: a loop over 32 bits works; for many calls, precompute a 256-entry byte-reversal table and swap the four bytes.

```
static uint8_t rev8[256];
void init_rev(void) {
  for (int i = 0; i < 256; i++)
    rev8[i] = (rev8[i >> 1] >> 1)
              | ((i & 1) << 7);
}
uint32_t rev32(uint32_t x) {
  return (uint32_t)rev8[x & 255] << 24
       | (uint32_t)rev8[(x >> 8) & 255]
<< 16
       | (uint32_t)rev8[(x >> 16) &
255] << 8
       | rev8[x >> 24];
}
```

The table recurrence reverses `i` by reversing `i >> 1`, moving it down one place and putting the low bit of `i` on top. **Complexity.** O(1) per call after an O(256) set-up. **Follow-ups.** The mask-and-swap ladder (swap halves, then pairs, then nibbles, bytes) needs no table.

Q1.48 Single number (every other value appears twice, or thrice)

GOOGLE · QUALCOMM · OA · ■■□ · T

In an array where every value appears exactly twice except one, find the odd one out. Then solve it when the others appear three times.

30-second answer: twice: XOR everything, since equal values cancel. Thrice: keep, per bit, a count modulo 3 using two accumulators.

```
int single2(const vector<int>& a) {
  int x = 0;
  for (int v : a) x ^= v;
  return x;
}
int single3(const vector<int>& a) {
  int ones = 0, twos = 0;
  for (int v : a) {
    ones = (ones ^ v) & ~twos;
    twos = (twos ^ v) & ~ones;
  }
  return ones;
}
```

In `single3`, `ones` holds the bits seen once (mod 3) and `twos` those seen twice; a third occurrence clears both. **Complexity.** O(*n*) time, O(1) space.

Q1.49 Bitwise AND of a range

JUNIPER · GOOGLE · TECH-1 · ■■□ · T

Return the AND of all integers in [`m`, `n`].

30-second answer: the answer is the common binary prefix of `m` and `n`, followed by zeros; shift both right until they are equal, then shift back.

```
int rangeAnd(int m, int n) {
  int s = 0;
  while (m != n) { m >>= 1; n >>= 1;
++s; }
  return m << s;
}
```

Any bit position that differs between `m` and `n` takes both values somewhere in the range, so it ANDs to zero, along with every lower bit. This is the same idea as the common prefix of two IP addresses. **Complexity.** O(32).

Q1.50 Add without +; divide without /

JUNIPER · QUALCOMM · TECH-1 · ■■□ · T

Add two integers with bit operations only. Then divide two integers without `*`, `/` or `%`.

30-second answer: addition is XOR (sum without carries) plus AND shifted left (carries), repeated until no carry remains; division is shift-and-subtract from the highest bit down.

```
int add(int a, int b) {
  unsigned x = a, y = b;
  while (y) {
    unsigned c = (x & y) << 1;
    x ^= y;
    y = c;
  }
  return (int)x;
}
```

```
int divide(int a, int b) {
  if (a == INT_MIN && b == -1)
    return INT_MAX;
  bool neg = (a < 0) != (b < 0);
  unsigned long long x = llabs((long
long)a),
                     y = llabs((long
long)b), q = 0;
  for (int i = 31; i >= 0; --i)
    if ((x >> i) >= y) {
      x -= y << i;
      q |= 1ull << i;
    }
  return (int)(neg ? -(long long)q
                   : (long long)q);
}
```

Complexity. O(32) each. **Mistakes.** Negating `INT_MIN` in 32-bit arithmetic (undefined); not handling `INT_MIN / -1`, whose true result does not fit.

Q1.51 Convert an IP range to the fewest CIDR blocks

CISCO · GOOGLE · TECH-2 · ■■■ · T

Given a start address and a count `n`, cover exactly those `n` consecutive addresses with the fewest CIDR blocks.

30-second answer: repeatedly take the largest block that is aligned at the current address and no bigger than what remains.

Approach. A block of size 2^*k* must start at a multiple of 2^*k*, so the largest allowed size at address `a` is its lowest set bit, `a & -a` (the whole space when `a` is zero). Shrink the size by halves until it fits in `n`, emit the block with prefix length 32 − *k*, and advance.

```
vector<pair<uint32_t,int>> rangeToCidr(
    uint32_t a, uint64_t n) {
  vector<pair<uint32_t,int>> out;
  while (n > 0) {
    uint64_t size = a ? (a & -a) :
(1ull << 32);
    while (size > n) size >>= 1;
    out.push_back({a, 32 -
__builtin_ctzll(size)});
    a += size;
    n -= size;
  }
  return out;
}
```

For 255.0.0.7 and *n* = 10: block 255.0.0.7/32 (alignment 1), then 255.0.0.8/29 (alignment 8, 8 ≤ 9), then 255.0.0.16/32 (alignment 16 shrunk to 1). **Complexity.** O(32) blocks at most, each O(32). **Mistakes.** Choosing the largest power of two below `n` without the alignment test, which yields blocks that are not valid prefixes.

Q1.52 The Internet checksum

CISCO · JUNIPER · BROADCOM · TECH-1 · ■■□ · T

Compute the 16-bit one's-complement checksum of a byte buffer, as used by IPv4, ICMP, UDP and TCP.

30-second answer: add the data as 16-bit big-endian words, fold the carries back into the low 16 bits, and complement the result.

```
uint16_t checksum(const uint8_t *p,
size_t n) {
  uint32_t sum = 0;
  for (; n > 1; p += 2, n -= 2)
    sum += (uint32_t)p[0] << 8 | p[1];
  if (n) sum += (uint32_t)p[0] << 8;
  while (sum >> 16)
    sum = (sum & 0xFFFF) + (sum >> 16);
  return (uint16_t)~sum;
}
```

For the bytes `45 00 00 73`, the words are 0x4500 and 0x0073, the sum is 0x4573 and the checksum is 0xBA8C. A receiver that sums the data including the checksum obtains 0xFFFF, which is the test. **Complexity.** O(*n*). **Mistakes.** Forgetting the odd trailing byte (padded with a zero low byte); folding the carry only once; reading the words in host byte order on a little-endian machine.

Q1.53 Allocate VLAN identifiers with a bitmap

ARISTA · BROADCOM · TECH-1 · ■■□ · T

Manage the 4096 VLAN IDs: allocate the lowest free one, free one, in constant time per word.

30-second answer: a bitmap of 64 words; find a word that is not all ones, take its lowest zero bit with a count-trailing-zeros instruction.

```
#define NW (4096 / 64)
static uint64_t used[NW];
int alloc_vlan(void) {
  for (int w = 0; w < NW; w++)
    if (~used[w]) {
      int b = __builtin_ctzll(~used[w]);
      used[w] |= 1ull << b;
      return w * 64 + b;
    }
  return -1;
}
void free_vlan(int id) {
  used[id / 64] &= ~(1ull << (id % 64));
}
```

Complexity. O(64) per allocation, O(1) per free. **Follow-ups.** Reserve IDs 0 and 4095 by presetting their bits; a two-level summary bitmap makes allocation O(1). **Mistakes.** Shifting a 32-bit `1` (use `1ull`).

Linked lists, stacks and queues

Q1.54 Intersection of two linked lists

NOKIA · TECH-1 · ■□□ · S

Two singly linked lists may merge into a common tail. Return the first shared node, or `NULL`, in O(1) extra space.

30-second answer: let each pointer walk its own list and then the other; both cover the same total length, so they meet at the intersection, or both reach `NULL` together.

```
Node *intersect(Node *a, Node *b) {
  Node *p = a, *q = b;
  while (p != q) {
    p = p ? p->next : b;
    q = q ? q->next : a;
  }
  return p;
}
```

If the lists have x and y private nodes and z shared ones, each pointer travels $x + y + z$ nodes before the meeting point, so the shared tail aligns. **Complexity.** O($m + n$) time, O(1) space. **Mistakes.** Comparing values instead of node addresses.

Q1.55 Remove duplicates from a sorted list, linear and circular

ARISTA · TECH-1 · ■■□ · S

Remove repeated values from a sorted singly linked list. Then do it for a sorted circular list.

30-second answer: compare each node with its successor and unlink the successor while they are equal; for the circular list, stop when the walk returns to the head and finally compare the last node with the head.

```
Node *dedup(Node *h) {
  for (Node *p = h; p && p->next; ) {
    if (p->val == p->next->val) {
      Node *d = p->next;
      p->next = d->next;
      free(d);
    } else p = p->next;
  }
  return h;
}
void dedup_circ(Node *h) {
  if (!h) return;
  Node *p = h;
  while (p->next != h) {
    if (p->val == p->next->val) {
      Node *d = p->next;
      p->next = d->next;
      free(d);
    } else p = p->next;
  }
  if (p != h && p->val == h->val) {
    Node *q = h;
    while (q->next != p) q = q->next;
    q->next = h;
    free(p);
  }
}
```

In the circular version a duplicate may wrap around (the tail equals the head); the head is kept and the tail removed. **Complexity.** O(n). **Mistakes.** Advancing `p` after a deletion (it would skip a third copy); looping `while (p)` on a circular list, which never ends.

Q1.56 Delete a node given only a pointer to it

ARISTA · TECH-1 · ■□□ · S

You are given a pointer to a node that is not the tail of a singly linked list, and no head. Delete it.

30-second answer: copy the successor's value into the node and unlink the successor.

```
void delete_node(Node *n) {
  Node *d = n->next;
  n->val = d->val;
  n->next = d->next;
  free(d);
}
```

Complexity. O(1). **Follow-ups.** The tail cannot be deleted this way, since it has no successor to copy; any other pointer held to the successor is now dangling. **Mistakes.** Trying to unlink `n` itself without access to its predecessor.

Q1.57 Remove the n-th node from the end

AMAZON · MICROSOFT · TECH-1 · ■■□ · T

Delete the n*-th node from the end of a singly linked list in one pass.*

30-second answer: advance one pointer *n* nodes ahead, then move both until the lead pointer reaches the last node; the trailing pointer is then the predecessor of the node to delete.

```
Node *remove_nth(Node *head, int n) {
  Node dummy = {0, head};
  Node *fast = &dummy, *slow = &dummy;
  while (n-- > 0 && fast) fast = fast->next;
  if (!fast) return head;     /* n too big */
  while (fast->next) {
    fast = fast->next;
    slow = slow->next;
  }
  Node *d = slow->next;
  slow->next = d->next;
  free(d);
  return dummy.next;
}
```

The dummy node lets the head itself be removed (*n* equal to the length). **Complexity.** O(*L*) time, O(1) space.

Q1.58 Palindrome linked list

AMAZON · TECH-1 · ■■□ · T

Decide whether the values of a singly linked list read the same in both directions, in O(1) extra space.

30-second answer: find the middle with fast and slow pointers, reverse the second half in place, compare it with the first half, and reverse it back.

```
bool is_pal(Node *h) {
  Node *slow = h, *fast = h;
  while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
  }
  Node *r = reverse(slow), *save = r;
  bool ok = true;
  for (Node *p = h; ok && r;
       p = p->next, r = r->next)
    if (p->val != r->val) ok = false;
  reverse(save);          /* restore the list */
  return ok;
}
```

For `1 → 2 → 3 → 2 → 1` the middle is 3, the reversed half is `1 → 2 → 3`, and the comparison `1=1, 2=2, 3=3` succeeds (the middle compares with itself). `reverse` is the function of §1.6 (p. 20). **Complexity.** O(*n*) time, O(1) space.

Q1.59 Add two numbers stored as linked lists

ARISTA · AMAZON · TECH-1 · ■■□ · S

Each list holds a non-negative number with the least significant digit first. Return their sum as a new list.

30-second answer: add digit by digit with a carry, as on paper, appending to a result list through a dummy head; continue while either list or a carry remains.

```
Node *add_lists(Node *a, Node *b) {
  Node dummy = {0, NULL}, *t = &dummy;
  int carry = 0;
  while (a || b || carry) {
    int s = carry + (a ? a->val : 0)
                  + (b ? b->val : 0);
    carry = s / 10;
    Node *n = malloc(sizeof *n);
    if (!n) return NULL;  /* leaks par-
tial */
    n->val = s % 10;
    n->next = NULL;
    t->next = n;
    t = n;
    if (a) a = a->next;
    if (b) b = b->next;
  }
  return dummy.next;
}
```

`[2,4,3] + [5,6,4]` gives `[7,0,8]` (342 + 465 = 807). **Complexity.** O(max(*m*, *n*)). **Follow-ups.** When the most significant digit comes first, reverse both inputs (or push digits on stacks), add, and reverse the result. **Mistakes.** Dropping a final carry (`5 + 5` must give `[0,1]`).

Q1.60 Reverse a doubly linked list; delete a given node

ARISTA · TECH-1 · ■□□ · S

Reverse a doubly linked list in place. Then delete a node given its pointer.

30-second answer: swap the `next` and `prev` pointers of every node and return the last node visited; deleting a node relinks its two neighbours and updates the head if the node was first.

```
typedef struct D { int val;
          struct D *prev, *next; } D;
D *dl_reverse(D *h) {
  D *cur = h, *last = NULL;
  while (cur) {
    D *nx = cur->next;
    cur->next = cur->prev;
    cur->prev = nx;
    last = cur;
    cur = nx;
  }
  return last;
}
D *dl_delete(D *h, D *n) {
  if (n->prev) n->prev->next = n->next;
  else h = n->next;
  if (n->next) n->next->prev = n->prev;
  free(n);
  return h;
}
```

Complexity. O(*n*) and O(1). **Mistakes.** Following `cur->next` after swapping (it now points backwards; hence the saved `nx`); forgetting the head and tail cases in deletion.

Q1.61 Reverse nodes in groups of k

ARISTA · TECH-2 · ■■■ · T

Reverse a list `k` nodes at a time; a final group shorter than `k` stays as it is.

30-second answer: check that `k` nodes remain, reverse the rest of the list recursively, then reverse this group onto the result.

```
Node *rev_k(Node *head, int k) {
  Node *p = head;
  for (int i = 0; i < k; i++) {
    if (!p) return head;   /* fewer than k */
    p = p->next;
  }
  Node *prev = rev_k(p, k), *cur = head;
  for (int i = 0; i < k; i++) {
    Node *nx = cur->next;
    cur->next = prev;
    prev = cur;
    cur = nx;
  }
  return prev;
}
```

The reversal loop is the ordinary one of §1.6 (p. 20) with `prev` starting at the already processed remainder. **Complexity.** O(*n*) time, O(*n*/*k*) stack; an iterative version keeps the tail of the previous group and uses O(1) space. **Mistakes.** Reversing a short final group.

Q1.62 Copy a list with random pointers

AMAZON · MICROSOFT · TECH-2 · ■■□ · T

Each node has `next` and `rnd` (any node or `NULL`). Make a deep copy.

30-second answer: interleave a copy after every original node, set each copy's `rnd` from its original's `rnd->next`, then split the two lists.

```
typedef struct R { int val;
        struct R *next, *rnd; } R;
R *copy_random(R *h) {   /* checks omitted */
  if (!h) return NULL;
  for (R *p = h; p; p = p->next->next) {
    R *c = malloc(sizeof *c);
    c->val = p->val;
    c->next = p->next;
    c->rnd = NULL;
    p->next = c;
  }
  for (R *p = h; p; p = p->next->next)
    if (p->rnd) p->next->rnd = p->rnd->next;
  R *nh = h->next;
  for (R *p = h; p; p = p->next) {
    R *c = p->next;
    p->next = c->next;
    if (c->next) c->next = c->next->next;
  }
  return nh;
}
```

Complexity. O(*n*) time, O(1) extra beyond the copy. **Follow-ups.** The alternative is a hash map from original to copy, which uses O(*n*) memory but is simpler to write.

Q1.63 Merge k sorted lists

AMAZON · GOOGLE · TECH-2 · ■■■ · T

Merge `k` sorted linked lists into one sorted list.

30-second answer: keep the head of each list in a min-heap; repeatedly take the smallest and push its successor.

```
struct LN { int val; LN *next; };
LN *mergeK(vector<LN*> ls) {
  auto cmp = [](LN* a, LN* b) {
    return a->val > b->val; };
  priority_queue<LN*, vector<LN*>,
                 decltype(cmp)> pq(cmp);
  for (LN* l : ls) if (l) pq.push(l);
  LN dummy{0, nullptr}, *t = &dummy;
  while (!pq.empty()) {
    LN* n = pq.top(); pq.pop();
    t->next = n; t = n;
    if (n->next) pq.push(n->next);
  }
  return dummy.next;
}
```

Complexity. O(N log k) for N total nodes. **Follow-ups.** Merging pairwise in rounds (divide and conquer) has the same bound without a heap. This is the same k-way merge that combines sorted runs of an external sort.

Q1.64 Min stack

AMAZON · MICROSOFT · OA · ■□□ · T

A stack with `push`, `pop`, `top` and `getMin`, all O(1).

30-second answer: store with every element the minimum of the stack up to that element.

```
class MinStack {
  vector<pair<int,int>> s;  // value,
min
public:
  void push(int x) {
    int m = s.empty() ? x
                      : min(x,
s.back().second);
    s.push_back({x, m});
  }
  void pop() { s.pop_back(); }
  int top() { return s.back().first; }
  int getMin() { return
s.back().second; }
};
```

Complexity. O(1) per operation, O(n) space. **Mistakes.** Keeping a single `min` variable, which cannot be restored after popping the minimum.

Q1.65 Queue using two stacks

MICROSOFT · AMAZON · OA · ■□□ · T

Implement a FIFO queue using only stack operations.

30-second answer: push onto an input stack; pop from an output stack, refilling it from the input stack only when it is empty. Each element moves at most twice.

```
class MyQueue {
  stack<int> in, out;
  void shift() {
    if (out.empty())
      while (!in.empty()) {
        out.push(in.top()); in.pop();
      }
  }
public:
  void push(int x) { in.push(x); }
  int pop() {
    shift(); int v = out.top();
    out.pop(); return v;
  }
  int peek() { shift(); return
out.top(); }
  bool empty() { return in.empty() &&
out.empty(); }
};
```

Complexity. Amortised O(1) per operation. **Mistakes.** Refilling `out` while it still holds elements, which breaks the order.

Q1.66 Evaluate a reverse Polish expression

AMAZON · OA · ■■□ · T

Evaluate tokens such as `["2","1","+","3","*"]` *(that is, (2 + 1) × 3 = 9). Division truncates towards zero.*

30-second answer: push numbers; on an operator pop two operands (the right one first), apply, and push the result.

```
int evalRPN(const vector<string>& t) {
  vector<long long> st;
  for (auto& s : t) {
    if (s.size() == 1 && strchr("+-*/",
s[0])) {
      long long b = st.back();
st.pop_back();
      long long a = st.back();
st.pop_back();
      char o = s[0];
      st.push_back(o == '+' ? a + b
                 : o == '-' ? a - b
                 : o == '*' ? a * b : a
/ b);
    } else st.push_back(stoll(s));
  }
  return (int)st.back();
}
```

Complexity. O(*n*). **Mistakes.** Swapping the operand order (matters for `-` and `/`); treating `"-3"` as an operator (hence the test on the token length).

Q1.67 Sliding window maximum

GOOGLE · AMAZON · TECH-2 · ■■■ · T

For every window of `k` *consecutive elements, output its maximum, in O(*n*).*

30-second answer: keep a deque of indices whose values decrease from front to back; the front is the maximum of the window. Drop the front when it leaves the window, and drop smaller values from the back before pushing the new index.

```
vector<int> maxWindow(const
vector<int>& a,
                      int k) {
  deque<int> dq;
  vector<int> r;
  for (int i = 0; i < (int)a.size();
++i) {
    if (!dq.empty() && dq.front() <= i
- k)
      dq.pop_front();
    while (!dq.empty() && a[dq.back()]
<= a[i])
      dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1)
r.push_back(a[dq.front()]);
  }
  return r;
}
```

For `[1,3,-1,-3,5,3,6,7]` and $k = 3$ the output is `[3,3,5,5,6,7]`. Each index enters and leaves the deque once, so the cost is O(*n*). **Mistakes.** Storing values rather than indices (the expiry test needs the index); using `<` so that equal values stay and the deque grows.

Trees and tries

Q1.68 Invert a binary tree; test for symmetry

GOOGLE · AMAZON · OA · ■□□ · T

Swap the left and right child of every node. Then decide whether a tree is a mirror image of itself.

30-second answer: recurse: swap at each node; for symmetry compare the left subtree with the mirror of the right subtree.

```
TreeNode* invert(TreeNode* t) {
  if (!t) return t;
  swap(t->left, t->right);
  invert(t->left);
  invert(t->right);
  return t;
}
bool mirror(TreeNode* a, TreeNode* b) {
  if (!a || !b) return a == b;
  return a->val == b->val
      && mirror(a->left, b->right)
      && mirror(a->right, b->left);
}
// symmetric iff root == null ||
//   mirror(root->left, root->right)
```

Complexity. O(n) time, O(h) stack. **Mistakes.** Comparing `a->left` with `b->left` in the mirror test.

Q1.69 Lowest common ancestor

MICROSOFT · AMAZON · TECH-1 · ■■□ · T

Find the lowest node that has both `p` and `q` in its subtree, first in a BST and then in a general binary tree.

30-second answer: in a BST walk down, going left when both keys are smaller and right when both are larger; the first node that splits them is the answer. In a general tree, recurse and return the node where both sides report a hit.

```
TreeNode* lcaBST(TreeNode* t, int p,
int q) {
  while (t) {
    if (p < t->val && q < t->val)
      t = t->left;
    else if (p > t->val && q > t->val)
      t = t->right;
    else return t;
  }
  return nullptr;
}
TreeNode* lca(TreeNode* t, TreeNode* p,
              TreeNode* q) {
  if (!t || t == p || t == q) return t;
  TreeNode* l = lca(t->left, p, q);
  TreeNode* r = lca(t->right, p, q);
  return l && r ? t : (l ? l : r);
}
```

Complexity. O(h) for the BST, O(n) for the general tree. **Mistakes.** Assuming both nodes exist in the general version; the code above then returns the one found.

Q1.70 Kth smallest element in a BST

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Return the k-th smallest key of a BST.

30-second answer: in-order traversal visits keys in ascending order; stop at the *k*-th visit using an explicit stack.

```
int kthSmallest(TreeNode* t, int k) {
  vector<TreeNode*> st;
  while (t || !st.empty()) {
    while (t) { st.push_back(t); t = t-
>left; }
    t = st.back(); st.pop_back();
    if (--k == 0) return t->val;
    t = t->right;
  }
  return -1;
}
```

Complexity. O($h + k$) time, O(h) space. **Follow-ups.** With frequent updates and queries, store the subtree size in each node and descend in O(h).

Q1.71 Diameter of a binary tree

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

Find the number of edges on the longest path between two nodes of a tree.

30-second answer: post-order recursion that returns the node's height to its parent while updating a global best with the left height plus the right height.

```
int diam(TreeNode* t, int& best) {
  if (!t) return 0;
  int l = diam(t->left, best);
  int r = diam(t->right, best);
  best = max(best, l + r);
  return 1 + max(l, r);
}
// call: int best = 0; diam(root, best);
```

The value returned upward (a height) differs from the value recorded (a path through the node); mixing them up is the classic bug. **Complexity.** O(*n*) time, O(*h*) space.

Q1.72 Serialise and deserialise a binary tree

GOOGLE · AMAZON · TECH-2 · ■■■ · T

Convert a binary tree to a string and back, losing no structure.

30-second answer: pre-order traversal with an explicit marker for each null child; the same order reads it back recursively.

```
void ser(TreeNode* t, string& o) {
  if (!t) { o += "#,"; return; }
  o += to_string(t->val) + ",";
  ser(t->left, o);
  ser(t->right, o);
}
TreeNode* de(istringstream& in) {
  string s;
  getline(in, s, ',');
  if (s == "#") return nullptr;
  auto* t = new TreeNode{stoi(s)};
  t->left = de(in);
  t->right = de(in);
  return t;
}
```

The tree `1 (2, 3)` becomes `1,2,#,#,3,#,#,`. Without the null markers the string would not determine the shape. **Complexity.** O(*n*). **Follow-ups.** Level-order form with a queue avoids deep recursion.

Q1.73 Implement a trie

GOOGLE · AMAZON · TECH-1 · ■■□ · S

Support `insert(word)`, `search(word)` *and* `startsWith(prefix)` *for lowercase words.*

30-second answer: a tree with 26 children per node and an end flag; each operation walks one edge per letter.

```
struct Trie {
  struct N { N* c[26] = {}; bool end =
false; };
  N root;
  void insert(const string& w) {
    N* t = &root;
    for (char ch : w) {
      N*& nx = t->c[ch - 'a'];
      if (!nx) nx = new N;
      t = nx;
    }
    t->end = true;
  }
  N* walk(const string& p) {
    N* t = &root;
    for (char ch : p) {
      t = t->c[ch - 'a'];
      if (!t) return nullptr;
    }
    return t;
  }
  bool search(const string& w) {
    N* t = walk(w); return t && t->end;
  }
  bool startsWith(const string& p) {
    return walk(p) != nullptr;
  }
};
```

Complexity. O(*L*) per operation; memory up to 26 pointers per node. **Follow-ups.** A hash map of children saves memory for large alphabets; deletion needs the pruning of the next problem. **Mistakes.** Treating a prefix as a word (`search("app")` when only `"apple"` is stored).

Q1.74 Delete a prefix from the routing trie

GOOGLE · JUNIPER · TECH-2 · ■■■ · T

Remove a route from the binary trie of §1.9 (p. 30)*, freeing nodes that no longer lead anywhere.*

30-second answer: recurse down the prefix; clear the next hop at its node; on the way back, free any child that has no route and no children.

```
static bool prune(Node *t, uint32_t p,
                  unsigned len, un-
signed i) {
  if (i == len) t->hop = -1;
  else {
    unsigned b = (p >> (31 - i)) & 1u;
    if (t->c[b] && prune(t->c[b], p,
len, i+1)) {
      free(t->c[b]);
      t->c[b] = NULL;
    }
  }
  return t->hop < 0 && !t->c[0] && !t-
>c[1];
}
void trie_delete(Node *root, uint32_t p,
                 unsigned len) {
  prune(root, p, len, 0);
}
```

The function returns whether its node is now useless; the root is never freed. Deleting 10.1.2.0/24 from the table of §1.9 (p. 30) removes the nodes below 10.1.0.0/16 but keeps the /16 route. **Complexity.** O(*len*). **Mistakes.** Freeing a node that still carries a shorter prefix's route; clearing the hop without pruning (a slow memory leak under route churn).

Q1.75 CIDR firewall rules: the most specific rule wins

GOOGLE · CISCO · TECH-2 · ■■■ · T

A rule list maps CIDR blocks to allow or deny. For a packet's source address, apply the most specific matching rule; if two rules have the same prefix, the earlier rule wins; with no match, deny.

30-second answer: store the action at the prefix's node in the trie of §1.9 (p. 30)**, ignore a later rule for an already stored prefix, and answer with a longest-prefix lookup.**

```
static Node *find_exact(Node *t,
uint32_t p,
                        unsigned len) {
  for (unsigned i = 0; t && i < len;
i++)
    t = t->c[(p >> (31 - i)) & 1u];
  return t;
}
bool add_rule(Node *root, uint32_t p,
              unsigned len, int act) {
  Node *e = find_exact(root, p, len);
  if (e && e->hop >= 0) return true;
  return trie_insert(root, p, len, act);
}
int decide(const Node *root, uint32_t
ip) {
  int a = trie_lookup(root, ip);
  return a < 0 ? 0 : a;   /* 1 allow, 0
deny */
}
```

With the rules `10.0.0.0/8 allow` and `10.1.0.0/16 deny`, the address 10.1.5.5 is denied and 10.2.0.1 is allowed. **Complexity.** O(32) per rule and per lookup. **Mistakes.** Using first-match-in-list semantics (a scan) when the specification says most specific; a rule `0.0.0.0/0` is the default and sits at the root.

Q1.76 Longest-prefix match without a trie

GOOGLE · TECH-2 · ■■□ · S

The interviewer rejects the trie: "use hash tables".

30-second answer: one hash table per prefix length, mapping the masked prefix to the next hop; probe from /32 down to /0 and return the first hit.

```
struct LPM {
  unordered_map<uint32_t, int> t[33];
  static uint32_t mk(uint32_t a, int
len) {
    return len ? a & (~0u << (32 -
len)) : 0;
  }
  void add(uint32_t p, int len, int
hop) {
    t[len][mk(p, len)] = hop;
  }
  int lookup(uint32_t a) {
    for (int len = 32; len >= 0; --len)
{
      auto it = t[len].find(mk(a, len));
      if (it != t[len].end()) return it-
>second;
    }
    return -1;
  }
};
```

Complexity. At most 33 hash probes per lookup; O(1) insertion. **Follow-ups.** Keep a list of the prefix lengths that occur (real tables use few distinct lengths) and probe only those; binary search over lengths with "marker" entries gives O(log 32) probes. **Mistakes.** The shift `~0u << 32` for the /0 case (handled by `len ?`).

Q1.77 Maximum XOR of two numbers

GOOGLE · TECH-2 · ■■■ · T

Find the maximum of `a[i] ^ a[j]` *over all pairs of a non-negative integer array in O(*n*).*

30-second answer: insert every number into a binary trie, most significant bit first; for each number walk the trie choosing, at each bit, the child with the opposite bit when it exists.

```
int findMaxXor(const vector<int>& a) {
  vector<array<int,2>> t(1);   // 0 =
root
  for (int x : a) {
    int u = 0;
    for (int i = 30; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (!t[u][b]) {
        t[u][b] = t.size();
        t.push_back({0, 0});
      }
      u = t[u][b];
    }
  }
  int best = 0;
  for (int x : a) {
    int u = 0, cur = 0;
    for (int i = 30; i >= 0; --i) {
      int b = ((x >> i) & 1) ^ 1;
      if (t[u][b]) { cur |= 1 << i; u =
t[u][b]; }
      else u = t[u][b ^ 1];
    }
    best = max(best, cur);
  }
  return best;
}
```

A higher bit set to one outweighs all lower bits, so the greedy choice of the opposite bit is optimal. **Complexity.** O(31 *n*) time and space. **Mistakes.** Starting at bit 31 for non-negative `int` (harmless but wasted), or building the trie with node pointers that reallocate (use indices, as here).

Heaps, large data and graphs

Q1.78 Kth largest element

AMAZON · GOOGLE · OA · ■■□ · T

Return the k*-th largest value of an unsorted array (counting duplicates).*

30-second answer: keep a min-heap of the *k* largest values seen; its root is the answer.

```
int kthLargest(const vector<int>& a,
int k) {
  priority_queue<int, vector<int>,
                 greater<int>> h;
  for (int x : a) {
    h.push(x);
    if ((int)h.size() > k) h.pop();
  }
  return h.top();
}
```

Complexity. O(*n* log *k*) time, O(*k*) space. **Follow-ups.** `nth_element` (quickselect) gives O(*n*) on average but modifies the array; a heap also handles a stream. **Mistakes.** Using a max-heap of size *k* (the trap of §1.10 (p. 33)).

Q1.79 Median of a data stream

GOOGLE · AMAZON · TECH-2 · ■■■ · T

Support `add(x)` *and* `median()` *on a growing stream.*

30-second answer: a max-heap for the lower half and a min-heap for the upper half, kept equal in size (or one larger below); the median sits at the roots.

```
class Median {
  priority_queue<int> lo;   // lower
half
  priority_queue<int, vector<int>,
                 greater<int>> hi;
public:
  void add(int x) {
    lo.push(x);
    hi.push(lo.top()); lo.pop();
    if (hi.size() > lo.size()) {
      lo.push(hi.top()); hi.pop();
    }
  }
  double median() {   // after >= 1 add
    return lo.size() > hi.size()
      ? lo.top()
      : ((double)lo.top() + hi.top()) /
2;
  }
};
```

Each `add` sends the new value through `lo` to `hi`, which keeps every value of `lo` no larger than every value of `hi`, and then rebalances. **Complexity.** O(log *n*) per `add`, O(1) per `median`.

Q1.80 Top 10 source IPs by bytes in a 10 GB flow log

GOOGLE · AMAZON · TECH-2 · ■■■ · T

A flow log of `src_ip bytes` lines is far larger than memory. Report the ten sources with the most bytes.

30-second answer: if the distinct addresses fit in memory, one pass with a hash map of totals and a size-10 min-heap suffices; if not, hash-partition the log into files by address, total each file separately, and merge the per-file top tens.

Approach. Memory is set by the number of *distinct* keys, not by the log size: 10^7 addresses need about 10^7 × (4 + 8) bytes for the totals, which is modest. If even that is too large, partition: a hash of the address picks one of *P* files, so every record of an address lands in the same file, and each file then fits in memory. The top ten overall are among the union of the per-file top tens. When an approximate answer is acceptable, a count-min sketch with a heap of candidates uses fixed memory.

```
using P = pair<uint64_t, uint32_t>;
vector<P> top10(
    unordered_map<uint32_t, uint64_t>&
m) {
  priority_queue<P, vector<P>,
greater<P>> h;
  for (auto& [ip, b] : m) {
    h.push({b, ip});
    if (h.size() > 10) h.pop();
  }
  vector<P> r;
  while (!h.empty()) {
    r.push_back(h.top()); h.pop();
  }
  reverse(r.begin(), r.end());
  return r;
}
```

Complexity. O(*N*) to read, O(*d* log 10) for *d* distinct keys, and two passes of disk I/O when partitioning. **Mistakes.** Partitioning by line number or by byte range (one address then appears in many files and its total is split); sorting all *d* totals when only ten are needed.

Q1.81 Reorder packets by sequence number

ARISTA · BROADCOM · TECH-1 · ■■□ · T

Packets arrive out of order with sequence numbers. Deliver the data in sequence order, as soon as the next expected number is available, ignoring duplicates.

30-second answer: hold arrivals in a min-heap keyed by sequence number; after each arrival release the top while it equals the next expected number.

```
struct Reorder {
  using Pk = pair<uint32_t, string>;
  uint32_t next = 0;
  priority_queue<Pk, vector<Pk>,
                 greater<Pk>> pq;
  vector<string> push(uint32_t seq,
                      string data) {
    vector<string> out;
    pq.push({seq, move(data)});
    while (!pq.empty() &&
           pq.top().first <= next) {
      if (pq.top().first == next) {
        out.push_back(pq.top().second);
        ++next;
      }
      pq.pop();   // delivered or du-
plicate
    }
    return out;
  }
};
```

Arrivals 1, 0, 2 deliver nothing, then `[d0, d1]`, then `[d2]`. **Complexity.** O(log *n*) per packet. **Follow-ups.** Real sequence numbers wrap; compare with `(int32_t)(a - b) < 0`. A bounded window and a timeout decide when to skip a lost packet. **Mistakes.** Letting a duplicate with a smaller number block the heap top forever.

Q1.82 Minimum dice throws to finish a snake-and-ladder board

CISCO · OA · ■■□ · S

A board has cells 1 to N. *Landing on a ladder foot or snake head moves you to its other end. A throw advances 1 to 6 cells. Find the fewest throws from cell 1 to cell* N.

30-second answer: model cells as vertices and each throw as an edge of weight one, so breadth-first search from cell 1 gives the minimum.

```
int minThrows(const vector<int>& jump) {
  int N = jump.size() - 1;  // jump[c] > 0: to
  vector<int> dist(N + 1, -1);
  queue<int> q;
  dist[1] = 0; q.push(1);
  while (!q.empty()) {
    int u = q.front(); q.pop();
    if (u == N) return dist[u];
    for (int d = 1; d <= 6 && u + d <= N; ++d) {
      int v = jump[u + d] > 0
              ? jump[u + d] : u + d;
      if (dist[v] < 0) {
        dist[v] = dist[u] + 1;
        q.push(v);
      }
    }
  }
  return -1;
}
```

With $N = 30$, ladders $3 \to 22$, $5 \to 8$, $11 \to 26$, $20 \to 29$ and snakes $27 \to 1$, $21 \to 9$, $17 \to 4$, $19 \to 7$, the answer is 3: throw 2 (to 3, up to 22), then 6 (to 28), then 2. **Complexity.** $O(6N)$. **Mistakes.** Plain DFS (finds a path, not the shortest); applying a second jump after landing on a jump's end (one throw gives one jump).

Q1.83 Number of islands

AMAZON · MICROSOFT · OA · ■■□ · T

In a grid of `'1'` *(land) and* `'0'` *(water), count the groups of land cells connected horizontally or vertically.*

30-second answer: scan the grid; on each unvisited land cell, count one island and flood-fill it (DFS or BFS) so its cells are not counted again.

```
int numIslands(vector<string>& g) {
  int R = g.size(), C = R ? g[0].size() : 0;
  int n = 0;
  function<void(int,int)> fill =
      [&](int r, int c) {
    if (r < 0 || c < 0 || r >= R || c >= C
        || g[r][c] != '1') return;
    g[r][c] = '0';   // mark visited
    fill(r + 1, c); fill(r - 1, c);
    fill(r, c + 1); fill(r, c - 1);
  };
  for (int r = 0; r < R; ++r)
    for (int c = 0; c < C; ++c)
      if (g[r][c] == '1') { ++n; fill(r, c); }
  return n;
}
```

Complexity. $O(RC)$ time; the recursion depth can reach RC, so for large grids use the explicit stack of §1.11 (p. 34). **Mistakes.** No visited marking (infinite recursion); checking bounds after indexing.

Q1.84 Can all tasks be finished (course schedule)

AMAZON · GOOGLE · OA · ■■□ · T

There are `n` *tasks and pairs* `[a, b]` *meaning "task* `b` *must precede task* `a`*". Is there an order that satisfies all pairs?*

30-second answer: build the dependency graph and run Kahn's topological sort; all tasks can be finished exactly when every vertex is output, i.e. there is no cycle.

```
bool canFinish(int n,
               const
vector<vector<int>>& pre) {
  vector<vector<int>> g(n);
  for (auto& p : pre)
g[p[1]].push_back(p[0]);
  return (int)topo(n, g).size() == n;
}
```

`topo` is the function of §1.11 (p. 34); returning its order instead of a boolean answers the "find a valid order" variant. **Complexity.** $O(V + E)$. **Mistakes.** Reversing the edge direction (harmless for the yes/no answer, wrong for the order).

Q1.85 Network delay time

CISCO · GOOGLE · TECH-1 · ■■□ · T

A signal starts at router `k`*. Directed links* `[u, v, w]` *take* `w` *time units. How long until all* `n` *routers (numbered 1 to* `n`*) have the signal, or −1 if some never do?*

30-second answer: run Dijkstra from `k`; the answer is the largest shortest distance, or −1 if any router is unreachable.

```
int networkDelay(
    const vector<vector<int>>& times,
    int n, int k) {
  Graph g(n + 1);
  for (auto& t : times)
    g[t[0]].push_back({t[1], t[2]});
  auto d = dijkstra(g, k);
  long long worst = 0;
  for (int i = 1; i <= n; ++i)
    worst = max(worst, d[i]);
  return worst >= LLONG_MAX / 4
         ? -1 : (int)worst;
}
```

`Graph` and `dijkstra` are those of §1.11 (p. 34). The signal reaches the last router at the *maximum* of the shortest arrival times, not their sum. **Complexity.** O((*V* + *E*) log *V*). **Mistakes.** Using BFS on weighted links; forgetting that vertex 0 does not exist.

Q1.86 Cheapest route with at most k stops

CISCO · AMAZON · TECH-2 · ■■■ · T

Given directed priced links `[u, v, price]`*, find the cheapest route from* `src` *to* `dst` *using at most* `k` *intermediate stops, or −1.*

30-second answer: Bellman–Ford with exactly *k* + 1 rounds, each using only the previous round's distances, so a path never exceeds *k* + 1 links.

```
int cheapest(int n,
vector<vector<int>>& f,
             int src, int dst, int k) {
  vector<Edge> es;
  for (auto& x : f)
    es.push_back({x[0], x[1], x[2]});
  auto d = bellmanFord(n, es, src, k +
1);
  return d[dst] >= LLONG_MAX / 4
         ? -1 : (int)d[dst];
}
```

`Edge` and `bellmanFord` are those of §1.11 (p. 34); the round count is the hop limit, as in a protocol that counts hops. **Complexity.** O((*k* + 1)*E*). **Mistakes.** Dijkstra with a plain distance array (the cheapest route may use too many stops, and a dearer route with fewer stops is then lost).

Q1.87 First link that creates a loop

CISCO · JUNIPER · TECH-1 · ■■□ · T

Links between nodes 1 to `n` *are added in order. Report the first link that connects two nodes that are already connected.*

30-second answer: union–find over the nodes; the first link whose `unite` returns false closes a loop.

```
vector<int> firstLoop(
    int n, const vector<vector<int>>&
e) {
  DSU d(n + 1);
  for (auto& x : e)
    if (!d.unite(x[0], x[1])) return x;
  return {};
}
```

`DSU` is the structure of §1.11 (p. 34). In a switched network the same test finds the link that a spanning-tree protocol must block. **Complexity.** O(*E* α(*n*)). **Follow-ups.** For a graph that is a tree plus one extra link, the answer is the last link of the input that lies on the cycle (the variant asked on some sites).

Q1.88 Make a network connected

CISCO · AMAZON · TECH-1 · ■■□ · T

`n` *computers and cables* `[a, b]`*. In one operation you move a cable to join two computers. Return the fewest operations to connect everything, or −1 if impossible.*

30-second answer: connecting *c* components needs *c* − 1 spare cables; there are enough exactly when there are at least *n* − 1 cables, so count the components with union–find.

```
int makeConnected(
    int n, const vector<vector<int>>&
c) {
  if ((int)c.size() < n - 1) return -1;
  DSU d(n);
  int comps = n;
  for (auto& x : c)
    if (d.unite(x[0], x[1])) --comps;
  return comps - 1;
}
```

Every cable that does not merge two components is spare, and with at least *n* − 1 cables there are always enough spares. **Complexity.** O(*E* α(*n*)).

Q1.89 Links whose failure splits the network (bridges)

GOOGLE · ARISTA · TECH-2 · ■■■ · T

Find every link whose removal disconnects the network (a bridge*).*

30-second answer: depth-first search records the entry time `tin` of each vertex and `low`, the earliest entry time reachable from its subtree using one back edge; a tree edge (u, v) is a bridge when `low[v] > tin[u]`.

```
vector<vector<int>> bridges(int n,
    const vector<vector<int>>& edges) {
  vector<vector<int>> g(n), out;
  for (auto& e : edges) {
    g[e[0]].push_back(e[1]);
    g[e[1]].push_back(e[0]);
  }
  vector<int> tin(n, -1), low(n, 0);
  int timer = 0;
  function<void(int,int)> dfs =
      [&](int u, int par) {
    tin[u] = low[u] = timer++;
    for (int v : g[u]) {
      if (v == par) continue;
      if (tin[v] >= 0)
        low[u] = min(low[u], tin[v]);
      else {
        dfs(v, u);
        low[u] = min(low[u], low[v]);
        if (low[v] > tin[u])
          out.push_back({u, v});
      }
    }
  };
  for (int i = 0; i < n; ++i)
    if (tin[i] < 0) dfs(i, -1);
  return out;
}
```

If nothing in the subtree below `v` can reach `u` or above except through the edge (u, v), the edge is a bridge. **Complexity.** O($V + E$). **Mistakes.** Skipping a parent edge when links are duplicated (parallel links are never bridges, so a multigraph needs the edge identifier, not the parent vertex).

Q1.90 Minimum cost to connect all points

GOOGLE · AMAZON · TECH-2 · ■■□ · T

Points on a plane are joined by cables costing the Manhattan distance between endpoints. Find the least total cost that connects all points.

30-second answer: this is a minimum spanning tree on a dense graph; Prim's algorithm in O(n^2), or Kruskal on sorted pair edges with union–find.

```
int minCostConnect(
    const vector<vector<int>>& p) {
  int n = p.size(), total = 0;
  vector<int> d(n, INT_MAX);
  vector<char> in(n, 0);
  d[0] = 0;
  for (int it = 0; it < n; ++it) {
    int u = -1;
    for (int i = 0; i < n; ++i)
      if (!in[i] && (u < 0 || d[i] <
d[u]))
        u = i;
    in[u] = 1;
    total += d[u];
    for (int v = 0; v < n; ++v)
      if (!in[v])
        d[v] = min(d[v],
          abs(p[u][0] - p[v][0]) +
          abs(p[u][1] - p[v][1]));
  }
  return total;
}
```

`d[v]` is the cheapest cable from the tree to `v`; each round adds the nearest outside point. **Complexity.** O(n^2) time, O(n) space. **Follow-ups.** For sparse graphs, Kruskal costs O(E log E).

Q1.91 Number of provinces (connected components)

NOKIA · AMAZON · OA · ■■□ · S

A matrix `m[i][j] = 1` *states that nodes* `i` *and* `j` *are directly connected. Count the groups of connected nodes.*

30-second answer: depth-first search with an explicit stack from every unvisited node; the number of starts is the number of groups.

```
int provinces(const
vector<vector<int>>& m) {
  int n = m.size(), cnt = 0;
  vector<char> seen(n, 0);
  for (int s = 0; s < n; ++s) {
    if (seen[s]) continue;
    ++cnt;
    vector<int> st{s};
    seen[s] = 1;
    while (!st.empty()) {
      int u = st.back(); st.pop_back();
      for (int v = 0; v < n; ++v)
        if (m[u][v] && !seen[v]) {
          seen[v] = 1;
          st.push_back(v);
        }
    }
  }
  return cnt;
}
```

Complexity. O(n^2) for the matrix form. **Follow-ups.** Union–find gives the same count and also handles edges that arrive over time.

Backtracking, searching, intervals and dynamic programming

Q1.92 Permutations and combination sum

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

Generate all permutations of distinct integers. Then, from positive integers, list all combinations that sum to a target, each value reusable.

30-second answer: choose, explore, un-choose. For permutations, swap each remaining element into position `i` and recurse. For combinations, either take `c[i]` again or skip to `c[i+1]`.

```
void perm(vector<int>& a, int i,
          vector<vector<int>>& out) {
  if (i == (int)a.size()) {
    out.push_back(a); return;
  }
  for (int j = i; j < (int)a.size();
++j) {
    swap(a[i], a[j]);
    perm(a, i + 1, out);
    swap(a[i], a[j]);   // un-choose
  }
}
void comb(const vector<int>& c, int i,
          int rem, vector<int>& path,
          vector<vector<int>>& out) {
  if (rem == 0) { out.push_back(path);
return; }
  if (rem < 0 || i == (int)c.size()) re-
turn;
  path.push_back(c[i]);
  comb(c, i, rem - c[i], path, out);
  path.pop_back();
  comb(c, i + 1, rem, path, out);
}
```

Complexity. O($n \cdot n!$) for permutations; combinations are bounded by the search tree. **Mistakes.** Forgetting the second `swap` or the `pop_back`; for repeated inputs, duplicates need a sorted array and a skip rule.

Q1.93 Search in a rotated sorted array

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

A sorted array of distinct values has been rotated at an unknown point (`4 5 6 7 0 1 2`). Find the index of a target in O(log n*).*

30-second answer: in each step one half is sorted; test whether the target lies in the sorted half and discard the other half.

```
int searchRot(const vector<int>& a, int
x) {
  int lo = 0, hi = (int)a.size() - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == x) return mid;
    if (a[lo] <= a[mid]) {   // left
sorted
      if (a[lo] <= x && x < a[mid])
        hi = mid - 1;
      else lo = mid + 1;
    } else {                 // right
sorted
      if (a[mid] < x && x <= a[hi])
        lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
}
```

Searching 0 in `4 5 6 7 0 1 2`: at mid = 3 (value 7) the left half `4..7` is sorted and excludes 0, so `lo = 4`; at mid = 5 (value 1) the left half `0 1` is sorted and contains 0, so `hi = 4`; mid = 4 then finds index 4. **Complexity.** O(log n). **Mistakes.** `<` instead of `<=` in `a[lo] <= a[mid]` when `lo == mid`; duplicates break the test (worst case O(n)).

Q1.94 Merge overlapping IP address ranges

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Given ranges as strings `"a.b.c.d-e.f.g.h"`*, merge overlapping ranges and print them in order.*

30-second answer: convert each end to a 32-bit integer, sort and merge as intervals (§1.13 (p. 40)**), and convert back.**

```
extern "C" {
bool parse_ipv4(const char*, uint32_t*);
void format_ipv4(uint32_t, char[16]);
}
vector<string> mergeRanges(
    const vector<string>& in) {
  auto fmt = [](long long x) {
    char b[16];
    format_ipv4((uint32_t)x, b);
    return string(b);
  };
  vector<array<long long, 2>> v;
  for (auto& s : in) {
    size_t d = s.find('-');
    uint32_t a = 0, b = 0;
    parse_ipv4(s.substr(0, d).c_str(),
&a);
    parse_ipv4(s.substr(d + 1).c_str(),
&b);
    v.push_back({a, b});
  }
  vector<string> out;
  for (auto& r : mergeIntervals(v))
    out.push_back(fmt(r[0]) + "-" +
fmt(r[1]));
  return out;
}
```

`10.0.0.1-10.0.0.9` and `10.0.0.5-10.0.0.20` merge to `10.0.0.1-10.0.0.20`. **Complexity.** O(*n* log *n*). **Mistakes.** Comparing the strings (lexicographic order is not numeric order); a signed 32-bit integer for the address (use 64-bit or unsigned). **Follow-ups.** Ask whether adjacent ranges (`…9` and `…10`) should merge.

Q1.95 Peak number of simultaneous flows

CISCO · AMAZON · TECH-1 · ■■□ · T

Given flow intervals `[start, end)`*, find the largest number that overlap at one instant (the meeting-rooms problem).*

30-second answer: sweep line: turn each interval into a +1 event at its start and a −1 event at its end, sort the events, and track the running total.

```
int maxConcurrent(
    const vector<array<int,2>>& iv) {
  vector<pair<int,int>> ev;
  for (auto& x : iv) {
    ev.push_back({x[0], +1});
    ev.push_back({x[1], -1});
  }
  sort(ev.begin(), ev.end());
  int cur = 0, best = 0;
  for (auto& e : ev) {
    cur += e.second;
    best = max(best, cur);
  }
  return best;
}
```

Sorting the pairs puts −1 before +1 at equal times, so an interval ending at *t* frees its slot before another starting at *t* takes it. **Complexity.** O(*n* log *n*). **Follow-ups.** A min-heap of end times gives the same count and the actual assignment.

Q1.96 Climbing stairs and house robber (one-dimensional DP)

AMAZON · MICROSOFT · OA · ■□□ · T

(a) Count the ways to climb `n` stairs taking 1 or 2 steps at a time. (b) Choose non-adjacent houses to maximise the total loot.

30-second answer: (a) `ways[i] = ways[i-1] + ways[i-2]`; (b) the best up to house *i* either skips it or takes it with the best up to *i* − 2. Two variables replace the table.

```
int climb(int n) {
  int a = 1, b = 1;   // ways[0], ways[1]
  for (int i = 2; i <= n; ++i) {
    int c = a + b; a = b; b = c;
  }
  return b;
}
int rob(const vector<int>& h) {
  int take = 0, skip = 0;
  for (int x : h) {
    int t = skip + x;
    skip = max(skip, take);
    take = t;
  }
  return max(take, skip);
}
```

`climb(5)` is 8. For houses `[2,7,9,3,1]` the answer is 12 (2 + 9 + 1). **Complexity.** O(*n*) time, O(1) space. **Mistakes.** Taking the maximum of adjacent pairs greedily.

Q1.97 Longest increasing subsequence

GOOGLE · MICROSOFT · TECH-2 · ■■□ · T

Find the length of the longest strictly increasing subsequence (not necessarily contiguous).

30-second answer: keep `t[i]`, the smallest possible last value of an increasing subsequence of length `i + 1`; each new value replaces the first entry that is not smaller, found by binary search.

```
int lis(const vector<int>& a) {
  vector<int> t;
  for (int x : a) {
    auto it = lower_bound(t.begin(),
                          t.end(), x);
    if (it == t.end()) t.push_back(x);
    else *it = x;
  }
  return t.size();
}
```

For `[10, 9, 2, 5, 3, 7, 101, 18]` the array `t` evolves 10; 9; 2; 2,5; 2,3; 2,3,7; 2,3,7,101; 2,3,7,18, so the length is 4. Note that `t` is not itself a subsequence of the input. **Complexity.** O(*n* log *n*). The quadratic DP, `dp[i] = 1 + max dp[j]` over smaller `a[j]`, is the form to state first. **Mistakes.** `upper_bound` (allows equal values, giving the non-decreasing variant).

Q1.98 Edit distance

GOOGLE · MICROSOFT · TECH-2 · ■■■ · T

Find the fewest single-character insertions, deletions and replacements that turn one string into another.

30-second answer: `d[i][j]` is the distance between the first `i` characters of `s` and the first `j` of `t`; if the last characters match it equals `d[i-1][j-1]`, otherwise one plus the best of replace, delete and insert.

```
int editDistance(const string& s,
                 const string& t) {
  int n = s.size(), m = t.size();
  vector<vector<int>> d(n + 1,
                        vector<int>(m +
1));
  for (int i = 0; i <= n; ++i) d[i][0]
= i;
  for (int j = 0; j <= m; ++j) d[0][j]
= j;
  for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
      d[i][j] = s[i-1] == t[j-1]
        ? d[i-1][j-1]
        : 1 + min({d[i-1][j-1],
                   d[i-1][j], d[i][j-
1]});
  return d[n][m];
}
```

`"kitten"` to `"sitting"` costs 3 (two replacements and one insertion). **Complexity.** O(*nm*) time and space, reducible to O(min(*n*, *m*)) space by keeping two rows. **Follow-ups.** The longest common subsequence uses the same table with `d[i][j] = d[i-1][j-1] + 1` on a match and `max(d[i-1][j], d[i][j-1])` otherwise.

Design, rate limiting and finite-state machines

Q1.99 LRU cache with a time-to-live (an ARP cache), thread-safe

MICROSOFT · AMAZON · TECH-2 · ■■■ · S

Extend the LRU cache of §1.15 (p. 43)*: every entry expires* `ttl` *time units after it was written, and the cache is used by several threads.*

30-second answer: store an expiry time in each node and treat an expired entry as a miss; guard every method with one mutex, because even `get` **reorders the list.**

```
class ArpCache {
  struct Item { int key; string mac;
                long long expires; };
  size_t cap; long long ttl;
  list<Item> items;
  unordered_map<int,
      list<Item>::iterator> pos;
  mutex m;
public:
  ArpCache(size_t c, long long t)
      : cap(c), ttl(t) {}
  optional<string> get(int ip,
                       long long now) {
    lock_guard<mutex> g(m);
    auto it = pos.find(ip);
    if (it == pos.end()) return nullopt;
    if (it->second->expires <= now) {
      items.erase(it->second);
      pos.erase(it);
      return nullopt;
    }
    items.splice(items.begin(), items,
                 it->second);
    return it->second->mac;
  }
  void put(int ip, string mac,
           long long now) {
    lock_guard<mutex> g(m);
    if (!cap) return;
    auto it = pos.find(ip);
    if (it != pos.end()) {
      items.erase(it->second);
      pos.erase(it);
    } else if (items.size() == cap) {
      pos.erase(items.back().key);
      items.pop_back();
    }
    items.push_front({ip, move(mac),
                      now + ttl});
    pos[ip] = items.begin();
  }
};
```

The caller supplies `now`, which makes the class testable and lets it use a monotonic clock. Expiry is lazy: an expired entry is removed when it is looked up. **Complexity.** O(1) average per call. **Follow-ups.** A background sweep, or a min-heap of expiry times, frees memory for entries never looked up again; shard the cache by key hash, each shard with its own mutex, to reduce contention; a reader-writer lock does not help because `get` writes. **Mistakes.** Locking only `put`; refreshing the expiry on `get` when the specification says the TTL runs from the write.

Q1.100 LFU cache

AMAZON · GOOGLE · TECH-2 · ■■■ · T

Like an LRU cache, but evict the least frequently used key; among equals, evict the least recently used. `get` *and* `put` *must be O(1).*

30-second answer: map each key to its value, count and list position; keep one list of keys per count, and the minimum count so that the victim is the back of that list.

```
class LFU {
  struct E { int v, f;
             list<int>::iterator it; };
  int cap, minf = 0;
  unordered_map<int, E> kv;
  unordered_map<int, list<int>> fl;
  void touch(int k) {
    E& e = kv[k];
    fl[e.f].erase(e.it);
    if (fl[e.f].empty()) {
      fl.erase(e.f);
      if (minf == e.f) ++minf;
    }
    ++e.f;
    fl[e.f].push_front(k);
    e.it = fl[e.f].begin();
  }
public:
  explicit LFU(int c) : cap(c) {}
  int get(int k) {
    if (!kv.count(k)) return -1;
    touch(k);
    return kv[k].v;
  }
  void put(int k, int v) {
    if (!cap) return;
    if (kv.count(k)) {
      kv[k].v = v; touch(k); return;
    }
    if ((int)kv.size() == cap) {
      int old = fl[minf].back();
      fl[minf].pop_back();
      if (fl[minf].empty())
fl.erase(minf);
      kv.erase(old);
    }
    minf = 1;
    fl[1].push_front(k);
    kv[k] = {v, 1, fl[1].begin()};
  }
};
```

Complexity. O(1) average per operation. **Mistakes.** Not updating `minf` when a frequency list empties; resetting `minf` to 1 only after eviction, not on a new insertion; erasing the key from `kv` but not its list node.

Q1.101 Fixed-window versus sliding-window rate limiting

MICROSOFT · AMAZON · TECH-1 · ■■□ · T

Implement a limit of `lim` requests per window of `w` time units in two ways, and show a request trace on which the fixed-window version lets through twice the limit.

30-second answer: a fixed window keeps one counter per aligned window and resets it at the boundary; a sliding log keeps the timestamps of accepted requests and drops those older than `w`.

```
struct Fixed {
  long long w, win = -1;
  int lim, n = 0;
  Fixed(long long w, int lim)
      : w(w), lim(lim) {}
  bool allow(long long t) {
    if (t / w != win) { win = t / w; n
= 0; }
    return n < lim && ++n;
  }
};
struct SlidingLog {
  long long w; size_t lim;
  deque<long long> q;
  SlidingLog(long long w, size_t lim)
      : w(w), lim(lim) {}
  bool allow(long long t) {
    while (!q.empty() && q.front() <= t
- w)
      q.pop_front();
    if (q.size() >= lim) return false;
    q.push_back(t);
    return true;
  }
};
```

With *lim* = 5 and *w* = 10: five requests at time 9 and five at time 11. The fixed window starts a new window at time 10, so it accepts all ten (ten requests in two time units). The sliding log still holds the five requests of time 9 at time 11 (since 9 > 11 − 10), so it rejects the second batch. **Complexity.** Fixed: O(1) time and space. Log: O(1) amortised time, O(*lim*) space. **Follow-ups.** The token bucket of §1.15 (p. 43) is a middle course: two numbers of state and bounded bursts.

Q1.102 Rate-limit repeated log messages

MICROSOFT · AMAZON · OA · ■□□ · T

A logger prints a message only if the same message has not been printed in the last 10 seconds. Timestamps are non-decreasing.

30-second answer: a hash map from message to the time of its last print.

```
class Logger {
  unordered_map<string, int> last;
public:
  bool shouldPrint(int t, const string&
m) {
    auto it = last.find(m);
    if (it != last.end() && t - it-
>second < 10)
      return false;
    last[m] = t;
    return true;
  }
};
```

Complexity. O(1) average. **Follow-ups.** The map grows without bound; drop entries older than ten seconds with a queue of (time, message) pairs from the front. **Mistakes.** Updating the time of a message that was suppressed (the window would then never end for a steady stream).

Q1.103 Insert, delete and get-random in O(1)

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Design a set with `insert(x)`, `remove(x)` and `getRandom()` (uniform over the elements), all O(1) on average.

30-second answer: keep the elements in a vector for random access and a hash map from value to its index; remove by moving the last element into the removed slot.

```
class RandomSet {
  vector<int> v;
  unordered_map<int, int> pos;
public:
  bool insert(int x) {
    if (pos.count(x)) return false;
    pos[x] = v.size();
    v.push_back(x);
    return true;
  }
  bool remove(int x) {
    auto it = pos.find(x);
    if (it == pos.end()) return false;
    int last = v.back(), i = it->second;
    v[i] = last;
    pos[last] = i;
    v.pop_back();
    pos.erase(x);
    return true;
  }
  int getRandom() { return v[rand() %
v.size()]; }
};
```

Moving the last element avoids shifting the vector. When `x` is itself the last element, `v[i] = last` and `pos[last] = i` are harmless self-assignments, and `pos.erase(x)` then removes it. **Complexity.** O(1) average. **Mistakes.** Erasing from the middle of the vector (O(*n*)); forgetting to update the index of the moved element.

::: {.p1} ::: pack ## Revision Spread {#rv1}

Q1.104 Valid number (a finite-state scan)

JUNIPER · MICROSOFT · TECH-2 · ■■■ · T

Decide whether a string is a valid decimal number: optional sign, digits with an optional point (at least one digit overall), and an optional exponent `e` or `E` with an optional sign and at least one digit.

30-second answer: scan in the order sign, integer part, point and fraction, exponent; each stage is a state, and a stage needing a digit fails if none follows.

```
bool isNumber(const string& s) {
  size_t i = 0, n = s.size();
  bool dig = false;
  auto D = [&](size_t k) {
    return k < n && isdigit((unsigned
char)s[k]);
  };
  if (i < n && (s[i] == '+' || s[i] ==
'-')) ++i;
  while (D(i)) { ++i; dig = true; }
  if (i < n && s[i] == '.') {
    ++i;
    while (D(i)) { ++i; dig = true; }
  }
  if (!dig) return false;
  if (i < n && (s[i] == 'e' || s[i] ==
'E')) {
    ++i;
    if (i < n && (s[i] == '+' || s[i]
== '-')) ++i;
    if (!D(i)) return false;
    while (D(i)) ++i;
  }
  return i == n;
}
```

Accepted: `"2"`, `".5"`, `"3."`, `"-1.2e+10"`. Rejected: `"."`, `"e3"`, `"1e"`, `"1e+"`, `"--1"`, `"1 "`. The same logic as a table over (state, character class) is the form to present for a state-machine question: the states are start, sign, integer, point, fraction, exponent, exponent sign and exponent digits. **Complexity.** O(*n*). **Mistakes.** Accepting a bare point, or an exponent with no digits.

THE 15-MINUTE VERSION

Ten facts. (1) Read the constraints first: $n \le 10^5$ means O(n log n), $n \le 10^4$ means O(n^2). (2) Say the six steps aloud: understand, examples, brute force, optimise, code, test. (3) Sorted input suggests two pointers or binary search; "contiguous" suggests a window or prefix sums. (4) Mask for prefix p: `p ? ~0u << (32-p) : 0`; network is `ip & mask`, broadcast is `net | ~mask`. (5) Delete from a list with `Node **pp`; test `fast` before `fast->next`. (6) A trie reads one bit per level; longest-prefix match keeps the deepest marked node and costs O(32). (7) BFS marks on push; Dijkstra skips stale entries and needs non-negative weights. (8) LRU is a hash map to doubly-linked-list nodes; every list change is mirrored in the map. (9) Token bucket: `tokens = min(B, tokens + r·Δt)`, pass if `tokens ≥ size`; integers and a monotonic clock. (10) Binary search: half-open `[lo, hi)`, `mid = lo + (hi-lo)/2`, never `lo = mid`.

Three pictures to redraw. The binary trie with a walk that remembers the deepest mark (§1.9 (p. 30)); the pointer-to-pointer delete and the reversal rewiring (§1.6 (p. 20)); the LRU map-plus-list diagram (§1.15 (p. 43)).

Constraint to complexity

Largest n	Aim for	Typical tools
10–12	O(n!)	permutations, backtracking
20–25	O(2^n)	subsets, bitmask
100	O(n^4)	small DP
500	O(n^3)	Floyd–Warshall
2000–5000	O(n^2)	2-D DP, all pairs
10^5–2×10^5	O(n log n)	sort, heap, binary search
10^6	O(n)	windows, prefix sums, hashing
10^9 and up	O(log n) or O(1)	binary search on the answer, formula

A judge performs about 10^8 simple operations per second; in Python step down one row. Values up to 10^9 mean no array indexed by value, and sums of 10^5 such values need 64 bits.

Keyword to pattern

If the problem says	Think	See
sorted; pair or triple with sum	two pointers, binary search	§1.2 (p. 10)
longest or shortest contiguous ... with a condition	variable sliding window	§1.2 (p. 10)
subarray of size k	fixed window	§1.2 (p. 10)
subarray sum equals k, negatives	prefix sums and a map	§1.2 (p. 10)
seen before, count, group, duplicates	hash set or map	§1.3 (p. 14)
valid, parse, split IP or MAC	single-scan validator	§1.4 (p. 16)
mask, flag, single number, power of two	bit tricks	§1.5 (p. 17)
cycle, middle, k-th from end	fast and slow pointers	§1.6 (p. 20)
reverse, merge, delete in a list	rewire with `prev`, dummy head, `**pp`	§1.6 (p. 20)
matching, undo, next greater	stack, monotonic stack	§1.7 (p. 24)
fixed-size FIFO, driver buffer	ring buffer with a count	§1.7 (p. 24)
window maximum	monotonic deque	§1.7 (p. 24)
tree depth, path, subtree	DFS, post-order return	§1.8 (p. 27)
level by level	BFS with a queue	§1.8 (p. 27)

prefix, autocomplete, IP route lookup	trie	§1.9 (p. 30)
top *k*, median of a stream, merge *k* lists	heap	§1.10 (p. 33)
fewest hops; grid regions	BFS; flood fill	§1.11 (p. 34)
cheapest path, non-negative	Dijkstra	§1.11 (p. 34)
at most *k* edges; negative weights	Bellman–Ford	§1.11 (p. 34)
prerequisites, order of tasks	topological sort	§1.11 (p. 34)
loop in a network, components	union–find	§1.11 (p. 34)
all combinations or permutations	backtracking	§1.12 (p. 39)
minimise the maximum, "at most *D* days"	binary search on the answer	§1.13 (p. 40)
overlapping ranges, IP ranges	sort by start, merge	§1.13 (p. 40)
number of ways, min cost, can reach	dynamic programming	§1.14 (p. 42)
O(1) get and recency	hash map and doubly linked list	§1.15 (p. 43)
allow *r* per second with bursts	token bucket	§1.15 (p. 43)
validate a sequence of protocol events	state table	§1.16 (p. 47)

Big-O at a glance

Structure	Access	Search	Insert	Delete
array	O(1)	O(*n*)	O(*n*)	O(*n*)
dynamic array	O(1)	O(*n*)	O(1) amortised at end	O(*n*)
singly linked list	O(*n*)	O(*n*)	O(1) after a known node	O(1) after the predecessor
stack, queue, deque, ring	end only	O(*n*)	O(1)	O(1)
hash table	–	O(1) avg, O(*n*) worst	O(1) avg	O(1) avg
BST balanced / skewed	–	O(log *n*) / O(*n*)	same	same
binary heap	min: O(1)	O(*n*)	O(log *n*)	O(log *n*)
trie, key length *L*	–	O(*L*)	O(*L*)	O(*L*)
union–find	–	O(α(*n*))	O(α(*n*))	–

Sort	Time (typical)	Worst	Extra space	Stable
insertion	O(n^2), O(*n*) if nearly sorted	O(n^2)	O(1)	yes
merge	O(*n* log *n*)	O(*n* log *n*)	O(*n*)	yes
quick	O(*n* log *n*)	O(n^2)	O(log *n*)	no
heap	O(*n* log *n*)	O(*n* log *n*)	O(1)	no
counting, radix	O(*n* + *k*)	same	O(*k*)	yes

C++ `std::sort` is introsort (not stable); `stable_sort` is a merge sort; Python's `sorted` is Timsort (stable).

Graph algorithm	Time
BFS, DFS, topological sort	O(*V* + *E*)
Dijkstra with a binary heap	O((*V* + *E*) log *V*)
Bellman–Ford	O(*VE*)
Floyd–Warshall	O(V^3)
Kruskal; Prim with a heap	O(*E* log *E*); O(*E* log *V*)

Recurrences: $T(n) = 2T(n/2) + O(n)$ gives O(*n* log *n*); $T(n) = T(n/2) + O(1)$ gives O(log *n*); $T(n) = 2T(n-1) + O(1)$ gives O(2^n). Landmarks: $\log_2 10^6 \approx 20$, $\log_2 10^9 \approx 30$, $2^{10} \approx 10^3$, `INT_MAX` ≈ 2.1×10^9, `LLONG_MAX` ≈ 9.2×10^{18}.

Edge-case checklist

- **Values:** duplicates, all equal, negatives, zero, `INT_MIN`, `INT_MAX`, overflow of a sum or product, `abs(INT_MIN)`.
- **Order:** sorted, reverse sorted, rotated.
- **Strings:** empty, spaces, case, one character, leading zeros.
- **Lists:** `NULL` head, one node, cycle, operating on the head or the tail.
- **Trees:** `NULL` root, single node, skewed (depth n), duplicate keys.
- **Graphs:** disconnected, self-loop, parallel edges, cycle, unreachable target.
- **Intervals:** touching endpoints, nested intervals, ask whether adjacent ranges merge.
- **Binary search:** target below all, above all, absent, at both ends.
- **Networking:** `01.2.3.4`, `256.1.1.1`, a missing octet, /0 and /32, network and broadcast addresses, port 0 and 65535.

Code skeletons in one line each

- Prefix mask `p ? ~0u << (32-p) : 0`; same subnet `(a & m) == (b & m)`; IP to integer `a << 24 | b << 16 | c << 8 | d`.
- Bit idioms: `x & (x-1)` clears the lowest set bit; `x & -x` isolates it; `x && !(x & (x-1))` tests a power of two; `(x + a - 1) & ~(a - 1)` aligns up.
- List: `Node **pp = &head; while (*pp && (*pp)->val != k) pp = &(*pp)->next;` then `*pp = (*pp)->next`.
- Reverse: save `next`, set `head->next = prev`, advance `prev` and `head`.
- LPM walk: `best = root->hop; for i in 0..31: t = t->c[bit]; if (t && t->hop >= 0) best = t->hop`.
- Dijkstra: heap of (distance, vertex); `if (d > dist[u]) continue;` relax edges.
- Union–find: `find` with path compression; `unite` returns false if already joined.
- LRU: map key to list node; `get` moves the node to the front; `put` evicts the back and erases its map entry.
- Token bucket: refill on demand from a monotonic clock, cap at B, pass if the level covers the packet.

Flashcards

Q	A
Complexity of a loop that halves n? §1.1 (p. 9)	O(log n)
Why is `vector::push_back` O(1) amortised?	Copies at doublings total less than $2n$
Two Sum in O(n)? §1.1 (p. 9)	Hash map value to index; look up `target - a[i]`, then insert
Variable window: `if` or `while` to shrink? §1.2 (p. 10)	`while`, until the window is valid again
Seed of the prefix-sum map? §1.2 (p. 10)	`{0: 1}`, for subarrays starting at index 0
Why is `m[key]` unsafe for lookup in C++? §1.3 (p. 14)	It inserts a default value
Why reject `01.2.3.4`? §1.4 (p. 16)	Leading zeros are ambiguous (octal)
Is `strncpy` safe? §1.4 (p. 16)	No: it does not terminate when truncating
Hosts in a /27? §1.5 (p. 17)	2^5 − 2 = 30
Why does `x & 1 == 0` fail? §1.5 (p. 17)	== binds tighter; write `(x & 1) == 0`
Why special-case /0 in a mask?	Shifting a 32-bit value by 32 is undefined
Popcount by loop? §1.5 (p. 17)	`x &= x - 1` until zero
Broadcast from network and mask?	`net | ~mask`
Why use `Node **pp` for delete? §1.6 (p. 20)	The head needs no special case
Floyd: why does resetting to the head find the cycle start? §1.6 (p. 20)	$a = (k-1)L + c$, so both pointers arrive together
Order of the loop test in Floyd?	`fast` first, then `fast->next`
Ring buffer, full or empty? §1.7 (p. 24)	Keep a count (or waste a slot, or use free-running indices)
Monotonic stack stores?	Indices, not values
BST delete with two children? §1.8 (p. 27)	Copy the in-order successor, delete it from the right subtree
Why does a sorted insertion hurt a BST?	The tree becomes a chain; operations cost O(n)
Validate a BST correctly?	Pass open bounds down, or check that in-order is increasing
LPM data structure and cost? §1.9 (p. 30)	Binary trie, O(32) per lookup, independent of the table size
LPM without a trie?	One hash table per length, probe /32 down to /0

Heap array parent and children? §1.10 (p. 33)	Parent $(i-1)/2$; children $2i+1$ and $2i+2$
Heap for the k largest?	A min-heap of size k
BFS: when to mark visited? §1.11 (p. 34)	When pushed
Dijkstra stale check?	`if (d > dist[u]) continue;`
Dijkstra with a negative edge?	Wrong; use Bellman–Ford
Bellman–Ford with a hop limit?	Run k rounds, relaxing from a copy
OSPF versus RIP, algorithmically?	Dijkstra on the whole graph versus distributed Bellman–Ford
Union–find: `unite` returns false means?	The edge closes a loop
Topological sort detects a cycle how?	Fewer than V vertices are output
Backtracking steps? §1.12 (p. 39)	Choose, explore, unchoose
Binary-search midpoint? §1.13 (p. 40)	`lo + (hi - lo) / 2`
Binary search on the answer needs?	A monotone yes/no test
Merge intervals: sort by?	Start; compare with the last merged end
The five DP steps? §1.14 (p. 42)	State, recurrence, base, order, answer cell
Why is greedy wrong for coins {1,3,4}, amount 6?	Greedy gives 4+1+1; the best is 3+3
Why is LRU O(1)? §1.15 (p. 43)	The map finds the node; the node unlinks itself from a doubly linked list
Common LRU bug?	Evicting from the list but not the map
Token-bucket refill formula?	`tokens = min(B, tokens + r·Δt)`
Fixed-window flaw?	Up to twice the limit across a window boundary
Table-driven FSM benefit? §1.16 (p. 47)	Transitions are data, reviewable and testable

:::

PART I · PILLAR 2

2

Programming: C, Python, C++

Network software is written close to the machine; we learn to see the bytes behind every line.

2.1 Memory layout of a process	MUST
2.2 Types, sizes and conversions	MUST
2.3 Pointers and pointer arithmetic	MUST
2.4 Arrays, pointers and strings	MUST
2.5 Declarations, const and function pointers	MUST
2.6 Storage classes and qualifiers	MUST
2.7 Structs, unions, padding and alignment	MUST
2.8 Bit-fields and bit manipulation	MUST
2.9 Endianness and byte order	MUST
2.10 Dynamic memory and leaks	MUST
2.11 The preprocessor and macros	MUST
2.12 Undefined behaviour	MUST
2.13 Writing the library: memcpy to itoa	MUST
2.14 Parsing an IPv4 header	MUST
2.15 The gotcha gallery: 36 output traps	MUST
2.16 Python: the data model	SHOULD
2.17 Python: comprehensions and generators	SHOULD
2.18 Python: struct, re, subprocess, files, exceptions	MUST

2.19 C++: references, classes and RAII	SHOULD
2.20 C++: virtual functions and the vtable	SHOULD
2.21 C++: STL containers and smart pointers	SHOULD

2.0 Why this pillar matters, and how to read it

Switch operating systems, routing daemons and data-plane software are written largely in C and C++, because they must control memory layout and run without a garbage collector. Automation around them is mostly Python. Interviewers therefore test C hardest: pointers, structures, bytes and output prediction. Part A (sections 2.1–2.15) is the bulk of the pillar and is entirely Must. Part B covers the Python language only, and Part C the C++ that interviews reach.

Three boundaries keep the book free of repetition. The sockets API belongs to Pillar 7 (p. 285), shell scripting to Pillar 8, Python's network libraries to Pillar 10, and linked-list and tree algorithms to Pillar 1 (p. 7). Here we meet only the language features those chapters rely on.

All code assumes 64-bit Linux (the LP64 data model), `gcc` 13 and an x86-64 processor, unless stated otherwise. Compile with `gcc -Wall -Wextra -g file.c -o file`. Every output printed below was produced by running the code under these assumptions. Where the standard leaves behaviour open, the text says so and does not print an output.

> BIG IDEA
>
> A C program is a sequence of bytes at addresses; every type, pointer and structure is a rule for interpreting those bytes, and every trap in this pillar is a place where the rule and the bytes disagree.

2.1 Memory layout of a process MUST · ■□□

When Linux starts a program it gives the process a private *virtual address space*, a range of addresses that the hardware maps to physical memory page by page. The kernel divides this space into regions called *segments* or *sections*, and the C compiler decides which region each object lives in. Knowing the regions answers a surprising share of interview questions, among them "where does a static local live?" and "why did this big array crash my program?".

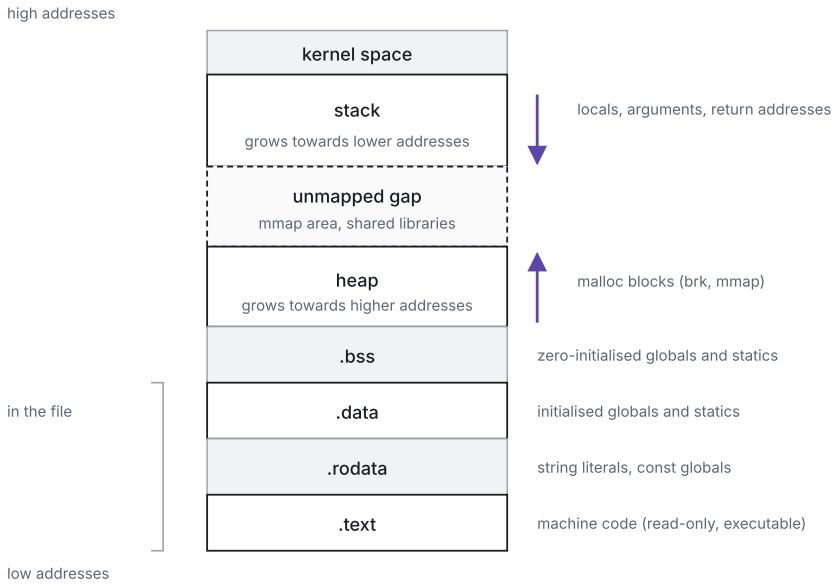


Figure 2.1 The layout of a Linux process. The three lowest sections come from the executable file; .bss occupies no space in the file.

Figure 2.1 shows the regions from the high addresses downwards. The *text* section holds machine code and is read-only. The *rodata* section holds string literals and `const` globals. The *data* section holds globals and statics with a non-zero initial value, and the *BSS* (block started by symbol) holds those initialised to zero, which is why the file records only their total size. The *heap* serves `malloc`, and the *stack* serves function calls. Between them lies unmapped space, so that neither can grow into the other without a fault.

Object	Section	Lifetime	Initial value
`int g = 7;` (global)	.data	whole program	7
`int g;` (global)	.bss	whole program	0
`static int s;` (file or function scope)	.bss (.data if initialised)	whole program	0
`const int k = 3;` (global)	.rodata	whole program	3
`"hello"` (string literal)	.rodata	whole program	the bytes
`int x;` (local)	stack	until the block ends	indeterminate
`malloc(n)`	heap	until `free`	indeterminate

WORKED EXAMPLE

Classify the objects in a small program.

1. Compile the program below and print the address of one object from each row of the table.
2. Read the addresses in order. Code, literals and globals sit close together near the bottom, the heap lies above them, and the stack is far higher.
3. Check the sizes with `size`: the BSS figure counts the zero-initialised bytes, while `text` includes the read-only data.

```
int g_init = 7;  int g_zero;  const int g_ro = 3;
int main(void) {
    static int s_init = 9;      /* .data */
    int local = 1;              /* stack */
    char *lit = "hello";        /* pointer: stack, bytes: .rodata */
    int *h = malloc(sizeof *h); /* heap */
    printf("%p %p %p\n", (void *)&g_ro, (void *)h, (void *)&local);
    free(h); (void)s_init; (void)lit; return 0;
}
```

Result: on one run the three addresses were `0x5595455f9004`, `0x55954f44b2a0` and `0x7fffa525c4f4`, in increasing order. Exact values change on every run because of address-space randomisation; only the order is stable.

TRAP

Claim: a `static` local array is stored on the stack like any other local. *Why it is wrong:* `static` changes the lifetime, not just the visibility, so the array is placed in .data or .bss and exists for the whole run. *Correct statement:* making a large local array `static` removes the stack-overflow risk, but the function is then no longer re-entrant, because every call shares the same bytes.

INTERVIEW LENS

QUALCOMM · ARISTA · CISCO

> “What kind of variable is stored where?” is a reported Qualcomm question, and Arista has asked candidates to print and classify the addresses of a global, a static, a local, a heap object and a literal. A strong answer draws Figure 2.1 from memory, names the section for each object, and adds that the stack and heap grow towards each other. Follow-ups: what is the BSS for, and what happens on stack overflow (a segmentation fault from the guard page).

CHECK YOURSELF

1. In which section does `static int n = 0;` inside a function live? [1]
2. Why does the BSS take no room in the executable file? [2]
3. Where is the pointer `char *p = "abc";` stored, and where are the characters? [3]

2.2 Types, sizes and conversions MUST · ■□□

The size of `int` is not fixed by the language. The standard guarantees only that `char` is one byte and that `sizeof(short) <= sizeof(int) <= sizeof(long)`. On 64-bit Linux (the *LP64* model) `int` is 4 bytes while `long` and pointers are 8; on 32-bit systems (*ILP32*) all three are 4. The full size table is in the Revision Spread (p. 143). Code that handles packet fields must therefore use the fixed-width types of `<stdint.h>`: `uint8_t`, `uint16_t`, `uint32_t` and `uint64_t`, printed with the macros `PRIu32` and its relatives from `<inttypes.h>`.

Arithmetic does not happen in the narrow type. Two rules govern every mixed expression. First, the *integer promotions* widen `char` and `short` operands to `int`. Second, the *usual arithmetic conversions* bring the two operands to a common type: if one is `unsigned int` and the other `int`, the signed one is converted to unsigned. This is the origin of several famous surprises.

Integer promotion — in an expression, any type narrower than `int` is first widened to `int`, so `char + char` is computed as `int`.

```
uint8_t x = 200, y = 100;
printf("%d\n", x + y);                 /* 300, not 44: computed as int */
unsigned int u = 1; int b = -1;
printf("%s\n", b < u ? "true" : "false"); /* false: -1 becomes 4294967295 */
```

The first line prints `300` because both operands are promoted; the sum is truncated to 44 only when stored back into a `uint8_t`. The second prints `false` because `-1` converted to `unsigned int` is `UINT_MAX`. Hence, comparing a signed and an unsigned value is a defect unless one has first proved the signed value is non-negative; `gcc -Wextra` warns about it.

Signedness of `char` is implementation-defined: it is signed on x86 and unsigned on most ARM Linux builds. Thus `char c = 200;` stores -56 on x86 and 200 on ARM, and any code that indexes a table with a plain `char` is a latent bug. Use `unsigned char` or `uint8_t` for bytes.

Overflow behaves differently by signedness. Unsigned arithmetic wraps modulo 2^n and is fully defined, which is exactly what a 32-bit byte counter needs: with `uint32_t before, after`, the expression `after - before` yields the correct delta even after the counter wraps. Signed overflow is undefined behaviour (2.12 (p. 105)).

2.3 Pointers and pointer arithmetic MUST · ■□□

A *pointer* is a variable that holds an address. If `int x = 5;` then `&x` is the address of `x`, and for `int *p = &x;` the expression `*p` is the object at that address, so `*p = 9;` changes `x`. A pointer that points nowhere holds `NULL`, and dereferencing it is undefined behaviour that in practice raises a segmentation fault. A pointer of type `T *` knows the size of `T`, and this size drives all pointer arithmetic.

BIG IDEA

Adding `n` to a `T *` moves the address by `n * sizeof(T)` bytes, so pointer arithmetic counts elements, not bytes.

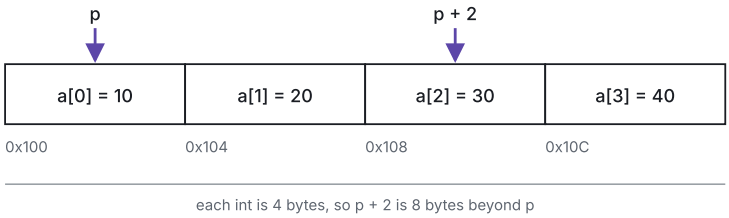


Figure 2.2 With `int *p = a;`, the pointer `p + 2` holds 0x108, because the address advances by two elements of four bytes.

The rules are short. Adding or subtracting an integer moves the pointer by that many elements. Subtracting two pointers into the same array gives the number of elements between them, of type `ptrdiff_t`. Comparing pointers is meaningful only within one array. A `void *` carries no element size, so it cannot be dereferenced or incremented in standard C; it converts implicitly to and from any object pointer type in C, which is why `malloc` returns it. To step through raw bytes, cast to `unsigned char *`.

```
int a[4] = {10, 20, 30, 40};
int *p = a;
printf("%d %d\n", *(p + 2), p[3]);                    /* 30 40 */
printf("%ld\n", (long)(&a[3] - &a[0]));               /* 3: elements   */
printf("%ld\n", (long)((char *)(p + 1) - (char *)p)); /* 4: bytes      */
```

A function receives copies of its arguments, so to modify the caller's variable it must receive its address. The same idea with one more level gives the pointer to pointer, needed whenever a function must change a pointer itself, as when it allocates memory for the caller.

```
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int make_buffer(char **out, size_t n) {   /* caller's pointer is changed */
    *out = malloc(n);
    return *out ? 0 : -1;
}
```

TRAP

Claim: `p + 1` always adds one to the address. *Why it is wrong:* the increment is scaled by the pointee size, so for `int *p` the address grows by 4, and for `struct ethhdr *` by 14. *Correct statement:* to move by bytes, cast to `unsigned char *` first, as in `(uint8_t *)p + 1`.

CHECK YOURSELF

1. If `long *q` holds 0x2000, what does `q + 3` hold? [4]
2. What does `&a[3] - &a[0]` evaluate to for `int a[4]`, and what is its type? [5]

2.4 Arrays, pointers and strings MUST · ■□□

An array is a contiguous block of elements, and a pointer is a single address; they are different things that the language lets one mix. In nearly every expression the name of an array *decays* to a pointer to its first element, so `a[i]` is defined as `*(a + i)`. The exceptions are the operand of `sizeof`, the operand of `&`, and a string literal that initialises an array. Because `a[i]` means `*(a + i)` and addition commutes, even `i[a]` is legal and means the same.

```
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d %d %zu\n", m[1][2], *(*(m + 1) + 2), sizeof m);  /* 6 6 24 */
printf("%zu %zu\n", sizeof(m[0]), sizeof(&m[0]));           /* 12 8   */
```

A two-dimensional array is an array of arrays laid out row by row, so `m[0]` is itself an array of 3 ints (12 bytes) and `&m[0]` is a pointer to it (8 bytes). Decay has one consequence that every interviewer tests: an array *parameter* is really a pointer. In `void f(int a[10])` the `10` is ignored and `sizeof(a)` is 8. Hence a function that needs the length must receive it as a separate argument.

A C *string* is an array of `char` ending with a zero byte `'\0'`. Its length excludes the terminator, while its storage includes it. Two declarations look alike and differ completely, as Figure 2.3 shows.

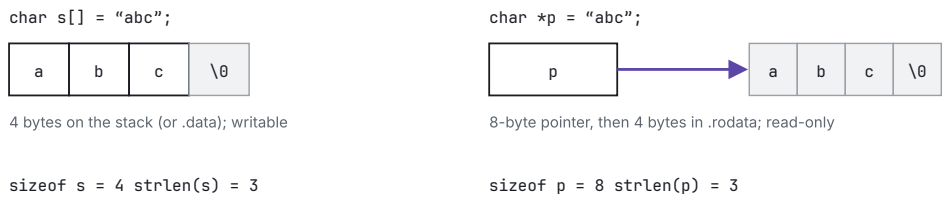


Figure 2.3 An array owns its characters; a pointer to a literal merely refers to read-only characters.

Writing through `p[0] = 'A'` in the second form is undefined behaviour and normally faults, because the literal is in read-only memory. With the array form the write is legal, since the array is a private, writable copy.

The standard string functions all walk to the terminator. `strlen(s)` counts characters and is O(n). `strcpy(d, s)` copies until and including the terminator and does not check that `d` is large enough; this is the classic buffer overflow. `strncpy(d, s, n)` is not its safe version: if `s` has `n` or more characters it writes no terminator, and if it has fewer it pads `d` with zeros up to `n`. The reliable idiom is `snprintf(d, sizeof d, "%s", s)`, which always terminates. `strcmp` returns zero for equal strings and otherwise a negative or positive value, so `if (strcmp(a, b))` is true when the strings *differ*.

> **LAB**
>
> `LAB 2.1 · 15 MIN`
>
> ```
> gcc -Wall -g -o strdemo strdemo.c      # strdemo.c: the two declarations of Figure 2.3
> ./strdemo
> gcc -g -fsanitize=address -o strdemo strdemo.c && ./strdemo
> ```

With `p[0] = 'A'` uncommented, the plain build prints `Segmentation fault`. The sanitised build prints `AddressSanitizer: SEGV on unknown address` and a stack trace whose top frame names the line.

What you should notice: the array form never faults, and the sanitiser reports the line, which is quicker than reading a core dump.

CHECK YOURSELF

1. What does `sizeof(a)` return inside `void f(int a[10])` on a 64-bit machine? [6]
2. After `strncpy(d, "abcdef", 3)`, is `d` a valid string? [7]

2.5 Declarations, const and function pointers MUST · ■■□

C declarations are read from the identifier outwards: start at the name, look right, then left, and respect the parentheses. In `int *const p` the name `p` is a constant, and it is a pointer to `int`. The qualifier `const` applies to the thing immediately on its left, or to the type on its right when nothing stands on its left. Three forms cover every interview question.

Declaration	Reads as	May change `*p`?	May change `p`?
`const int *p` (same as `int const *p`)	`p` is a pointer to a constant `int`	no	yes
`int *const p`	`p` is a constant pointer to `int`	yes	no
`const int *const p`	constant pointer to a constant `int`	no	no

TRAP

Claim: `const char *s` means the string cannot change, so the pointer cannot move either. *Why it is wrong:* here `const` binds to `char`, the pointee; the pointer is free to move, as in `s++`. *Correct statement:* to forbid moving the pointer one writes `char *const s`, and to forbid both, `const char *const s`. Functions that only read a buffer should take `const T *`, which documents intent and lets the compiler reject accidental writes.

A *function pointer* holds the address of code. Its declaration mirrors a function prototype with the name wrapped as `(*name)`: `int (*fp)(int, int);` declares `fp` as a pointer to a function taking two `int` and returning `int`. The parentheses matter, because `int *fp(int, int)` declares a function returning `int *`. A `typedef` makes such types readable, and calling through the pointer needs no dereference: `fp(2, 3)` and `(*fp)(2, 3)` are the same.

Function pointers appear in two interview patterns. The first is the *callback*: the library calls code the caller supplies, as `qsort` does with its comparator. The second is the *dispatch table*: an array of function pointers indexed by a small integer, replacing a long `switch`. Network code uses it to route a packet to a handler by IP protocol number.

1. .bss, because the initial value is zero; `static int n = 5;` would be in .data.↩
2. Its contents are known to be zero, so the file records only the size and the loader supplies zeroed pages.↩
3. The pointer follows the rule for its own storage (stack for a local, .data for a global); the characters are in .rodata.↩
4. 0x2018, since 3 elements of 8 bytes add 24 bytes.↩
5. 3, of type `ptrdiff_t` (a signed integer); the unit is elements, not bytes.↩
6. 8, because the parameter is a pointer; the length 10 is ignored.↩
7. Not necessarily: three characters were copied and no terminator was written, so `d` is terminated only if its fourth byte happened to be zero.↩

```c
typedef int (*handler_fn)(const uint8_t *pkt, size_t len);

static int on_icmp(const uint8_t *p, size_t n) { (void)p; printf("icmp %zu\n",
n); return 0; }
static int on_tcp (const uint8_t *p, size_t n) { (void)p; printf("tcp %zu\n",
n); return 0; }
static int on_udp (const uint8_t *p, size_t n) { (void)p; printf("udp %zu\n",
n); return 0; }

static handler_fn table[256] = { [1] = on_icmp, [6] = on_tcp, [17] = on_udp };

int dispatch(uint8_t proto, const uint8_t *pkt, size_t len) {
    handler_fn fn = table[proto];        /* unset entries are NULL        */
    if (!fn) { printf("proto %u: no handler\n", proto); return -1; }
    return fn(pkt, len);
}
```

Calling `dispatch(6, buf, 40)`, `dispatch(17, buf, 8)` and `dispatch(47, buf, 0)` prints `tcp 40`, `udp 8` and `proto 47: no handler`. The designated initialisers `[1] = on_icmp` set only the named slots, and a static array zeroes the rest, so the `NULL` test is reliable. The lookup costs one indexed load and one indirect call, independent of the number of protocols.

The comparator of `qsort` receives `const void *` and must cast back. The safe return expression is `(x < y) - (x > y)`, because the shortcut `x - y` overflows for extreme values, which is undefined behaviour (2.12 (p. 105)).

```c
int cmp_desc(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x < y) - (x > y);              /* descending order */
}
/* int v[] = {3, 9, 1, 7};  qsort(v, 4, sizeof v[0], cmp_desc);  ->  9 7 3 1 */
```

Sorting a routing table needs two keys, longest prefix first and then address ascending. The comparator tests the length first and falls through to the address only on a tie: `if (x->len != y->len) return (int)y->len - (int)x->len;`. With prefixes `10.0.0.0/8`, `192.168.1.0/24`, `10.1.0.0/16` and `192.168.0.0/24` the sorted order is `192.168.0.0/24`, `192.168.1.0/24`, `10.1.0.0/16`, `10.0.0.0/8`.

> INTERVIEW LENS
>
> CISCO · JUNIPER
>
> “What are function pointers? Show how you would dispatch packets by protocol number.” is reported as an interview topic at both companies. A strong answer shows the `typedef`, the table with designated initialisers, the `NULL` check, and says why a table beats a `switch` when handlers are registered at run time. Follow-up: how would a module register its own handler (a `register_handler(proto, fn)` function that writes the slot).

> CHECK YOURSELF
>
> 1. What does `const char *const s` forbid? [1]
> 2. Why is `int *fp(int, int)` not a function pointer? [2]

2.6 Storage classes and qualifiers MUST · ■■□

The *storage class* of an object fixes its lifetime and linkage, and a *qualifier* restricts how the compiler may treat accesses. Both are interview staples, and the word `static` alone has three meanings.

`static`. (i) On a local variable it gives the variable static storage duration: one instance, created before `main`, zero-initialised, keeping its value between calls. (ii) On a global variable or function it gives *internal linkage*: the name is visible only inside that translation unit (one `.c` file after preprocessing), so two files may each define `static int count;` without a clash. (iii) A `static` array inside a function therefore moves from the stack to .data or .bss, as noted in 2.1 (p. 91).

`extern`. It declares an object defined elsewhere and allocates nothing. A header should declare (`extern int g;`) and exactly one `.c` file should define (`int g = 0;`). Defining a variable in a header gives each including file its own copy, or a linker error for duplicates; marking it `static` in the header silently gives every file a private copy, which is almost never intended.

`auto` **and** `register`. `auto` is the default for locals and is never written. `register` is a historical hint that forbids taking the object's address; compilers now ignore it for optimisation.

`const`. It makes writes through that name an error. A global `const` usually lives in .rodata, so casting `const` away and writing to it is undefined behaviour. A `const` local is still on the stack but the compiler may assume it never changes.

`volatile`. It tells the compiler that an object may change in ways the code does not show, so every access in the source must really happen, in order. Three situations need it: a memory-mapped hardware register, a flag set by a signal handler or interrupt service routine, and a variable modified by `setjmp`/`longjmp` paths. Consider the loop `while (!flag) ;` waiting for an interrupt.

```
int flag;            void wait_plain(void) { while (!flag) ; }
volatile int vflag;  void wait_vol(void)   { while (!vflag) ; }
```

LAB

LAB 2.2 · 10 MIN

```
gcc -O2 -S -o - v.c | grep -v '^\s*\.'
```

```
wait_plain:  movl flag(%rip),%eax ; testl %eax,%eax ; jne .L1 ; jmp .L3
wait_vol:    movl vflag(%rip),%eax ; testl %eax,%eax ; je .L6 ; ret
```

What you should notice: `wait_plain` loads `flag` once and then spins on `jmp .L3` without reading memory again, so it never sees the change. `wait_vol` branches back to the load on each pass (the `.L6` label), because every access to a `volatile` object must happen.

A `const volatile` object is legal and useful: the program may not write it, but hardware may change it, as with a read-only status register. In an embedded driver one writes `volatile const uint32_t *status = (volatile const uint32_t *)0x40021000;`.

TRAP

> *Claim:* `volatile` makes a shared flag safe between two threads. *Why it is wrong:* `volatile` prevents the compiler from caching a value but provides neither atomicity nor memory ordering between processors, so a read may see a stale value or a torn one on a multi-core machine. *Correct statement:* use `_Atomic` (C11) or the primitives of Pillar 7 for threads; reserve `volatile` for hardware registers and signal handlers (and there prefer `volatile sig_atomic_t`).

`restrict` and `inline`. `restrict` is a promise by the programmer that, during the pointer's lifetime, no other pointer reaches the same object; `memcpy` is declared with `restrict` parameters, which is exactly why overlapping use is undefined. `inline` suggests substituting the function body at the call site; the compiler decides. Together with `static` it is the type-safe alternative to a function-like macro (2.11 (p. 104)). The two tables that summarise qualifiers and storage classes are in the Revision Spread (p. 143).

> CHECK YOURSELF
>
> 1. Two files each contain `static int n;`. Does the linker report a clash? [3]
> 2. Why can a `volatile` read not be removed by the optimiser even when its result is unused? [4]

2.7 Structs, unions, padding and alignment MUST · ■■□

A `struct` groups members laid out in declaration order. The compiler may not reorder them, but it may insert unused bytes called *padding*, because processors read an `n`-byte value most efficiently, or only correctly, at an address divisible by `n`. This divisibility rule is the member's *alignment requirement*. Three rules give the layout of any structure.

> BIG IDEA
>
> Each member sits at the next offset divisible by its own alignment, and the structure's size is rounded up to a multiple of its largest member's alignment so that arrays of the structure stay aligned.

On LP64 the alignment of each basic type equals its size: `char` 1, `short` 2, `int` 4, `long` and pointers 8, `double` 8. Figure 2.4 applies the rules to one structure and to its reordering.

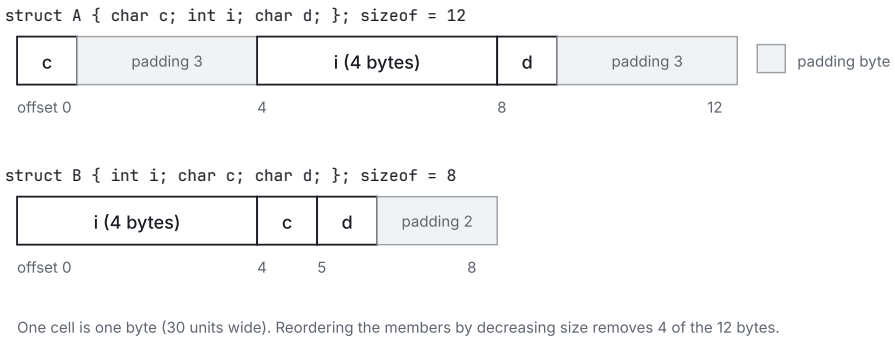


Figure 2.4 Padding in `struct A` and its reordering. Shaded cells are padding and hold unspecified values.

In `struct A` the `int` cannot start at offset 1, so three padding bytes follow `c`. The last `char` ends at offset 9, but the size must be a multiple of 4, so three more bytes follow it. Placing the members in decreasing size order, as in `struct B`, needs only two. The same method answers

the reported Qualcomm questions: for `struct { char a; int b; short c; }` the offsets are 0, 4 and 8 and the size is 12; reordered as `int, short, char` it is 8. For `struct { short s; long l; char c; }` the offsets on LP64 are 0, 8 and 16 and the size is 24; on a 32-bit x86 system `long` is 4 bytes with alignment 4, so the offsets are 0, 4 and 8 and the size is 12.

A `union` stores all its members at offset 0, so its size is the largest member rounded up to the strictest alignment. For `union { int i; char c[5]; }` the size is 8, not 5. Only the member last written is meaningful in the portable sense; reading another member reinterprets the bytes (*type punning*), which C99 permits through unions but C++ does not. The usual use is the *tagged union*, a structure with a tag and a union of alternatives, which stores one of several message formats in the space of the largest.

```
struct msg { enum { MSG_HELLO, MSG_DATA } type;
             union { struct { uint32_t id; }            hello;
                     struct { uint16_t len; uint8_t b[64]; } data; } u; };
```

Padding can be removed with `__attribute__((packed))` or `#pragma pack(1)`, which gives `struct A` the size 6. The price is misaligned member access: x86 handles it at some cost, but strict-alignment processors may fault or take a trap, and taking a pointer to a packed member is a warning for this reason. Packing is therefore acceptable for describing a layout that a protocol dictates, provided members are read with `memcpy` or byte shifts. Two further hazards are worth knowing. Comparing structures with `memcmp` is wrong, because the padding bytes are unspecified. Sending a structure straight over a socket is wrong, because padding, member sizes and byte order all differ between machines (2.9 (p. 101)); fields must be serialised one by one.

The standard macro `offsetof(type, member)` from `<stddef.h>` gives a member's byte offset. Its inverse, `container_of`, recovers the enclosing structure from a pointer to a member, and is the basis of the intrusive lists of the Linux kernel.

```
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))
```

WORKED EXAMPLE

Size of a structure without `sizeof` (a reported Qualcomm question).

1. Declare one object `struct A a;`.
2. Take the address one element past it, `&a + 1`; pointer arithmetic scales by the whole structure.
3. Cast both addresses to `char *` and subtract, so that the unit becomes bytes.

Result: `(char *)(&a + 1) - (char *)&a` **evaluates to 12 for** `struct A`**.**

INTERVIEW LENS

QUALCOMM · CISCO · BROADCOM

Structure versus union, padding with a worked size, and reordering to minimise size are the most frequently reported C questions. A strong answer states the three rules, draws the byte map as in Figure 2.4, and mentions packing and its cost. The follow-up is nearly always "can you send this over the network?".

CHECK YOURSELF

1. What is `sizeof(struct { char a; short b; char c; })` on LP64? [5]
2. What is the minimum size after reordering? [6]

2.8 Bit-fields and bit manipulation MUST · ■■□

Embedded and protocol code reads and writes individual bits constantly. The four basic operations on an unsigned integer `x` use a mask with a single 1 at bit `n`: set with `x |= 1u << n`, clear with `x &= ~(1u << n)`, toggle with `x ^= 1u << n`, and test with `(x >> n) & 1u`. The `u` suffix matters, because `1 << 31` shifts into the sign bit of an `int`, which is undefined (2.12 (p. 105)). As macros, with every argument parenthesised (2.11 (p. 104)):

```
#define SET_BIT(x, n)    ((x) |=  (1u << (n)))
#define CLEAR_BIT(x, n)  ((x) &= ~(1u << (n)))
#define TOGGLE_BIT(x, n) ((x) ^=  (1u << (n)))
#define TEST_BIT(x, n)   (((x) >> (n)) & 1u)
```

A multi-bit *field* is read by shifting and masking, and written by clearing the field before inserting the value. This read-modify-write helper is the classic register-access question.

```
static inline uint32_t get_field(uint32_t reg, unsigned shift, unsigned width) {
    return (reg >> shift) & ((1u << width) - 1u);
}
static inline uint32_t set_field(uint32_t reg, unsigned shift, unsigned width,
uint32_t val) {
    uint32_t mask = ((1u << width) - 1u) << shift;     /* width < 32 assumed */
    return (reg & ~mask) | ((val << shift) & mask);
}
```

Two idioms recur. Kernighan's population count loops `while (x) { x &= x - 1; n++; }`, because `x & (x - 1)` clears the lowest set bit, so the loop runs once per set bit. A power-of-two test is `x != 0 && (x & (x - 1)) == 0`, since a power of two has a single set bit. Further bit-level exercises appear in Pillar 1 (p. 7).

Bit-fields let the compiler do the shifting: `struct { unsigned ihl:4, version:4; }`. They are convenient, yet the standard leaves the order of fields within a byte, the byte order of larger units and the behaviour of fields straddling units to the implementation. The experiment below shows the same value assigned to the same two fields in two declaration orders.

```
struct bf1 { unsigned int version:4, ihl:4; };   /* version declared first */
struct bf2 { unsigned int ihl:4, version:4; };   /* ihl declared first     */
/* with version = 4, ihl = 5, first byte on x86-64 gcc:  bf1 -> 0x54,  bf2 ->
0x45 */
```

On this platform gcc places the first-declared field in the low-order bits, so only `bf2` produces the wire byte `0x45` of an IPv4 header, and the correct declaration order is reversed on a big-endian target. Hence, bit-fields are a poor choice for wire formats and are best kept for private data and hardware registers of a known compiler; portable code extracts fields with shifts and masks, as in 2.14 (p. 108).

2.9 Endianness and byte order MUST · ■■□

A 32-bit integer occupies four bytes, and *endianness* is the order in which they are stored. A *big-endian* machine stores the most significant byte at the lowest address; a *little-endian* machine

stores the least significant byte first. x86 and most ARM systems run little-endian, whereas the Internet protocols are defined in big-endian order, termed *network byte order*.

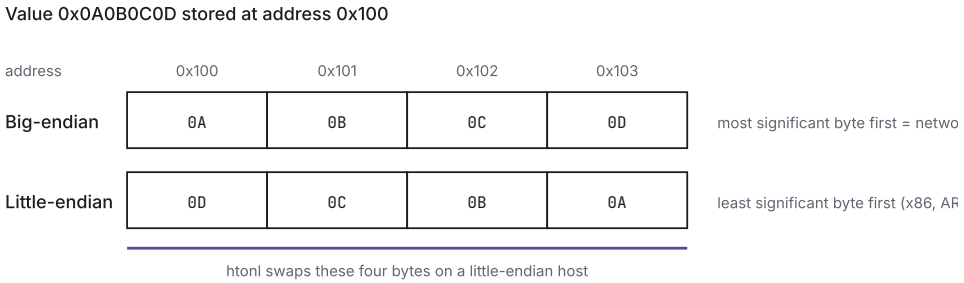


Figure 2.5 The same 32-bit value in memory on a big-endian and a little-endian machine.

Byte order concerns bytes, not bits: the bits inside a byte are not reordered by any API. The headers `<arpa/inet.h>` provide four conversions: `htons` and `htonl` (host to network, 16- and 32-bit) and `ntohs` and `ntohl` (network to host). On a big-endian host they do nothing, so code that calls them is correct on both kinds of machine. If the call is forgotten, the program works against itself and fails against others: the port 80 is `0x0050`, which a little-endian host stores as the bytes `50 00`, and a peer reading those bytes in network order sees `0x5000`, which is 20480.

```
uint16_t port = 80;
printf("%04x %04x\n", port, htons(port));              /* 0050 5000 */
uint8_t w[2] = {0x00, 0x50};  uint16_t raw;  memcpy(&raw, w, 2);
printf("%u %u\n", raw, ntohs(raw));                    /* 20480 80  */
```

To detect the host order, store 1 in an integer and inspect the byte at the lowest address. The union form is defined behaviour; reading through an `unsigned char *` also works, because character types may alias any object.

```
int is_little_endian(void) {
    union { unsigned int i; unsigned char c[4]; } u = { .i = 1 };
    return u.c[0] == 1;                /* least significant byte stored first
*/
}
uint32_t swap32(uint32_t x) {
    return (x >> 24) | ((x >> 8) & 0x0000FF00u) | ((x << 8) & 0x00FF0000u) | (x
<< 24);
}
```

For `x = 0x0A0B0C0D`, `swap32(x)` returns `0x0D0C0B0A`, which is what `htonl` produces on this host. A final rule governs reading wire data: do not cast a byte pointer to `uint32_t *` and dereference, because the address may be misaligned and the cast violates aliasing rules. Assemble the value from bytes, `(p[0] << 24) | (p[1] << 16) | (p[2] << 8) | p[3]`, which is correct on every host without any conversion call. The socket calls that use `htons` for ports are treated in Pillar 7 (p. 285).

> **INTERVIEW LENS**
>
> CISCO · JUNIPER · ARISTA · BROADCOM
>
> “Write a program to detect endianness” and “why do we need `htons`?” are reported at Cisco, and byte-order questions are common at every systems employer. A strong answer draws Figure 2.5, writes the union test, implements `swap32` with shifts, and explains that the conv-

ersion functions are portable no-ops on big-endian hosts. Follow-up: are the bits inside a byte also swapped (no, bits are not addressable, so only bytes move).

CHECK YOURSELF

1. What are the four bytes of 0x12345678 in memory on x86? [7]
2. Why is `*(uint32_t *)buf` a poor way to read a header field? [8]

2.10 Dynamic memory and leaks MUST · ■■□

Automatic variables vanish when their block ends, so data that must outlive a call, or whose size is known only at run time, comes from the heap. `malloc(n)` returns a block of `n` uninitialised bytes or `NULL`; `calloc(k, size)` returns `k * size` zeroed bytes and fails cleanly if the product overflows; `realloc(p, n)` resizes a block, possibly moving it; `free(p)` returns it. All blocks come from the allocator, which keeps a few header bytes before each block, and `free(NULL)` is defined to do nothing.

```
int *a = malloc(100 * sizeof *a);      /* sizeof *a stays correct if the type
changes */
if (!a) return -1;                     /* always test */
int *tmp = realloc(a, 200 * sizeof *a);
if (!tmp) { free(a); return -1; }      /* on failure the old block is still
valid */
a = tmp;
free(a); a = NULL;                     /* NULL makes a second free harmless */
```

The idiom with `tmp` is mandatory. In `p = realloc(p, n)` a failure overwrites `p` with `NULL` and the only reference to the old block is lost. When `realloc` cannot grow a block in place it allocates a new one, copies the contents and frees the old block, so every pointer into the old block becomes invalid.

Five defects account for nearly all heap bugs, and each has a tool that finds it.

Defect	What happens	Tool that reports it
Memory leak	the last pointer to a block is lost or `free` is never called	valgrind, LeakSanitizer
Use after free	the block is read or written after `free`	AddressSanitizer, valgrind
Double free	`free` on the same pointer twice; the allocator's lists are corrupted	AddressSanitizer, glibc abort message
Buffer overflow	a write past the end of the block	AddressSanitizer, valgrind
Dangling pointer	a pointer outlives its object, whether freed heap or an expired local	AddressSanitizer (heap and stack)

A *leak* costs nothing in a short program and exhausts memory in a router process that runs for months; it is therefore treated as a defect. The program below loses its only pointer.

```
char *a = malloc(16);
strcpy(a, "leak");
a = NULL;                  /* last reference lost */
```

LAB

1. Both modifying the characters through `s` and reassigning `s` itself.↩
2. Without parentheses around `*fp` the declarator binds as a function returning `int *`; a pointer needs `int (*fp)(int, int)`.↩
3. No; `static` gives internal linkage, so each file has its own `n`.↩
4. The access may itself have an effect, such as clearing a status bit in a device register, so the standard treats it as observable behaviour.↩
5. Offsets 0, 2, 4, so the members end at 5; rounding to a multiple of 2 gives 6.↩
6. Order `short, char, char` gives offsets 0, 2, 3, ending at 4, and the size is 4.↩
7. `78 56 34 12`, in increasing address order, since x86 is little-endian.↩
8. The address may be misaligned, the cast breaks the aliasing rules, and the value is in network order and still needs `ntohl`.↩

LAB 2.3 · 10 MIN

```
gcc -g -fsanitize=address -o leak leak.c && ./leak
```

```
ERROR: LeakSanitizer: detected memory leaks
Direct leak of 16 byte(s) in 1 object(s) allocated from:
    #1 ... in main leak.c:4
SUMMARY: AddressSanitizer: 16 byte(s) leaked in 1 allocation(s).
```

What you should notice: the report names the size and the line of the `malloc`. With valgrind, `valgrind --leak-check=full ./leak` prints the same fact as “definitely lost: 16 bytes in 1 blocks”.

Returning a pointer to a local is the stack counterpart. `char *f(void) { char buf[16] = "x"; return buf; }` returns the address of storage that is dead the moment `f` returns; the compiler warns, and the behaviour is undefined. The remedies are to let the caller pass the buffer, to allocate on the heap, or to return a `static` buffer (not re-entrant).

TRAP

Claim: after `free(p)` the pointer is `NULL`, or at least safe to test. *Why it is wrong:* `free` receives a copy of `p` and cannot change the caller’s variable, so `p` still holds the old address. *Correct statement:* assign `p = NULL` yourself after `free`; then a second `free(p)` is harmless and a dereference faults immediately instead of corrupting memory silently.

CHECK YOURSELF

1. Why does `calloc(n, 4)` differ from `malloc(n * 4)` apart from zeroing? [1]
2. What happens to other pointers into a block after `realloc` moves it? [2]

2.11 The preprocessor and macros MUST · ■■□

Before compilation the *preprocessor* performs textual substitution; it knows nothing of types or scopes. A macro `#define NAME text` replaces each occurrence of `NAME` by `text`, and a function-like macro `#define F(x) text` does so with its arguments pasted in unevaluated. Everything that goes wrong follows from this single fact.

```
#define SQ(x)  x*x
#define SQ2(x) ((x)*(x))
SQ(2+3)    /* 2+3*2+3  = 11 */      SQ2(2+3)  /* ((2+3)*(2+3)) = 25 */
10/SQ(2)   /* 10/2*2   = 10 */      #define MAX(a,b) ((a)>(b)?(a):(b))
MAX(i++, j)/* i++ is evaluated twice: with i = 1, j = 0 the result is 2 and i
becomes 3 */
```

Hence three rules: parenthesise every parameter and the whole body; never pass an argument with side effects; and prefer a `static inline` function when types are known, because it evaluates its arguments once and is type-checked. A macro remains the right tool for code that must stay generic, for `__FILE__` and `__LINE__` capture, and for conditional compilation.

A multi-statement macro must be wrapped as `do { ... } while (0)`. Without it, `#define SWAP(a,b) t=a; a=b; b=t;` used as `if (c) SWAP(x,y); else ...` breaks the `if` and fails to compile, because only the first statement belongs to the `if`. The wrapper turns the body into one statement that takes the trailing semicolon.

Two operators help. The `#` operator turns an argument into a string and `##` pastes two tokens. To stringify the *value* of a macro one needs two levels, because `#` stops expansion.

```
#define N 4
#define STR(x)  #x
#define XSTR(x) STR(x)
#define CAT(a,b) a##b
STR(N)  /* "N" */    XSTR(N)  /* "4" */    int CAT(x,1) = 5;   /* declares x1 */
```

Include guards prevent a header being processed twice: wrap it in `#ifndef NET_H`, `#define NET_H`, …, `#endif`, or use `#pragma once`. Conditional compilation (`#ifdef DEBUG`) selects code at build time.

The *X-macro* keeps a list in one place and generates several tables from it, so that an enumeration and its printable names can never fall out of step. It is a favourite way to write protocol state tables.

```
#define STATES(X) X(CLOSED) X(LISTEN) X(ESTABLISHED)
#define AS_ENUM(n) S_##n,
#define AS_STR(n)  #n,
enum state { STATES(AS_ENUM) S_COUNT };              /* S_CLOSED, S_LISTEN,
..., 3 */
static const char *names[] = { STATES(AS_STR) };     /* "CLOSED", "LISTEN", ...
*/
/* names[S_LISTEN] is "LISTEN" */
```

> INTERVIEW LENS
>
> CISCO · QUALCOMM
>
> “What is wrong with `#define SQUARE(x) x*x`? What happens with `SQUARE(i++)`?” and “macro versus inline function” are reported Cisco questions. A strong answer shows `SQ(2+3)` giving 11, the parenthesised fix, and that `SQ2(i++)` multiplies `i++` by `i++`, an unsequenced double modification and therefore undefined. Follow-up: when is a macro better than an inline function (generic code, stringification, `__LINE__`).

> CHECK YOURSELF
>
> 1. What does `XSTR(N)` give when `N` is defined as `4`, and why does `STR(N)` differ? [3]
> 2. Why wrap a multi-statement macro in `do { } while (0)`? [4]

2.12 Undefined behaviour MUST · ■■□

The standard distinguishes three kinds of unspecified outcome. *Implementation-defined* behaviour varies but must be documented, such as the signedness of `char` or the result of right-shifting a negative number. *Unspecified* behaviour is a choice among allowed outcomes that need not be documented, such as the order in which function arguments are evaluated. *Undefined behaviour* (UB) places no requirement at all on the program: it may crash, print the expected result, or be compiled to something else entirely.

> BIG IDEA
>
> The compiler assumes that undefined behaviour never occurs and optimises on that assumption, so a program with UB can fail in ways unrelated to the faulty line.

The common sources are the following. (i) Signed integer overflow. (ii) Dereferencing `NULL`, a dangling or an out-of-bounds pointer, including one past the end. (iii) Reading an uninitialised automatic variable. (iv) Modifying an object twice between sequence points, as in `i = i++ + ++i`. (v) Shifting by a negative count or by the type's width or more, and left-shifting into the sign bit. (vi) Modifying a string literal or a `const` object. (vii) Accessing an object through a pointer of an incompatible type (the *strict aliasing* rule). (viii) Using memory after `free`, or freeing twice. (ix) Falling off the end of a non-`void` function and using the result. (x) A data race between threads.

The consequences are visible in generated code. The function `int f(int x) { return x + 1 > x; }` is compiled by gcc to `movl $1, %eax; ret`: signed overflow cannot happen, so the comparison is always true, although for `x = INT_MAX` the machine addition would wrap. A check written as `if (x + 1 < x) overflow();` can be removed in the same way, so the correct test is `if (x == INT_MAX)`, or arithmetic done in unsigned types.

> LAB
>
> `LAB 2.4 · 10 MIN`
>
> ```
> gcc -g -fsanitize=undefined -o ub ub.c && ./ub     # prints x + 1 with x = INT_MAX
> ```
>
> ```
> ub.c:3:75: runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
> -2147483648
> ```
>
> **What you should notice:** the plain build prints only the wrapped value. The sanitiser reports the file, line and operands, so that a silent wrap becomes a located error. Run test suites with `-fsanitize=address,undefined` before concluding a program is correct.

An interview answer about UB has three parts: name the category, say why the standard leaves it open (portability to processors that trap, and permission to optimise), and give the tool that detects it. Unsigned overflow, by contrast, is defined and wraps, which makes it the right type for counters, hashes and sequence numbers.

> CHECK YOURSELF
>
> 1. Is `a[i] = i++;` defined? [5]
> 2. Is `-8 >> 1` defined? [6]

2.13 Writing the library: memcpy to itoa MUST · ■■□

Interviewers ask for the standard functions because each hides one subtle decision. The versions below are written for clarity; production libraries copy a word at a time but must give the same results.

```c
size_t my_strlen(const char *s) { const char *p = s; while (*p) p++; return
(size_t)(p - s); }

char *my_strcpy(char *d, const char *s) {
    char *r = d;
    while ((*d++ = *s++)) ;               /* copies the terminator, then the
test is 0 */
    return r;
}

int my_strcmp(const char *a, const char *b) {
    while (*a && *a == *b) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;   /* compare as unsigned bytes
*/
}
```

`strcmp` compares as `unsigned char`, which is why the cast is present: with a plain signed `char`, a byte above 127 would compare below an ASCII letter. The pitfalls of `my_strcpy` are the missing size check and overlapping arguments.

`memcpy` copies `n` bytes and requires that the regions do not overlap. `memmove` allows overlap, so it must choose the copying direction: if the destination lies *before* the source, copy forwards; if it lies inside the source region *after* its start, copy backwards, otherwise a forward copy would overwrite source bytes that have not yet been read.

```c
void *my_memcpy(void *dst, const void *src, size_t n) {
    unsigned char *d = dst; const unsigned char *s = src;
    while (n--) *d++ = *s++;
    return dst;
}
void *my_memmove(void *dst, const void *src, size_t n) {
    unsigned char *d = dst; const unsigned char *s = src;
    if (d == s || n == 0) return dst;
    if ((uintptr_t)d < (uintptr_t)s || (uintptr_t)d >= (uintptr_t)s + n) {
        while (n--) *d++ = *s++;           /* forwards: safe when no harmful
overlap */
    } else {
        d += n; s += n;
        while (n--) *--d = *--s;           /* backwards: destination overlaps
the tail */
    }
    return dst;
}
```

With `char s[16] = "abcdef"`, calling `my_memmove(s+2, s, 4)` leaves `"ababcd"`, and `my_memmove(s, s+2, 4)` on a fresh copy leaves `"cdefef"`. Applying `my_memcpy` (or the real `memcpy`) to the first case is undefined, and a forward byte loop would give `"abababa"` because the first copied bytes are overwritten before they are read. Casting through `uintptr_t` before comparing pointers avoids comparing unrelated objects, which the standard does not define.

`atoi` hides four decisions: skip leading whitespace, accept one sign, stop at the first non-digit, and refuse to overflow. The version below clamps, as the common LeetCode variant requires; the overflow test is written so that it never overflows itself.

```c
int my_atoi(const char *s) {
    while (*s == ' ' || (*s >= '\t' && *s <= '\r')) s++;
    int neg = 0;
    if (*s == '-' || *s == '+') neg = (*s++ == '-');
    int v = 0;
    while (*s >= '0' && *s <= '9') {
        int d = *s++ - '0';
        if (v > (INT_MAX - d) / 10) return neg ? INT_MIN : INT_MAX;   /* v*10+d
would overflow */
        v = v * 10 + d;
    }
    return neg ? -v : v;
}
```

`my_atoi("  -42xyz")` returns -42, `"+7"` returns 7, `"2147483648"` returns 2147483647, `"-2147483649"` returns -2147483648 and `"abc"` returns 0. The test `v > (INT_MAX - d) / 10` is equivalent to `v * 10 + d > INT_MAX` but is computed without overflow.

`itoa` is not standard, and the task is to build it. Digits come out least-significant first, so they are collected and reversed. The sign needs care: `-n` overflows for `INT_MIN`, so the magnitude is computed in unsigned arithmetic, where negation is defined.

```c
char *my_itoa(int n, char *buf) {            /* buf must hold at least 12 bytes
*/
    char tmp[12]; int i = 0;
    unsigned u = n < 0 ? 0u - (unsigned)n : (unsigned)n;
    do { tmp[i++] = (char)('0' + u % 10); u /= 10; } while (u);   /* do-while: 0
gives "0" */
    char *o = buf;
    if (n < 0) *o++ = '-';
    while (i) *o++ = tmp[--i];
    *o = '\0';
    return buf;
}
```

For 0, -1234 and `INT_MIN` the results are `0`, `-1234` and `-2147483648`. The buffer size is 12 because `INT_MIN` has ten digits, a sign and the terminator. The conversion of an address to dotted-quad text and back is the same exercise with four numbers and is solved in the Problem Set (p. 128).

> CHECK YOURSELF
>
> 1. Why does `my_memmove(s+2, s, 4)` copy backwards? [7]
> 2. Why compute `0u - (unsigned)n` rather than `-n`? [8]

2.14 Parsing an IPv4 header from a byte buffer MUST · ■■□

A packet arrives as an array of bytes. The task is to extract the fields safely: check the length before reading, read multi-byte fields in network order, and verify the checksum. The meaning of each field belongs to Pillar 3 (p. 146); here we consider only offsets and C technique.

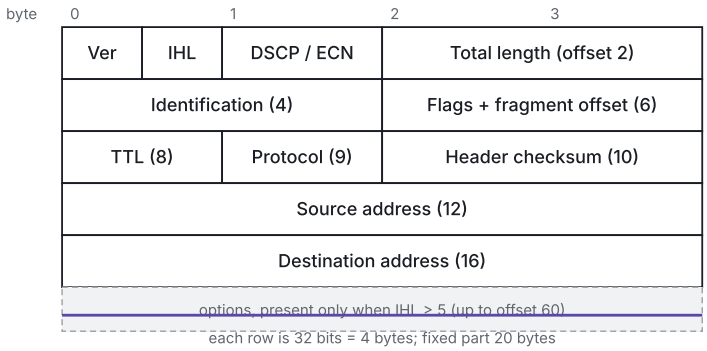


Figure 2.6 The IPv4 header as a C programmer reads it. Offsets in parentheses are in bytes; every multi-byte field is big-endian.

Two designs exist. One may declare a packed structure and cast the buffer to it, which is short but wrong for the reasons of 2.7 (p. 99): the buffer may be misaligned, bit-fields vary, and every field still needs `ntohs`. The robust design reads each field from its offset with small helpers that assemble bytes, and fills an ordinary structure. Version and header length share byte 0 and are split with a shift and a mask.

```c
struct ipv4 { uint8_t version, ihl; uint16_t total_len, id, frag;
              uint8_t ttl, proto; uint16_t csum; uint32_t src, dst; };

static inline uint16_t rd16(const uint8_t *p) { return (uint16_t)((p[0] << 8) |
p[1]); }
static inline uint32_t rd32(const uint8_t *p) {
    return ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) | ((uint32_t)p[2] <<
8) | p[3];
}

uint16_t inet_csum(const uint8_t *p, size_t n) {     /* ones-complement sum, RFC
1071 */
    uint32_t sum = 0;
    for (; n > 1; p += 2, n -= 2) sum += rd16(p);
    if (n) sum += (uint32_t)p[0] << 8;               /* odd trailing byte, pad-
ded with 0 */
    while (sum >> 16) sum = (sum & 0xFFFF) + (sum >> 16);   /* fold carries back
in */
    return (uint16_t)~sum;
}

int ipv4_parse(const uint8_t *buf, size_t len, struct ipv4 *h) {
    if (len < 20) return -1;                         /* too short to read the
fixed part */
    h->version = buf[0] >> 4;  h->ihl = buf[0] & 0x0F;
    if (h->version != 4 || h->ihl < 5) return -2;    /* wrong version or im-
possible IHL */
    size_t hlen = (size_t)h->ihl * 4;                /* IHL counts 32-bit words
*/
    if (len < hlen) return -1;
    h->total_len = rd16(buf + 2);
    if (h->total_len < hlen || h->total_len > len) return -3;
    h->id = rd16(buf + 4);  h->frag = rd16(buf + 6);
    h->ttl = buf[8];  h->proto = buf[9];  h->csum = rd16(buf + 10);
    h->src = rd32(buf + 12);  h->dst = rd32(buf + 16);
    return inet_csum(buf, hlen) == 0 ? 0 : -4;       /* a valid header sums to
zero */
}
```

The helper `u32_to_dotted` prints an address held in host order: `snprintf(out, 16, "%u.%u.%u.%u", ip >> 24, (ip >> 16) & 255, (ip >> 8) & 255, ip & 255)`. For the header bytes `45 00 00 73 00 00 40 00 40 11 b8 61 c0 a8 00 01 c0 a8 00 c7`, supplied in a buffer of at least 115 bytes, `ipv4_parse` returns 0 and the structure holds version 4, IHL 5 (20 bytes), total length 115, TTL 64, protocol 17 (UDP), source 192.168.0.1 and destination 192.168.0.199. With the checksum field zeroed, `inet_csum` returns `0xb861`, the stored value. Passing a length of 19 returns -1, and changing the TTL byte to `0x3f` without updating the checksum returns -4.

The order of the checks is the point of the exercise. Every read is guarded by a prior length test, and `total_len` is trusted only after it has been compared with the number of bytes actually received. A parser that reads `ihl` and then indexes options without checking is the template of a remote vulnerability. The more general loop for variable-length options, a type-length-value parser, appears in the Problem Set (p. 128).

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA · BROADCOM

> “Parse this header from a hex dump” is reported as a networking-software exercise, and “Can you send a C struct directly over a socket?” as a conceptual follow-up. A strong answer reads fields by offset, validates lengths first, converts byte order, and explains why the packed-structure cast is fragile. Follow-ups: how to extend the parser to TCP or UDP (the next header starts at offset `ihl * 4`), and how to verify the checksum incrementally.

CHECK YOURSELF

1. A packet has `buf[0] = 0x47`. What are the version and the header length in bytes? [9]
2. Why must `len` be tested before `total_len` is trusted? [10]

1. `calloc` detects overflow of the multiplication and returns `NULL`, whereas `n * 4` may wrap and yield a block that is too small.↩
2. They dangle, because the old block has been freed.↩
3. `"4"`; the argument of `STR` is not macro-expanded before `#` is applied, whereas the extra level in `XSTR` expands it first.↩
4. It makes the body a single statement, so the macro behaves correctly after `if` without braces and with a trailing semicolon.↩
5. No; `i` is read and modified without sequencing, so the behaviour is undefined.↩
6. It is implementation-defined, not undefined; gcc performs an arithmetic shift and yields -4.↩
7. The destination begins inside the source region, so a forward copy would overwrite source bytes before they are read.↩
8. `-INT_MIN` overflows a signed `int`, which is undefined; unsigned negation is defined and gives 2147483648.↩
9. Version 4 (high nibble) and IHL 7, so the header is 28 bytes.↩
10. The field comes from the sender and may claim more bytes than were received; using it unc-

2.15 The gotcha gallery: 36 output traps MUST · ■■□

hecked allows reads beyond the buffer.↩

Output-prediction questions are a fixed ritual in online assessments at Cisco and Juniper and in Qualcomm and Arista interviews: a snippet of three to eight lines, and the question “what does this print?”. The skill is to classify the snippet first as *defined*, *implementation-defined* or *undefined*, and to compute an output only in the first two cases. This section is the single home of the book’s C traps. Each entry gives the code, the output on 64-bit Linux with gcc (so `int` is 4 bytes and `char` is signed), and the reason. Entries marked UB print nothing definite; any output seen is an accident.

> TRAP
> *Claim:* when a snippet compiles and runs, whatever it prints is “the answer”. *Why it is wrong:* a program with undefined behaviour has no correct output, and the same source may print different results at `-O0` and `-O2`, or on the next compiler release. *Correct statement:* in an interview, say “undefined behaviour” and name the rule; computing a number from an unsequenced expression is the failure the question is designed to find.

Expressions and sequencing

T1. Two writes to one variable.

```
int i = 1;
i = i++ + ++i;
```

Answer: undefined. `i` is modified twice with no sequence point between the modifications.

T2. `sizeof` does not evaluate.

```
int i = 5;
printf("%zu %d\n", sizeof(i++), i);
```

Answer: `4 5`. The operand of `sizeof` is not evaluated (except for variable-length arrays), so `i` is unchanged.

T3. `printf` returns a count.

```
printf("%d\n", printf("hello"));
```

Answer: `hello5` followed by a newline. The inner call prints five characters and returns 5, which the outer call then prints.

T4. Signed meets unsigned.

```
puts(-1 < 1u ? "true" : "false");
```

Answer: `false`. The usual arithmetic conversions turn `-1` into `UINT_MAX`, which is larger than 1 (2.2 (p. 93)).

T5. Plain `char` above 127.

```
char c = 200;
printf("%d\n", c);
```

Answer: `-56` on x86-64, where `char` is signed (implementation-defined). On ARM Linux, where `char` is unsigned, it prints 200.

T6. Array parameter.

```
void f(int a[10]) { printf("%zu\n", sizeof(a)); }
int main(void) { int a[10]; f(a); printf("%zu\n", sizeof a); }
```

Answer: `8` then `40`. Inside `f` the parameter is a pointer (8 bytes on LP64); in `main` the array has 10 elements of 4 bytes.

T7. Macro without parentheses.

```
#define SQ(x) x*x
printf("%d %d\n", SQ(2+3), 10/SQ(2));
```

Answer: `11 10`. The first expands to `2+3*2+3`; the second to `10/2*2`, which evaluates left to right as `(10/2)*2`.

T8. Fall-through.

```
int x = 2;
switch (x) { case 1: putchar('a');
             case 2: putchar('b');
             case 3: putchar('c'); break;
             default: putchar('d'); }
```

Answer: `bc`. Execution enters at `case 2` and continues into `case 3` until `break`.

T9. Comma operator.

```
int x = (1, 2, 3);
printf("%d\n", x);
```

Answer: `3`. The comma operator evaluates its operands left to right and yields the last. Without the parentheses, `int x = 1, 2;` is a compile error, since `2` would be read as a second declarator.

T10. Short-circuit evaluation.

```
int a = 0, b = 5;
if (a && b++) { }
printf("%d %d\n", a, b);
```

Answer: `0 5`. Because `a` is false, the right operand of `&&` is never evaluated. The same applies to `||` when the left side is true.

T11. A static local keeps its value.

```
int cnt(void) { static int c; return ++c; }
for (int i = 0; i < 3; i++) printf("%d ", cnt());
```

Answer: `1 2 3`. The static `c` is zero-initialised once, before `main`, and survives between calls. Printing three calls in a single `printf` would be unspecified, because the order of evaluating arguments is unspecified.

Pointers and declarations

T12. Increment and dereference.

```
int a[] = {10, 20, 30}, *p = a;
int x = *p++;        /* post-increment the pointer, read the old target */
int y = (*p)++;      /* increment the value that p points to           */
int z = ++*p;        /* pre-increment the value                        */
printf("%d %d %d %d %ld\n", x, y, z, a[1], (long)(p - a));
```

Answer: `10 20 22 22 1`. Postfix `++` binds tighter than `*`, so `*p++` is `*(p++)`: the pointer moves, the old element is read. In `(*p)++` and `++*p` the value changes and the pointer does not.

T13. Array address versus element address.

```
int a[5];
printf("%ld %ld\n", (long)((char *)(a + 1) - (char *)a),
                    (long)((char *)(&a + 1) - (char *)a));
```

Answer: `4 20`. `a + 1` steps over one `int`; `&a` has type `int (*)[5]`, so `&a + 1` steps over the whole array.

T14. Where `const` binds.

```
int x = 1, y = 2;
const int *p = &x;   *p = 3;     /* line A */
p = &y;                          /* line B */
int *const q = &x;   q = &y;     /* line C */
```

Answer: line A is a compile error (`assignment of read-only location '*p'`), line B is accepted, and the assignment in line C is a compile error (`assignment of read-only variable 'q'`). `const int *` protects the pointee; `int *const` protects the pointer (2.5 (p. 96)).

T15. Writing to a literal.

```
char *p = "arista";  p[0] = 'A';
char  q[] = "arista"; q[0] = 'A';
```

Answer: the first write is undefined and normally ends in a segmentation fault, because the literal lives in .rodata. The second is defined: `q` is a writable copy.

T16. Pointer to a dead local.

```
char *name(void) { char buf[8] = "eth0"; return buf; }
printf("%s\n", name());
```

Answer: undefined. `buf` ceases to exist when `name` returns. The compiler warns (`returns address of local variable`), and the output may look right, garbage, or crash.

T17. Postfix decrement in recursion.

```
int sum(int n) { return n == 0 ? 0 : n + sum(n--); }
printf("%d\n", sum(3));
```

Answer: the program overflows the stack and ends with a segmentation fault. `n--` passes the *old* value, so the argument never decreases and the recursion never reaches 0. The fix is `sum(n - 1)`.

T18. Padding.

```
struct A { char c; int i; char d; };
struct B { int i; char c; char d; };
printf("%zu %zu\n", sizeof(struct A), sizeof(struct B));
```

Answer: `12 8`. See Figure 2.4 on padding (p. 99): three padding bytes after `c` and three after `d` in `A`, two at the end of `B`.

T19. Union overlay.

```
union { int i; char c[4]; } u = { 1 };
printf("%zu %d\n", sizeof u, u.c[0]);
```

Answer: `4 1` on a little-endian machine, since the least significant byte of 1 is stored first. On a big-endian machine the second number is 0.

T20. A flag polled in a loop. Read the assembly in Lab 2.2 of 2.6 (p. 98): without `volatile`, `while (!flag);` is compiled to read `flag` once. This is a question about the optimiser, not about the printed value.

Numbers

T21. Overflow, signed and unsigned.

```
printf("%u\n", UINT_MAX + 1u);
int m = INT_MAX;  m = m + 1;
```

Answer: the first line prints `0`, because unsigned arithmetic wraps modulo 2^32 (defined). The second is undefined; the UBSan message appears in Lab 2.4 of 2.12 (p. 105).

T22. Floating-point equality.

```
printf("%d\n", 0.1 + 0.2 == 0.3);
float f = 0.1;  printf("%d\n", f == 0.1);
```

Answer: `0` and `0`. The decimal fractions are not exactly representable in binary, and the sum differs from `0.3` in the last bit. In the second case `f` holds the nearest `float`, which differs from the nearest `double` that `0.1` denotes. Compare floating-point values with a tolerance.

T23. Type of a character constant.

```
printf("%d %zu\n", 'A', sizeof('A'));
```

Answer: `65 4`. A character constant has type `int` in C; in C++ it has type `char` and the size is 1.

T24. Shifts.

```
printf("%d\n", -8 >> 1);
int a = 1 << 31;   unsigned b = 1u << 32;
```

Answer: the first prints `-4`: right-shifting a negative value is implementation-defined, and gcc shifts arithmetically. `1 << 31` shifts into the sign bit of an `int` and is undefined; `1u << 32` shifts by the full width and is undefined.

T25. `strlen` versus `sizeof`.

```
char s[] = "abc";
printf("%zu %zu\n", strlen(s), sizeof s);
```

Answer: `3 4`. `strlen` stops before the terminator; the array includes it.

T26. Uninitialised local.

```
int n;
printf("%d\n", n);
```

Answer: undefined for an automatic variable (an indeterminate value). Had `n` been `static` or global, it would be 0. Compilers warn with `-Wall -O2`.

T27. Double free.

```
char *p = malloc(8);  free(p);  free(p);
```

Answer: undefined. With glibc 2.39 the program aborts with `free(): double free detected in tcache 2`. Use after free and a missing `free` are the two related defects (2.10 (p. 103)).

T28. A countdown that never ends.

```
for (unsigned i = 3; i >= 0; i--) printf("%u ", i);
```

Answer: the loop prints `3 2 1 0 4294967295 4294967294 ...` without end, because an unsigned value is always `>= 0` and `0 - 1` wraps. Use `for (unsigned i = n; i-- > 0; )`.

T29. Leading zero.

```
printf("%d %d\n", 010, 010 + 1);
```

Answer: `8 9`. A leading `0` makes an octal constant, and `08` would not compile.

T30. Types in the conditional operator.

```
puts((1 ? -1 : 0u) > 0 ? "pos" : "neg");
```

Answer: `pos`. The two branches are converted to a common type, which is `unsigned int`, so `-1` becomes `UINT_MAX` even though the first branch is chosen.

T31. A mismatched format specifier.

```
printf("%d\n", 3.5);          printf("%s\n", 5);
```

Answer: undefined. `printf` reads its arguments according to the format, not according to what was passed. The compiler warns with `-Wformat`, which is part of `-Wall`.

T32. Division and remainder of negative numbers.

```
printf("%d %d\n", -7 / 2, -7 % 2);
```

Answer: `-3 -1`. Since C99 division truncates towards zero, and the remainder takes the sign of the dividend, so that `(a / b) * b + a % b == a`.

T33. Assignment in a condition.

```
int x = 0;
if (x = 5) puts("yes");
printf("%d\n", x);
```

Answer: `yes` and then `5`. The expression `x = 5` assigns and has the value 5, which is true. The compiler suggests parentheses with `-Wall`.

T34. Precedence of `==` over `&`.

```
int x = 4;
printf("%d\n", x & 1 == 0);
```

Answer: `0`. The comparison binds tighter than `&`, so the expression is `x & (1 == 0)`, which is `4 & 0`. The intended test is `(x & 1) == 0`; the precedence table is in the Revision Spread (p. 143).

T35. Partial initialisation, and `i[a]`.

```
int a[5] = {1};
for (int i = 0; i < 5; i++) printf("%d ", a[i]);
int b[] = {10, 20, 30};   printf("%d\n", 2[b]);
```

Answer: `1 0 0 0 0` followed by `30`. Members without an initialiser are zeroed, and `2[b]` is `*(2 + b)`, the same as `b[2]`.

T36. Where does each variable live? The classification of globals, statics, locals, literals and `malloc` blocks is the table in 2.1 (p. 91), with the effect of `static` on a large local array in the trap that follows it.

> INTERVIEW LENS
>
> CISCO · JUNIPER · QUALCOMM · ARISTA
>
> A set of ten to fifteen of these, usually T1, T2, T7, T10 to T19, T25 and T28, makes up the C section of many online assessments. Qualcomm adds `const` placement, structure against union, padding, `volatile` and `static` (T11, T14 to T20); Arista adds the debugging variants, namely uninitialised variables (T26), returning a local (T16), `n--` in recursion (T17) and writing to a literal (T15). A strong answer names the category first and then the mechanism. If the output is undefined, one gives the rule that is broken and the tool that detects it (UBSan, ASan or a compiler warning).

> LAB
>
> `LAB 2.5 · 20 MIN`
>
> Type the six snippets T2, T4, T12, T13, T18 and T25 into one file, predict each result on paper, then run:
>
> ```
> gcc -Wall -Wextra -O0 -o gal gal.c && ./gal
> ```

Repeat with `-O2`. The defined traps print identical output at both levels; add T1 and T26 and observe the compiler warnings and, possibly, different output.

What you should notice: warnings from `-Wall -Wextra` already flag several entries (T1, T16, T26, T31, T33), and T14 is rejected outright, so reading them is the cheapest debugging step.

CHECK YOURSELF

1. Which entries of this section rely on implementation-defined behaviour rather than undefined behaviour? [1]
2. Why is T11 safe with a loop but not with three calls in one `printf`? [2]

2.16 Python: the data model SHOULD · ■□□

This part treats the Python language only. The libraries for networks and devices are in Pillar 10, sockets in Pillar 7 (p. 285), and shell pipelines in Pillar 8. Examples assume Python 3.10 or later.

In Python every value is an object, and a variable is a *name* bound to an object, not a box that holds a value. Assignment `b = a` therefore binds a second name to the same object. Objects are either *mutable* (list, dict, set, bytearray) or *immutable* (int, float, str, bytes, tuple), and the difference explains most interview traps.

BIG IDEA

Names refer to objects; assignment copies the reference, never the object, so a change made through one name is visible through every name bound to the same mutable object.

```
a = [1, 2]; b = a; c = a[:]            # b is an alias; c is a shallow copy
b.append(3)
print(a, c, a is b, a == c, a is c)    # [1, 2, 3] [1, 2] True False False
grid = [[0] * 2] * 2; grid[0][0] = 9
print(grid)                            # [[9, 0], [9, 0]]: both rows are one
list
t = (1, [2]); t[1].append(3); print(t) # (1, [2, 3]): the tuple holds the same
list
```

The operator `==` compares values and `is` compares identity. Use `is` only for `None` (`if x is None`), because the identity of small integers and strings is an implementation detail. A *shallow copy* (`a[:]`, `list(a)`, `a.copy()`) copies the outer container but shares the elements; `copy.deepcopy` copies recursively.

The two most reported Python traps are both consequences of the model. A default argument is evaluated once, when the function is defined, so a mutable default is shared by all calls. Removing items from a list while iterating over it skips elements, because the iterator tracks a position, not the items.

1. T5 (signedness of `char`) and T24 (right shift of a negative value); the others print a definite value or are undefined. ↩
2. The loop sequences the calls statement by statement, whereas the order of evaluating the arguments of one call is unspecified. ↩

```
def bad(x, acc=[]):                    # one list, created at definition time
    acc.append(x); return acc
print(bad(1), bad(2))                  # [1, 2] [1, 2]  (the same list twice)
def good(x, acc=None):
    if acc is None: acc = []
    acc.append(x); return acc
print(good(1), good(2))                # [1] [2]

nums = [1, 2, 2, 3]
for n in nums:
    if n == 2: nums.remove(n)
print(nums)                            # [1, 2, 3]: the second 2 was skipped
```

The fix for the second case is to build a new list, `nums = [n for n in nums if n != 2]` (2.17 (p. 119)). The first output line shows `[1, 2] [1, 2]` because both calls return the same list object, and `print` shows its final state.

Built-in containers have the complexity one expects from their implementations, which is a frequent question: `x in list` scans linearly and costs O(n), whereas `x in set` and `x in dict` hash the key and cost O(1) on average; `list.append` is amortised O(1), `list.insert(0, x)` is O(n), and `collections.deque` offers O(1) at both ends. Dictionary keys must be *hashable*, which means immutable: a tuple is a valid key, a list is not.

Special methods, written with double underscores, let a class join the language's protocols: `__repr__` for printing, `__eq__` and `__hash__` for equality and hashing, `__lt__` for sorting, `__len__`, `__iter__` and `__contains__` for containers. A class that defines `__eq__` must define `__hash__` consistently, or it becomes unhashable.

```
class Prefix:
    __slots__ = ("addr", "length")
    def __init__(self, addr, length): self.addr, self.length = addr, length
    def __repr__(self): return f"Prefix({self.addr!r}, {self.length})"
    def __eq__(self, o): return (self.addr, self.length) == (o.addr, o.length)
    def __hash__(self): return hash((self.addr, self.length))
    def __lt__(self, o): return (-self.length, self.addr) < (-o.length, o.addr)

ps = [Prefix("10.0.0.0", 8), Prefix("10.1.0.0", 16), Prefix("10.0.0.0", 8)]
print(sorted(ps), len(set(ps)))
# [Prefix('10.1.0.0', 16), Prefix('10.0.0.0', 8), Prefix('10.0.0.0', 8)] 2
```

Python's integers have arbitrary precision, so there is no overflow, and `//` floors (`-7 // 2` is `-4`, `-7 % 2` is `1`), unlike C, where division truncates towards zero (2.15 (p. 112), T32). The conversion `int("ff", 16)` gives 255, `int.from_bytes(b"\x00\x50", "big")` gives 80 and `(80).to_bytes(2, "big")` gives `b'\x00P'`.

> CHECK YOURSELF
>
> 1. What does `bool("0")` return, and why? [1]
> 2. Why is `{[1, 2]: "x"}` an error? [2]

2.17 Python: comprehensions and generators SHOULD · ■■□

A *comprehension* builds a list, set or dictionary from an iterable in one expression: `[x * x for x in range(5) if x % 2 == 0]` is `[0, 4, 16]`, `{x % 3 for x in range(10)}` is `{0, 1, 2}`, and `{w: len(w) for w in words}` maps each word to its length. A comprehension is

shorter than the equivalent loop and avoids the repeated attribute lookup of `append`, but one that needs more than one condition is clearer as a loop.

A *generator* produces its values one at a time instead of storing them. The generator expression `(x * x for x in range(5))` looks like a list comprehension in round brackets, and a function containing `yield` is a generator function: each call to `next` runs the body until the next `yield`, then pauses with its local state intact. A generator is *exhausted* after one pass.

```
sq = (x * x for x in range(5))
print(next(sq), next(sq), sum(sq), sum(sq))   # 0 1 29 0: the second sum finds
nothing left

def fib():
    a, b = 0, 1
    while True:                                # infinite, yet uses constant
memory
        yield a
        a, b = b, a + b
print(list(islice(fib(), 8)))                 # [0, 1, 1, 2, 3, 5, 8, 13]
(from itertools import islice)
```

The benefit is memory. Reading a 50 GB log into a list needs 50 GB, whereas a generator pipeline holds one line at a time. A `for` loop over a file object already reads lazily, so a pipeline chains generators, each consuming the previous one.

```
from collections import Counter

def read_lines(path):
    with open(path) as f:                      # closed even if the consumer
stops early
        for line in f:
            yield line.rstrip("\n")

def src_ips(lines):
    for line in lines:
        yield line.split(None, 1)[0]           # first whitespace-separated
field

counts = Counter(src_ips(read_lines("access.log")))
print(counts.most_common(3))                   # the three most frequent ad-
dresses with counts
```

For lines `10.0.0.1 GET /a`, `10.0.0.2 GET /b`, `10.0.0.1 POST /c`, `10.0.0.3 GET /a`, `10.0.0.1 GET /d`, `10.0.0.2 GET /e` the call prints `[('10.0.0.1', 3), ('10.0.0.2', 2), ('10.0.0.3', 1)]`. The memory used is proportional to the number of *distinct* addresses, not to the file size, which is the answer to the "what if the file is 50 GB?" follow-up. If even the distinct set is too large, one sorts externally or uses a counting sketch, as discussed in Pillar 1 (p. 7). For the top `k` only, `heapq.nlargest(k, counts.items(), key=lambda kv: kv[1])` costs O(n log k).

> INTERVIEW LENS
>
> GOOGLE · AMAZON · MICROSOFT
>
> "Extract the IP addresses from a log and sort by frequency; what if the file is 50 GB?" is a reported Google question. A strong answer streams with a generator, counts with a `Counter`, and states the memory bound and the point at which one would switch to external sorting.

> Follow-ups: the difference between `most_common(k)` and a heap, and why a list comprehension over the whole file would fail.

CHECK YOURSELF

1. How does `[f(x) for x in xs]` differ from `(f(x) for x in xs)` in memory? [3]
2. What does a generator return when called a second time after exhaustion? [4]

2.18 Python: struct, re, subprocess, files, exceptions MUST · ■■□

Binary data with `struct`. The module `struct` converts between Python values and packed bytes, using a format string. The first character selects byte order and alignment: `!` means network (big-endian) order with no padding, `<` little-endian, and no prefix means *native* order with C-style padding. The letters are `B` (1 byte), `H` (2), `I` (4), `Q` (8) and `Ns` for `N` raw bytes. Unpacking the IPv4 header of 2.14 (p. 108) takes one line.

```
hdr = bytes.fromhex("450000730000400040011b861c0a80001c0a800c7")
fmt = "!BBHHHBBH4s4s"                          # 1+1+2+2+2+1+1+2+4+4 = 20 bytes
v_ihl, tos, total, ident, frag, ttl, proto, csum, src, dst = struct.unpack(fmt,
hdr)
print(v_ihl >> 4, v_ihl & 15, total, ttl, proto, hex(csum))   # 4 5 115 64 17
0xb861
print(".".join(map(str, src)), struct.calcsize(fmt))           # 192.168.0.1 20
print(struct.pack("!HI", 80, 1), struct.calcsize("BI"), struct.calcsize("!BI"))
# b'\x00P\x00\x00\x00\x01' 8 5
```

The last line shows the padding rule of 2.7 (p. 99) in Python: `"BI"` uses native alignment and occupies 8 bytes, while `"!BI"` is packed into 5. For a wire format the prefix `!` is always correct.

Regular expressions with `re`. A regular expression describes a text pattern. The functions are `re.search` (first match anywhere), `re.match` (match at the start), `re.findall` and `re.sub`. Named groups `(?P<name>...)` make a log parser self-documenting, and `re.compile` avoids recompiling the pattern in a loop.

```
log = "Oct 29 10:01:02 sw1 %LINK-3-UPDOWN: Interface Gi0/1, changed state to
down"
pat = re.compile(r"(?P<host>\S+) %(?P<fac>[A-Z]+)-(?P<sev>\d)-(?P<mn>[A-Z_]+): "
                 r"Interface (?P<ifc>\S+), changed state to (?P<st>up|down)")
m = pat.search(log)
print(m["host"], m["sev"], m["ifc"], m["st"])      # sw1 3 Gi0/1 down
```

Use raw strings (`r"..."`) so that backslashes reach the regular-expression engine. Quantifiers are greedy by default, so `.*` consumes as much as it can; `.*?` is the minimal form. When the input has fixed whitespace-separated columns, `line.split()` is faster and clearer than a regular expression, as in the table-parsing exercise of the Problem Set (p. 128).

Running programs with `subprocess`. `subprocess.run` starts a command, waits, and returns a result. Pass the command as a *list* so that no shell is involved and no argument can be interpreted as shell syntax; this is the main defence against command injection. `check=True` raises `CalledProcessError` on a non-zero exit status, `capture_output=True` collects the output, `text=True` decodes it, and `timeout` raises `TimeoutExpired` for a hung command.

```
r = subprocess.run(["echo", "hi"], capture_output=True, text=True, check=True,
timeout=5)
print(repr(r.stdout), r.returncode)                # 'hi\n' 0
try:
    subprocess.run(["false"], check=True)
except subprocess.CalledProcessError as e:
    print("rc", e.returncode)                       # rc 1
```

Files. Open files with a `with` block, which closes the file even when an exception occurs. `for line in f` reads lazily; `f.read()` loads everything. Open in text mode (`"r"`) for logs and in binary mode (`"rb"`) for packets and captures. The encoding defaults to the platform's, so give `encoding="utf-8"` for portable code.

Exceptions. `try` runs the code, `except E as e` handles a named error type, `else` runs only when no exception occurred, and `finally` always runs. Catch the narrowest type that you can handle; a bare `except:` also swallows `KeyboardInterrupt`. To translate an error and keep the original, use `raise New(...) from e`.

```
class ParseError(Exception): pass
def parse(s):
    try:
        return int(s)
    except ValueError as e:
        raise ParseError(f"bad number {s!r}") from e
try: parse("x")
except ParseError as e: print(e, type(e.__cause__).__name__)   # bad number 'x'
ValueError
```

> TRAP
>
> *Claim:* `subprocess.run(f"ping -c1 {host}", shell=True)` is fine because the hostnames come from our own inventory. *Why it is wrong:* the string is parsed by the shell, so a value such as `8.8.8.8; rm -rf ~` runs a second command, and inventories do get corrupted. *Correct statement:* pass an argument list, `["ping", "-c1", host]`, without `shell=True`.

> CHECK YOURSELF
>
> 1. What is the byte string produced by `struct.pack("!H", 80)`? [5]
> 2. When does the `else` clause of a `try` statement run? [6]

2.19 C++: references, classes and RAII SHOULD · ■■□

C++ is C with abstractions that cost nothing when unused. Interviews ask for a small, repeatable core: references, constructors and destructors, resource ownership, and move semantics. Examples assume `g++ -std=c++17`.

A *reference* is a second name for an existing object: `int &r = x; r = 9;` changes `x`. Unlike a pointer, a reference must be bound when created, cannot be rebound, and cannot be null. Its main use is a parameter: `void f(const Packet &p)` passes without copying and without allowing modification, and `void g(Packet &p)` lets the callee modify the caller's object.

A *class* bundles data and functions and controls access with `private` and `public`. A *constructor* initialises an object and a *destructor* `~T()` runs when the object's lifetime ends, whether the scope exits normally or by an exception. This guaranteed call is the basis of **RAII** (resource

acquisition is initialisation): acquire a resource in a constructor and release it in the destructor, so that the resource cannot leak on any path.

> BIG IDEA
>
> Tie every resource, whether memory, file, lock or socket, to the lifetime of an object, because the destructor runs on every exit path, including exceptions.

```
struct Guard {
    const char *name;
    explicit Guard(const char *n) : name(n) { std::printf("acquire %s\n",
name); }
    ~Guard() { std::printf("release %s\n", name); }
    Guard(const Guard &) = delete;            // a guard must not be copied
    Guard &operator=(const Guard &) = delete;
};
void work(bool fail) {
    Guard a("A");  Guard b("B");
    if (fail) throw std::runtime_error("boom");
    std::puts("work done");
}
```

Calling `work(false)` prints `acquire A`, `acquire B`, `work done`, `release B`, `release A`. Calling `work(true)` inside a `try` prints `acquire A`, `acquire B`, `release B`, `release A` and then, in the handler, `caught boom`. Destructors run in reverse order of construction, and they run during the unwinding caused by the exception. The standard library applies the same pattern: `std::lock_guard` holds a mutex, `std::ifstream` closes its file, `std::vector` frees its memory.

The compiler generates five special members when asked or needed: the copy constructor, copy assignment, move constructor, move assignment and destructor. The *rule of zero* says to write none of them and let members such as `std::vector` and `std::unique_ptr` manage themselves. The *rule of three/five* says that if a class defines a destructor, copy operation or move operation to manage a raw resource, it should define or delete all five. A *move* transfers a resource from a temporary or an `std::move`d object instead of copying it, leaving the source empty but valid.

```
class File {
    std::FILE *f_;
public:
    explicit File(const char *p) : f_(std::fopen(p, "r")) { if (!f_) throw
std::runtime_error("open"); }
    ~File() { if (f_) std::fclose(f_); }
    File(File &&o) noexcept : f_(o.f_) { o.f_ = nullptr; }   // ownership moves
    File(const File &) = delete;                             // two owners would
close twice
};
```

> TRAP
>
> *Claim:* in C++ it is enough to call `delete` at the end of the function. *Why it is wrong:* an exception or an early `return` before that line skips it, and the memory leaks. *Correct statement:* never write a bare `new` in application code; use `std::make_unique<T>()` or a container, so that the destructor frees the memory on every path.

> CHECK YOURSELF
>
> 1. In which order are local objects destroyed? [7]

> 2. Why delete the copy constructor of a class that owns a `FILE *`? [8]

2.20 C++: virtual functions and the vtable SHOULD · ■■□

Inheritance lets a derived class reuse a base class, and *polymorphism* lets code written against the base call the derived behaviour. By default C++ binds a member function at compile time, according to the static type of the pointer. Declaring the function `virtual` makes the call dispatch at run time, according to the dynamic type of the object.

```
struct Shape  { virtual double area() const { return 0; }
                virtual ~Shape() { std::puts("~Shape"); } };
struct Circle : Shape { double r; explicit Circle(double r) : r(r) {}
                double area() const override { return 3 * r * r; }
                ~Circle() override { std::puts("~Circle"); } };

Shape *s = new Circle(2);
std::printf("%g\n", s->area());      // 12: Circle::area, chosen at run time
delete s;                            // ~Circle, then ~Shape
```

The standard implementation is the *vtable*. For each class with virtual functions the compiler builds one table of function addresses, and each object carries a hidden pointer to its class's table, the *vptr*, stored in the object (typically at offset 0). A virtual call loads the vptr, indexes the table and calls through the result, which costs one extra load and an indirect call. Figure 2.7 shows two objects sharing one table per class.

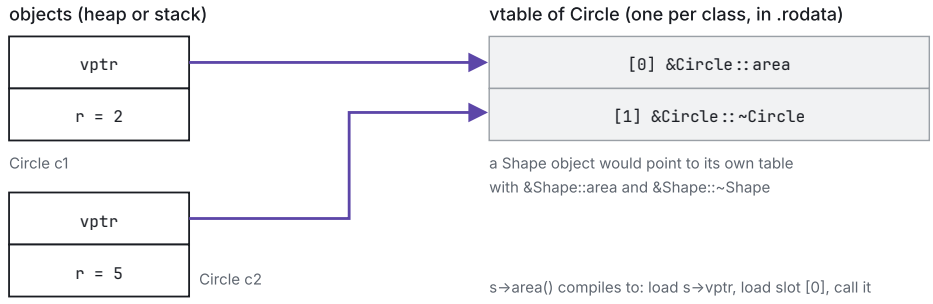


Figure 2.7 Two `Circle` objects point to the same vtable; the call chooses its target through the pointer, not the type of `s`.

A consequence is visible in the size of objects. `struct Plain { int x; void f() {} };` is 4 bytes, because non-virtual functions are not stored in the object, while `struct Virt { int x; virtual void f() {} };` is 16 bytes on LP64: an 8-byte vptr, the 4-byte `int` and 4 bytes of padding. An empty-looking `Shape` with only virtual functions is 8 bytes, just the vptr.

> **TRAP**
>
> *Claim:* a base class needs no virtual destructor if the derived class adds no data. *Why it is wrong:* `delete` on a `Base *` that points to a `Derived` calls only `~Base` when the destructor is not virtual, which is undefined behaviour and skips the derived cleanup. *Correct statement:* any class meant to be used polymorphically gets a `virtual` destructor. The experiment with `NShape` and `NCircle` (no `virtual`) prints only `~NShape`, and `s->area()` returns the base result 0, not 3.

Two related hazards. *Slicing*: assigning a `Circle` to a `Shape` object, as in `Shape sl = c;`, copies only the base part; `sl.area()` then returns 0, the base version, because `sl` is a real `Shape`, so polymorphism needs a pointer or reference. *Calling virtual functions in constructors*: while a base constructor runs the object is still a base, so the base version is called. A `= 0` after the declaration makes a function *pure virtual* and the class *abstract*, which cannot be instantiated and serves as an interface.

> INTERVIEW LENS
>
> NOKIA · CISCO · MICROSOFT
>
> "What is a virtual function and how is it implemented? Why a virtual destructor?" is a reported C++ question. A strong answer draws Figure 2.7, states the cost (one pointer per object, one indirect call), and explains the destructor rule with the `delete base_ptr` case. Follow-ups: the size of an object with virtual functions, what a pure virtual function is, and why virtual calls resist inlining.

> CHECK YOURSELF
>
> 1. How many vtables exist for ten `Circle` objects? [9]
> 2. What does `sizeof(Shape)` report on LP64 for a class with only virtual functions? [10]

2.21 C++: STL containers and smart pointers SHOULD · ■■□

The Standard Template Library supplies containers, iterators and algorithms. The interview skill is to choose the container by the operations needed, and to state the cost. The table gives the usual figures; "amortised" means averaged over a sequence of operations, and "average" for hashed containers assumes a good hash function.

Container	Access	Insert / erase	Find	Notes
`vector`	O(1) by index	back: amortised O(1); middle: O(n)	O(n)	contiguous; best default
`deque`	O(1) by index	both ends O(1); middle O(n)	O(n)	blocks of memory; no contiguity
`list`	O(n)	O(1) given an iterator	O(n)	doubly linked; stable iterators
`set`, `map`	none by position	O(log n)	O(log n)	red-black tree; sorted order
`unordered_set`, `unordered_map`	none	average O(1), worst O(n)	average O(1), worst O(n)	hash table; no order
`priority_queue`	top O(1)	push, pop O(log n)	none	binary heap; max-heap by default

`std::map` keeps keys sorted, so iteration yields `a=1 b=2` for keys inserted as `b`, `a`, and range queries (`lower_bound`) are possible; `unordered_map` is faster for plain lookup. `vector::push_back` doubles the capacity when full: in libstdc++ 1000 insertions cause 11 reallocations, which is why the cost is amortised O(1). `map::operator[]` inserts a default element if the key is absent, so use `count` or `find` to test membership without modifying the map.

> TRAP
>
> *Claim:* a pointer or iterator into a `vector` stays valid while the vector grows. *Why it is wrong:* when the capacity is exceeded, the vector allocates a new block, copies the elements

> and frees the old block, so every saved pointer, reference and iterator dangles. *Correct statement:* after any operation that may reallocate (`push_back`, `insert`, `reserve`), re-obtain iterators, or store indices instead. In the test `v{1,2,3}; p = &v[0]; v.reserve(100);` the comparison `p == &v[0]` is false.

Smart pointers apply RAII to heap objects. A `std::unique_ptr<T>` has exactly one owner, costs nothing beyond the raw pointer, cannot be copied and can be moved; create it with `std::make_unique<T>(args)`. A `std::shared_ptr<T>` shares ownership through a reference count and frees the object when the last owner is destroyed. A `std::weak_ptr<T>` observes a `shared_ptr` object without owning it.

```
auto up  = std::make_unique<int>(5);
auto up2 = std::move(up);            // up is now null; ownership moved
auto sp  = std::make_shared<int>(7);
auto sp2 = sp;                       // sp.use_count() == 2
```

`shared_ptr` has a failure mode that garbage-collected languages do not: a reference cycle. If two nodes hold `shared_ptr` to each other, neither count reaches zero and both leak. In the test, the destructors of two cyclic `Node` objects never run, whereas replacing the back-pointer by a `weak_ptr` (as in a doubly linked list's `prev`) prints `~NodeW` twice. Hence the guideline: `unique_ptr` by default, `shared_ptr` only for genuinely shared ownership, and `weak_ptr` for back-references and caches.

> CHECK YOURSELF
>
> 1. Which container gives sorted iteration, and what does a lookup cost? [11]
> 2. Why do two `shared_ptr` nodes pointing at each other leak? [12]

1. `True`; any non-empty string is true, whatever its characters.↩
2. A list is mutable and therefore unhashable, so it cannot be a dictionary key.↩
3. The list comprehension builds all results at once; the generator expression computes one at a time on demand.↩
4. Nothing: the iteration is already finished, so `next` raises `StopIteration` and a `for` loop ends immediately.↩
5. `b'\x00P'`, the two bytes `00 50`, since `P` is the character 0x50.↩
6. Only when the `try` body finished without raising an exception.↩
7. In reverse order of construction, when the scope ends.↩

8. A copy would give two owners of one handle, and both destructors would close it, which is a double free.↩
9. One for the class `Circle`; the ten objects each hold a pointer to it.↩
10. 8, the size of the vptr, since the class has no data members.↩
11. `std::map` or `std::set`, with O(log n) per lookup.↩
12. Each holds a reference to the other, so neither count falls to zero and neither destructor runs.↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then the approach, code, follow-ups and common mistakes where they apply. Tags give the companies and round where the question was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard), and **S** when a specific question was reported by candidates or **T** when only the topic was reported and the wording is ours. Questions whose subject belongs to another pillar are listed at the end of the second part with a pointer.

Q2.1 What is the difference between a structure and a union? Give a use for a union.

QUALCOMM · CISCO · TECH-1 · ■□□ · S

30-second answer: a structure stores every member one after another; a union stores all members at the same address, so its size is that of its largest member and only one is valid at a time.

Answer. `struct { int i; char c; }` is 8 bytes on LP64; `union { int i; char c; }` is 4. Use a union for one storage that holds alternative message formats, selected by a tag (2.7 (p. 99)), or to view the same bytes as different types (endianness test). **Mistakes.** Reading a member other than the last written is reinterpretation of bytes, not conversion.

Q2.2 What is structure padding and why does the compiler add it?

QUALCOMM · CISCO · TECH-1 · ■□□ · S

30-second answer: the compiler inserts unused bytes so that each member starts at an address divisible by its alignment, because misaligned access is slower or faults on some processors.

Answer. `struct { char c; int i; char d; }` has offsets 0, 4, 8 and size 12, with 6 bytes of padding (Figure 2.4). The size is rounded to a multiple of the largest alignment so arrays of the structure stay aligned. **Follow-ups.** Reorder by decreasing size, or pack with `__attribute__((packed))` at the cost of misaligned access.

Q2.3 What is `sizeof(struct { char a; int b; short c; })`? Reorder it to minimise the size.

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: 12 bytes; ordering as `int b; short c; char a;` gives 8.

Approach. `a` at 0; `b` needs a multiple of 4, so 4; `c` needs a multiple of 2, so 8 to 10; round to a multiple of 4: 12. Reordered: `b` at 0, `c` at 4, `a` at 6, end 7, round to 8. **Mistake.** Adding the member sizes (7) and forgetting alignment.

Q2.4 What is the size of a structure with a `short`, a `long` and a `char` on 32-bit and 64-bit machines?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: 24 bytes on 64-bit Linux (LP64) and 12 on 32-bit x86, because `long` is 8 bytes there and 4 here.

Approach. For `{ short s; long l; char c; }`: LP64 offsets 0, 8, 16, end 17, rounded to 24. ILP32 offsets 0, 4, 8, end 9, rounded to 12. Order `long, short, char` gives 16 and 8. **Mistake.** Assuming `long` is 4 bytes everywhere; on 64-bit Windows (LLP64) it is 4, so the answer is machine-specific.

Q2.5 Find the size of a structure without using `sizeof`.

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: take two consecutive objects and subtract their addresses as bytes.

Code. For a variable: `(char *)(&x + 1) - (char *)&x`. For an array element: `(char *)&a[1] - (char *)&a[0]`. **Why.** `&x + 1` advances by one whole object, and the `char *` cast makes the difference count bytes. **Follow-up.** The macro form `((size_t)((T *)0 + 1))` relies on a null pointer and is formally undefined.

Q2.6 Worksheet: sizes of six structures on LP64.

QUALCOMM · OA · ■■□ · S

30-second answer: 24, 16, 12, 4, 8, 16.

Structure	Offsets	Size
`{char a; double d; char b;}`	0, 8, 16	24
`{double d; char a, b;}`	0, 8, 9	16
`{char a[3]; short s; int i;}`	0, 4, 8	12
`{short s; char c;}`	0, 2	4
`{int i; struct {char c; short s;} in;}`	0, 4 (inner size 4)	8
`{char *p; char c;}`	0, 8	16

Q2.7 What does `volatile` do? Give a scenario where you must use it.

CISCO · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: `volatile` tells the compiler that an object may change outside the program's control, so every source-level access must really be performed and not cached in a register or removed.

Answer. Scenarios: a memory-mapped device register, a flag set in a signal handler or interrupt routine, and variables across `setjmp`. Evidence: at `-O2` a plain `while (!flag);` loads `flag` once and spins on a `jmp`, whereas the `volatile` version reloads (Lab 2.2, 2.6 (p. 98)). **Mistake.** Using it for thread synchronisation (Q2.9).

Q2.8 Can a variable be both `const` and `volatile`? When?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: yes; the program must not write it, but hardware may change it, as with a read-only status register.

Answer. `volatile const uint32_t *status = (volatile const uint32_t *)0x40021000;` forbids stores through `status` and forces a fresh load on each read. `const` alone would allow the compiler to assume the value never changes.

Q2.9 Is `volatile` enough to share a flag between two threads?

ARISTA · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: no; it stops compiler caching but gives neither atomicity nor ordering between processors.

Answer. Another core may see the write late or out of order relative to other writes, and a multi-word update may be seen torn. Use `_Atomic` (C11) or `atomic_flag`, or a mutex with a condition variable (Pillar 7). **Follow-up.** For a single-core signal handler, `volatile sig_atomic_t` is the correct pairing.

Q2.10 Write a program to detect whether the machine is little- or big-endian.

CISCO · OA · ■□□ · S

30-second answer: store 1 in a 32-bit integer and read its first byte; 1 means little-endian.

Code. `union { unsigned i; unsigned char c[4]; } u = { .i = 1 }; return u.c[0] == 1;` An equivalent form is `*(unsigned char *)&one`, which is allowed because character types may alias anything. **Mistake.** Testing with bit shifts: shifts act on values and give the same result on both kinds of machine.

Q2.11 Why do we need `htons` and `htonl`? What breaks if we forget them?

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: the network uses big-endian order and x86 is little-endian, so multi-byte fields must be converted; otherwise the peer reads the bytes reversed.

Answer. Port 80 (`0x0050`) stored little-endian is `50 00`, which the peer reads as `0x5000`, i.e. 20480. On a big-endian host the functions do nothing, so the code is portable. **Mistakes.** Converting twice (which restores the wrong order), converting a value that is not in host order, and using `htons` on a 32-bit value.

Q2.12 Swap the byte order of a 32-bit integer without library calls.

CISCO · BROADCOM · TECH-1 · ■■□ · T

30-second answer: shift each byte to its mirrored position and OR them together.

Code. `(x >> 24) | ((x >> 8) & 0xFF00) | ((x << 8) & 0xFF0000) | (x << 24)` with `uint32_t x`; `swap16` is `(uint16_t)((x << 8) | (x >> 8))`. `swap32(0x0A0B0C0D)` is `0x0D0C0B0A`. **Mistake.** Using signed `int`, so that `x >> 24` sign-extends. GCC offers `__builtin_bswap32`.

Q2.13 Explain the memory layout of a C program. Where do globals, statics, locals, `malloc` memory and literals live?

ARISTA · QUALCOMM · TECH-1 · ■□□ · S

30-second answer: text holds code, rodata holds literals and const globals, data holds initialised globals and statics, BSS holds zero-initialised ones, the heap holds `malloc` blocks, and the stack holds locals and return addresses.

Answer. Draw Figure 2.1 in the order text, rodata, data, BSS, heap (growing up), gap, stack (growing down). A `static` local is in data or BSS. **Follow-ups.** Why BSS is not stored in the file; why the heap and stack grow towards each other; what happens on stack overflow (a fault on the guard page).

Q2.14 What is the difference between a stack and a heap allocation?

CISCO · QUALCOMM · TECH-1 · ■□□ · T

30-second answer: stack storage is automatic, fast and fixed in size per thread, and ends with the function; heap storage is explicit, slower, large, and lives until `free`.

Answer. The stack is typically 8 MB per thread on Linux (`ulimit -s`), so a large local array overflows it; the heap is limited by memory. Stack allocation is a pointer adjustment; `malloc` searches free lists and can fail. Heap errors are leaks and double frees; stack errors are overflow and dangling pointers to dead frames.

Q2.15 What causes a segmentation fault?

ARISTA · CISCO · TECH-1 · ■□□ · T

30-second answer: an access to an address that is unmapped or not permitted, such as a null or wild pointer, a write to read-only memory, or a stack overflow.

Answer. Typical C causes: dereferencing `NULL`, use after free, writing to a string literal, indexing far outside an array, and unbounded recursion. Method: compile with `-g -fsanitize=address`, rerun, and read the first frame of the report; or run under `gdb` and type `bt`.

Q2.16 What is the difference between a static global variable, a static local variable and a static function?

CISCO · BROADCOM · TECH-1 · ■□□ · S

30-second answer: a static local keeps its value across calls; a static global or function is visible only within its file.

Answer. Local: static storage duration, zero-initialised, one instance, lives in data or BSS. Global and function: internal linkage, so other files cannot link to the name and the same name may be reused in another file. **Mistake.** Putting `static int x;` in a header: each including file gets a private copy.

Q2.17 What is wrong with `#define SQUARE(x) x*x`? Fix it. What does `SQUARE(i++)` do?

CISCO · OA · ■□□ · S

30-second answer: the argument is pasted without parentheses, so `SQUARE(2+3)` is `2+3*2+3 = 11`; the fix is `((x)*(x))`, and `i++` is then evaluated twice, which is undefined.

Answer. Also `10/SQUARE(2)` expands to `10/2*2 = 10`, which shows why the whole body needs parentheses. For side effects use a `static inline` function, which evaluates its argument once (2.11 (p. 104)).

Q2.18 Macro versus inline function: when would you use each?

CISCO · TECH-1 · ■■□ · S

30-second answer: prefer `static inline` for typed computation; use a macro only for generic code, stringification, token pasting, source location capture and conditional compilation.

	Macro	`static inline`
Type checking	none	full
Argument evaluation	each use re-evaluates	once
Debugging	expanded text, no symbol	steppable
Generic over types	yes	no (C11 `_Generic` aside)

Q2.19 Predict the output: `int a[] = {1,2,3}; int *p = a; printf("%d %d", *p++, *++p);`

CISCO · OA · ■■□ · S

30-second answer: undefined behaviour; `p` is modified twice without sequencing, and the order of evaluating arguments is unspecified.

Answer. Say so first, then show the defined rewrite: `int x = *p++; int y = *++p;` gives `x = 1` and `y = 3`, printed as `1 3`, because the statements are sequenced and `p` visits `a[1]` between them.

Q2.20 Explain `const int *p`, `int *const p` and `const int *const p`. Quiz of six declarations.

CISCO · QUALCOMM · TECH-1 · ■□□ · I

30-second answer: read right to left; `const` applies to what is on its left, so the first protects the pointee, the second the pointer, the third both.

Declaration	`*p = x` allowed?	`p = &y` allowed?
`const int *p`	no	yes
`int const *p`	no	yes
`int *const p`	yes	no
`const int *const p`	no	no
`const char *s`	no	yes
`char *const s`	yes	no

Q2.21 What is a dangling pointer? A memory leak? How would you find each on Linux?

CISCO · TECH-1 · ■□□ · S

30-second answer: a dangling pointer refers to storage that has been freed or has gone out of scope; a leak is allocated storage with no remaining pointer. AddressSanitizer finds the first; valgrind or LeakSanitizer finds the second.

Answer. Build with `-fsanitize=address`: a use after free prints a report with the allocation and free stacks; a leak prints the allocation line (Lab 2.3). Prevention: set pointers to `NULL` after `free`, never return the address of a local, and own each block in exactly one place.

Q2.22 `malloc` versus `calloc`. What does `realloc` do if it cannot grow in place?

ARISTA · CISCO · OA · ■□□ · S

30-second answer: `malloc` returns uninitialised memory; `calloc` returns zeroed memory and checks the multiplication for overflow; `realloc` moves the block, copying the data and freeing the old one.

Answer. If it fails it returns `NULL` and leaves the old block valid, so `p = realloc(p, n)` leaks on failure; use a temporary. Pointers into the old block dangle after a move. `free(NULL)` is a no-op.

Q2.23 Implement `memmove`. Why is `memcpy` unsafe for overlapping regions?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: when the destination starts inside the source, a forward copy overwrites bytes before they are read, so `memmove` copies backwards in that case.

Code. The function of 2.13 (p. 106): if `d < s` or `d >= s + n` copy forwards, otherwise from the end. `memmove(s+2, s, 4)` on `"abcdef"` gives `"ababcd"`. Compare addresses through `uintptr_t`. **Follow-up.** `memcpy` may assume no overlap (`restrict`) and use wide copies.

Q2.24 Implement an aligned `malloc` that returns memory aligned to any power of two.

QUALCOMM · BROADCOM · CISCO · TECH-2 · ■■■ · T

30-second answer: allocate extra, round the address up to the alignment, and store the original pointer just before the returned address so that `aligned_free` can recover it.

```
void *aligned_malloc(size_t size,
size_t align) {      /* align: power of
two */
    void *raw = malloc(size + align - 1
+ sizeof(void *));
    if (!raw) return NULL;
    uintptr_t a = ((uintptr_t)raw +
sizeof(void *) + align - 1) & ~
(uintptr_t)(align - 1);
    ((void **)a)[-1] = raw;
/* remember the real block */
    return (void *)a;
}
void aligned_free(void *p) { if (p)
free(((void **)p)[-1]); }
```

Why. Adding `align - 1` guarantees an aligned address inside the block; the mask `~(align - 1)` rounds down to a multiple. Room for the saved pointer is reserved before rounding. Check: `aligned_malloc(100, 64)` returns an address with `a % 64 == 0`. **Follow-up.** C11 offers `aligned_alloc`; POSIX `posix_memalign`.

Q2.25 Write a function to delete a node from a singly linked list; then do it without special-casing the head.

ARISTA · TECH-1 · ■■□ · S

30-second answer: walk with a pointer to the link, not to the node, so the head and an inner node are handled identically.

```
void list_delete(struct node **pp, int
key) {
    while (*pp && (*pp)->v != key) pp =
&(*pp)->next;   /* pp points at the link
*/
    if (*pp) { struct node *dead = *pp;
*pp = dead->next; free(dead); }
}
```

Complexity. O(n) time, O(1) space. With list `1,2,3`, deleting 1, 3 and 9 leaves `2`. The classic version keeps a `prev` pointer and tests `if (!prev)` for the head. The full list library (reverse, cycle detection) belongs to Pillar 1 (p. 7).

Q2.26 What are function pointers? Show how to dispatch packets to handlers by protocol number.

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: a function pointer holds the address of code; a 256-entry array of them indexed by the protocol field replaces a `switch`, and an empty slot is `NULL`.

Answer. `typedef int (*handler_fn)(const uint8_t *, size_t); static handler_fn table[256] = { [6] = on_tcp, [17] = on_udp };` then `fn = table[proto]; if (fn) fn(pkt, len);`. Handlers can register at run time by writing a slot. Full code and output in 2.5 (p. 96).

Q2.27 What happens in recursion versus an infinite loop?

CISCO · TECH-1 · ■□□ · S

30-second answer: infinite recursion consumes a stack frame per call and ends with a stack-overflow crash, whereas an infinite loop uses constant space and runs forever.

Answer. The default stack of 8 MB holds on the order of 10^5 small frames. Compilers may turn tail calls into loops at `-O2`, in which case recursion also spins. Example: `sum(n--)` never reduces its argument (T17, 2.15 (p. 112)).

Q2.28 Is `i = i++ + ++i;` valid? What does it print?

CISCO · QUALCOMM · OA · ■□□ · T

30-second answer: it compiles but has undefined behaviour, because `i` is modified twice without a sequence point; no output can be called correct.

Answer. Name the rule, say that gcc's `-Wsequence-point` (in `-Wall`) warns, and state that different compilers or optimisation levels may give different values. Similar: `a[i] = i++`, `f(i++, i++)`.

Q2.29 What is `offsetof`? How does `container_of` work?

ARISTA · BROADCOM · TECH-2 · ■■■ · T

30-second answer: `offsetof(T, m)` is the byte offset of member `m` within `T`; `container_of` subtracts that offset from a pointer to the member to recover the enclosing structure.

Answer. `#define container_of(ptr, T, m) ((T *)((char *)(ptr) - offsetof(T, m)))`. If `struct pkt { int id; struct list_head node; }` and `n` points at a `node`, `container_of(n, struct pkt, node)` is the packet. This lets one generic list implementation link any structure (intrusive list), as in the Linux kernel.

Q2.30 Can you send a C structure directly over a socket? What could go wrong?

CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: not safely; padding, member sizes and byte order differ between machines and compilers, and pointers are meaningless to the receiver.

Answer. Serialise field by field into a byte buffer in network order, as in 2.14 (p. 108), and send the buffer. Packing removes padding but not endianness, and risks misaligned access. Also never send structures holding pointers. Sockets themselves are in Pillar 7 (p. 285).

Q2.31 Bit-fields for protocol headers: good idea or not?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: not for wire formats; the standard leaves field order and straddling to the implementation, so portable code uses shifts and masks.

Answer. The experiment of 2.8 (p. 101) shows `{version:4, ihl:4}` giving `0x54` and `{ihl:4, version:4}` giving `0x45` on x86-64 gcc; a big-endian compiler reverses this. Bit-fields suit private structures and registers on a known compiler. `(b >> 4)` and `(b & 0x0F)` work everywhere.

Q2.32 How would you test the memory bit-cells of a memory segment using C?

BROADCOM · TECH-2 · ■■■ · S

30-second answer: write walking-one patterns and their complements to every byte, read them back, and report the first mismatch; use a `volatile` pointer so the accesses really happen.

```c
int memtest(volatile uint8_t *p, size_t n) {
    for (int bit = 0; bit < 8; bit++) {
        uint8_t pat = (uint8_t)(1u << bit);
        for (size_t i = 0; i < n; i++) p[i] = pat;
        for (size_t i = 0; i < n; i++) if (p[i] != pat) return -1;
        for (size_t i = 0; i < n; i++) p[i] = (uint8_t)~pat;
        for (size_t i = 0; i < n; i++) if (p[i] != (uint8_t)~pat) return -1;
    }
    return 0;
}
```

Complexity. O(16 n) accesses. A healthy region returns 0. **Follow-ups.** A single pattern misses stuck-at and coupling faults, hence both polarities; address-line faults need a separate test that writes a unique value to addresses that are powers of two. Run it with caches disabled or on uncached memory, or the cache hides the cell.

Q2.33 Edit, compile and run this program from the Linux command line. It crashes; debug it.

ARISTA · TECH-1 · ■■□ · S

30-second answer: compile with `-g -Wall -Wextra -fsanitize=address,undefined`, run, and read the first report frame; if needed use `gdb` with `run`, `bt`, `frame`, `print`.

Approach. (i) Read the compiler warnings: uninitialised variables, returning a local, format mismatches. (ii) Run under ASan: it names the line of an out-of-bounds or use-after-free access. (iii) In `gdb ./a.out`: `run`, then at the crash `bt` (call chain), `frame 1`, `print *node`, `info locals`; `watch x` stops when `x` changes. (iv) Fix, rebuild, and rerun. **Mistake.** Adding `printf` calls without understanding, which can hide a race or an overflow by changing layout.

Q2.34 `uint8_t a = 200, b = 100;` What does `printf("%d", a + b)` print, and why?

CISCO · OA · ■□□ · T

30-second answer: 300, because both operands are promoted to `int` before the addition, so nothing wraps.

Answer. Only `uint8_t c = a + b;` would truncate to 44. Contrast `unsigned a = 1; int b = -1; b < a` which is false because `b` converts to unsigned (2.2 (p. 93)).

Q2.35 `char *s = "abc"` versus `char s[] = "abc"`: what differs?

ARISTA · QUALCOMM · OA · ■□□ · T

30-second answer: the pointer refers to read-only characters in .rodata and has size 8; the array is a writable 4-byte copy.

Answer. `sizeof` gives 8 and 4; writing `s[0]` faults in the first case. The pointer can be reassigned, the array name cannot. Figure 2.3.

Q2.36 What do `restrict` and `inline` mean?

BROADCOM · NOKIA · TECH-1 · ■■□ · T

30-second answer: `restrict` promises that no other pointer accesses the same object during the pointer's lifetime, enabling optimisation; `inline` suggests substituting the function body at call sites.

Answer. `memcpy(void *restrict, const void *restrict, size_t)` carries the promise, which is why overlap is undefined. A broken promise is undefined behaviour. `static inline` in a header avoids multiple definitions.

Q2.37 Why use `<stdint.h>` types for packet fields?

BROADCOM · QUALCOMM · TECH-1 · ■□□ · T

30-second answer: `int` and `long` change size between platforms, whereas `uint16_t` and `uint32_t` have exactly the width the protocol specifies.

Answer. A TCP port is `uint16_t`, an IPv4 address `uint32_t`, a byte `uint8_t`. Print with `PRIu32` from `<inttypes.h>`. Unsigned types also give defined wrap-around for sequence numbers.

Q2.38 What is the effect of making a big local array `static`?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: the array moves from the stack to .data or .bss, which removes the stack-overflow risk and keeps its contents between calls, but the function is no longer re-entrant or thread-safe.

Answer. The binary's BSS grows but its file size does not. Two callers, or two threads, share the same bytes. The alternative is `malloc`, freed by the caller.

Problem Set, continued: Python, C++ and coding items

Q2.39 Python: given `show ip interface brief` text, list the interfaces whose status or protocol is not up.

AMAZON NDE · TECH-1 · ■□□ · S

30-second answer: split each line into whitespace-separated fields, take the name from the first and the status and protocol from the last fields, and filter.

```
def parse_brief(text):
    out = {}
    for line in text.splitlines():
        f = line.split()
        if len(f) < 6 or not re.match(r"
[A-Za-z]+\d", f[0]):
            continue
# skips the header and blank lines
        out[f[0]] = (" ".join(f[4:-1]),
f[-1])         # status may be two words
    return out
bad = [i for i, sp in
parse_brief(text).items() if sp !=
("up", "up")]
```

For the three-interface sample (Gi0/1 up/up, Gi0/2 `administratively down`/down, Gi0/3 up/down) the result is `['GigabitEthernet0/2', 'GigabitEthernet0/3']`. **Mistake.** `split(" ")` instead of `split()`, which breaks on runs of spaces. **Follow-up.** Structured output (JSON) is preferable when the device offers it; see Pillar 10.

Q2.40 Python: extract IP addresses from a log and sort by frequency. What if the file is 50 GB?

GOOGLE · TECH-1 · ■■□ · S

30-second answer: stream the file line by line into a `Counter`; memory depends on the number of distinct addresses, not on the file size.

Answer. The pipeline `Counter(src_ips(read_lines(path))).most_common(k)` of 2.17 (p. 119) runs in one pass, O(n) time. If the distinct set does not fit, partition addresses by hash into several files and count each, or sort externally. For the top `k` only, `heapq.nlargest` costs O(n log k). **Mistake.** `f.readlines()`, which loads everything.

Q2.41 Python: time complexity of `x in list` versus `x in set`? Why?

GOOGLE · AMAZON · OA · ■□□ · I

30-second answer: O(n) for a list, because it compares each element; O(1) on average for a set, because it hashes the key and inspects one bucket.

Answer. Hence membership tests inside loops should use a set built once. The set requires hashable (immutable) elements, and its worst case is O(n) with many collisions.

Q2.42 Python: explain the mutable default argument bug.

GOOGLE · MICROSOFT · OA · ■□□ · I

30-second answer: a default value is evaluated once at definition time, so a default list or dict is shared by all calls that omit the argument.

Answer. `def f(x, acc=[])` returns `[1, 2]` for both `f(1)` and `f(2)` as printed after the second call. Fix: `acc=None` and `if acc is None: acc = []` (2.16 (p. 118)).

Q2.43 Python: what is the difference between `is` and `==`? What is a shallow copy?

MICROSOFT · GOOGLE · OA · ■□□ · T

30-second answer: `==` compares values and `is` compares object identity; a shallow copy duplicates the container but shares the elements.

Answer. `a = [[0]] * 2` makes two references to one inner list; `a[0].append(1)` changes both rows. Use `is` only against `None`. For independent nested copies use `copy.deepcopy` or a comprehension `[[0] for _ in range(2)]`.

Q2.44 Tell me about an automation script you wrote: what it did, how you tested it and how it failed.

AMAZON NDE · TECH-2 · ■■□ · S

30-second answer: describe one concrete script in four parts: the manual task it replaced, how it handled bad input and device errors, how you tested it safely, and one failure with the fix and what you now do differently.

Approach. Choose a script of your own, for example one that collects `show` output from several devices and reports mismatches. State the scale (devices, minutes saved). Testing: a dry-run mode, a lab or a single device first, unit tests on saved outputs. Failure: a timeout on one device that stopped the run, fixed with per-device `try/except`, a timeout and a summary. Use the STAR frame of Part II. **Mistake.** Claiming no failures.

Q2.45 C++: what is a virtual function and how is it implemented? Why a virtual destructor?

NOKIA · CISCO · TECH-1 · ■■□ · I

30-second answer: a virtual function is chosen by the object's dynamic type through a hidden vptr to a per-class vtable; the destructor must be virtual so that `delete` through a base pointer runs the derived destructor.

Answer. Figure 2.7. Cost: one pointer per object and one indirect call. A class with `virtual` members and an `int` is 16 bytes on LP64, without them 4. Without a virtual destructor, `delete base_ptr` runs only `~Base`, which is undefined. **Follow-up.** Pure virtual (`= 0`) makes the class abstract.

Q2.46 C++: what is RAII? Give an example with locks or files.

MICROSOFT · GOOGLE · TECH-1 · ■■□ · I

30-second answer: tie a resource to an object's lifetime, acquiring in the constructor and releasing in the destructor, so that every exit path, including an exception, releases it.

Answer. `std::lock_guard<std::mutex> g(m);` locks at construction and unlocks when `g` leaves scope; `std::ifstream` closes its file; `std::unique_ptr` frees memory. Destruction order is the reverse of construction (output of `work(true)` in 2.19 (p. 122)).

Q2.47 C++: `map` versus `unordered_map`: implementation and complexity?

GOOGLE · MICROSOFT · TECH-1 · ■□□ · I

30-second answer: `map` is a balanced tree, ordered, O(log n) per operation; `unordered_map` is a hash table, unordered, O(1) on average and O(n) worst case.

Answer. Choose `map` when sorted iteration or range queries (`lower_bound`) are needed, `unordered_map` for plain lookup. `operator[]` inserts a default value for a missing key; use `find` or `count` to test. Keys of `unordered_map` need a hash function.

Q2.48 C++: when are iterators and pointers into a `vector` invalidated?

GOOGLE · TECH-1 · ■■□ · T

30-second answer: whenever the vector reallocates (growth beyond capacity, `reserve`), and for elements at or after the point of an `insert` or `erase`.

Answer. With `v{1,2,3}`, `p = &v[0]` and `v.reserve(100)`, the comparison `p == &v[0]` is false. Fixes: reserve before taking pointers, keep indices, or use `list`. **Mistake.** Erasing inside a range-for; use `it = v.erase(it)` or the erase-remove idiom.

Q2.49 C++: explain `unique_ptr`, `shared_ptr` and `weak_ptr`. How do cycles leak?

MICROSOFT · GOOGLE · TECH-1 · ■■□ · T

30-second answer: `unique_ptr` is a sole, movable owner; `shared_ptr` shares ownership by reference count; `weak_ptr` observes without owning and breaks cycles.

Answer. Two `shared_ptr` nodes pointing at each other keep both counts at least 1, so neither destructor runs; making the back-link a `weak_ptr` frees both. Default to `unique_ptr`; create with `make_unique` or `make_shared`.

Q2.50 C++: what is object slicing, and what are the rules of zero, three and five?

NOKIA · MICROSOFT · TECH-2 · ■■□ · T

30-second answer: slicing copies only the base part of a derived object into a base object, losing derived data and virtual behaviour; the rules say which special members to write when a class owns a resource.

Answer. `Shape sl = circle;` makes `sl.area()` return the base result 0. Pass polymorphic objects by pointer or reference. Rule of zero: own resources through members such as `vector` and `unique_ptr` and write no special members. Rules of three and five: if you write a destructor, copy or move operation, define or delete all of copy constructor, copy assignment, destructor and (five) the two move operations.

Q2.51 Describe what happens on the stack when a function is called.

CISCO · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: the caller places arguments (in registers on x86-64, the rest on the stack) and a call instruction pushes the return address; the callee saves the frame pointer and reserves space for its locals; return reverses this.

Answer. A frame holds the return address, the saved frame pointer, locals and spills. A buffer overflow of a local array can overwrite the return address, which is why `gets` and unchecked `strcpy` are exploitable and why stack protectors place a canary value before the return address. Details of the call stack and virtual memory: Pillar 7 (p. 285).

Q2.52 Swap two integers in three ways. Why does the XOR swap fail when both pointers are the same?

QUALCOMM · TECH-1 · ■□□ · T

30-second answer: with a temporary, with addition and subtraction, and with XOR; the XOR swap sets the value to zero if both pointers alias the same object.

```
void swap_tmp(int *a, int *b)       {
int t = *a; *a = *b; *b = t; }
void swap_add(unsigned *a, unsigned *b)
{ *a = *a + *b; *b = *a - *b; *a = *a -
*b; }
void swap_xor(unsigned *a, unsigned *b)
{ *a ^= *b; *b ^= *a; *a ^= *b; }
```

Why. If `a == b`, the first XOR makes the object `x ^ x = 0`, and the rest keep it zero. The unsigned types avoid signed overflow in the addition version. The temporary version is the one to write: it is clearest and compilers optimise it.

Q2.53 Count the set bits of a 32-bit value; test for a power of two.

QUALCOMM · TECH-1 · ■□□ · S

30-second answer: `x &= x - 1` clears the lowest set bit, so count the iterations; a power of two has exactly one set bit.

```
int popcount(uint32_t x) { int n = 0;
while (x) { x &= x - 1; n++; } return
n; }
int is_pow2(uint32_t x)  { return x && !
(x & (x - 1)); }
```

Complexity. O(number of set bits). `popcount(0xF0F0)` is 8, `is_pow2(64)` is 1 and `is_pow2(96)` is 0. GCC: `__builtin_popcount`.

Q2.54 Reverse a string in place.

BROADCOM · OA · ■□□ · T

30-second answer: swap characters from both ends with two pointers until they meet.

```
void reverse(char *s) {
    if (!*s) return;
    char *i = s, *j = s + strlen(s) - 1;
    while (i < j) { char t = *i; *i++ =
*j; *j-- = t; }
}
```

`"router"` becomes `"retuor"`. **Mistakes.** Passing a string literal (read-only; use an array) and computing `strlen(s) - 1` for the empty string, where `size_t` wraps.

Q2.55 Convert a dotted quad to a `uint32_t` with validation, and back.

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: parse four decimal fields of at most three digits and value at most 255, separated by single dots, shifting each into place; reject anything else.

```
int dotted_to_u32(const char *s,
uint32_t *out) {
    uint32_t addr = 0; int parts = 0;
    for (;;) {
        if (*s < '0' || *s > '9') re-
turn -1;
        unsigned v = 0; int digits = 0;
        while (*s >= '0' && *s <= '9') {
            v = v * 10 + (unsigned)
(*s++ - '0');
            if (++digits > 3 || v >
255) return -1;
        }
        addr = (addr << 8) | v;
        if (++parts == 4) break;
        if (*s++ != '.') return -1;
    }
    if (*s != '\0') return -1;
    *out = addr; return 0;
}
```

`"192.168.1.10"` returns 0 with `0xC0A8010A` (host order; apply `htonl` for the wire). `"1.2.3"`, `"1.2.3.256"` and `"1.2.3.4.5"` return -1. The reverse is `u32_to_dotted` in 2.14 (p. 108). **Mistake.** Accepting empty fields (`1..2.3`) or trailing junk.

Q2.56 Implement a circular buffer with power-of-two size.

BROADCOM · QUALCOMM · TECH-1 · ■■□ · I

30-second answer: keep two free-running counters and mask them with `size - 1`; the buffer is empty when they are equal and full when they differ by the size.

```
#define RB_SIZE 8
/* power of two */
struct ring { uint8_t buf[RB_SIZE]; un-
signed head, tail; };
int rb_put(struct ring *r, uint8_t b) {
    if (r->head - r->tail == RB_SIZE)
return -1;   /* full  */
    r->buf[r->head++ & (RB_SIZE - 1)] =
b; return 0;
}
int rb_get(struct ring *r, uint8_t *b) {
    if (r->head == r->tail) return -1;
/* empty */
    *b = r->buf[r->tail++ & (RB_SIZE -
1)]; return 0;
}
```

Ten puts into an empty ring succeed 8 times; the first get returns 0. Unsigned subtraction keeps `head - tail` correct after the counters wrap. **Why the mask.** `& (N-1)` replaces `% N` and wastes no slot. One producer and one consumer need no lock if the counters are atomic.

Q2.57 Write a bounds-safe TLV (type-length-value) parser that rejects truncated and overlong items.

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: before reading each item, check that two header bytes remain, then that the declared length fits in what remains; subtract rather than add, to avoid overflow.

```
int tlv_walk(const uint8_t *p, size_t
len) {
    size_t off = 0;
    while (off < len) {
        if (len - off < 2) return -1;
/* no room for type and length */
        unsigned t = p[off], l = p[off
+ 1];
        if (l > len - off - 2) return
-1;          /* value would run past the
end */
        printf("type %u len %u\n", t,
l);
        off += 2 + l;
    }
    return 0;
}
```

For `{1,2,0xAA,0xBB,2,0}` it prints `type 1 len 2` and `type 2 len 0`, returning 0; for `{1,5,0xAA}` it returns -1. **Mistake.** Testing `off + 2 + l > len`, which can overflow for wide length fields. Use a 16- or 32-bit length the same way.

Q2.58 Build a MAC learning table with a hash and ageing.

ARISTA · BROADCOM · TECH-2 · ■■□ · T

30-second answer: hash the 6-byte MAC into a bucket of chained entries; a frame refreshes or inserts the entry for its source MAC, lookups return the port, and an ageing sweep removes entries not seen for the timeout.

```
struct mac_ent { uint8_t mac[6];
uint16_t port; uint32_t seen; struct
mac_ent *next; };
static struct mac_ent *tab[8];
static unsigned h(const uint8_t *m) {
uint32_t x = 2166136261u; for (int i =
0; i < 6; i++) { x ^= m[i]; x *=
16777619u; } return x % 8; }
void learn(const uint8_t *m, uint16_t
port, uint32_t now) {
    struct mac_ent **pp = &tab[h(m)];
    for (; *pp; pp = &(*pp)->next)
        if (!memcmp((*pp)->mac, m, 6))
{ (*pp)->port = port; (*pp)->seen =
now; return; }
    struct mac_ent *e = calloc(1,
sizeof *e); if (!e) return;
    memcpy(e->mac, m, 6); e->port =
port; e->seen = now; *pp = e;
}
void age(uint32_t now, uint32_t max) {
    for (int i = 0; i < 8; i++)
        for (struct mac_ent **pp =
&tab[i]; *pp; ) {
            if (now - (*pp)->seen >
max) { struct mac_ent *d = *pp; *pp = d-
>next; free(d); }
            else pp = &(*pp)->next;
        }
}
```

Learning MAC A on port 3 at time 100, B on port 4 at 200, and A again on port 5 at 210, a lookup gives 5 and 4; `age(305, 100)` removes B (105 s old) and keeps A (95 s). The lookup loop is the same as in `learn`. A station moving between ports is handled by the update branch. Why the switch behaves this way: Pillar 4 (p. 213).

Q2.59 Write a fixed-size pool allocator for packet buffers.

BROADCOM · CISCO · TECH-2 · ■■■ · T

30-second answer: carve a static array into equal blocks and keep free blocks on a singly linked list threaded through the blocks themselves; allocation and free are O(1) pointer swaps.

```
#define POOL_N 4
#define BUF_SZ 64
static union blk { union blk *next;
uint8_t data[BUF_SZ]; } pool[POOL_N];
static union blk *freelist;
void  pool_init(void)   { for (int i =
0; i < POOL_N; i++) { pool[i].next =
freelist; freelist = &pool[i]; } }
void *pool_alloc(void)  { union blk *b
= freelist; if (b) freelist = b->next;
return b; }
void  pool_free(void *p){ union blk *b
= p; b->next = freelist; freelist = b; }
```

The union lets a free block store the link where an allocated block stores data, so there is no per-block overhead. After four allocations the fifth returns `NULL`; after `pool_free(y)` the next allocation returns `y`. **Benefits.** No fragmentation, no `malloc` latency, bounded memory. **Follow-ups.** Thread safety (a lock or per-core pools) and alignment of `BUF_SZ`.

Q2.60 Write a re-entrant tokeniser in the style of `strtok_r`.

BROADCOM · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: keep the scan position in a caller-supplied pointer instead of a static variable; skip leading delimiters, mark the token's end with a NUL, and save the position after it.

```
char *my_strtok_r(char *s, const char
*delim, char **save) {
    if (!s) s = *save;
    s += strspn(s, delim);
/* skip leading delimiters */
    if (!*s) { *save = s; return NULL; }
    char *tok = s;
    s += strcspn(s, delim);
/* find the end of the token */
    if (*s) *s++ = '\0';
    *save = s;
    return tok;
}
```

On `"a,b;;c"` with delimiters `",;"` the tokens are `[a][b][c]`; consecutive delimiters give no empty token. The input is modified, so it must be writable. Plain `strtok` keeps its state in a hidden static, so it is not thread-safe.

Q2.61 Write a `printf`-lite supporting `%d %x %s %c` using `stdarg.h`.

BROADCOM · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: walk the format string; copy ordinary characters, and on `%` read the matching argument with `va_arg` and print it in the right base.

```
int my_printf(const char *fmt, ...) {
    va_list ap; va_start(ap, fmt); int n
= 0;
    for (; *fmt; fmt++) {
        if (*fmt != '%') {
putchar(*fmt); n++; continue; }
        switch (*++fmt) {
        case 'd': { int v = va_arg(ap,
int); char b[12]; int i = 0;
                    unsigned u = v < 0
? 0u - (unsigned)v : (unsigned)v;
                    do { b[i++] = (char)
('0' + u % 10); u /= 10; } while (u);
                    if (v < 0) {
putchar('-'); n++; }
                    while (i) {
putchar(b[--i]); n++; } break; }
        case 'x': { unsigned u =
va_arg(ap, unsigned); char b[9]; int i
= 0;
                    do { b[i++] =
"0123456789abcdef"[u & 15]; u >>= 4; }
while (u);
                    while (i) {
putchar(b[--i]); n++; } break; }
        case 's': { const char *s =
va_arg(ap, const char *); while (*s) {
putchar(*s++); n++; } break; }
        case 'c': putchar(va_arg(ap,
int)); n++; break;
        case '%': putchar('%'); n++;
break;
        default:  putchar('%');
putchar(*fmt); n += 2;
        }
    }
    va_end(ap); return n;
}
```

`my_printf("%s=%d %x %c %% %d\n", "ttl", 64, 255, 'z', -5)` prints `ttl=64 ff z % -5` and returns 17. `char` is read as `int` because of the default argument promotions. **Mistake.** Reading with the wrong type, which is undefined, exactly as in a real `printf`.

Q2.62 Python: count the lines containing `ERROR`; build a dictionary from `show` output lines.

GOOGLE · AMAZON · OA · ■□□ · T

30-second answer: a generator expression with `sum` counts lazily; a dictionary comprehension pairs the first two fields of each line.

```
n = sum(1 for line in open(path) if
"ERROR" in line)          # 2 for lines
ERROR a, ok, ERROR b
states = {k: v for k, v in (l.split()
[:2] for l in lines)}    # {'Gi0/1':
'up', 'Gi0/2': 'down'}
```

Use `with open(path) as f` in production so the file is closed. For the dictionary, skip blank lines first, since `l.split()[:2]` of an empty line unpacks to the wrong length.

Q2.63 Python: unpack an IPv4 header with `struct` and verify its checksum.

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: unpack with `"!BBHHHBBH4s4s"`, then add all 16-bit words with end-around carry; a valid header sums to `0xFFFF`.

```
def csum_ok(hdr: bytes) -> bool:
    if len(hdr) % 2: hdr += b"\x00"
    total = sum(struct.unpack(f"!
{len(hdr) // 2}H", hdr))
    while total >> 16: total = (total &
0xFFFF) + (total >> 16)    # fold the
carries
    return total == 0xFFFF
```

For the header of 2.14 (p. 108) it returns `True`; clearing the checksum bytes (positions 10 and 11) gives `False`. The unpack is in 2.18 (p. 121). **Mistake.** Native format `"BBHH..."` instead of `!`, giving the wrong order and padding.

Q2.64 Python: a log-to-`Counter`-to-top-k report with `argparse`.

GOOGLE · TECH-1 · ■■□ · S

30-second answer: parse arguments, stream the file into a `Counter` keyed by the first field, and print `most_common(k)`.

```
def main(argv=None):
    ap =
argparse.ArgumentParser(description="Top
talkers")
    ap.add_argument("logfile");
ap.add_argument("-k", type=int,
default=3)
    a = ap.parse_args(argv)
    c = Counter()
    with open(a.logfile) as f:
        for line in f:
            parts = line.split()
            if parts: c[parts[0]] += 1
# skip blank lines
    for ip, n in c.most_common(a.k):
print(f"{n:6d}  {ip}")
```

On a file with `1.1.1.1` three times, `2.2.2.2` twice and `3.3.3.3` once, `-k 2` prints `3 1.1.1.1` and `2 2.2.2.2`. `argparse` supplies `--help` and error messages free. Add `if __name__ == "__main__": main()`.

Q2.65 Python: a streaming parser for a 10 GB flow log, reporting the top sources by bytes.

GOOGLE · AMAZON NDE · TECH-2 · ■■■ · T

30-second answer: a generator yields (source, bytes) pairs and skips malformed lines; a dictionary accumulates totals; `heapq.nlargest` selects the top k.

```
def flow_bytes(lines):
    for l in lines:
        p = l.split()
        if len(p) == 3 and
p[2].isdigit(): yield p[0], int(p[2])
# src dst bytes

tot = defaultdict(int)
for src, b in flow_bytes(open(path)):
tot[src] += b
top = heapq.nlargest(2, tot.items(),
key=lambda kv: kv[1])
```

With lines `10.0.0.1 10.0.0.9 1500`, `10.0.0.2 10.0.0.9 40`, `10.0.0.1 10.0.0.8 900`, `10.0.0.3 10.0.0.9 2000` and a malformed line, the result is `[('10.0.0.1', 2400), ('10.0.0.3', 2000)]`. Memory is proportional to the number of sources. **Follow-up.** For billions of sources use sampling or a count-min sketch; neither is needed at interview depth.

Q2.66 C++: implement `RingBuffer<T, N>` as a template.

BROADCOM · QUALCOMM · TECH-2 · ■■□ · T

30-second answer: the C ring buffer of Q2.56 with the element type and capacity as template parameters and a compile-time check that `N` is a power of two.

```
template <typename T, std::size_t N>
class RingBuffer {
    static_assert(N > 0 && (N & (N -
1)) == 0, "N must be a power of two");
    std::array<T, N> buf_{};
    std::size_t head_ = 0, tail_ = 0;
// free-running counters
public:
    bool push(const T &v) { if (size()
== N) return false; buf_[head_++ & (N -
1)] = v; return true; }
    bool pop(T &out)      { if
(empty()) return false; out =
buf_[tail_++ & (N - 1)]; return true; }
    std::size_t size() const { return
head_ - tail_; }
    bool empty() const { return head_
== tail_; }
};
```

For `RingBuffer<int, 4>`, six pushes give four successes; the first `pop` yields 0 and `size()` is then 3. Storage is inside the object, so there is no allocation.

Q2.67 C++: implement a move-only `unique_ptr`-lite.

MICROSOFT · GOOGLE · TECH-2 · ■■■ · T

30-second answer: own a raw pointer, delete it in the destructor, delete the copy operations and implement the move operations by stealing the pointer and nulling the source.

```
template <typename T> class UniquePtr {
    T *p_;
public:
    explicit UniquePtr(T *p = nullptr)
noexcept : p_(p) {}
    ~UniquePtr() { delete p_; }
    UniquePtr(const UniquePtr &) = de-
lete;
    UniquePtr &operator=(const UniquePtr
&) = delete;
    UniquePtr(UniquePtr &&o) noexcept :
p_(o.p_) { o.p_ = nullptr; }
    UniquePtr &operator=(UniquePtr &&o)
noexcept {
        if (this != &o) { delete p_; p_
= o.p_; o.p_ = nullptr; }
        return *this;
    }
    T &operator*() const { return *p_; }
    T *operator->() const { return p_; }
    T *release() { T *t = p_; p_ = null-
ptr; return t; }
    explicit operator bool() const { re-
turn p_ != nullptr; }
};
```

For an object that prints in its constructor and destructor, `UniquePtr<Tr> b = std::move(a);` leaves `a` empty and `b` owning, and "dtor" is printed exactly once. **Mistakes.** Forgetting the self-assignment check and `noexcept` on the move operations (containers use the copy path otherwise).

Questions solved in other pillars. Several topics reported for this pillar's companies belong elsewhere and are solved there once. The mutex and deadlock question ("What is a mutex? How do you prevent deadlock?"), thread-safe flags and the socket API belong to Pillar 7 (p. 285). The Bash one-liner for the ten most frequent source addresses, and the scripting items, belong to Pillar 8. The linked-list library, the least-recently-used cache and the longest-prefix-match trie belong to Pillar 1 (p. 7). Python libraries for devices (`ipaddress`, `netmiko`, concurrent reachability checks) belong to Pillar 10.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Sections: text, rodata, data, BSS, heap, stack; a `static` local is in data or BSS. (2) `p + n` moves `n * sizeof(*p)` bytes; an array parameter is a pointer. (3) `const` binds to what is on its left: `const int *p` protects the pointee, `int *const p` the pointer. (4) Padding: each member at a multiple of its alignment; size rounded to the largest alignment; reorder by decreasing size. (5) Network order is big-endian; `0x0A0B0C0D` is `0D 0C 0B 0A` in little-endian memory. (6) `volatile` is not atomic; signed overflow, null dereference, unsequenced writes and shifts by the width are undefined. (7) Macros: parenthesise everything, never pass side effects, wrap statements in `do { } while (0)`. (8) `realloc` through a temporary; `free` then `NULL`; ASan for memory errors, UBSan for undefined behaviour, valgrind for leaks. (9) Parse packets by offset with length checks first; never cast a byte buffer to a structure. (10) Python names are references, defaults are evaluated once, generators are lazy; C++ releases resources in destructors, and a polymorphic base needs a virtual destructor.

Three figures to redraw. The process layout (Figure 2.1 (p. 91)), the padding byte map (Figure 2.4 (p. 99)) and the byte-order diagram (Figure 2.5 (p. 101)).

Sizes of the basic types, in bytes

Type	ILP32	LP64 (Linux)	LLP64 (Win64)
`char`	1	1	1
`short`	2	2	2
`int`	4	4	4
`long`	4	8	4
`long long`	8	8	8
pointer, `size_t`	4	8	8
`float`, `double`	4, 8	4, 8	4, 8

Alignment equals size for scalars on LP64. `uint8_t` .. `uint64_t` are exact on all three.

Operator precedence, high to low

Level	Operators	Assoc.
1	`()` `[]` `.` `->` `x++` `x--`	L to R
2	`!` `~` `++x` `--x` unary `+` `-` `*p` `&x` `sizeof` cast	R to L
3	`*` `/` `%`	L to R
4	`+` `-`	L to R
5	`<<` `>>`	L to R
6	`<` `<=` `>` `>=`	L to R
7	`==` `!=`	L to R
8 to 10	& then ^ then |	L to R
11, 12	&& then ||	L to R
13	`?:`	R to L
14	`=` `+=` `<<=` ...	R to L
15	`,`	L to R

Infamous: `*p++` is `*(p++)`; `x & 1 == 0` is `x & (1 == 0)`; `a << 1 + 2` is `a << 3`; `a + b << c` is `(a + b) << c`.

Format specifiers

Spec	Type
`%d` `%i`	`int`
`%u` `%x` `%o`	`unsigned` (decimal, hex, octal)
`%ld` `%lu`	`long`, `unsigned long`
`%lld` `%llu`	`long long`
`%zu` `%zd`	`size_t`, `ssize_t`
`%c` `%s`	`char`, string
`%f` `%e` `%g`	`double`
`%p`	`void *`
`%02x` `%5.2f` `%-8s` `%*d`	zero-pad, width.precision, left-align, width from argument
`PRIu32` `PRId64`	`<inttypes.h>` for `uint32_t`, `int64_t`

A mismatch is undefined; `uint8_t` promotes to `int`, so `%d` is correct for it.

Storage classes

Declaration	Scope	Lifetime	Linkage	Default
local (`auto`)	block	block	none	indeterminate
`static` local	block	program	none	0
global	file and others	program	external	0
`static` global or function	file	program	internal	0
`extern`	declares, no storage	n/a	external	n/a
`malloc` block	by pointer	until `free`	n/a	indeterminate

Qualifiers

Qualifier	Promise or effect	Does not mean
`const`	no writes through this name	not stored in ROM unless global
`volatile`	every access really happens, in order	not atomic, not a barrier
`restrict`	no other pointer to the same object	checked by the compiler
`_Atomic`	indivisible access, ordering	free of locks
`inline`	body may be substituted	guaranteed inlining
`register`	historical hint; address not allowed	faster today

Padding rule, endianness, macros

Offset of a member is the next multiple of its alignment; `sizeof` is a multiple of the largest alignment. `{char; int; char}` is 12; `{int; char; char}` is 8; `{char a[3]; short; int}` is 12. `packed` removes padding at the price of misaligned access. `htons` and `htonl` convert host to network order and are no-ops on big-endian machines. Safe macro: `#define MAX(a,b) ((a)>(b)?(a):(b))` still evaluates an argument twice.

Memory bug to tool

Symptom	Tool
leak	`valgrind --leak-check=full`, LeakSanitizer
overflow, use after free	`-fsanitize=address`
undefined behaviour	`-fsanitize=undefined`
crash, wrong value	`gdb`: `run`, `bt`, `print`, `watch`
warnings	`-Wall -Wextra -Werror`

STL: which container?

Use `vector` unless you can say why not. Need sorted order or range queries: `map` or `set` (O(log n)). Need fastest lookup and no order: `unordered_map` (O(1) average). Need stable iterators and cheap insertion in the middle: `list`. Need both ends: `deque`. Need the largest element repeatedly: `priority_queue`. Complexity table: 2.21 (p. 125).

Python quick reference

`struct` formats: `!` network order, `<` little, `B` 1 byte, `H` 2, `I` 4, `Q` 8, `Ns` N bytes; native alignment only without a prefix. `list`: append O(1) amortised, `insert(0, x)` O(n), `x in` O(n). `set` and `dict`: O(1) on average, keys hashable. `Counter.most_common(k)`, `defaultdict(list)`, `deque` for queues, `heapq.nlargest`. `subprocess.run([...], check=True, capture_output=True, text=True, timeout=t)`; never `shell=True` with data. `//` floors, `%` takes the divisor's sign.

Flashcards

Question	Answer
`sizeof(struct{char; int; short;})` on x86-64?	12
`sizeof` of `{short; long; char;}` on LP64 and ILP32?	24 and 12

Size of `union { int i; char c[5]; }`?	8
`const int *p`: what is constant?	the pointee
`int *const p`: what is constant?	the pointer
Where does a static local live?	data or BSS
Where does `"abc"` live?	rodata (read-only)
Why is BSS free in the file?	zero contents; only the size is stored
`0x0A0B0C0D` in little-endian memory?	`0D 0C 0B 0A`
Network byte order?	big-endian
Is `volatile` atomic?	no
Can a variable be `const volatile`?	yes: read-only hardware register
`sizeof(a)` for an array parameter?	pointer size, 8 on LP64
`strlen("abc")` and `sizeof("abc")`?	3 and 4
Result of `-1 < 1u`?	false
`uint8_t` 200 + 100?	300 (promoted to `int`)
`char c = 200` prints?	-56 where `char` is signed
`SQ(2+3)` with `x*x`?	11
Why `do { } while (0)` in macros?	one statement, safe after `if`
`STR(N)` versus `XSTR(N)`?	name versus expanded value
Signed overflow?	undefined
Unsigned overflow?	defined, wraps modulo 2^n
`1 << 31` on `int`?	undefined
`i = i++ + ++i;`?	undefined

`memcpy` versus `memmove`?	`memmove` handles overlap
Safe `realloc` pattern?	assign to a temporary first
`strncpy` guarantee?	none: may not terminate
`free` twice?	undefined; glibc aborts
Static array in a function: effect?	off the stack; not re-entrant
Offset of the header version nibble?	byte 0, high four bits
Valid IPv4 header checksum sum?	`0xFFFF` (zero after complement)
Fix for `for (unsigned i = n; i >= 0; i--)`?	`for (unsigned i = n; i-- > 0; )`
What does `container_of` compute?	enclosing struct from member pointer
`*p++` means?	`*(p++)`: read, then advance
Python default `acc=[]`: problem?	one list shared by all calls
`x in list` versus `x in set`?	O(n) versus O(1) average
`!` in a `struct` format?	network (big-endian) order
Why `subprocess` with a list?	no shell parsing, no injection
C++ vptr: how many per object?	one, pointing to the class vtable
Why virtual destructor?	`delete` via base runs derived destructor
`map` versus `unordered_map`?	O(log n) ordered versus O(1) average
Smart pointer for sole ownership?	`unique_ptr`
Break a `shared_ptr` cycle with?	`weak_ptr`

PART I · PILLAR 3

3

Computer Networks Fundamentals

Every packet tells a story; we learn to read it, layer by layer.

3.1 Networks, devices and delay	MUST
3.2 The OSI and TCP/IP models, and encapsulation	MUST
3.3 The data-link layer: Ethernet, switching, error control and ARQ	MUST
3.4 IPv4 addressing, subnetting, VLSM and CIDR	MUST
3.5 The IPv4 header, fragmentation, MTU and ICMP	MUST
3.6 ARP, DHCP and NAT: the glue protocols	MUST
3.7 IPv6 essentials	SHOULD
3.8 Routing concepts in one page	MUST
3.9 Transport: UDP, TCP, flow control and congestion control	MUST
3.10 The application layer: DNS, HTTP, TLS and QUIC	SHOULD
3.11 Capstone: what happens when you type google.com	MUST
3.12 Where the topics are asked, and a mini-project	OPTIONAL

3.1 Networks, devices and delay MUST · ■□□

A computer network is a set of devices that exchange data over shared links so that they can share resources: files, printers, an internet connection, a database. Every network question in an interview reduces to two things, namely *where the bits go* and *how long they take*, and this section supplies the vocabulary for both.

> BIG IDEA
>
> A network moves bits between devices, and splitting that job into independent layers is what allows each part to be replaced without disturbing the rest.

Sizes and shapes

Networks are named by their reach. A *LAN* (local area network) covers a room, a floor or a campus and is owned by one organisation. A *MAN* covers a city. A *WAN* (wide area network) joins sites across cities or countries and usually runs over links leased from a carrier. The *Internet* is the network of networks that joins all of them using one common protocol, IP.

The *topology* is the pattern in which devices are joined. In a *bus* all devices hang from one shared cable; in a *star* each hangs from a central device; in a *ring* each has two neighbours; in a *full mesh* every pair is joined directly. The last is robust but costs n(n − 1)/2 links for n devices, so ten sites already need 45 links and a hundred need 4,950. Hence real networks are stars of stars, with a few meshed links at the core.

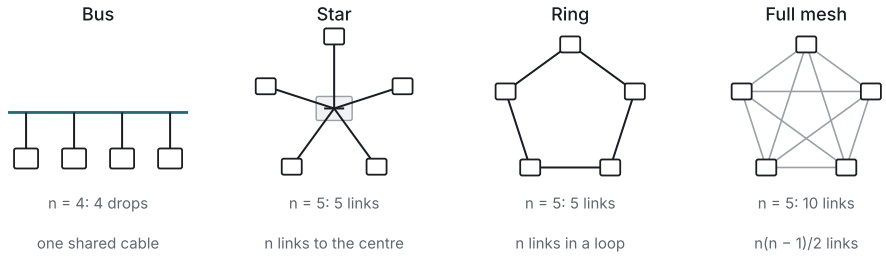


Figure 3.1 Four physical topologies and their link counts. A full mesh grows quadratically, which is why large networks are built as stars of stars.

Devices

Five boxes appear in almost every diagram, and an interviewer expects one sentence on each. A *hub* is a layer-1 repeater: every bit that enters a port leaves all the other ports, so all attached devices share one wire. A *switch* learns which device sits behind which port and forwards a frame only to the port concerned. A *router* joins different networks and forwards packets by their IP destination. A *layer-3 switch* is a switch with routing built into its hardware, which is how campus networks route between VLANs at line rate. A *firewall* permits or drops traffic according to rules, and an *access point* (AP) bridges Wi-Fi radio to the wired network.

Two terms follow from these devices. A *collision domain* is the set of ports whose transmissions can collide on a shared medium; a hub extends it, and every switch port ends it. A *broadcast domain* is the set of devices that receive a frame sent to the broadcast address; a switch extends it, and only a router ends it. Note that modern switched Ethernet is full-duplex, so collisions no longer occur in practice, yet the counting question remains a standard interview exercise.

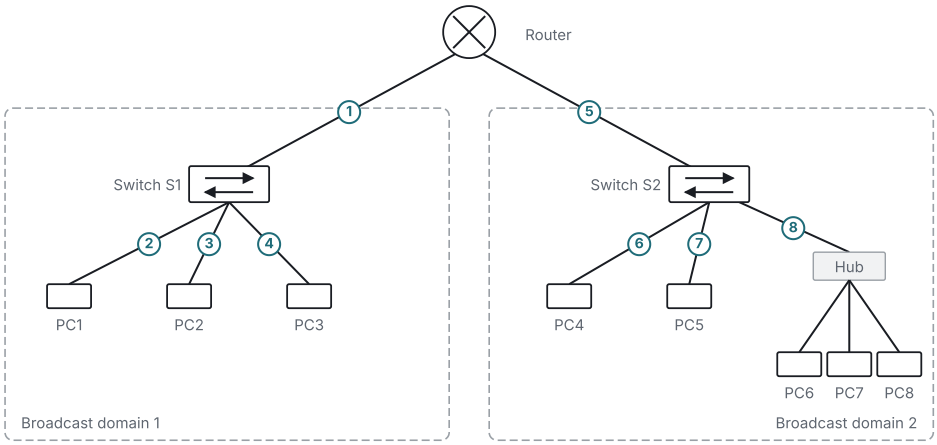


Figure 3.2 Counting domains. The router does not forward broadcasts, so there are two broadcast domains (dashed). Every switch port is its own collision domain, while the hub and PC6 to PC8 share one wire. Link 8 is therefore one domain, and the total is 8 collision domains, numbered on the links.

WORKED EXAMPLE

Count the domains in the figure.

1. A router interface never forwards broadcasts, so each of its two sides is a separate broadcast domain: S1 with PC1–PC3, and S2 with PC4–PC5, the hub and PC6–PC8. That gives 2.
2. Each switch port is a separate collision domain. S1 has four ports in use (router, PC1, PC2, PC3), giving 4.
3. S2 has four ports in use (router, PC4, PC5, hub). The hub and PC6–PC8 share one wire and so form a single domain behind S2's hub port, giving 4.

Result: 2 broadcast domains and 8 collision domains.

Circuit and packet switching

A telephone network of the old kind used *circuit switching*: a path was reserved from end to end for the whole call, whether or not anyone was speaking. The Internet uses *packet switching*: the data is cut into packets, each carrying its destination address, and routers forward them independently. Packet switching wastes nothing on silence and shares links statistically, at the price that packets can queue, be delayed or be lost. Every later topic in this pillar is a mechanism for living with that price.

Four kinds of delay

A packet travelling from host to host accumulates delay at every hop from four sources. *Processing delay* is the time a router takes to read the header and choose the output port; it is microseconds. *Queueing delay* is the time spent waiting for the output link to become free, and it varies with load; it is the cause of jitter, the variation of delay from one packet to the next. *Transmission delay* is the time to push all the bits of the packet onto the link, L/R for a packet of L bits on a link of R bit/s. *Propagation delay* is the time a bit takes to travel the length of the link, d/v, where v is about 2×10^8 m/s in copper or fibre, two-thirds of the speed of light in vacuum.

Bandwidth vs throughput — bandwidth is what the link could carry, in bits per second; throughput is what you actually got. A motorway has six lanes (bandwidth) but you still arrive late in a jam.

Transmission delay depends on packet size and link speed; propagation delay depends only on distance. Confusing the two is the commonest error in this topic.

WORKED EXAMPLE

Send a 1 MB file over a 2,000 km link. Take 1 MB as 8×10^6 bits and ignore queueing and headers.

1. At 10 Mb/s: transmission = $8 \times 10^6 / 10^7 = 0.8$ s.
2. Propagation = 2×10^6 m / 2×10^8 m/s = 0.010 s = 10 ms, whatever the speed of the link.
3. At 10 Mb/s the total is 0.8 + 0.01 ≈ 0.81 s.
4. At 1 Gb/s: transmission = $8 \times 10^6 / 10^9$ = 8 ms, so the total is 8 + 10 = 18 ms.

Result: 0.81 s and 18 ms. Once the link is fast, distance, not bandwidth, sets the floor on latency; no upgrade can beat 10 ms for these 2,000 km.

TRAP

Claim: "A faster link makes every transfer faster by the same factor." *Why it is wrong:* only transmission delay shrinks with bandwidth; propagation delay is fixed by distance and the speed of light in the medium. *Correct statement:* a hundred-fold faster link cuts a large transfer a hundred-fold, but a small request across an ocean barely improves.

INTERVIEW LENS

CISCO · NOKIA

Asked as "hub versus switch versus router", the interviewer wants collision and broadcast domains stated per device, then one scenario, such as why a loop of hubs is harmless but a loop of switches is not (a topic for Pillar 4). A strong answer draws the figure above and counts, without being asked. See Q3.2 in the Problem Set (p. 196).

LAB

`LAB 3.1 · 15 MIN` Cisco Packet Tracer. Build two small networks: four PCs on a hub, and four PCs on a switch. Switch to *Simulation* mode, filter on ICMP, and send one ping from PC0 to PC1 in each network.

What you should notice: on the hub, the event list shows the echo request copied to every other port and the request is seen by PC2 and PC3, which discard it. On the switch, the first request is flooded (the switch has not yet learned PC1), but the reply, and every later ping, travels only between the two ports concerned.

CHECK YOURSELF

1. How many links does a full mesh of 12 routers need? [1]
2. A frame is sent from a PC to another PC in the same VLAN through two switches. How many collision domains does it cross? [2]
3. Which of the four delays grows when the network is congested? [3]

3.2 The OSI and TCP/IP models, and encapsulation MUST · ■□□

Sending a web page involves cables, addresses, reliability, encryption and a document format, and no single protocol does all of this. The standard answer is to divide the work into *layers*, each offering a service to the layer above and using the service of the layer below. A layer on one machine is concerned only with its *peer* layer on the other machine, which it talks to through a header that it adds to the data.

BIG IDEA

> Each layer wraps the data from above in its own header so that its peer layer at the far end can act on it, and this wrapping is why a change at one layer does not disturb the others.

Two models

The *OSI model* (Open Systems Interconnection) has seven layers and is used as vocabulary: engineers say “a layer-2 problem” or “a layer-7 load balancer”. The *TCP/IP model* describes what is actually deployed and has four layers, in which the top three OSI layers merge into one application layer and the bottom two into one link layer. Textbooks that prefer five layers split the link layer again, giving the five-layer model used in this book from now on.

OSI mnemonic — layers 7 to 1: *All People Seem To Need Data Processing* (Application, Presentation, Session, Transport, Network, Data link, Physical). Reverse it for 1 to 7: *Please Do Not Throw Sausage Pizza Away*.

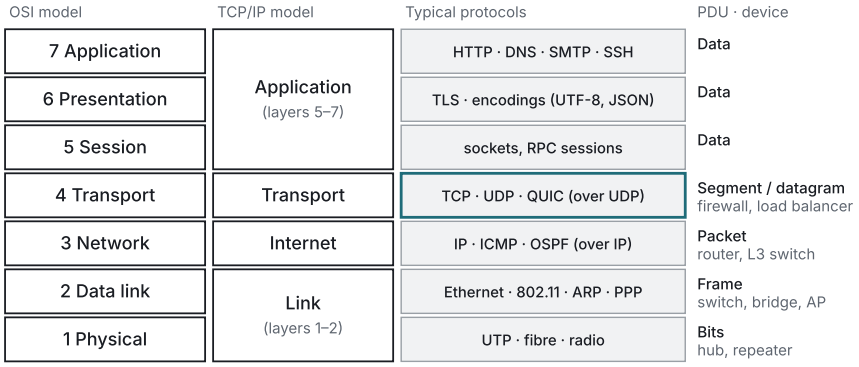


Figure 3.3 The seven-layer OSI model beside the four-layer TCP/IP model. The highlighted row is where TCP and UDP live; the stack below it is the network, the stack above it is the application.

Each layer has a name for its unit of data, the *protocol data unit* (PDU): data at the application layer, a *segment* (TCP) or *datagram* (UDP) at the transport layer, a *packet* at the network layer, a *frame* at the data-link layer, and *bits* on the wire. Interviewers notice when these names are used loosely.

Protocols do not always sit tidily. ARP is carried directly in Ethernet frames yet resolves IP addresses, so it is often called a “layer 2.5” protocol. ICMP is a layer-3 protocol, even though it travels inside IP packets. TLS lives between the transport and application layers and has no home in the TCP/IP model. QUIC runs over UDP but provides what TCP provides, plus encryption. Hence the question “which layer is X?” is best answered with “it is carried in Y and serves Z”.

Encapsulation

When an application writes data to a socket, the transport layer prepends a TCP header, giving a segment; the network layer prepends an IP header, giving a packet; the link layer prepends an Ethernet header and appends a trailer, the *frame check sequence* (FCS), giving a frame. This is *encapsulation*; the receiver strips the headers in reverse order, termed *decapsulation*. A layer knows which protocol is above it through a *demultiplexing key* in its header: the EtherType field in Ethernet (0x0800 for IPv4, 0x86DD for IPv6, 0x0806 for ARP), the protocol field in IP (6 for TCP, 17 for UDP, 1 for ICMP), and the destination port in TCP and UDP.

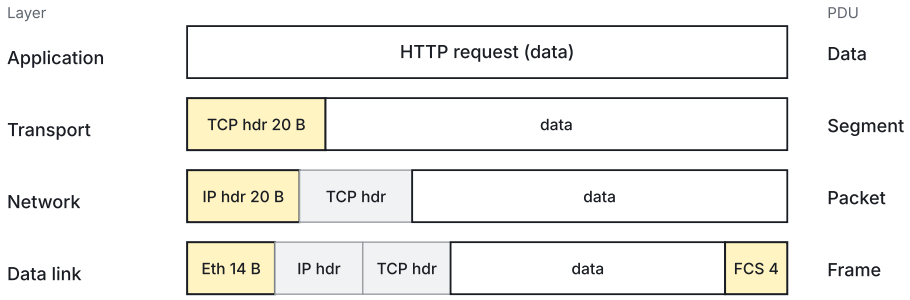


Figure 3.4 Encapsulation. Each layer wraps the data it receives from above in its own header (highlighted). Only the data-link layer also adds a trailer, the frame check sequence (FCS). The receiver reverses the process, which is termed decapsulation.

The headers cost bandwidth, and an interviewer may ask by how much. We measure it below.

WORKED EXAMPLE

Overhead of an HTTP request carrying 100 bytes of data, and of a full-size segment.

1. Header bytes: TCP 20 + IPv4 20 + Ethernet 14 + FCS 4 = 58 bytes.
2. On the wire there is also an 8-byte preamble and start delimiter and a 12-byte inter-frame gap, which together occupy the medium for 20 bytes. The total per frame is 100 + 58 + 20 = 178 byte-times.
3. Efficiency = 100 / 178 ≈ 56%.
4. With 1,460 bytes of data (the largest that fits a 1,500-byte IP packet), the total is 1,460 + 78 = 1,538 byte-times, and the efficiency is 1,460 / 1,538 ≈ 94.9%.

Result: 56% for the small request and 94.9% for the full-size segment. Small packets waste capacity, which is why bulk transfers use the largest segment the path allows.

What changes where

The stack diagram below shows a packet crossing one router. The hosts build and read all layers, but the router climbs only to layer 3: it removes the incoming frame, decrements the IP time-to-live, looks up the destination, and builds a new frame for the next link. The layer-4 segment inside is never read. Hence the rule that interviewers love, which is stated in Section 3.6 with numbers: *the MAC addresses change at every hop, whereas the IP addresses do not (unless NAT intervenes).*

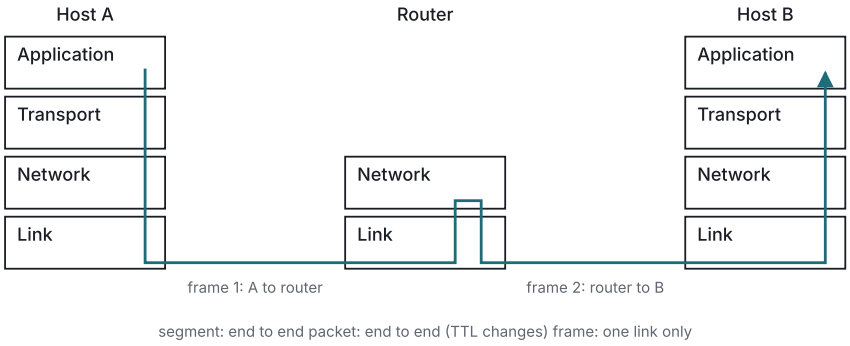


Figure 3.5 A packet crossing one router. The segment is built and read only at the two hosts; the router climbs just to layer 3, then rewrites the frame for the next link.

TRAP

Claim: "A router decrypts and reads the TCP header, which is how it knows the port." *Why it is wrong:* an ordinary router forwards on the IP destination alone and never touches the segment. *Correct statement:* firewalls, NAT devices and load balancers do read layer 4, and are therefore described as layer-4 devices; a plain router is a layer-3 device.

INTERVIEW LENS

CISCO · NOKIA · AMAZON · MICROSOFT · JUNIPER

The question is "name the layers with a protocol and a device each" (Q3.1) or, harder, "which layer does a load balancer work at?". A strong answer gives both L4 (TCP connections, as for a network load balancer) and L7 (HTTP headers, as for an application load balancer), and notes the cost of each.

LAB

`LAB 3.2 · 20 MIN` Wireshark. Run `curl -s http://example.com -o /dev/null` while capturing, stop, and apply the display filter `http`. Select the request and expand the packet-details pane.

```
Frame 14: 142 bytes on wire
Ethernet II, Src: 52:54:00:12:34:56, Dst: 52:54:00:12:35:02   (type 0x0800)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 93.184.216.34  (proto 6)
Transmission Control Protocol, Src Port: 49822, Dst Port: 80, Len: 76
Hypertext Transfer Protocol: GET / HTTP/1.1
```

What you should notice: the four headings are the four layers of Figure 3.4 from the bottom upward; the EtherType, the IP protocol number and the destination port are the three demultiplexing keys. The headers total 66 bytes of the 142 captured: 14 (Ethernet), 20 (IPv4) and 32 (TCP, 20 plus 12 bytes of timestamp options), leaving 76 bytes of HTTP.

CHECK YOURSELF

1. At which layer does a switch operate, and which address does it read? [4]
2. Why is ARP called "layer 2.5"? [5]
3. A frame carries 14 + 20 + 20 header bytes and 1,000 bytes of data. What fraction of the frame (ignoring FCS and preamble) is data? [6]

3.3 The data-link layer: Ethernet, switching, error control and ARQ

MUST · ■■□

The data-link layer delivers frames across a single link, between two devices that are directly connected. It must decide where one frame ends and the next begins (*framing*), which neighbour a frame is for (*addressing*), whether the frame was damaged (*error detection*) and, on some links, how to repair damage (*ARQ*, automatic repeat request). Ethernet, which dominates wired networks, answers the first three; the fourth is done mostly by higher layers, but it is the standard exam topic for sliding-window protocols and reappears in TCP.

BIG IDEA

Layer 2 gets a frame across one link: the MAC address picks the receiver, the CRC detects damage, and a switch forwards by learning where each address lives.

1. 12 × 11 / 2 = 66.↩
2. Three: PC to switch 1, switch 1 to switch 2, switch 2 to PC. Each link is its own domain.↩
3. Queueing delay, because packets wait for the output link; it is also the source of jitter.↩
4. Layer 2; the destination MAC address.↩
5. It is carried in an Ethernet frame (layer 2) but it maps layer-3 addresses to layer-2 addresses.↩
6. 1,000 / 1,054 ≈ 94.9%.↩

The physical layer in one page

Bits travel as electrical signals over copper, as light over fibre or as radio waves. *Unshielded twisted pair* (UTP) is the cable in offices: Cat5e carries 1 Gb/s and Cat6a carries 10 Gb/s to 100 m. *Fibre* carries more over longer distances: *multimode* (MMF) reaches a few hundred metres inside a building, and *single-mode* (SMF) reaches tens of kilometres. Transceivers plug into the switch as SFP (1 Gb/s), SFP+ (10), SFP28 (25) and QSFP (40 to 400).

Two ends of a link must agree on speed and *duplex*. Full duplex means both ends transmit at once; half duplex means they take turns. Auto-negotiation normally settles both, but if one end is forced to full duplex and the other is left on auto, the auto end falls back to half duplex. This *duplex mismatch* is a classic fault: the link comes up and pings work, but throughput collapses under load, the half-duplex side reports late collisions and the full-duplex side reports CRC errors and runts.

Ethernet frames and MAC addresses

Every Ethernet port has a 48-bit *MAC address*, written as six hexadecimal bytes. The first three bytes form the *OUI*, assigned to the manufacturer; the last three are chosen by the manufacturer. Two bits of the first byte carry meaning: the lowest bit (I/G) marks a group address, so any address with an odd first byte, including `ff:ff:ff:ff:ff:ff`, is multicast or broadcast; the next bit (U/L) marks an address that an administrator has set instead of the hardware default.

Unicast, multicast, broadcast — one receiver, a group of receivers, everyone on the segment. Broadcast is the all-ones address.

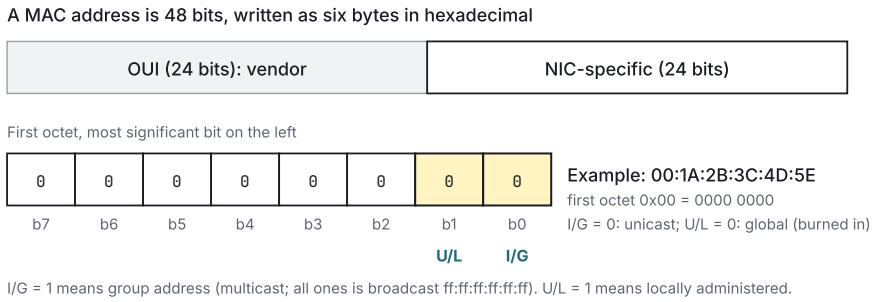


Figure 3.6 MAC address structure. The two low bits of the first octet are flags; the U/L bit is the one flipped when an IPv6 interface ID is built from a MAC (Section 3.7).

The frame format that nearly all networks use is *Ethernet II*. The destination address comes first so that a switch can start forwarding before the rest has arrived. The 2-byte *type* field says what the payload is (0x0800 for IPv4, for example). The payload must be at least 46 bytes, and short payloads are padded, for a reason derived below. The trailer is the 4-byte FCS, a CRC-32 computed over the frame. The *maximum transmission unit* (MTU), the largest payload, is 1,500 bytes.

A *VLAN tag* of 4 bytes, defined in 802.1Q, may be inserted after the source address. It starts with the value 0x8100 (the tag protocol identifier), followed by a 3-bit priority, a drop-eligible bit and a 12-bit VLAN identifier, so up to 4,094 usable VLANs exist. VLANs themselves are treated in Pillar 4.

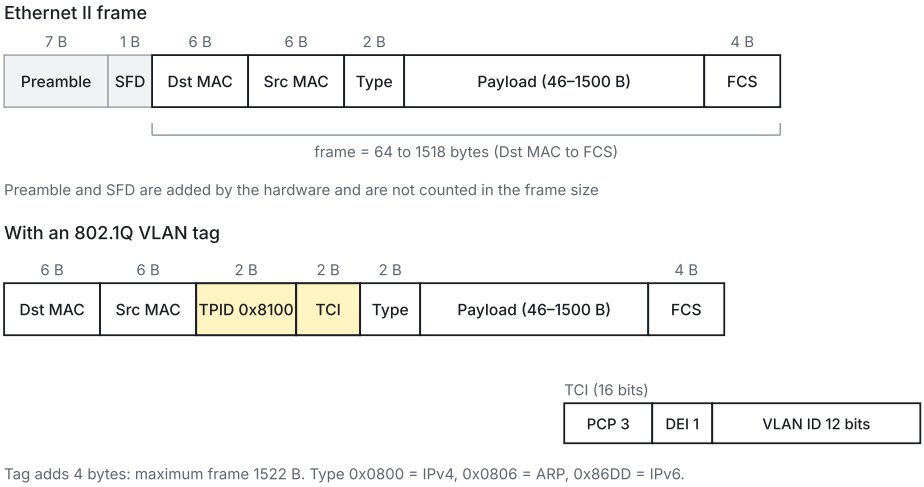


Figure 3.7 Byte-accurate Ethernet II frame (top) and the same frame with an 802.1Q tag (below). The minimum of 64 bytes is the reason for padding short payloads up to 46 bytes.

CSMA/CD and the 64-byte minimum

On the original shared cable, any station could transmit when it heard silence, and two stations could start together. *CSMA/CD* (carrier sense multiple access with collision detection) handles this: a station listens first, transmits if the cable is idle, keeps listening while transmitting, and on hearing a collision stops, sends a short jam signal and waits a random time that grows after each repeated collision (*binary exponential backoff*). For the sender to detect a collision it must still be transmitting when the damaged signal returns from the far end, which sets a lower limit on the frame length.

> WORKED EXAMPLE
>
> **Why is the minimum frame 64 bytes?** A 10 Mb/s Ethernet spans at most 2,500 m, and signals travel at 2×10^8 m/s.
>
> 1. One-way propagation: $T_p = 2{,}500 / (2 \times 10^8) = 12.5$ µs.
> 2. In the worst case, the colliding signal from the far station reaches the sender after one more trip: the sender must still transmit for $2T_p = 25$ µs.
> 3. Bits transmitted in 25 µs at 10 Mb/s: $25 \times 10^{-6} \times 10 \times 10^6 = 250$ bits.
> 4. The standard rounds up to 512 bits (64 bytes, a 51.2 µs slot time) to allow for repeater delays, the jam signal and a safety margin.
>
> **Result: the minimum frame must satisfy $L \geq 2T_p \times R$, giving 250 bits here and 512 bits in the standard. At 1 Gb/s the same rule would demand a much longer frame, so gigabit half-duplex uses carrier extension; full-duplex links have no collisions at all.**

Switching

A *switch* builds a *MAC address table* (also called the CAM table, after the content-addressable memory that stores it) by learning. For each arriving frame it does three things. It reads the source address and records “this address is behind this port”. It then looks up the destination: if the destination is known, the frame is *forwarded* to that port only; if the destination port is the arrival port, it is *filtered* (dropped); if the destination is broadcast, multicast or unknown, the frame is *flooded* out of every port except the arrival port. Entries are removed after an *ageing time* (300 s on Cisco switches), so a device that is moved is relearned.

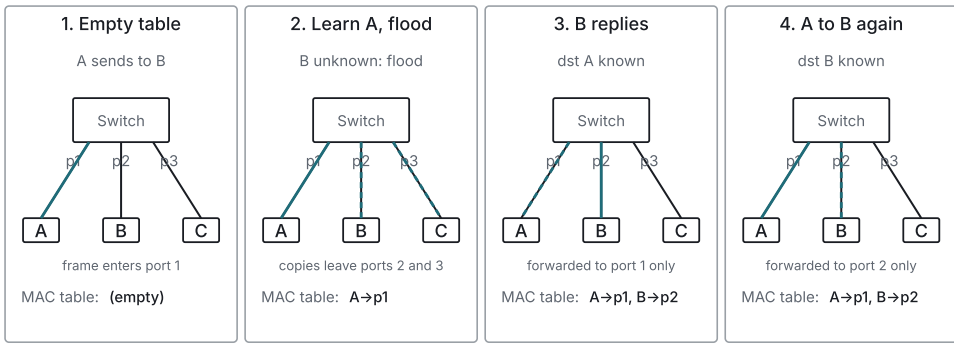


Figure 3.8 A switch learns by reading the source address of every frame and forwards by looking up the destination. Solid accent lines: the frame entering; dashed accent lines: where it leaves.

Flooding an unknown unicast frame is not the same as a broadcast: the frame is still addressed to one MAC, and the receivers other than that one discard it at layer 2. A switch does, however, treat both the same way.

If the table fills, a switch cannot learn and floods every frame for unknown addresses, so it behaves like a hub. An attacker who sends frames with thousands of invented source addresses exploits exactly this to see other people's traffic (*MAC flooding*; mitigation in Pillar 9).

Switches also differ in when they start forwarding. A *store-and-forward* switch receives the whole frame, checks the FCS and then forwards; it adds a delay of one frame time (1,500 bytes take 12 µs at 1 Gb/s and 1.2 µs at 10 Gb/s) but never forwards a corrupted frame. A *cut-through* switch begins forwarding as soon as it has read the destination address, after the 6-byte destination address, so its latency is tens of nanoseconds and independent of frame size, at the price of forwarding damaged frames, which the receiver then discards. Data-centre switches offer cut-through because latency, not error filtering, matters most there.

Framing and error detection

The bits of a frame must be delimited. Ethernet uses a preamble and start delimiter; serial links often use a flag byte. If the flag could appear inside the data, it must be hidden by *stuffing*. In *bit stuffing* (HDLC) the flag is `01111110`, and the sender inserts a 0 after every run of five 1s in the data, which the receiver removes; the data `01111110` is sent as `011111010`. In *byte stuffing* an escape byte is inserted before any flag or escape byte inside the data.

Errors are detected by adding redundancy. A *parity bit* makes the number of 1s even (or odd) and detects any odd number of flipped bits, but misses even numbers. A *checksum* adds the data in words, as the IP header checksum does (Section 3.5). A *cyclic redundancy check* (CRC) treats the data as a polynomial over arithmetic modulo 2 and divides it by a fixed *generator* polynomial; the remainder is the check value. It detects all burst errors no longer than the degree of the generator and almost all longer ones, which is why Ethernet uses the 32-bit generator of CRC-32.

> WORKED EXAMPLE
>
> **CRC for data 1101011011 with generator 10011 ($x^4 + x + 1$).**
>
> 1. The generator has 5 bits, so its degree is 4: append four zeros to the data, giving 11010110110000.
> 2. Divide modulo 2. Wherever the leading bit is 1, XOR the generator under it, aligned to that bit; there is no borrowing. The register after each XOR is shown below.

3. The last four bits are the remainder: 1110. Replace the appended zeros with it and transmit 11010110111110.
4. The receiver divides the whole received word by the same generator. If no bit is damaged the remainder is 0000.

```
start       11010110110000
xor at  0   10011000000000
result      01001110110000
xor at  1   01001100000000
result      00000010110000
xor at  6   00000010011000
result      00000000101000
xor at  8   00000000100110
result      00000000001110      remainder = 1110
```

Result: frame 11010110111110; the receiver's check gives remainder 0. Addition and subtraction are both XOR, which is why the sender can simply replace the zeros with the remainder.

CRC only detects errors. *Hamming codes* correct them. In a Hamming(7,4) code, four data bits are protected by three parity bits placed at positions 1, 2 and 4 of a 7-bit word, each covering the positions whose binary index contains its bit: p1 covers 1, 3, 5, 7; p2 covers 2, 3, 6, 7; p4 covers 4, 5, 6, 7. After a single-bit error the three parity checks, read as a binary number (the *syndrome*), name the position of the faulty bit.

WORKED EXAMPLE

Hamming(7,4) with even parity for data 1011.

1. Place the data at positions 3, 5, 6, 7: d3 = 1, d5 = 0, d6 = 1, d7 = 1.
2. p1 = d3 xor d5 xor d7 = 1 xor 0 xor 1 = 0. p2 = d3 xor d6 xor d7 = 1 xor 1 xor 1 = 1. p4 = d5 xor d6 xor d7 = 0 xor 1 xor 1 = 0.
3. The codeword, positions 1 to 7, is 0110011.
4. Suppose position 5 flips in transit, giving 0110111. The p1 group (1, 3, 5, 7) now sums to an odd number, so c1 = 1; the p2 group (2, 3, 6, 7) is even, so c2 = 0; the p4 group (4, 5, 6, 7) is odd, so c4 = 1.
5. The syndrome c4c2c1 = 101 = 5, so flip position 5 back.

Result: syndrome 5 locates the error, and the corrected word is 0110011.

Reliable delivery on a link: ARQ

Some links, such as Wi-Fi or satellite, must repair losses themselves; the same ideas reappear in TCP (Section 3.9). In *Stop-and-Wait* the sender transmits one frame and waits for its acknowledgement; a timer expires if none arrives, and the frame is resent. It is simple but wasteful on long links. *Go-Back-N* (GBN) lets the sender have a *window* of N unacknowledged frames; the receiver accepts only the next expected frame, discards the rest and repeats the last acknowledgement, and on timeout the sender resends the lost frame and everything after it. *Selective Repeat* (SR) has the receiver buffer out-of-order frames and acknowledge each, so that only the lost frame is resent, at the price of buffers and more complex bookkeeping.

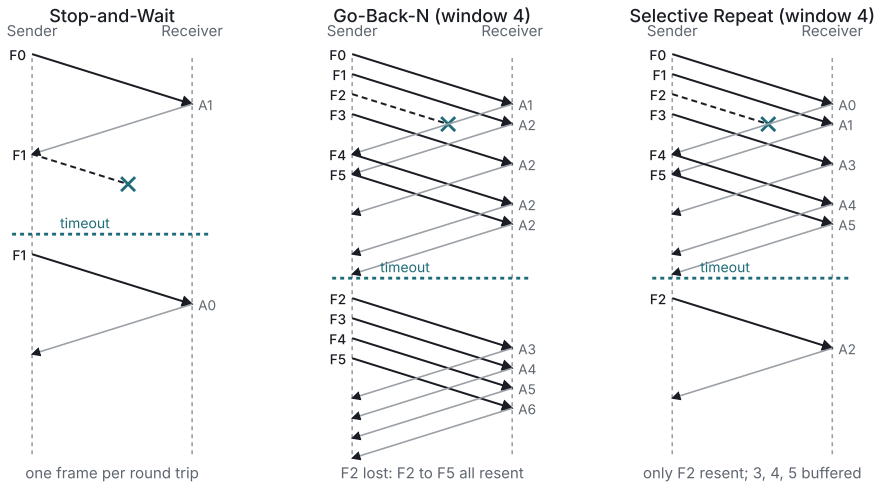


Figure 3.9 Three ARQ schemes after the loss of frame 2. Acknowledgements are shown by the next frame expected (cumulative) in Stop-and-Wait and Go-Back-N, and per frame (A3 means frame 3 received) in Selective Repeat. Go-Back-N discards out-of-order frames and repeats A2; Selective Repeat buffers them.

Let $a = T_p / T_t$ be the ratio of the one-way propagation time to the frame transmission time. The sender can send for T_t, then idles for a round trip of $2T_p$; a window that covers the whole $1 + 2a$ frame times keeps the link busy. Sequence numbers are written in n bits, so they wrap modulo 2^n, and the window must be small enough that an old frame is never mistaken for a new one: in GBN the window may be at most $2^n - 1$, whereas in SR it may be at most 2^{n-1}, because the sender's and receiver's windows must not overlap.

WORKED EXAMPLE

A 1 Mb/s link carries 1,000-bit frames, and the one-way propagation delay is 20 ms.

1. $T_t = 1{,}000 / 10^6 = 1$ ms, so $a = 20$ ms $/ 1$ ms $= 20$.
2. Stop-and-Wait efficiency: $\eta = 1 / (1 + 2a) = 1 / 41 \approx 2.4\%$. The link carries one frame in every 41 ms.
3. A window of $1 + 2a = 41$ frames gives 100% (and any window $W < 41$ gives $\eta = W / 41$).
4. Sequence numbers for $W = 41$: GBN needs $2^n - 1 \geq 41$, so $2^n \geq 42$ and $n = 6$. SR needs $2^{n-1} \geq 41$, so $n - 1 \geq 6$ and $n = 7$.

Result: 2.4%; a window of 41 gives full utilisation; 6 bits for GBN and 7 bits for SR.

TRAP

Claim: "A CRC corrects errors, and a longer CRC corrects more." *Why it is wrong:* a CRC is a detection code; the receiver can only tell that a frame is bad. *Correct statement:* Ethernet drops a frame whose FCS fails and leaves recovery to the higher layers (TCP resends); correction is the job of codes such as Hamming or the forward error correction in Wi-Fi and cellular.

INTERVIEW LENS

QUALCOMM · BROADCOM · NOKIA · ARISTA · CISCO

Qualcomm, Broadcom and Nokia interviews like the numerical problems: the CRC, the window limits and Stop-and-Wait efficiency (Q3.6 to Q3.8). Cisco and Arista ask about learning, flooding and the full CAM table (Q3.5), and Arista adds cut-through (Q3.9). A strong answer states the formula before the arithmetic and checks the units.

LAB

`LAB 3.3 · 25 MIN` Three small tasks. (i) In Packet Tracer, connect three PCs to one switch, run `show mac address-table` before and after a ping from PC0 to PC1, then `clear mac address-table dynamic`. (ii) In Wireshark, apply `eth.dst == ff:ff:ff:ff:ff:ff` and list the protocols that appear. (iii) In Python, check CRC-32.

```
Switch# show mac address-table dynamic
 Vlan    Mac Address       Type        Ports
    1    0001.6446.e5a2    DYNAMIC     Fa0/1
    1    0003.e4a1.b3c4    DYNAMIC     Fa0/2
```

```
import zlib
print(hex(zlib.crc32(b"123456789")))   # 0xcbf43926, the standard check value
```

What you should notice: before the ping the table is empty, and after it holds exactly the source addresses seen, one per port. The broadcast filter shows ARP and DHCP; broadcast frames are never sources, so they never appear in the table.

CHECK YOURSELF

1. A host sends a frame to an address that is not in the switch's table. What does the switch do, and what does it learn? [1]
2. Which Ethernet field makes the receiver pass the payload to IPv4 rather than ARP? [2]
3. For Stop-and-Wait, what happens to the efficiency if the propagation delay is doubled? [3]

3.4 IPv4 addressing, subnetting, VLSM and CIDR MUST · ■■□

An IPv4 address is a 32-bit number that identifies a network interface and tells a router where to send packets for it. It is written as four decimal numbers, one per byte (an *octet*), separated by dots: 172.16.45.200. Subnetting, the art of dividing an address block into smaller networks, is the most frequently examined skill in this book, and it is a mechanical one that anybody can perform in under a minute after a short drill.

BIG IDEA

The mask splits a 32-bit address into a network part and a host part, and subnetting is nothing more than moving that split to fit the number of hosts needed.

Network part, host part, mask

Every address has two parts: the *network* part, shared by all devices in the same subnet, and the *host* part, which distinguishes devices inside it. The *subnet mask* marks the split with ones over the network bits and zeros over the host bits. The *prefix length* is the count of ones, so 255.255.248.0 is written /21 (eight plus eight plus five ones). Two addresses are on the same subnet exactly when they agree on all network bits, which a host tests by bitwise AND with its own mask. In every subnet two addresses are special: the *network address* (host bits all zero) names the subnet itself, and the *broadcast address* (host bits all one) reaches every host in it. Hence a subnet with h host bits holds 2^h addresses and $2^h - 2$ usable hosts.

1. It floods the frame out of every port except the arrival port, and it learns the *source* address on the arrival port.↩
2. The EtherType field: 0x0800 for IPv4, 0x0806 for ARP.↩
3. a doubles, so η = 1 / (1 + 2a) roughly halves when a is large (1/41 becomes 1/81).↩

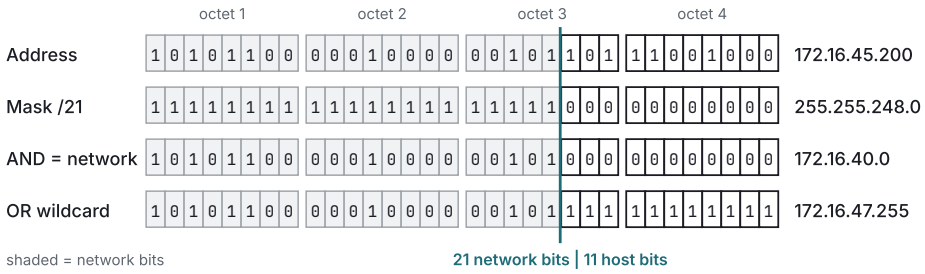


Figure 3.10 The 32-bit ruler for 172.16.45.200/21. The mask line falls inside the third octet, between bit 21 and bit 22; everything left of it is the network, everything right of it is the host.

WORKED EXAMPLE

Network, broadcast, range and hosts for 172.16.45.200/21, by the block-size method.

1. Mask: 21 ones is 255.255.248.0. The interesting octet is the third, where the mask is 248.
2. Block size = 256 − 248 = 8. Subnets in the third octet start at multiples of 8: 0, 8, … 40, 48.
3. The address has 45 in the third octet, and 40 ≤ 45 < 48, so the network address is 172.16.40.0.
4. The broadcast address is one below the next network: 172.16.47.255.
5. Usable range: 172.16.40.1 to 172.16.47.254. Host bits = 32 − 21 = 11, so hosts = $2^{11} - 2$ = 2,046.
6. Wildcard mask (the mask inverted, used in OSPF and ACLs) = 0.0.7.255.

Result: network 172.16.40.0, broadcast 172.16.47.255, 2,046 usable hosts, wildcard 0.0.7.255.

The master table below is worth memorising for the rows from /24 to /30, and the rest follow by the doubling rule: each bit added to the prefix halves the block.

Block-size trick — in the interesting octet, block = 256 − mask octet. List the multiples of the block; the address sits in one of them. Prefix /25 to /30 gives blocks 128, 64, 32, 16, 8, 4 in the fourth octet, and /17 to /23 gives the same series in the third.

Prefix	Mask	Wildcard	Block size (octet)	Addresses	Usable hosts
/8	255.0.0.0	0.255.255.255	1 (octet 1)	16,777,216	16,777,214
/12	255.240.0.0	0.15.255.255	16 (octet 2)	1,048,576	1,048,574
/16	255.255.0.0	0.0.255.255	1 (octet 2)	65,536	65,534
/17	255.255.128.0	0.0.127.255	128 (octet 3)	32,768	32,766
/18	255.255.192.0	0.0.63.255	64 (octet 3)	16,384	16,382
/19	255.255.224.0	0.0.31.255	32 (octet 3)	8,192	8,190
/20	255.255.240.0	0.0.15.255	16 (octet 3)	4,096	4,094
/21	255.255.248.0	0.0.7.255	8 (octet 3)	2,048	2,046
/22	255.255.252.0	0.0.3.255	4 (octet 3)	1,024	1,022
/23	255.255.254.0	0.0.1.255	2 (octet 3)	512	510
/24	255.255.255.0	0.0.0.255	1 (octet 3)	256	254
/25	255.255.255.128	0.0.0.127	128 (octet 4)	128	126
/26	255.255.255.192	0.0.0.63	64 (octet 4)	64	62
/27	255.255.255.224	0.0.0.31	32 (octet 4)	32	30

/28	255.255.255.240	0.0.0.15	16 (octet 4)	16	14
/29	255.255.255.248	0.0.0.7	8 (octet 4)	8	6
/30	255.255.255.252	0.0.0.3	4 (octet 4)	4	2
/31	255.255.255.254	0.0.0.1	2 (octet 4)	2	2 (RFC 3021)
/32	255.255.255.255	0.0.0.0	1 (octet 4)	1	1 (host route)

Classes, private ranges and special addresses

Originally the first bits of an address fixed its size: class A (first octet 1 to 126, /8), class B (128 to 191, /16), class C (192 to 223, /24), class D (224 to 239, multicast) and class E (240 to 255, reserved). The scheme wasted addresses, because an organisation needing 300 hosts had to take a class B with 65,534, and it bloated the routing tables. It was replaced in 1993 by *CIDR* (classless inter-domain routing), in which the prefix length is carried with the address and may be any value. The classes survive only in vocabulary and in some default behaviour of old protocols.

The special ranges below appear in many interview questions.

Range	Meaning
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private addresses (RFC 1918): not routed on the Internet, reused in every home and company
127.0.0.0/8	Loopback; 127.0.0.1 is the machine itself
169.254.0.0/16	Link-local (APIPA): self-assigned when DHCP fails
100.64.0.0/10	Carrier-grade NAT (shared address space)
0.0.0.0	“This host” at start-up, and the default route as 0.0.0.0/0
255.255.255.255	Limited broadcast, never forwarded by a router
224.0.0.0/4	Multicast
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Reserved for documentation; used in this book’s examples

Two prefixes break the rule “usable hosts = $2^h - 2$”. A /31 has two addresses and, by RFC 3021, both are usable on a point-to-point link, because such a link has no need of a network or broadcast address; this saves half of the addresses on every router-to-router link. A /32 names a single host or interface, and appears as a loopback address or a *host route*.

Splitting and VLSM

Fixed-length subnetting cuts a block into equal subnets. To obtain at least s subnets, borrow b host bits with $2^b \geq s$; the prefix grows by b.

> WORKED EXAMPLE
>
> **Divide 192.168.10.0/24 into six subnets.**
>
> 1. Six subnets need $2^b \geq 6$, so b = 3 (eight subnets). New prefix: 24 + 3 = /27, mask 255.255.255.224.
> 2. Block size = 256 − 224 = 32. Host bits = 5, so $2^5 - 2 = 30$ hosts per subnet.
> 3. The subnets start at .0, .32, .64, .96, .128, .160, .192, .224. For the second, network 192.168.10.32, usable .33 to .62, broadcast .63.

Result: eight /27 subnets of 30 hosts, block size 32; two are left over for growth.

Equal subnets waste addresses when the needs differ. *Variable-length subnet masking* (VLSM) gives each subnet the prefix it needs. The rule is to sort requirements from largest to smallest, allocate the largest first, and place each subnet on a multiple of its own block size so that blocks never straddle.

WORKED EXAMPLE

VLSM from 192.168.10.0/24 for LANs of 100, 50 and 25 hosts and two point-to-point WAN links.

1. 100 hosts: $2^7 - 2 = 126 \geq 100$, so /25. Allocate 192.168.10.0/25: usable .1 to .126, broadcast .127.
2. 50 hosts: $2^6 - 2 = 62 \geq 50$, so /26. Next free address .128: 192.168.10.128/26, usable .129 to .190, broadcast .191.
3. 25 hosts: $2^5 - 2 = 30 \geq 25$, so /27. Next free .192: 192.168.10.192/27, usable .193 to .222, broadcast .223.
4. Each WAN needs 2 addresses: /30. 192.168.10.224/30 (usable .225, .226) and 192.168.10.228/30 (usable .229, .230).
5. What remains is .232 to .255, which is 192.168.10.232/29 and 192.168.10.240/28, both spare.

Result: /25, /26, /27, /30, /30 with 24 addresses spare; allocating the small ones first would have forced the /25 to start at .128 and the plan would not fit.

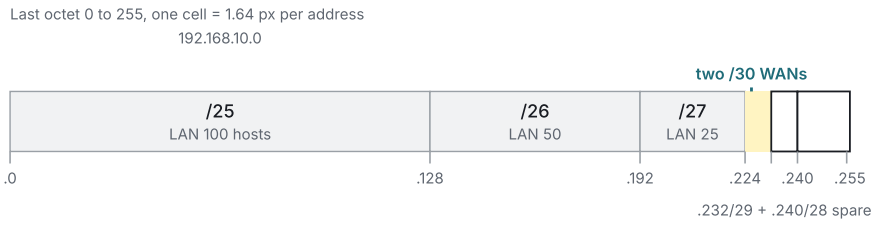


Figure 3.11 The VLSM plan as an address-space bar. The largest subnet is placed first, and each later one starts on a multiple of its own block size; what remains at the top is spare.

Summarisation and the same-subnet test

A router can announce one *summary* (supernet) instead of many routes when the subnets are contiguous and aligned. The method is to write the networks in binary, and find the longest common prefix.

WORKED EXAMPLE

Summarise 10.1.4.0/24 to 10.1.7.0/24, and test 10.1.5.0 to 10.1.8.0.

1. The third octets 4, 5, 6, 7 are 00000100, 00000101, 00000110, 00000111. They share the first six bits, 000001.
2. The prefix is 16 + 6 = 22: the summary is 10.1.4.0/22, which covers exactly 4 × 256 addresses and nothing else.
3. For 10.1.5.0 to 10.1.8.0 the third octets 5, 6, 7, 8 are 00000101 … 00001000, which share only the first four bits, 0000. The smallest single prefix is 10.1.0.0/20, which also claims 10.1.0.0 to 10.1.3.255 and 10.1.9.0 to 10.1.15.255. That over-claims addresses the router does not have.

> **Result: 10.1.4.0/22 summarises the first set exactly; the second cannot be summarised without over-claiming, and must be advertised as three routes: 10.1.5.0/24, 10.1.6.0/23 and 10.1.8.0/24.**

The same-subnet test is a cousin of this: two hosts are on one subnet if their addresses agree on the mask's network bits. For 10.10.10.130/26 and 10.10.10.190/26, the block size is 64 and both fall in 128 to 191, so they share a subnet; 10.10.10.193 falls in 192 to 255, so it does not. Interviewers at Cisco and Arista sometimes ask for this as code (Q3.14).

Finally, a word on how routers use all of these prefixes: when several routes match a destination, the router chooses the one with the *longest prefix*, the most specific. The algorithm and its data structure are treated in Pillar 1 (the trie), and the idea is applied in Section 3.8.

TRAP

Claim: "Every subnet has $2^n - 2$ usable hosts, so a /31 has none and a /32 has none." *Why it is wrong:* the "minus two" removes the network and broadcast addresses, which a /31 point-to-point link does not need (RFC 3021). *Correct statement:* a /31 gives two usable addresses on a point-to-point link, a /32 identifies one host, and $2^h - 2$ applies from /30 downwards.

INTERVIEW LENS

CISCO · AMAZON · JUNIPER · ARISTA · NOKIA

The question is "network, broadcast and range for X/Y without a calculator" (Q3.10), or a VLSM design (Q3.12). The examiner watches the method: state the block size first, then the multiples, then the answer. A strong candidate also mentions /31 and checks that the VLSM plan fits before presenting it.

LAB

`LAB 3.4 · 30 MIN` (i) On paper, solve the five drills below against a stopwatch; the target is 45 seconds each. (ii) In Python, check your answers with the standard library.

```
import ipaddress
n = ipaddress.ip_network("172.16.45.200/21", strict=False)
print(n.network_address, n.broadcast_address, n.num_addresses - 2,
n.hostmask)
```

```
172.16.40.0 172.16.47.255 2046 0.0.7.255
```

What you should notice: the library agrees with the block-size method. Then rewrite the same calculator with only integer shifts and masks (`mask = (0xFFFFFFFF << (32 - p)) & 0xFFFFFFFF`) and compare; that version is the one to write in an interview.

CHECK YOURSELF

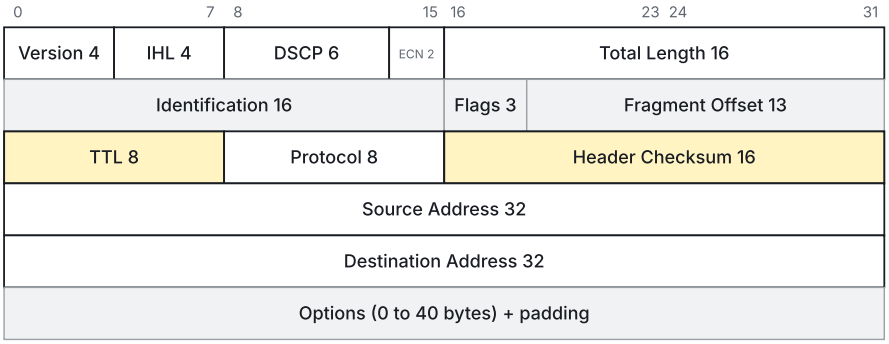
1. Network and broadcast of 10.20.30.77/26. [1]
2. Usable range of 192.168.5.130/25. [2]
3. How many usable hosts in 172.20.0.0/19? [3]
4. Are 200.1.1.65/27 and 200.1.1.95/27 on the same subnet? Is the second a valid host address? [4]
5. The smallest prefix that holds 500 hosts. [5]

3.5 The IPv4 header, fragmentation, MTU and ICMP MUST · ■■□

Every IPv4 packet starts with a header that holds what a router needs in order to forward it, and little else. Knowing the header field by field lets one read a hex dump, explain why traceroute works and diagnose the fault that makes some websites refuse to load.

> BIG IDEA
>
> The IP header carries just enough for each router to forward the packet, and when a link is too small for the packet, the packet is cut into fragments or, if forbidden to fragment, dropped with an error message.

The header, field by field



Yellow: rewritten at every router hop (TTL, hence the checksum).
Grey: fragmentation fields (set by the sender; changed only when a router fragments), and options.
Flags: bit 0 reserved, DF = do not fragment, MF = more fragments. IHL counts 32-bit words, so 5 means 20 bytes.

Figure 3.12 The IPv4 header, 32 bits wide, as it appears on the wire. The minimum is 20 bytes (five rows); the Total Length covers header and data.

Version is 4. *IHL* (header length) counts 32-bit words, so the usual value is 5. *DSCP* and *ECN* carry the traffic class and congestion marks (Pillar 4 and Section 3.9). *Total length* is the size of header plus data in bytes, so a packet is at most 65,535 bytes. *Identification*, *flags* and *fragment offset* support fragmentation, described below. *TTL* (time to live) is decremented by every router and the packet is discarded at zero, which stops packets circulating for ever in a routing loop. *Protocol* names the payload: 1 for ICMP, 6 for TCP, 17 for UDP, 89 for OSPF. The *header checksum* protects the header only; it is the 16-bit one's-complement sum of all header words, and because TTL changes, every router must recompute it. The two 32-bit addresses close the fixed part.

> WORKED EXAMPLE
>
> **Decode** `45 00 00 3c 1c 46 40 00 40 06 9c bc c0 a8 00 68 c0 a8 00 01`.
>
> 1. `45`: version 4, IHL 5, so the header is 20 bytes. `00`: DSCP 0, ECN 0.
> 2. `00 3c`: total length 60 bytes. `1c 46`: identification 0x1c46.
> 3. `40 00`: flags and offset. 0x4000 is 0100 0000 0000 0000 in binary, so DF = 1, MF = 0, offset 0.
> 4. `40`: TTL 64. `06`: protocol 6 (TCP). `9c bc`: header checksum.
> 5. `c0 a8 00 68` is 192.168.0.104 and `c0 a8 00 01` is 192.168.0.1, the source and destination.

1. Block 64, 77 lies in 64 to 127: network 10.20.30.64, broadcast 10.20.30.127.↩
2. Block 128: network .128, broadcast .255, usable 192.168.5.129 to 192.168.5.254.↩
3. 13 host bits, so $2^{13} - 2 = 8{,}190$.↩
4. Block 32, both lie in 64 to 95, so they share a subnet, but .95 is its broadcast address and is not a valid host address.↩
5. $2^9 - 2 = 510 \geq 500$ and $2^8 - 2 = 254$ is too few, so 9 host bits: /23.↩

6. Verify the checksum: add the ten 16-bit words, 0x4500 + 0x003c + 0x1c46 + 0x4000 + 0x4006 + 0x9cbc + 0xc0a8 + 0x0068 + 0xc0a8 + 0x0001 = 0x2FFFD. Fold the carry back in: 0xFFFD + 0x2 = 0xFFFF.

Result: a 60-byte TCP packet from 192.168.0.104 to 192.168.0.1, TTL 64, DF set; the one's-complement sum is 0xFFFF, so the header is intact.

A receiver verifies the checksum by summing all words including the checksum; an undamaged header gives 0xFFFF. To compute it, set the field to zero, sum, fold the carries and complement: here the sum without the checksum is 0x6343 and its complement is 0x9cbc.

Fragmentation, MTU and MSS

Every link has a *maximum transmission unit* (MTU), the largest IP packet it carries: 1,500 bytes on Ethernet and 1,492 on PPPoE. If a packet is larger, a router may *fragment* it, cutting the data into pieces that each carry a copy of the IP header. The Identification field is the same in all fragments, and the *fragment offset* gives the position of each piece in units of 8 bytes, so every piece but the last must have a data size that is a multiple of 8. The *MF* (more fragments) flag is 1 on all but the last. Only the destination reassembles, because routers on the path may take different routes for different fragments.

WORKED EXAMPLE

Fragment a 4,000-byte datagram (20-byte header, 3,980 bytes of data) for a link with MTU 1,500.

1. Room for data per fragment: 1,500 − 20 = 1,480 bytes. That is a multiple of 8 (1,480 = 185 × 8), so it can be used as is.
2. Number of fragments: 3,980 / 1,480 = 2.69, so three: 1,480 + 1,480 + 1,020 = 3,980 bytes.
3. Offsets in 8-byte units: 0, 1,480 / 8 = 185, 2,960 / 8 = 370.

Fragment	Data	Total Length	Offset field	MF
1	1,480	1,500	0	1
2	1,480	1,500	185	1
3	1,020	1,040	370	0

Result: three fragments with offsets 0, 185, 370; the last has MF = 0 and a Total Length of 1,040. If a later link has a smaller MTU, each fragment is fragmented again, with offsets relative to the original.

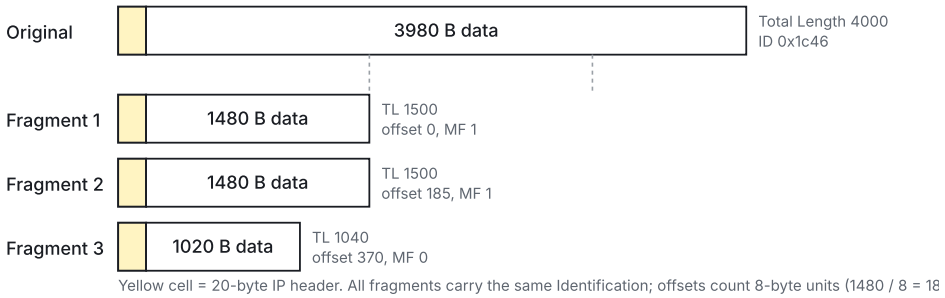


Figure 3.13 Fragmenting a 4,000-byte datagram for a link with MTU 1,500. Only the destination reassembles; routers on the way treat each fragment as a separate packet.

Fragmentation is costly: one lost fragment forces the loss of the whole datagram, and routers and firewalls dislike fragments. Hence modern hosts avoid it. They set the *DF* (don't fragment) flag and use *path MTU discovery* (PMTUD): a router that cannot forward a DF packet drops it and returns ICMP type 3 code 4, "fragmentation needed", with the next-hop MTU, and the sender then shrinks its packets. TCP expresses the result as the *maximum segment size* (MSS), the largest amount of TCP data in one segment, which excludes the IP and TCP headers: MSS = MTU − 20 − 20 = 1,460 on Ethernet. MTU and MSS are different: MTU includes the IP header and excludes the Ethernet header and FCS; MSS excludes both IP and TCP headers.

PMTUD fails when something blocks the ICMP message. The symptom is characteristic: the TCP handshake completes, small requests succeed, but large responses hang, because only full-size segments exceed the MTU. A common cause is a PPPoE or tunnel link with an MTU of 1,492 or less behind a firewall that drops all ICMP. The standard remedy is *MSS clamping*: the router rewrites the MSS option in passing SYN packets so that the endpoints never send oversize segments, for instance `ip tcp adjust-mss 1452` on a PPPoE interface (1,492 − 40).

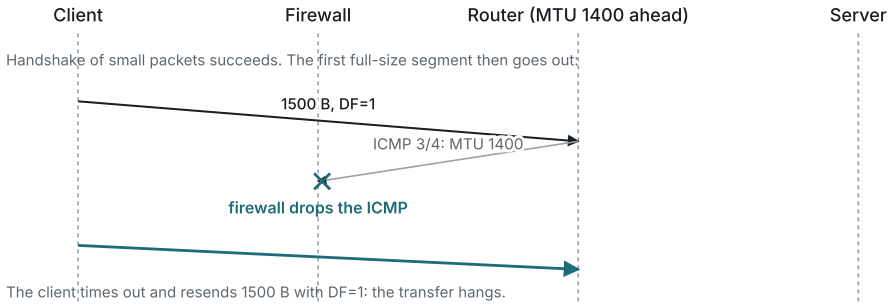


Figure 3.14 Path-MTU black hole. The router cannot forward a 1,500-byte packet with DF set onto its MTU-1,400 link, so it sends ICMP type 3 code 4; a firewall that blocks all ICMP discards it, and the client never learns to send smaller segments.

ICMP, ping and traceroute

The *Internet Control Message Protocol* (ICMP) carries errors and diagnostics, inside IP packets with protocol number 1. The types to know are: 0 echo reply and 8 echo request (ping); 3 destination unreachable, whose code says which kind (0 network, 1 host, 3 port, 4 fragmentation needed with DF set); 5 redirect; and 11 time exceeded.

Traceroute exploits the TTL. It sends probes with TTL 1, 2, 3 and so on. The first router decrements TTL 1 to zero, discards the probe and returns ICMP time exceeded, whose source address names that router; the next probe reaches the second router, and so forth until a probe reaches the destination, which answers differently. Linux sends UDP probes to ports from 33434 upwards, and the destination replies ICMP port unreachable (type 3 code 3); Windows `tracert` sends ICMP echo requests and the destination replies with an echo reply. A line of `* * *` means no answer arrived in time: the router may not generate time exceeded, rate-limit it or sit behind a firewall that drops the probe or its reply; it does not by itself mean that the path is broken.

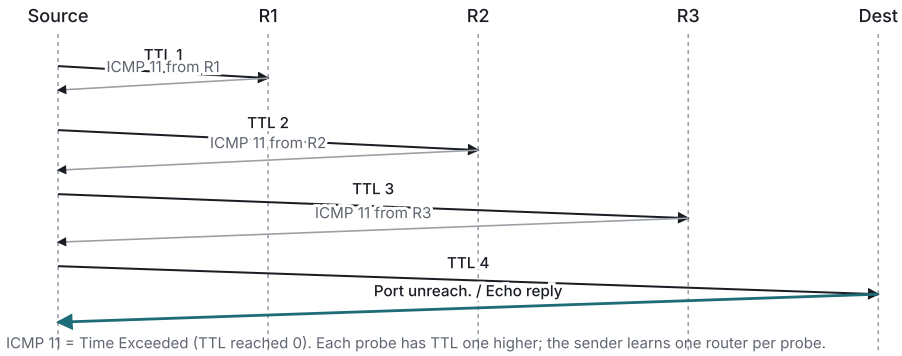


Figure 3.15 How traceroute works. A router that decrements TTL to zero discards the packet and returns an ICMP Time Exceeded message, revealing its own address. The last hop answers with a different message, because the probe finally arrives.

TRAP

Claim: "Fragment offset 1480 means the second fragment starts at byte 1480." *Why it is wrong:* the field counts 8-byte units, so a value of 1480 would mean byte 11,840. *Correct statement:* the second fragment of the example above has offset field 185, which is byte 1,480.

INTERVIEW LENS

AMAZON · MICROSOFT · CISCO · QUALCOMM · BROADCOM · JUNIPER

Q3.15 (fragmentation arithmetic) and Q3.16 (MTU versus MSS) are routine. Q3.17 is the scenario "ping works, the TCP handshake completes, the page never loads". A strong answer names PMTUD, says what to look for (ICMP 3/4 absent; retransmissions of the same large segment; a successful `ping -M do -s 1372`), and gives both fixes (allow ICMP type 3 code 4; clamp the MSS).

LAB

`LAB 3.5 · 20 MIN` On Linux, probe the path MTU with DF set. The ICMP header adds 8 bytes and the IP header 20, so a 1,472-byte payload is exactly a 1,500-byte packet.

```
$ ping -c1 -M do -s 1472 8.8.8.8
1480 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=9.8 ms
$ ping -c1 -M do -s 1473 8.8.8.8
ping: local error: message too long, mtu=1500
$ traceroute -n 8.8.8.8          # then: traceroute -I -n 8.8.8.8
 1  192.168.1.1  0.9 ms
 2  * * *
```

What you should notice: the largest unfragmented payload is 1,472 bytes, confirming the 1,500-byte MTU. In the traceroute, the `-I` (ICMP) option may succeed where the default UDP probes give `* * *` at a firewall; capture both in Wireshark with `ip.ttl == 1` to see the probes and the type-11 replies.

CHECK YOURSELF

1. A 1,500-byte packet (20-byte header) must cross a link with MTU 576. How many fragments, and what are their data sizes? [1]
2. Why is the header checksum recomputed at every hop? [2]
3. Which ICMP message makes PMTUD work, and what does it contain? [3]

3.6 ARP, DHCP and NAT: the glue protocols MUST · ■■□

Three small protocols turn a plugged-in laptop into a working Internet host. ARP finds the MAC address of the next hop, DHCP supplies the laptop's address, mask, gateway and DNS server, and NAT lets a private network share one public address. They are tested together because a single "packet walk" question exercises all three.

> BIG IDEA
>
> IP addresses say where a packet is going, MAC addresses say who gets it on the next link, and ARP, DHCP and NAT are the three devices that keep those two worlds consistent.

ARP

A host that wants to send an IP packet knows the destination IP address but needs a destination MAC address for the frame. If the destination is on the same subnet it needs the destination's MAC; otherwise it needs the MAC of the *default gateway*, the router that leads off the subnet. The *Address Resolution Protocol* (ARP) finds it: the host broadcasts "who has 192.168.1.1? tell 192.168.1.10" (opcode 1, destination MAC `ff:ff:ff:ff:ff:ff`), and only the owner replies by unicast (opcode 2) with its MAC. Both sides record the mapping in an *ARP cache*, which expires after a time that ranges from seconds on Linux to four hours on Cisco IOS.

Mnemonic — ARP asks "Who has this IP?" by broadcast and is answered by unicast; DHCP's first message, Discover, is also a broadcast, because the sender has nothing else to use.

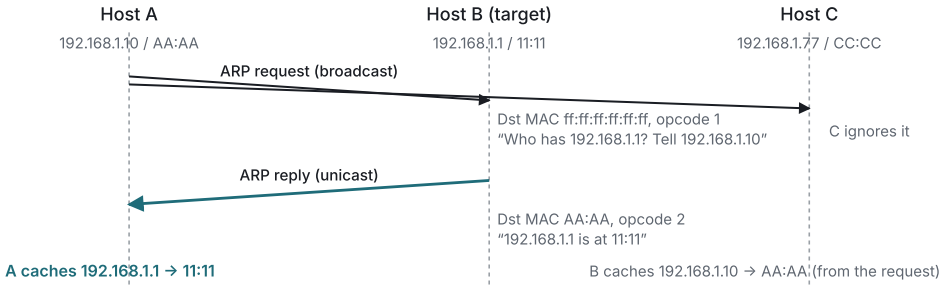


Figure 3.16 ARP resolves a next-hop IP address to a MAC address. The request is broadcast; only the owner of the target address replies, by unicast. MAC addresses are abbreviated to two bytes.

A *gratuitous ARP* is an ARP message in which the sender asks for its own address. A host sends it after configuring an address to detect a duplicate (if anyone replies, the address is taken) and to update other hosts' caches; a router in a first-hop redundancy group sends it on failover so that the switch and hosts learn that the virtual address has moved. *Proxy ARP* is a router answering ARP requests for addresses it can reach on behalf of the real owner. It lets a host with a wrong mask work, and so hides misconfiguration; it enlarges ARP tables and is normally disabled. *ARP spoofing*, the forging of replies, is treated in Pillar 9.

DHCP

DHCP (Dynamic Host Configuration Protocol) hands out addresses on lease. A new host has no address and does not know where the server is, so it must use broadcasts, and the exchange has four messages, remembered as DORA. The client broadcasts a **D**iscover (UDP source port 68, destination 67, source IP 0.0.0.0, destination 255.255.255.255). Every server replies with an **O**ffer holding an address and lease time. The client broadcasts a **R**equest naming the server it chose (option 54) and the address wanted (option 50), which also tells the other servers to withdraw. The chosen server confirms with an **A**cknowledgement, which carries the options: 1 subnet

mask, 3 default gateway, 6 DNS servers, 51 lease time. If no server answers, the client may give itself a link-local 169.254.x.y address, which signals a DHCP failure to anyone reading `ipconfig`.

The lease is renewed in stages. At T1, 50% of the lease, the client unicasts a Request to its server; if that fails, at T2, 87.5%, it broadcasts to any server; if the lease runs out, it starts again with Discover.

Broadcasts do not cross routers, but a company seldom has one DHCP server per subnet. A *relay agent*, configured on the router with `ip helper-address`, receives the broadcast, stores its own interface address in the *giaddr* field, and forwards the message by unicast to the server. The server uses giaddr to choose the right pool and sends its reply to that address, and the relay delivers it to the client.

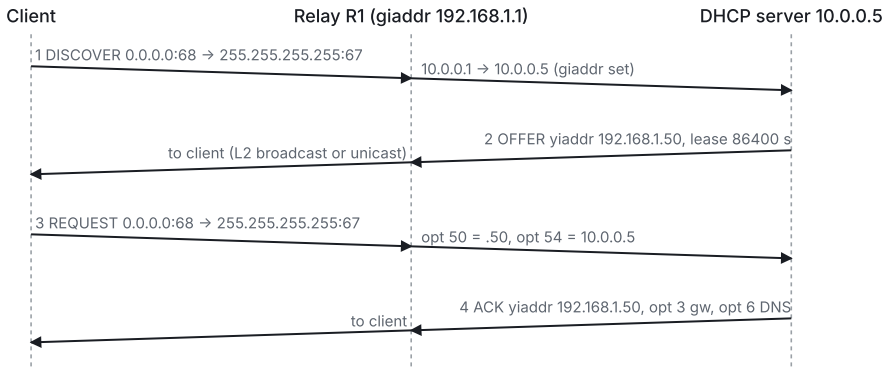


Figure 3.17 DHCP DORA through a relay. The client has no address and no idea where the server is, so Discover and Request are broadcasts (UDP 68 to 67). The relay turns them into unicasts, stamping its own interface address in the giaddr field so the server can pick a pool and reply.

NAT and PAT

Network address translation lets private addresses (Section 3.4) reach the Internet by rewriting the headers at a border device. In *static NAT* one private address is mapped to one public address permanently, which suits a server. In *dynamic NAT* the router maps hosts to addresses from a pool, one-to-one, for as long as they are in use. In *port address translation* (PAT, also *NAT overload*), by far the most common form, thousands of hosts share one public address, and the router tells their connections apart by the source port; it records each mapping in a *translation table*.

Cisco names the four addresses by two questions: is the host inside or outside, and is the address viewed as local (from inside) or global (from outside).

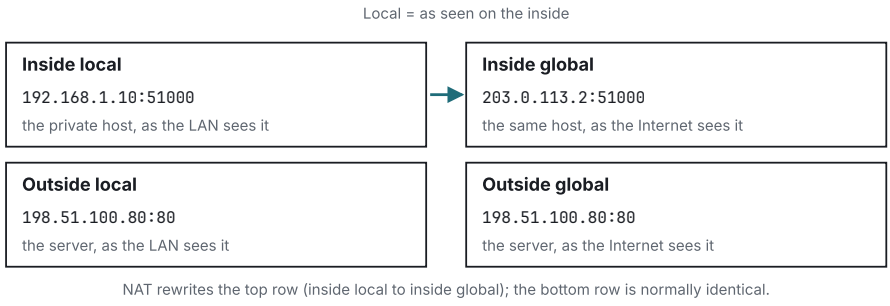


Figure 3.18 The four NAT address names. “Inside” and “outside” say where the host lives; “local” and “global” say from which side the address is viewed.

NAT has costs. A host behind it cannot be reached from outside unless a mapping exists, and so *port forwarding* (a static entry for one port) is needed to publish a server. Protocols that carry addresses in their payload, such as active FTP and SIP, break unless the router rewrites the payload too (an *application-level gateway*). Two hosts behind different NATs cannot open a connection to each other without help (see Q3.24). *Carrier-grade NAT* (CGNAT), which uses 100.64.0.0/10, adds a second NAT inside the provider. IPv6 (Section 3.7) removes the shortage that made all this necessary.

The packet walk

The following worked example is the canonical "show me every hop" question. PC-A (192.168.1.10/24, MAC AA:AA) sends an HTTP request to a web server at 198.51.100.80. R1 has an inside interface 192.168.1.1 (MAC 11:11) and an outside interface 203.0.113.2 (MAC 12:12) with PAT. The ISP router has 203.0.113.1 (MAC 21:21) towards R1 and 198.51.100.1 (MAC 22:22) towards the server, whose MAC is BB:BB. MAC addresses are abbreviated to two bytes.

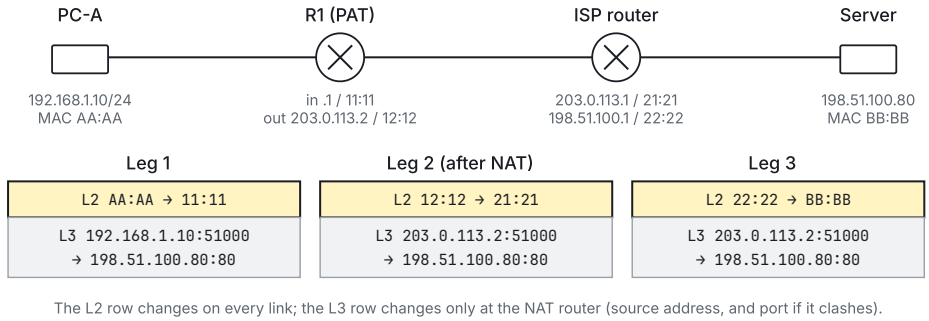


Figure 3.19 The packet walk of Section 3.6. The yellow row (MAC addresses) is rewritten at every hop; the grey row (IP address and port) is rewritten only where the NAT sits. The return packets use the same cards with source and destination swapped, and R1 translates the destination back.

WORKED EXAMPLE

Follow a TCP SYN from PC-A, source port 51000, to the server, port 80.

1. PC-A compares 198.51.100.80 with its own subnet 192.168.1.0/24. The destination is outside, so the frame must go to the default gateway, 192.168.1.1. PC-A has no cache entry and sends an ARP request; R1 replies "192.168.1.1 is at 11:11".
2. **Leg 1.** Frame: source MAC AA:AA, destination 11:11. Packet: 192.168.1.10:51000 to 198.51.100.80:80.
3. R1 decapsulates, decrements the TTL, looks up the route (default via 203.0.113.1) and applies PAT: the source 192.168.1.10:51000 becomes 203.0.113.2:51000, and the mapping is stored. R1 resolves 203.0.113.1 by ARP (21:21) if necessary.
4. **Leg 2.** Frame: 12:12 to 21:21. Packet: 203.0.113.2:51000 to 198.51.100.80:80.
5. The ISP router decrements TTL, finds 198.51.100.0/24 connected, and ARPs for the server (BB:BB).
6. **Leg 3.** Frame: 22:22 to BB:BB. Packet: unchanged, 203.0.113.2:51000 to 198.51.100.80:80.
7. The reply reverses the cards: server to 203.0.113.2:51000, then at R1 the table is consulted and the destination is rewritten to 192.168.1.10:51000, and the frame goes to AA:AA.

```
R1# show ip nat translations
Inside global      Inside local        Outside local     Outside global
203.0.113.2:51000  192.168.1.10:51000  198.51.100.80:80  198.51.100.80:80
```

Result: the MAC addresses change on every leg; the IP addresses are constant except at R1, which rewrites only the source address and port outward and the destination address and port inward. The TTL falls by one at each of the two routers.

TRAP

Claim: "The destination MAC stays the same from PC-A to the server, and the IP addresses change at every router." *Why it is wrong:* it is the other way round; a MAC address has meaning only on one link. *Correct statement:* L2 addresses are rewritten at every hop, L3 addresses end to end, except where NAT rewrites them.

INTERVIEW LENS

CISCO · AMAZON · GOOGLE · MICROSOFT · JUNIPER

Q3.23 is the walk above, usually with PAT. Interviewers push with "what if PC-A has no ARP entry for the gateway?", "what if the server is on the same subnet?" (no gateway: ARP for the server itself) and "what if two hosts use source port 51000?" (R1 picks another global port). Q3.20 and Q3.21 test DHCP with and without a relay.

LAB

`LAB 3.6 · 30 MIN` In Packet Tracer, configure R1 as DHCP server for 192.168.1.0/24 and PAT on its outside interface, then use `debug ip nat`. In Wireshark, filter `arp || dhcp` while renewing a lease with `sudo dhclient -r && sudo dhclient`; on Linux, view the cache with `ip neigh` and provoke a conflict with `arping`.

```
$ ip neigh show 192.168.1.1
192.168.1.1 dev eth0 lladdr 00:11:22:33:44:55 REACHABLE
1 DHCP Discover - Transaction ID 0x3a7c1e02
2 DHCP Offer    - Your IP address: 192.168.1.50
3 DHCP Request  - Requested IP: 192.168.1.50
4 DHCP ACK      - Lease 86400 s; Router 192.168.1.1
```

What you should notice: the first two packets use the Ethernet broadcast, and the DHCP transaction ID ties all four together. After the ACK, the client sends a gratuitous ARP; `show ip nat translations` fills only when traffic leaves the inside.

CHECK YOURSELF

1. Which MAC address does PC-A put as destination when it pings 8.8.8.8? [4]
2. A lease is 8 hours. When does the client first try to renew, and when does it start broadcasting? [5]
3. Why does a DHCP relay need to set giaddr? [6]

1. Data = 1,480; room per fragment = 576 − 20 = 556, rounded down to a multiple of 8 = 552. 1,480 / 552 = 2.68, so

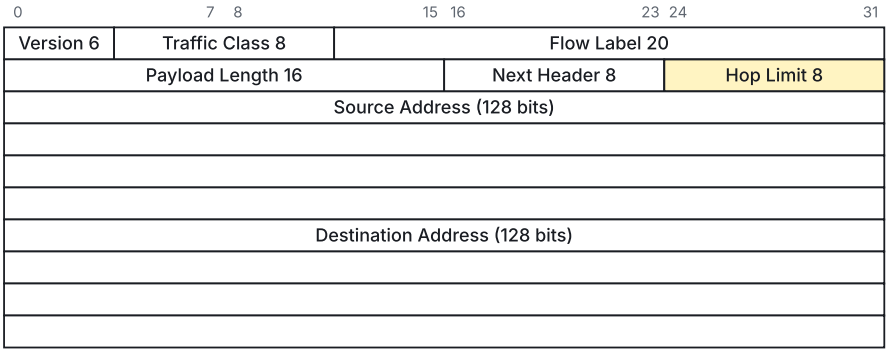
3.7 IPv6 essentials SHOULD · ■■□

IPv4 has about 4.3 billion addresses, and they ran out at the registries between 2011 and 2015. IPv6 widens the address to 128 bits, a space so large that every subnet can be given 2^{64} addresses. Its designers also simplified the header and removed several IPv4 mechanisms: there is no broadcast, no header checksum and no fragmentation by routers, and ARP is replaced by a neighbour discovery protocol built on ICMPv6. For Qualcomm, Nokia and Juniper this section is a Must.

> BIG IDEA
>
> IPv6 solves the shortage by making addresses 128 bits and solves the overhead by making the header fixed at 40 bytes, leaving everything else to extension headers and to ICMPv6.

Notation and address types

An IPv6 address is eight groups of four hexadecimal digits separated by colons. Two rules shorten it: leading zeros in a group may be dropped, and one run of consecutive all-zero groups may be replaced by `::`, once only, because two would make the address ambiguous. If two runs tie, compress the longest, and the leftmost if they are equal.



Fixed 40 bytes. Hop Limit (yellow) is the IPv4 TTL. No header checksum, no fragmentation fields, no options: those moved to extension headers chained by Next Header (6 = TCP, 17 = UDP, 58 = ICMPv6, 44 = Fragment).

Figure 3.20 The IPv6 header. Compared with IPv4 (Figure 3.12), the checksum, the header length, the three fragmentation fields and the options are gone; the addresses are four times larger and a Flow Label is added.

Type	Prefix	Notes
Global unicast (GUA)	2000::/3	Routable on the Internet; sites receive a /48 and subnets are /64
Unique local (ULA)	fc00::/7 (in practice fd00::/8)	Private, like RFC 1918
Link-local	fe80::/10	Valid on one link only; every interface has one
Multicast	ff00::/8	ff02::1 all nodes, ff02::2 all routers; replaces broadcast
Loopback, unspecified	::1/128, ::/128	Like 127.0.0.1 and 0.0.0.0
Documentation	2001:db8::/32	Used in this book

An *anycast* address is one assigned to several machines, and a packet goes to the nearest; it is how public DNS services and CDNs are reached.

three fragments: 552, 552, 376.↩

2. Because TTL, which is inside the header, changes at every hop.↩
3. ICMP type 3 code 4 (fragmentation needed and DF set); it carries the next-hop MTU and the start of the original packet.↩
4. The MAC address of its default gateway, learned by ARP, not any address for 8.8.8.8.↩
5. T1 = 50%, so after 4 hours it unicasts to the server; T2 = 87.5%, so after 7 hours it broadcasts to any server.↩
6. It tells the server which subnet the client is on, so that the right pool is used, and where to send the reply.↩

WORKED EXAMPLE

Compress `2001:0DB8:0000:0000:0000:FF00:0042:8329`, and `2001:db8:0:0:1:0:0:1`.

1. Drop leading zeros: `2001:DB8:0:0:FF00:42:8329`. The longest run of zero groups is three groups; replace it with `::`.
2. Lower case is conventional: `2001:db8::ff00:42:8329`.
3. In `2001:db8:0:0:1:0:0:1` there are two runs of two zero groups. They tie, so the left-most is compressed: `2001:db8::1:0:0:1`.

Result: 2001:db8::ff00:42:8329 and 2001:db8::1:0:0:1. Compressing both runs, as in `2001:db8::1::1`, is illegal.

Interface identifiers and SLAAC

Hosts in the same /64 need to choose the remaining 64 bits, the *interface identifier*. The older method, EUI-64, derives it from the MAC address: split the MAC in the middle, insert `FF:FE`, and invert the universal/local bit of the first byte.

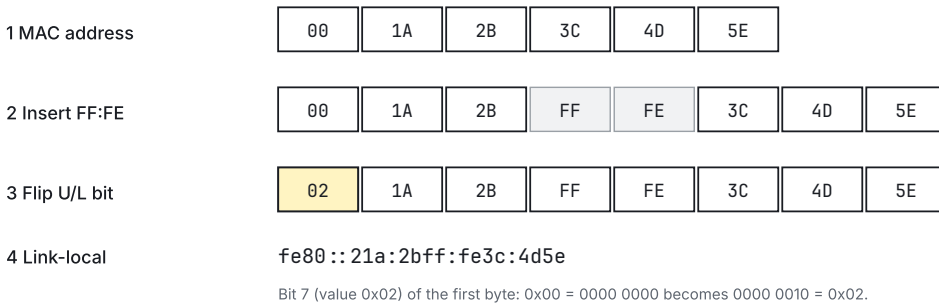


Figure 3.21 Building an EUI-64 interface identifier from a MAC address. The middle step splits the MAC between the vendor and device halves; the flip inverts the universal/local bit, so that a burned-in address is marked "universal" with a 1.

WORKED EXAMPLE

Link-local address and solicited-node multicast for MAC 00:1A:2B:3C:4D:5E.

1. Split the MAC after 3 bytes and insert FFFE: 00:1A:2B:FF:FE:3C:4D:5E.
2. Invert bit 7 of the first byte: 0x00 becomes 0x02. Interface ID = 021A:2BFF:FE3C:4D5E.
3. Link-local: `fe80::21a:2bff:fe3c:4d5e` (leading zero of 021A dropped).
4. The *solicited-node multicast* address is `ff02::1:ff` followed by the last 24 bits of the unicast address: `ff02::1:ff3c:4d5e`.
5. Its Ethernet multicast MAC is `33:33` followed by the last 32 bits of the IPv6 address: `33:33:ff:3c:4d:5e`.

Result: fe80::21a:2bff:fe3c:4d5e; solicited-node ff02::1:ff3c:4d5e; MAC 33:33:ff:3c:4d:5e.

Because EUI-64 exposes the hardware address and lets one be tracked between networks, modern systems use random, changing identifiers (*privacy extensions*) for global addresses. *Stateless address autoconfiguration* (SLAAC) lets a host build an address without a server: it forms a link-local address, checks for a duplicate (*DAD*, duplicate address detection), sends a Router Solicitation, receives a Router Advertisement carrying the prefix, and combines prefix and identifier. Two flags in the advertisement send the host to DHCPv6: M (managed) means "get the add-

ress by stateful DHCPv6", and O (other) means "build the address by SLAAC, but get DNS and other options by stateless DHCPv6". The prefix itself carries an A flag that permits SLAAC.

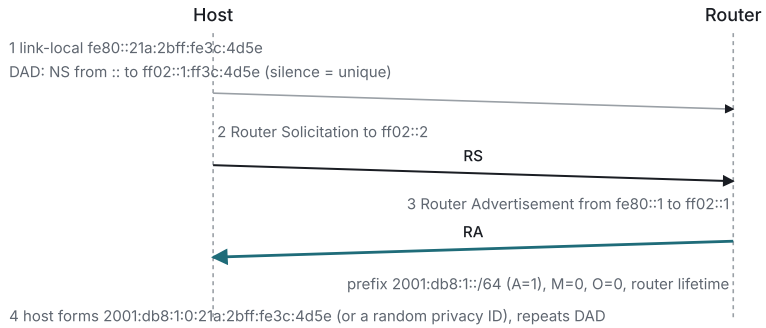


Figure 3.22 SLAAC. The host needs no server: it derives a link-local address, checks it for duplicates (DAD), asks for a router, and combines the advertised /64 prefix with its own interface identifier.

Neighbour discovery

NDP (neighbour discovery protocol) is a set of five ICMPv6 messages: router solicitation (type 133), router advertisement (134), neighbour solicitation (135), neighbour advertisement (136) and redirect (137). Address resolution is by 135 and 136: instead of a broadcast the host sends the neighbour solicitation to the *solicited-node multicast* address of the target, which only the few hosts sharing the last 24 bits listen to, and the target replies with its MAC in a neighbour advertisement. The network interrupts uninterested hosts far less than ARP does.

Header, fragmentation and transition

The 40-byte header has a fixed layout, and everything optional is moved into *extension headers* chained by the Next Header field. Routers never fragment IPv6 packets: only the source may, using a Fragment extension header, so a router with a too-large packet drops it and returns ICMPv6 "packet too big" (type 2). Every IPv6 link must carry at least 1,280 bytes. Hence ICMPv6 must not be blocked wholesale: unlike IPv4, without "packet too big", path MTU discovery fails, and without NDP nothing works at all.

Networks migrate in three ways. *Dual stack* runs both protocols side by side and is the preferred approach. *Tunnels* carry IPv6 packets inside IPv4 packets across an IPv4-only stretch. *NAT64* with *DNS64* lets IPv6-only hosts reach IPv4-only servers: DNS64 synthesises an AAAA record from the server's A record using the prefix 64:ff9b::/96, and NAT64 translates the packet.

WORKED EXAMPLE

Split 2001:db8:acad::/48 into /64 subnets.

1. The prefix borrows 64 − 48 = 16 bits, so there are 2^{16} = 65,536 subnets.
2. The subnet ID is the fourth group. The first three subnets are 2001:db8:acad:0::/64, 2001:db8:acad:1::/64 and 2001:db8:acad:2::/64.
3. The last is when the subnet ID is all ones: 2001:db8:acad:ffff::/64.

Result: 65,536 subnets, from 2001:db8:acad::/64 to 2001:db8:acad:ffff::/64. Subnetting IPv6 is easier than IPv4: it moves on hexadecimal digit boundaries (4 bits).

TRAP

Claim: "The EUI-64 transformation inverts the last bit of the MAC address." *Why it is wrong:* the bit flipped is the universal/local bit, bit 7 of the first byte (value 0x02), not the last bit of

the address. *Correct statement:* 00:1A:… becomes 02:1A:…, and `::` may be used only once in an address.

INTERVIEW LENS

QUALCOMM · NOKIA · JUNIPER · CISCO · AMAZON · GOOGLE

Q3.26 to Q3.29 are mechanical (compress, EUI-64, solicited-node, SLAAC versus DHCPv6), and each is solved in the Problem Set. Q3.30, “why don’t IPv6 routers fragment, and what breaks if ICMPv6 is blocked?”, rewards the candidate who links it back to PMTUD in Section 3.5.

LAB

`LAB 3.7 · 20 MIN` On Linux, inspect the addresses, neighbours and the router advertisement; in Wireshark use the filter `icmpv6.type == 134`.

```
$ ip -6 addr show dev eth0
    inet6 2001:db8:1:0:21a:2bff:fe3c:4d5e/64 scope global dynamic mngtmpaddr
    inet6 fe80::21a:2bff:fe3c:4d5e/64 scope link
$ ping -c1 ff02::1%eth0
```

What you should notice: the link-local address has the EUI-64 form derived above (some systems print a random identifier instead). The ping to ff02::1 is answered by every IPv6 host on the link, which is the multicast replacement for a broadcast ping.

CHECK YOURSELF

1. How many bits are in an IPv6 subnet ID if a site has a /48 and uses /64 subnets? [1]
2. Which multicast address does a router listen to for solicitations? [2]
3. Why does IPv6 need no ARP? [3]

3.8 Routing concepts in one page MUST · ■■□

Forwarding is the per-packet action of a router: look up the destination and send the packet out of one interface. *Routing* is the process that fills the table used for the lookup, either by hand (static routes) or by protocols that exchange information. The routing table is searched by *longest-prefix match*: of all entries whose prefix matches the destination, the router chooses the one with the longest mask, and the default route 0.0.0.0/0 matches everything with length zero, so it is chosen last. The data structure for a fast lookup is the trie of Pillar 1; this section needs only the rule.

WORKED EXAMPLE

Look up four destinations in a table with 10.0.0.0/8 to R1, 10.1.0.0/16 to R2, 10.1.1.0/24 to R3 and 0.0.0.0/0 to R4.

1. 10.1.1.5 matches /8, /16 and /24, and the longest is /24: next hop R3.
2. 10.1.2.5 matches /8 and /16 only (the third octet is not 1): R2.
3. 10.2.0.1 matches only /8: R1.
4. 11.0.0.1 matches nothing but the default: R4.

Result: R3, R2, R1, R4.

Routing protocols divide into two families by what they tell their neighbours. A *distance-vector* protocol sends its neighbours a list of destinations with a distance to each, and every router computes shortest paths by the Bellman-Ford rule; a *link-state* protocol floods the description of

every router's links to all routers, and each runs Dijkstra's algorithm on the full map. The algorithms themselves are solved in Pillar 1 (shortest paths) and the protocols are configured in Pillar 4 (RIP, OSPF, EIGRP, BGP).

	Distance-vector (RIP)	Link-state (OSPF, IS-IS)
Knowledge	Neighbours' distances only	Complete topology map
Algorithm	Bellman-Ford	Dijkstra (SPF)
Traffic	Periodic table to neighbours	Flooded updates on change
Convergence	Slow; count-to-infinity risk	Fast
Memory, CPU	Low	Higher

BGP, the protocol between organisations, is a third kind, *path-vector*: each route carries the list of autonomous systems it crossed, which prevents loops. An *autonomous system* (AS) is a network under one administration with an AS number; routing inside an AS uses an *interior* protocol and routing between ASes uses BGP.

The weakness of distance-vector is the *count to infinity*. A router believes a neighbour's claim that a network is reachable, and when the real route breaks, two routers may each point to the other for a route that no longer exists, adding one to the metric at each exchange.

WORKED EXAMPLE

Network N hangs off router C in the chain A, B, C; the link to N fails.

1. Before failure the metrics to N are C = 1, B = 2, A = 3.
2. C loses N but, before it announces anything, B's regular update says "N is 2 away". C accepts: N = 3 via B.
3. B hears from C that N is 3 and updates to 4; C then hears 4 and takes 5; the values grow 5, 6, 7 and so on until RIP's infinity of 16, when all three declare N unreachable.
4. With *split horizon*, B does not advertise N back to C, because B learned it from C; C hears nothing and N expires at once. *Poison reverse* sends N with metric 16 back instead. *Triggered updates* spread the bad news immediately, and *hold-down timers* ignore worse news for a while.

Result: without safeguards the metric climbs one step per exchange up to 16; split horizon removes the loop between two neighbours. Larger loops still need hold-down and triggered updates, one reason why link-state protocols replaced RIP.

CHECK YOURSELF

1. A table holds 192.168.0.0/16 and 192.168.4.0/22. Which entry forwards 192.168.5.9? [4]
2. Which family floods the topology, and which algorithm does each router run? [5]

1. 64 − 48 = 16 bits.↩
2. ff02::2, the all-routers address.↩
3. Neighbour discovery (ICMPv6 types 135 and 136, sent to the solicited-node multicast address) resolves IPv6 addresses to MAC addresses.↩

3.9 Transport: UDP, TCP, flow control and congestion control MUST · ■■■

The network layer delivers packets from one machine to another, but it does not say which program should receive them, nor does it promise that they arrive, in order, once. The transport layer adds exactly these things. It is also the layer where interviewers spend the most time, because a TCP question can start at a header field and end at the behaviour of a data-centre fabric. This section is therefore the longest in the pillar: it covers ports, UDP, the TCP header, connection set-up and teardown, the state machine and reliability here, and flow control and congestion control in the next pages.

> BIG IDEA
>
> UDP is a thin service that adds ports and a checksum to IP, while TCP builds a reliable, ordered, flow-controlled and congestion-controlled byte stream on top of an unreliable network by using sequence numbers, acknowledgements, timers and two windows.

Ports and sockets

An IP address names a machine; a *port number*, a 16-bit value, names a program on it. A *socket* is one endpoint, an address-and-port pair, and a TCP connection is identified by the four values (source IP, source port, destination IP, destination port), the *4-tuple* (with the protocol, the 5-tuple). Hence one server port such as 443 can serve millions of clients at once, because every client gives a different tuple. Ports below 1,024 are *well-known* (80 for HTTP, 443 for HTTPS, 53 for DNS, 22 for SSH; the full list is in the Revision Spread), 1,024 to 49,151 are *registered*, and 49,152 to 65,535 are *ephemeral*, picked by the operating system for the client side of a connection (Linux uses 32,768 to 60,999 by default). The sockets API that programs use is treated in Pillar 7.

UDP

The *User Datagram Protocol* is the smallest possible transport: the 8-byte header, shown with the TCP header below, holds the two ports, a length and a checksum that covers the data and parts of the IP header. There is no connection, no retransmission, no ordering and no congestion control: a datagram is sent and may arrive, or not. That is exactly right when a late packet is worthless, or when the application has its own recovery: DNS queries (one question, one answer), voice and video calls, online games and QUIC, the modern transport that the web is moving to (Section 3.10). UDP also suits broadcast and multicast, which a connection cannot.

4. 192.168.4.0/22 covers 192.168.4.0 to 192.168.7.255, so it is the longest match.↩
5. Link-state; Dijkstra's shortest-path-first.↩

The TCP header

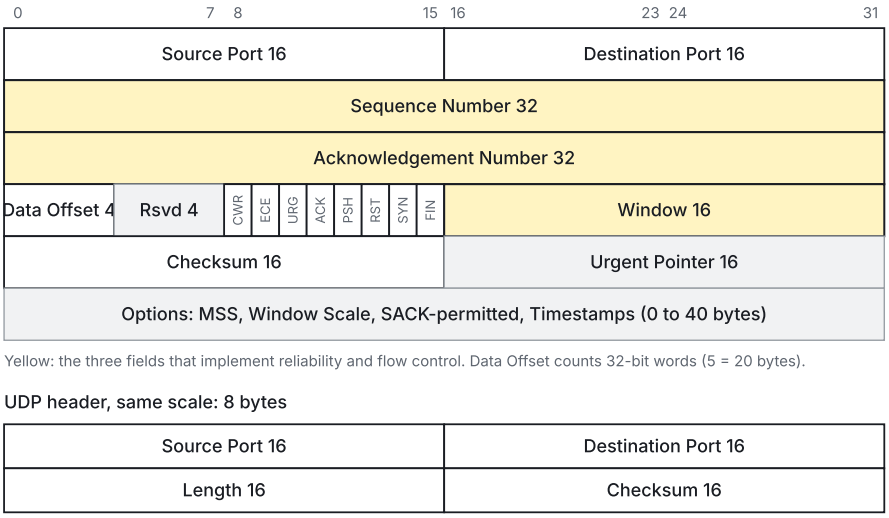


Figure 3.23 The TCP header (20 bytes plus options) above the UDP header (8 bytes). The extra twelve bytes of TCP buy sequence numbers, acknowledgements, a window and control flags.

Source and *destination ports* identify the endpoints. The *sequence number* is the byte-stream position of the first data byte in the segment; the *acknowledgement number* is the next byte the sender of the acknowledgement expects, so it confirms everything before it (a *cumulative* acknowledgement). *Data offset* gives the header length in 32-bit words, since options vary. The eight flag bits are: CWR and ECE, used for explicit congestion notification; URG, nearly obsolete; ACK, set on every segment after the first SYN; PSH, a hint to deliver to the application now; RST, which aborts the connection; SYN, which synchronises sequence numbers at the start; and FIN, which ends one direction. The *window* field is the receiver's advertised free buffer in bytes (Section 3.9, flow control). The checksum covers header, data and an IP pseudo-header.

The *options* appear mostly on SYN segments: the *maximum segment size* (MSS, the largest data per segment, usually 1,460), *window scale* (a shift count that multiplies the window, needed on fast long paths), *SACK-permitted* (enables selective acknowledgements) and *timestamps* (used to measure round-trip time and to protect sequence numbers on fast links).

Connection set-up: the three-way handshake

TCP connections have a beginning and an end, and both sides keep *state* (variables such as the next sequence number, the window and timers) while the connection lasts. The *three-way handshake* creates that state at both ends. The client sends a SYN with its *initial sequence number* (ISN); the server answers with a SYN,ACK that acknowledges the client's ISN and announces its own; the client acknowledges that in turn. The ISN is not zero but a random number: it prevents old segments of a former connection from being accepted by the new one, and it makes it hard for an attacker to guess sequence numbers and inject data (RFC 6528).

WORKED EXAMPLE

Handshake, data and teardown with ISNs 1000 (client) and 5000 (server).

1. SYN: seq = 1000. The SYN consumes one sequence number, so the next client byte is 1001.
2. SYN,ACK: seq = 5000, ack = 1001. The server's SYN also consumes one number.

3. ACK: seq = 1001, ack = 5001. The connection is established on both sides.
4. The client sends 500 bytes, seq = 1001, ack = 5001. The server replies with a pure ACK, seq = 5001, ack = 1501 (1,001 + 500). The pure ACK consumes no sequence number.
5. The client closes: FIN,ACK with seq = 1501. The server ACKs with ack = 1502, because the FIN counts as one byte.

Result: SYN (1000), SYN,ACK (5000, ack 1001), ACK (1001, ack 5001); after 500 bytes the acknowledgement is 1501; the FIN advances it to 1502. A SYN and a FIN each use one sequence number; a pure ACK uses none.

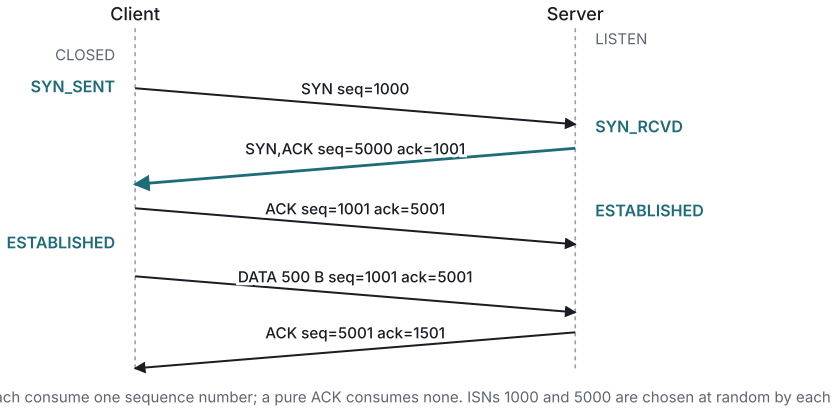


Figure 3.24 The three-way handshake with the initial sequence numbers of the worked example. The client is in SYN_SENT until the SYN,ACK arrives; the server is in SYN_RCVD until the final ACK arrives.

Two messages would not be enough, because the server could not tell whether its own SYN,ACK had arrived, nor distinguish a fresh SYN from a delayed duplicate of an old one. The third segment confirms to the server that the client heard it, and that the request is current. If two hosts send SYNs at the same moment, a *simultaneous open*, they still end up with one connection after exchanging SYN,ACKs.

Why three? — A and B each need to be sure the other heard their ISN. Two messages leave one direction unconfirmed; four are not needed, because the server combines its ACK and SYN in one segment.

A server must remember each half-open connection, and this is exploited by a *SYN flood*: an attacker sends a stream of SYNs from forged addresses and never answers the SYN,ACKs, filling the queue of half-open connections until real clients are refused. *SYN cookies* defeat it by keeping no state: the server encodes a hash of the 4-tuple, a secret and a coarse timestamp (and a code for the MSS) in the sequence number of its SYN,ACK, and forgets the connection. A legitimate client's ACK returns that number plus one; the server verifies the hash and rebuilds the connection. Only genuine clients cost the server any memory, at the price that some options of the SYN, such as the window scale, may be lost.

Teardown and TIME_WAIT

Each direction of a connection is closed separately: a FIN means "I have no more to send", and the other side acknowledges it. The result is the *four-way teardown* of the figure below, in which the second and third messages may be merged. The side that has received a FIN but not yet closed itself can keep sending (a *half-close*), which is why the server's ACK and FIN are separate in general.

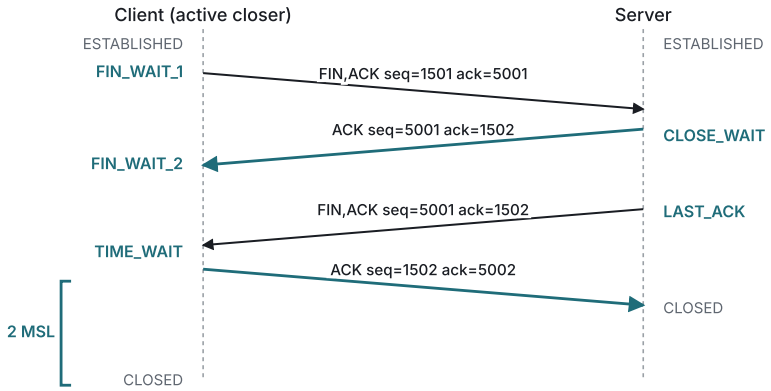


Figure 3.25 Four-way teardown. Each direction is closed separately, so the server may keep sending between its ACK and its own FIN (half-close). The side that sends the first FIN waits 2 MSL in TIME_WAIT before releasing the connection.

The side that sends the first FIN, the *active closer*, ends in the *TIME_WAIT* state and waits for twice the *maximum segment lifetime* (2 MSL; Linux uses a fixed 60 seconds) before it forgets the connection. There are two reasons for this. If the final ACK is lost, the peer retransmits its FIN, and the active closer must still be there to acknowledge it. And any stray segments of the old connection, still wandering the network, must die before the same 4-tuple is reused, or they could be mistaken for data of the new connection.

The passive closer sits in *CLOSE_WAIT* between receiving the FIN and calling `close()`. That state lasts only as long as the application takes to notice the closed connection; a server with thousands of CLOSE_WAIT sockets therefore has a bug in its own code, which forgets to close sockets, and no network setting will cure it. TIME_WAIT, in contrast, is normal; it becomes a nuisance only on a busy client or proxy, which opens many short connections to one destination and runs out of ephemeral ports (about 28,000 ports and 60 seconds allow roughly 470 new connections per second to a single destination). The remedies are connection reuse, more source addresses or ports and, on Linux, `tcp_tw_reuse` for outgoing connections.

A TCP stack sends an *RST* (reset) when it receives a segment for which no connection exists, which includes a SYN to a port where nothing listens, so that the sender fails at once instead of timing out. An RST is also sent when an application aborts (closing a socket with unread data, or with a zero linger time), when a firewall or an intrusion-prevention box injects one to kill a flow, and when a segment violates the connection state, such as data arriving after the peer has crashed and restarted.

The state machine

The behaviour above is summarised by a *state machine* of eleven states. Reading the figure along the solid line gives the client’s life, and along the dashed line the server’s.

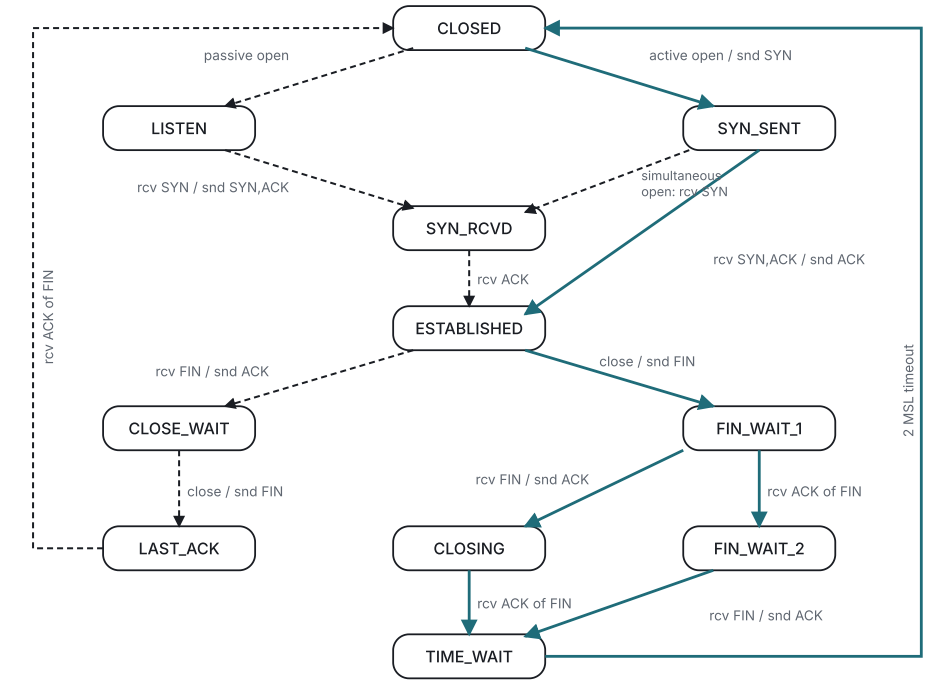


Figure 3.26 The TCP state machine (the eleven states of RFC 9293). Arrows are labelled "event / action". The three states that matter in operations are ESTABLISHED, CLOSE_WAIT (the application has not closed) and TIME_WAIT (the active closer waiting 2 MSL).

> TRAP
>
> *Claim:* "TIME_WAIT is on the client, and a server never has it." *Why it is wrong:* TIME_WAIT belongs to whichever side closes first, the *active closer*. *Correct statement:* a web server that closes idle connections itself accumulates TIME_WAIT sockets, while a client that closes first has them on its side; the passive closer goes through CLOSE_WAIT and LAST_ACK.

Reliability: acknowledgements, timers and fast retransmit

TCP numbers bytes, and the receiver acknowledges the next byte it wants. To avoid an acknowledgement for every segment, a receiver may delay the ACK, typically until a second full segment arrives or about 40 ms (Linux) to 200 ms (older systems) have passed. If a segment is lost, later segments arrive out of order and each one makes the receiver repeat the same acknowledgement, a *duplicate ACK*.

The sender starts a timer when it sends data. The *retransmission timeout* (RTO) must exceed the round-trip time (RTT), or packets are resent needlessly, but not by much, or recovery is slow. It is therefore computed from measured samples by the method of Jacobson and Karels (RFC 6298). With sample R, a smoothed average SRTT and a mean deviation RTTVAR, the update is RTTVAR = (1 − β) RTTVAR + β |SRTT − R|, then SRTT = (1 − α) SRTT + α R, with α = 1/8 and β = 1/4, and RTO = SRTT + 4 RTTVAR. *Karn's algorithm* adds that samples from retransmitted segments are ignored, since it is unclear which transmission the ACK answers; and after each timeout the RTO is doubled (*exponential backoff*).

> WORKED EXAMPLE
>
> **RTO update from SRTT = 100 ms and RTTVAR = 25 ms with samples 120 ms and 80 ms.**

1. First sample R = 120: |SRTT − R| = 20; RTTVAR = 0.75 × 25 + 0.25 × 20 = 23.75 ms; SRTT = 0.875 × 100 + 0.125 × 120 = 102.5 ms. RTO = 102.5 + 4 × 23.75 = 197.5 ms.
2. Second sample R = 80: |102.5 − 80| = 22.5; RTTVAR = 0.75 × 23.75 + 0.25 × 22.5 = 23.44 ms; SRTT = 0.875 × 102.5 + 0.125 × 80 = 99.69 ms. RTO = 99.69 + 4 × 23.44 = 193.4 ms.

Result: RTO = 197.5 ms, then 193.4 ms. RTTVAR is updated first, using the old SRTT; real stacks then clamp the RTO to a minimum, 200 ms on Linux (RFC 6298 recommends 1 s).

Waiting for a timeout is slow, so TCP also uses *fast retransmit*: when three duplicate ACKs arrive, the sender retransmits the missing segment at once. The number three is a compromise. One or two duplicates can be produced by simple reordering in the network, which would cause needless retransmissions; waiting for more costs time on small windows. *Selective acknowledgements* (SACK) refine this: the receiver lists, in an option, the blocks of data it holds above the hole, so that the sender repairs several holes in one round trip and resends nothing that has arrived.

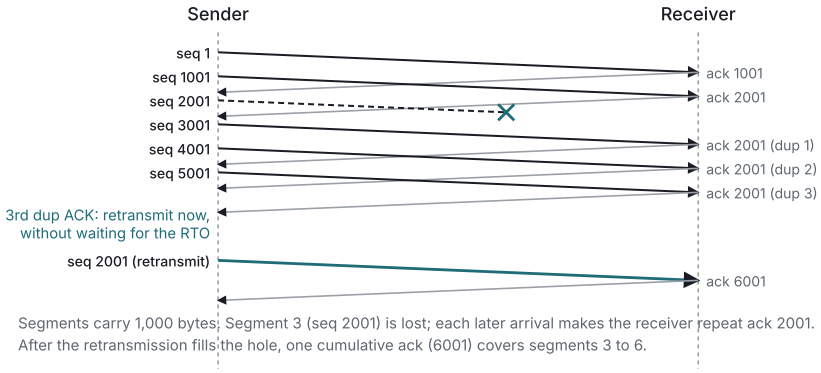


Figure 3.27 Fast retransmit. Three duplicate acknowledgements are strong evidence of loss rather than reordering, so the sender repairs the hole after about one round trip instead of waiting for the retransmission timeout.

INTERVIEW LENS

CISCO · AMAZON · MICROSOFT · GOOGLE · QUALCOMM · JUNIPER

Q3.32 to Q3.35 and Q3.42 (handshake, why three-way, TIME_WAIT, CLOSE_WAIT, SYN flood) are among the most reliably asked questions in the whole book, and each can be answered in thirty seconds. The follow-up decides the grade: “which side holds TIME_WAIT?” and “what would you check on a server with thousands of CLOSE_WAIT?” separate those who have memorised a diagram from those who have debugged a service. Q3.44 asks when a stack sends RST.

LAB

`LAB 3.9a · 25 MIN` Capture a handshake and a close with `tcpdump`, then provoke an RST and a CLOSE_WAIT leak.

```
$ sudo tcpdump -nn -S -i any 'tcp port 8080'
IP 10.0.2.15.49822 > 203.0.113.10.8080: Flags [S],  seq 1000, win 64240, op-
tions [mss 1460,sackOK,TS val 1 ecr 0,nop,wscale 7]
IP 203.0.113.10.8080 > 10.0.2.15.49822: Flags [S.], seq 5000, ack 1001, win
65160, options [mss 1460,sackOK,TS,nop,wscale 7]
IP 10.0.2.15.49822 > 203.0.113.10.8080: Flags [.],  ack 5001, win 502
$ sudo hping3 -S -p 9 -c 1 203.0.113.10        # a closed port
IP 203.0.113.10.9 > 10.0.2.15.2412: Flags [R.], seq 0, ack 1, win 0
$ ss -tan state close-wait                      # server that never calls
close()
Recv-Q Send-Q  Local Address:Port   Peer Address:Port
1      0       10.0.2.15:8080       10.0.2.99:51234
```

(The sequence numbers are shortened; real ISNs are random 32-bit values and `-S` shows them absolute.) **What you should notice:** `S` is SYN, `S.` is SYN,ACK and `.` is ACK. The closed port answers a SYN with `R.`, the RST,ACK described above. In Wireshark, *Statistics → TCP Stream Graphs → Stevens* plots sequence number against time; the preference *Relative sequence numbers* reproduces 0 and 1 instead of the random ISNs. The CLOSE_WAIT socket appears as soon as the client closes and stays until the server process closes its end (Pillar 7).

CHECK YOURSELF

1. A client with ISN 7000 sends a SYN, and later 300 bytes. What ACK does the server send after the data? [1]
2. Which endpoint is in TIME_WAIT if the server calls `close()` first? [2]
3. Three duplicate ACKs arrive for ack = 4001. What does the sender do? [3]

Flow control: protecting the receiver

Flow control stops a fast sender from overrunning a slow receiver. The receiver advertises in every segment how many more bytes its buffer can take, the *receive window* (rwnd), and the sender never has more than that many bytes unacknowledged. Because the sender may transmit a whole window before it needs an acknowledgement, the scheme is termed a *sliding window*: the left edge advances with each acknowledgement, and the right edge advances with it.

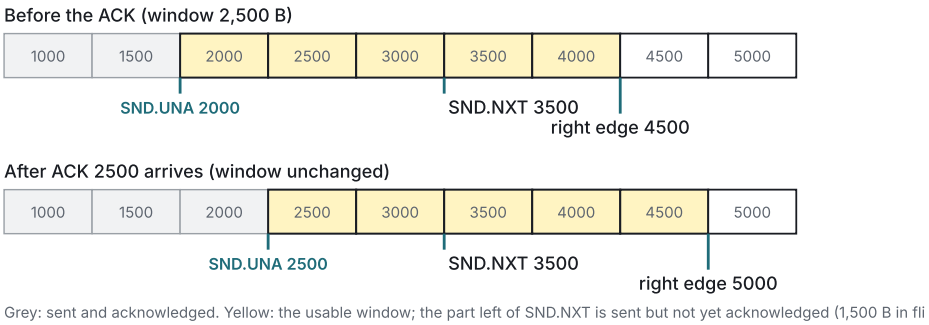


Figure 3.28 The sliding window on the byte stream, drawn in 500-byte cells. An acknowledgement slides the left edge right, and the right edge follows by the same amount if the receiver's advertised window is unchanged.

If the receiver's buffer fills completely it advertises a window of zero and the sender stops. To avoid deadlock should the window update be lost, the sender then sends tiny *window probes* on a *persist timer*, with growing intervals. A related fault is the *silly window syndrome*, in which a

1. The SYN used 7000, the data starts at 7001, so the ACK is 7001 + 300 = 7301.↩
2. The server, since it is the active closer.↩
3. It retransmits the segment that starts at byte 4001 immediately (fast retransmit) without waiting for the RTO, and enters fast recovery.↩

slow receiver advertises tiny windows and a sender fills them with tiny segments, so that headers dominate: the receiver should not advertise a window until it can offer a full segment or half its buffer, and the sender should not send tiny segments when it can wait.

Nagle's algorithm is the sender-side rule. While any sent data is unacknowledged, a small write is held back and merged with later writes until the acknowledgement arrives or a full segment accumulates. This saves many small packets, but combined with the receiver's delayed ACK it can stall an interactive application: if the program writes a request in two small pieces, the second piece waits for the ACK of the first, and the ACK waits, for up to 40 to 200 ms, for the receiver to delay it. The usual fix is to disable Nagle with the socket option `TCP_NODELAY`, or to assemble the request in one write.

Throughput limits and the window scale

Since at most one window is in flight per round trip, a connection cannot exceed throughput ≤ W / RTT, where W is the smaller of the windows. The window needed to keep a path full is the *bandwidth-delay product* (BDP), the capacity R times the round-trip time: the amount of data "in the pipe". The 16-bit window field holds at most 65,535 bytes, which the *window scale* option multiplies by 2^S, with s up to 14, giving windows to about 1 GiB.

WORKED EXAMPLE

A 64 KB window on a 100 ms path; then a 1 Gb/s path of the same length.

1. Ceiling with W = 65,535 B and RTT = 100 ms: 65,535 × 8 / 0.1 = 5.24 Mb/s, whatever the link speed.
2. For a 1 Gb/s path, BDP = 10^9 × 0.1 / 8 = 12.5 MB.
3. Required scale: 65,535 × 2^S ≥ 12.5 × 10^6, so 2^S ≥ 191, so s = 8 (2^8 = 256 gives 16.8 MB; s = 7 gives only 8.4 MB).
4. For a 1 Gb/s path with RTT 190 ms (Mumbai to Virginia): BDP = 23.75 MB, so s = 9 (33.5 MB).

Result: 5.24 Mb/s without scaling; a shift of 8 for the 100 ms path and 9 for the 190 ms path. The receive buffer, not just the option, must be that large.

Congestion control: protecting the network

Flow control protects the receiver; *congestion control* protects the network. In October 1986 the early Internet suffered *congestion collapse*: when queues filled, packets were dropped, senders retransmitted, the retransmissions filled the queues further, and throughput fell by a factor of a thousand. Van Jacobson's remedy, adopted in 1988, is that the sender keeps a second window, the *congestion window* (cwnd), which it grows when things go well and cuts when loss signals trouble. The sender may have at most min(cwnd, rwnd) bytes in flight. Loss is treated as the signal that a queue somewhere has overflowed, and cwnd is counted here in segments (MSS) for clarity.

The algorithm has two phases. In *slow start* cwnd starts small (1 segment in the classical description, 10 in modern stacks) and increases by one segment for every ACK, which doubles it every round trip, an exponential growth that is "slow" only compared with sending a whole window immediately. When cwnd reaches the *slow-start threshold* ssthresh, *congestion avoidance* begins, and cwnd grows by one segment per round trip, a linear increase. On loss the window is cut. This pattern, *additive increase, multiplicative decrease* (AIMD), is what lets many flows share a link fairly: each increase is equal, but each cut removes a share proportional to the flow's size, so unequal flows converge.

The variants differ in how they react to loss. *Tahoe*, the original, treats every loss alike: ssthresh = cwnd / 2, cwnd = 1, slow start again. *Reno* distinguishes the signals. A timeout is serious, so it acts like Tahoe. Three duplicate ACKs mean that later segments are still arriving, so the network is not dead: Reno sets ssthresh = cwnd / 2 and cwnd = ssthresh + 3 (the three duplicate ACKs each mark a segment that has left the network), retransmits the lost segment, and inflates cwnd by one per further duplicate ACK; when the new ACK arrives, cwnd deflates to ssthresh and congestion avoidance continues. This is *fast recovery*. *NewReno* improves recovery when several segments of one window are lost: it stays in fast recovery until all of them are acknowledged, treating a *partial ACK* as a sign of the next hole.

WORKED EXAMPLE

Trace cwnd (in segments) with initial cwnd = 1 and ssthresh = 16, when loss is detected at cwnd = 20.

1. Slow start, one value per RTT: RTT 1 to 5 give 1, 2, 4, 8, 16. At 16 = ssthresh the phase changes.
2. Congestion avoidance, +1 per RTT: RTT 6 to 9 give 17, 18, 19, 20.
3. Loss at cwnd = 20, so ssthresh = 20 / 2 = 10.
4. *Tahoe* (and any variant after a timeout): cwnd = 1, then slow start 1, 2, 4, 8, then 10 (capped at ssthresh), then 11, 12, 13 and so on.
5. *Reno* after three duplicate ACKs: cwnd = 10 + 3 = 13 during recovery, then 10 when the new ACK arrives, and then 11, 12, 13 … in congestion avoidance.

Result: ssthresh = 10 in both cases; Tahoe restarts from 1 and needs four extra round trips to reach the threshold, while Reno resumes at 10. Reno halves; a timeout resets to 1. Do not conflate the two.

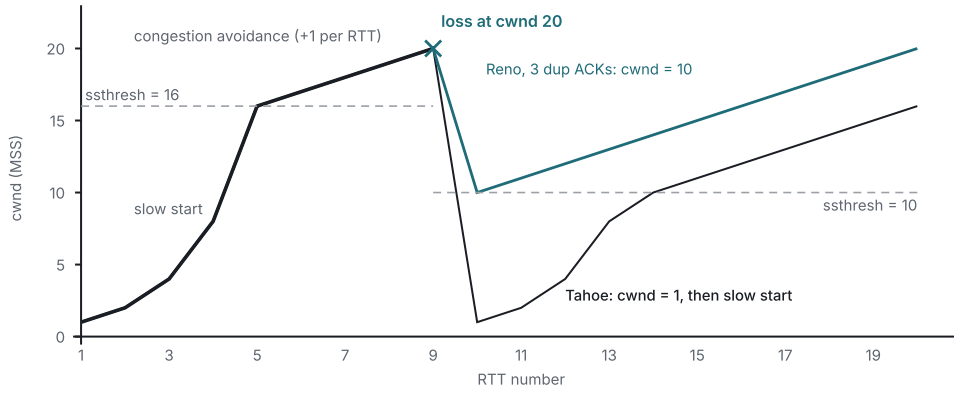


Figure 3.29 The congestion window over 20 round trips. Both variants share the first nine RTTs. At the loss, Tahoe (and any variant on a timeout) restarts from 1; Reno, having seen three duplicate ACKs, halves the window and continues in congestion avoidance.

Both Tahoe and Reno are loss-based and increase by one segment per round trip, which is too timid on a long fast path, where thousands of round trips are needed to recover after a single loss. Two modern algorithms address this in different ways.

CUBIC, the default in Linux (and in Windows and macOS), replaces the linear increase by a cubic function of the time since the last loss: $W(t) = C\,(t - K)^3 + W_{max}$, where W_{max} is the window when the loss occurred and $K = (W_{max}\,\beta / C)^{1/3}$ is the time needed to climb back to it. After a loss the window is cut to $0.7\,W_{max}$; it then rises quickly (the concave part), flattens around W_{max}

where the last loss occurred, and finally accelerates again to look for more capacity (the convex part). Since the function depends on time and not on the number of ACKs, flows with different round-trip times grow at the same rate, which is fairer than Reno.

BBR (bottleneck bandwidth and round-trip propagation time), from Google, drops the idea that loss is the signal. It builds a model of the path from two measurements: the maximum delivery rate seen recently, BtlBw, and the minimum round-trip time, RTprop. It then *paces* packets at about BtlBw and limits the data in flight to a small multiple of the BDP = BtlBw × RTprop, thus operating at the corner of the figure below, with a full pipe and an empty queue, instead of filling the buffer as loss-based algorithms do. Its phases are Startup (exponential search for BtlBw), Drain, ProbeBW (a cycle of eight round-trip phases with gains 1.25, 0.75 and six of 1, to detect more bandwidth) and ProbeRTT (a brief reduction every ten seconds to measure RTprop). BBR performs well on paths with random loss, where CUBIC collapses, and keeps queues short. Its first version, however, ignores loss and can take an unfair share when sharing a shallow buffer with CUBIC flows, which is why versions 2 and 3 add responses to loss and to ECN marks.

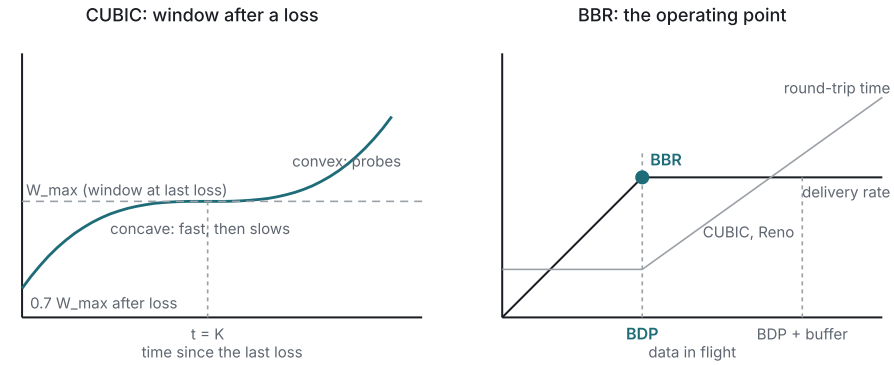


Figure 3.30 Left: CUBIC grows the window as a cubic function of the time since the last loss, flat around W_max. Right: delivery rate (black) saturates at the bottleneck bandwidth, while round-trip time (grey) starts to grow once the pipe is full; BBR operates at the corner, loss-based algorithms at the buffer limit.

Two further mechanisms lie close to this topic. *Explicit congestion notification* (ECN) lets a router mark a packet instead of dropping it when its queue grows; the receiver echoes the mark in the ECE flag and the sender reduces its window and answers with CWR, which signals congestion without loss. *Active queue management* (AQM), such as RED and CoDel, makes routers drop or mark early, so that queues stay short; it is taken up in Pillar 4's treatment of quality of service.

	UDP	TCP	QUIC
Runs over	IP	IP	UDP (port 443)
Connection set-up	None	1 RTT (SYN), plus TLS 1 to 2 RTT	1 RTT with TLS 1.3 built in; 0-RTT on resumption
Reliability and order	None	Whole stream, in order	Per stream
Congestion control	None (application's duty)	In the kernel	In user space (CUBIC, BBR)
Head-of-line blocking	Not applicable	Yes, across the stream	Per stream only
Header	8 bytes	20 to 60 bytes	UDP + QUIC header; payload encrypted

Typical use	DNS, voice, games	Web (HTTP/1.1, 2), email, SSH	HTTP/3

TRAP

Claim: “Flow control and congestion control are the same window, and TCP sends min(rwnd, cwnd) because the smaller one is safer.” *Why it is wrong:* the two windows protect different things, the receiver’s buffer (advertised in the header) and the network’s queues (estimated by the sender and never transmitted). *Correct statement:* the effective window is min(rwnd, cwnd); the slower of the two limits is the binding constraint, and each is changed by a different event.

INTERVIEW LENS

GOOGLE · MICROSOFT · AMAZON · QUALCOMM · CISCO

Q3.36 to Q3.41 are the TCP performance cluster: effective window, the cwnd trace (Q3.37), CUBIC versus BBR (Q3.38), the throughput ceiling (Q3.39), fast retransmit and SACK (Q3.40) and the Nagle stall (Q3.41). Google and Microsoft look for a mental model: which signal does the algorithm use (loss, or delay and bandwidth), and what does it assume about the buffer? Q3.43 is a coding variant: simulate a sliding-window sender; the solution is in the Problem Set.

LAB

`LAB 3.9b · 40 MIN` Compare CUBIC and BBR on an emulated path with two Linux network namespaces. The path has 50 ms of delay and 1% random loss.

```
sudo ip netns add c1; sudo ip netns add c2
sudo ip link add v1 type veth peer name v2
sudo ip link set v1 netns c1; sudo ip link set v2 netns c2
sudo ip -n c1 addr add 10.9.0.1/24 dev v1; sudo ip -n c2 addr add 10.9.0.2/24
dev v2
sudo ip -n c1 link set v1 up; sudo ip -n c2 link set v2 up
sudo ip netns exec c1 tc qdisc add dev v1 root netem delay 50ms loss 1%
sudo ip netns exec c2 iperf3 -s -D
sudo modprobe tcp_bbr
sudo ip netns exec c1 iperf3 -c 10.9.0.2 -t 20 -C cubic
sudo ip netns exec c1 iperf3 -c 10.9.0.2 -t 20 -C bbr
sudo ip netns exec c1 ss -ti dst 10.9.0.2      # during a run
```

```
bbr wscale:7,7 rto:252 rtt:51.2/1.1 mss:1448 cwnd:1043 bytes_retrans:...
pacing_rate 790Mbps delivery_rate 640Mbps
```

What you should notice: with 1% loss the Mathis estimate for a Reno-like flow, (MSS / RTT) × 1.22 / √p = (11,680 bit / 0.05 s) × 12.2, is about 2.9 Mb/s, and CUBIC lands in the same low range, whereas BBR is not governed by loss and reaches a large share of the veth capacity. Plot cwnd from repeated `ss -ti` samples to see CUBIC’s sawtooth against BBR’s flat, model-based window; set loss to 0 and watch the two converge. Exact numbers depend on the machine.

CHECK YOURSELF

1. cwnd = 12 segments, rwnd = 8 segments. How many segments may be in flight? [1]
2. After a timeout at cwnd = 30, what are ssthresh and cwnd? After three duplicate ACKs at cwnd = 30 under Reno? [2]
3. A path has 200 Mb/s and RTT 80 ms. What is its BDP, in bytes? [3]
4. Name the signal each of Reno, CUBIC and BBR uses. [4]

3.10 The application layer: DNS, HTTP, TLS and QUIC SHOULD

· ■■□

The application layer is where users' programs meet the network. This section covers the four protocols that every web request touches: DNS to find the address, HTTP to ask for the page, TLS to protect it and QUIC to carry it faster. Google, Microsoft and Amazon treat it as a Must; for the other companies it is background for the capstone that follows.

> BIG IDEA
>
> DNS turns names into addresses through a hierarchy of servers that cache their answers, and the HTTP, TLS and transport protocols beneath a web page have been reorganised, step by step, to need fewer round trips.

DNS

The *Domain Name System* is a distributed, hierarchical database that maps names to data, chiefly addresses. The hierarchy is read right to left: the *root* (written as a final dot) delegates to *top-level domains* such as `com.`, which delegate to organisations such as `google.com.`, whose own servers hold the records. Three roles take part. A *stub resolver*, inside every host, sends a single question to a configured resolver (the address is normally supplied by DHCP). A *recursive resolver*, run by the ISP, a company or a public service such as 8.8.8.8 or 1.1.1.1, does the work of finding the answer and caches it. An *authoritative server* holds the records of a zone and answers for it. The client's query is *recursive* ("find it for me"); the resolver's queries to the hierarchy are *iterative* ("tell me who to ask next"), because each server replies with an answer or a *referral*.

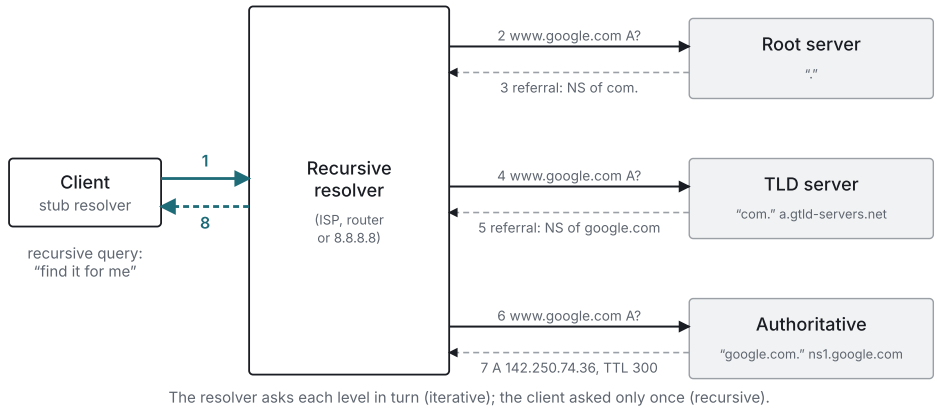


Figure 3.31 DNS resolution of www.google.com on a cold cache. Solid arrows are queries, dashed arrows replies. The resolver caches every reply for its TTL, so the next client is answered at step 1 without steps 2 to 7. The A record is an example value.

Every record has a type and a *time to live* (TTL), the number of seconds that a cache may keep it. The types to know are in the table. Caches also store failures: a *negative answer* (the name does not exist) is cached for a time derived from the zone's SOA record, which saves repeated queries for mistyped names.

Type	Holds	Note
A, AAAA	IPv4, IPv6 address	The usual answer

1. min(12, 8) = 8 segments.↩
2. Timeout: ssthresh = 15, cwnd = 1. Reno on three duplicate ACKs: ssthresh = 15, cwnd = 15 after recovery (18 during it).↩
3. 200 × 10^6 × 0.08 / 8 = 2 MB.↩
4. Reno and CUBIC react to loss (CUBIC with a cubic growth law after the cut); BBR uses measured bandwidth and minimum RTT.↩

CNAME	Alias to another name	Cannot coexist with other records on the same name
NS	Names of a zone's authoritative servers	Used in referrals
MX	Mail servers with a priority	
PTR	Name for an address	Reverse lookup, under in-addr.arpa
TXT	Free text	SPF, domain verification
SOA	Zone parameters, negative-cache time	One per zone
SRV, CAA	Service location; certificate authorities allowed	

DNS uses UDP port 53 for ordinary queries, because a question and answer fit in one datagram each. It falls back to TCP port 53 when an answer is too large and the server sets the truncation flag, and always for zone transfers between servers. The extension mechanism EDNS0 allows UDP answers beyond the original 512 bytes. DNSSEC adds digital signatures to records so that a resolver can detect forgery; DNS over TLS (port 853) and DNS over HTTPS (port 443) encrypt the queries. A CNAME cannot be placed at a zone apex such as `example.com.`, since the apex must also hold SOA and NS records, and a CNAME may not share its name with other records. Hosting providers therefore offer a flattened alias (ALIAS or ANAME), which the authoritative server resolves itself and returns as ordinary A records.

WORKED EXAMPLE

Follow `dig +trace www.google.com`, then watch the cache.

1. The resolver starts with the root hints and asks a root server. The reply is a referral: the NS records of `com.`, with a TTL of 172800 s (two days), such as `a.gtld-servers.net`.
2. It asks a `com.` server, which refers it to the NS records of `google.com.` (`ns1.google.com` and others), again with a TTL of 172800 s.
3. It asks `ns1.google.com`, which is authoritative and returns the A record, with a short TTL such as 300 s, because Google may change it often.
4. Query `dig www.google.com @8.8.8.8` twice, ten seconds apart: the first answer shows TTL 300 or less, and the second shows about 10 less: the resolver is serving its cache and decrementing the TTL.

Result: three iterative steps for a cold cache, none for a warm one. The answer is cached for 300 s; the NS records are cached for two days, so later lookups under `.com` skip the root and TLD steps.

HTTP

The *Hypertext Transfer Protocol* is a request-response protocol over TCP (or QUIC). A request names a *method* (GET fetches, POST submits, PUT replaces, DELETE removes, HEAD fetches headers only), a path and headers; the response carries a *status code* and a body. The code classes are 1xx informational, 2xx success (200), 3xx redirection (301 permanent, 302 temporary, 304 not modified), 4xx client error (400, 401, 403, 404, 429 too many requests) and 5xx server error (500, 502 bad gateway, 503 unavailable). HTTP/1.0 opened a new TCP connection per request. HTTP/1.1 keeps the connection open (*persistent connections*), but answers requests in order, so browsers open six or so connections per site.

HTTP/2 sends binary *frames* belonging to numbered *streams*, so that many requests share one TCP connection, interleaved, and compresses the headers (HPACK). It fixes the blocking at the HTTP level but not at the TCP level: a lost segment holds back every stream behind it. *HTTP/3*

therefore runs over *QUIC*, a transport built on UDP (port 443) in which each stream is delivered independently, loss recovery and congestion control run in the application, TLS 1.3 is part of the handshake, and a *connection ID* lets a connection survive a change of IP address, as when a phone moves from Wi-Fi to cellular. A client that has talked to the server before may send data in the first flight (*0-RTT*), at the price of replay risk for that data.

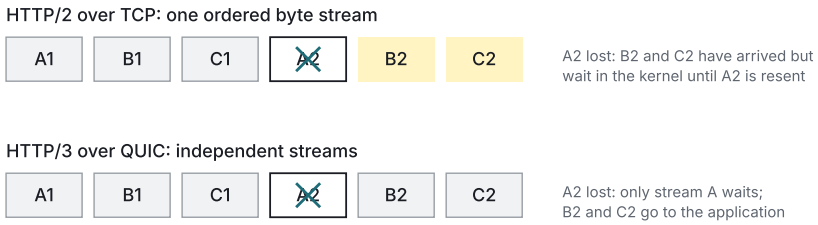


Figure 3.32 Head-of-line blocking. Three streams (A, B, C) share one connection, and the second packet of stream A is lost. Yellow: delivered late for no fault of its own.

TLS, in one paragraph

TLS encrypts and authenticates the connection. The client sends its supported versions and ciphers, a key share and the *server name indication* (SNI), which names the site wanted so that one address can serve many; the server answers with its key share and a *certificate* signed by a certificate authority that the client trusts, proving that the key belongs to the name, and both sides derive the session keys from the key shares. TLS 1.3 completes this in one round trip, whereas TLS 1.2 needs two. The details, certificate chains and attacks are treated in Pillar 9, Section 9.4; here only the round-trip count matters, because it adds to the time to first byte, as Section 3.11 shows.

Other application protocols appear in questions about ports. They are collected below, and the ports are repeated in the Revision Spread.

Protocol	Port	Purpose
SMTP; POP3; IMAP	25 (587 submission); 110; 143	Send mail; fetch and delete; synchronised mailbox
FTP	21 control, 20 data (active)	Active: the server connects back to the client. Passive: the client opens the data connection, which works through NAT
SSH; Telnet	22; 23	Encrypted remote shell; the same in clear text
SNMP; NTP	161/162; 123 (UDP)	Monitoring; time synchronisation

> INTERVIEW LENS
>
> GOOGLE · MICROSOFT · AMAZON
>
> The standard questions are recursive versus iterative DNS with TTLs and negative caching (Q3.46), CNAME versus A versus ALIAS (Q3.47), TLS 1.2 versus 1.3 with SNI (Q3.48) and head-of-line blocking in HTTP/2 versus HTTP/3 (Q3.49). Amazon adds operations: “which record would you change to move a site, and when will users see it?” (answer: the A record, after the old TTL expires; lower the TTL beforehand).

> LAB
>
> `LAB 3.10 · 25 MIN` Run the trace, then capture the protocols.

```
$ dig +trace www.google.com | grep -v '^;' | grep -E 'NS|A\s'
.            518400  IN  NS  a.root-servers.net.
com.         172800  IN  NS  a.gtld-servers.net.
google.com.  172800  IN  NS  ns1.google.com.
www.google.com. 300  IN  A   142.250.74.36
$ dig www.google.com @8.8.8.8 +noall +answer     # run twice, 10 s apart
www.google.com.  253  IN  A  142.250.74.36
$ curl -sI --http2 https://www.google.com | head -1
HTTP/2 200
```

What you should notice: the TTL falls between the two queries. In Wireshark, the filters `dns` and `tls.handshake.type == 1` show the query and the ClientHello, whose SNI extension holds the host name in clear text unless Encrypted Client Hello is used. For TLS decryption set the environment variable `SSLKEYLOGFILE` before starting the browser, and give the file to Wireshark under *Preferences → Protocols → TLS*. The addresses and TTLs are examples and change over time.

CHECK YOURSELF

1. A resolver has just cached `www.example.com` with TTL 600. The authoritative server changes the record. When do clients using that resolver see the change at the latest? [1]
2. Why can a resolver's reply to a large query arrive over TCP? [2]

3.11 Capstone: what happens when you type google.com and press Enter MUST · ■■□

This is the most commonly asked networking question in the book, because every concept in the pillar occurs in it. It is a poor question for a memorised monologue, since the interviewer will interrupt at the point where one is weakest. We therefore give a scaffold of eleven steps, then three versions of the answer at different depths, then the interruptions to expect.

BIG IDEA

A strong answer is ordered, layered and quantified: it follows the packets in time, names the protocol and the address at each step, and says roughly how long each step takes.

The eleven steps

1. **Parse and local checks.** The browser splits the URL into scheme, host and path, consults its HSTS list (a site that demands HTTPS is upgraded before any request), and looks in its own DNS cache, the operating system's cache and the hosts file.
2. **DNS.** If the name is unknown, the stub resolver sends a UDP query to port 53 of the resolver learnt from DHCP. The resolver, if its cache is cold, resolves iteratively through root, `com.` and `google.com.` servers (Section 3.10) and returns the A and AAAA records.
3. **ARP (or NDP).** The destination is off-subnet, so the frame goes to the default gateway; if its MAC address is not cached the host broadcasts an ARP request first (Section 3.6).
4. **TCP.** A three-way handshake to port 443 (about one round trip); or, if the browser knows that the site speaks HTTP/3, a QUIC handshake over UDP 443.
5. **TLS 1.3.** ClientHello with SNI, ServerHello with key share, certificate validation against trusted authorities, Finished messages: one more round trip.

6. **HTTP.** The browser sends `GET / HTTP/2`. Google's address is *anycast*, so the packets reach the nearest front end, which may answer `301` or `200`.
7. **At every router:** the TTL is decremented and the checksum updated, a longest-prefix match chooses the next hop, and the Ethernet header is rewritten (Section 3.8). At the home router, PAT rewrites the source address and port (Section 3.6).
8. **Inside Google:** the front end terminates TLS and forwards to a load balancer and then to backend servers (Pillar 5).
9. **The response** returns by the reverse path, through the NAT table to the laptop; the sender's initial window of about ten segments determines how much arrives in the first round trip (Section 3.9).
10. **Rendering.** The browser parses the HTML and requests the subresources (images, scripts, styles), which reuse the open connection or open new streams.
11. **Reuse and teardown.** The connection stays open for later requests (keep-alive, HTTP/2 multiplexing) and is closed after an idle period with FIN or, for QUIC, with a close frame.

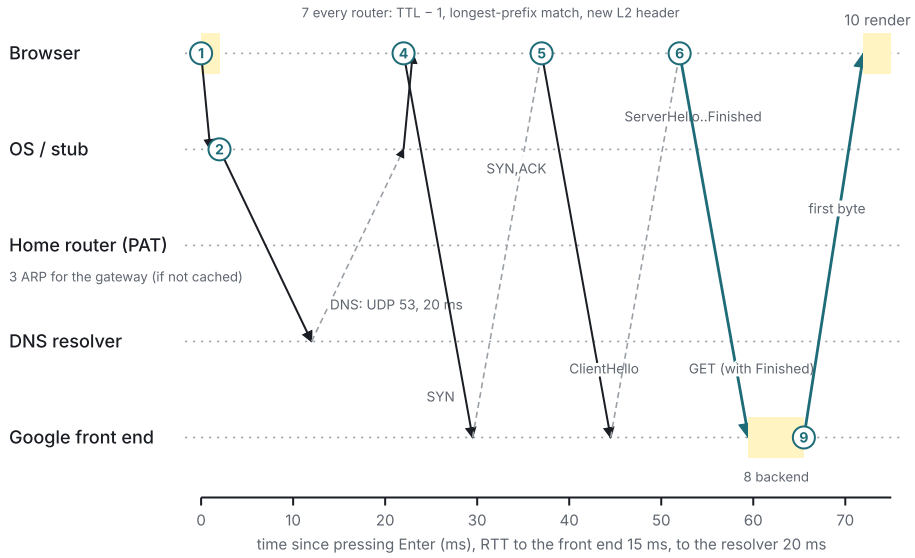


Figure 3.33 The google.com walk on a time axis (illustrative values). Numbers match the eleven steps of Section 3.11; steps 7 and 11 happen continuously. The accent arrows are the HTTP request and the first byte of the response.

Version 1: the 60-second answer

"The browser checks its caches and HSTS list, then DNS turns `www.google.com` into an IP address: my stub resolver asks the resolver given by DHCP, which asks root, `com.` and Google's servers if it has no cached answer. My host sees that the address is outside my subnet, so it ARPs for the default gateway and sends frames to its MAC address. A TCP handshake opens a connection to port 443, then a TLS handshake agrees keys and checks Google's certificate. The browser sends an HTTP GET; each router along the way decrements the TTL and forwards by longest-prefix match, rewriting the Ethernet header, and my home router translates my private address by PAT. Google's nearest front end answers, the reply comes back through the NAT table, and the browser renders the page, reusing the connection for the other files."

Version 2: the 5-minute answer

Add the following to the scaffold, in this order. Say that DNS usually asks for both A and AAAA, and that a browser prefers IPv6 but races both ("happy eyeballs") so that a broken IPv6 path costs

milliseconds, not seconds. At the link layer, name the frame fields: destination MAC is the gateway's, source MAC is the laptop's, and the IP header has TTL 64 and DF set. At TCP, give the sequence numbers' behaviour (random ISN, SYN consuming one number), the options in the SYN (MSS 1,460, window scale, SACK, timestamps) and the initial congestion window of 10 segments. At TLS, state that the client lists `h2` and `http/1.1` in ALPN, so the protocol is chosen inside the handshake, and that a 1.3 handshake costs one round trip. Finish with the numbers of the timeline: with a 15 ms round trip to the front end and 20 ms to a cold DNS resolver, DNS costs 20 ms, TCP 15 ms and TLS 15 ms, so the request leaves at about 50 ms and the first byte arrives at about 70 ms; QUIC saves one round trip by merging transport and TLS set-up, and a warm DNS cache saves nearly all of the first 20 ms.

WORKED EXAMPLE

Packet x-ray: the first TLS ClientHello leaving the laptop. Values are typical, with relative sequence numbers.

Layer	Fields
Ethernet II	dst 52:54:00:12:35:02 (gateway), src 52:54:00:12:34:56 (laptop), type 0x0800
IPv4	10.0.2.15 to 142.250.74.36, TTL 64, protocol 6, DF set, ID 0x1c46, total length 569
TCP	49822 to 443, flags PSH,ACK, seq 1, ack 1, window 502, header 32 bytes (timestamps)
TLS record	content type 22 (handshake), record version 0x0301, length 512
ClientHello	version 0x0303, 32 random bytes, ciphers (TLS_AES_128_GCM_SHA256 first), SNI `www.google.com`, ALPN `h2, http/1.1`, supported_versions 1.3, key_share x25519

1. Payload: the TLS record is 5 + 512 = 517 bytes.
2. TCP segment: 32 + 517 = 549 bytes; IP total length: 20 + 549 = 569.
3. Frame: 14 + 569 = 583 bytes (plus the 4-byte FCS on the wire).

Result: the whole frame holds only 517 bytes of the application's data. Note that the SNI is visible to anyone on the path.

Version 3: the 15-minute answer, with interruptions

At this depth, the interviewer interrupts. The answers below are the expected ones; each points to its home section.

"What if DNS is cached?" The query never leaves the machine (browser or OS cache), or stops at the resolver (about 1 ms on a home network), and the 20 ms of the example disappears. The TTL decides how long this lasts (Section 3.10).

"What if the gateway's MAC is unknown?" The host holds the packet, broadcasts an ARP request on the LAN, and sends the frame when the reply arrives; this adds a fraction of a millisecond. If nobody replies, ARP fails and the connection attempt reports "host unreachable" after a few seconds (Section 3.6).

"What if the certificate has expired, or names another host?" The handshake completes at the TCP level but the client aborts at validation with a fatal alert and the browser shows a warning; no HTTP is sent. For a site with HSTS the user cannot click through (Pillar 9).

"How does the reply find my laptop behind NAT?" The reply is addressed to the router's public address and port; the router looks up the port in its translation table, rewrites the destination to the laptop's private address and port, and sends the frame to the laptop's MAC (Section 3.6).

"What changes with HTTP/3?" Steps 4 and 5 merge into one QUIC round trip over UDP 443; streams are independent; and the browser learned of HTTP/3 from an Alt-Svc header or an HTTPS DNS record (Section 3.10).

"What if the first packet is bigger than the path MTU?" The ClientHello is small, but a large response is not: TCP segments of at most the MSS avoid it, and a blocked ICMP 3/4 would cause the black hole of Section 3.5.

"Why is the first page load slower than the second?" The first load pays for DNS, TCP and TLS set-up and a small congestion window; the second reuses the connection, whose window has grown (Section 3.9).

"Where does the load balancer sit?" After TLS termination, at the front end; the choice of backend is made by a layer-7 or layer-4 balancer (Pillar 5).

> INTERVIEW LENS
>
> GOOGLE · MICROSOFT · AMAZON · CISCO · JUNIPER · QUALCOMM
>
> Q3.45 is asked at nearly every company; Google, Microsoft and Amazon expect depth. The grading criteria are the order of events, the correct protocol and address at each step (MAC at the link, IP end to end, port at the transport layer) and numbers. The cheapest way to impress is to volunteer the failure cases yourself, in the order of the interruptions above.

> LAB
>
> `LAB 3.11 · 40 MIN` Record the whole sequence in one capture. Flush the caches first, then open the page.
>
> ```
> $ resolvectl flush-caches ; sudo ip neigh flush all     # Windows: ipconfig
> /flushdns ; arp -d *
> $ sudo tcpdump -nn -w walk.pcap -i any &  curl -sv https://www.google.com -o
> /dev/null
> $ tshark -r walk.pcap -Y 'arp or dns or tcp.flags.syn==1 or tls.handshake' -T
> fields -e frame.number -e _ws.col.Protocol -e _ws.col.Info
> ```
>
> Find the frame number of each step: (1) none, it is local; (2) the DNS query and response; (3) the ARP request and reply; (4) SYN, SYN-ACK, ACK; (5) ClientHello, ServerHello; (6) the HTTP/2 request; (9) the first data segment of the response; (11) FIN. **What you should notice:** the gaps between frames reproduce the timeline of the figure, scaled to your own network, and the ARP frames appear before the DNS query.

> CHECK YOURSELF
>
> 1. Which two steps of the walk would disappear on a second visit within a minute? [3]
> 2. At which step does the browser first learn the server's IP address? [4]

3.12 Where the topics are asked, and a mini-project OPTIONAL · ■■□

The table below shows how often each topic is reported at each company, from aggregated, self-reported interviews and role descriptions. It is an indication, not a guarantee. H marks a topic that is frequently reported or central to the role, M one that is sometimes asked, and L one that is rare.

The companies are Cisco (CSC), Qualcomm (QCM), Google (GOO), Microsoft (MSF), Amazon network development (AMZ), Juniper (JNP), Arista (ARI), Nokia (NOK) and Broadcom (BRC).

Topic	CSC	QCM	GOO	MSF	AMZ	JNP	ARI	NOK	BRC
OSI, TCP/IP, encapsulation	H	H	M	H	H	H	M	H	M
The google.com walk	H	M	H	H	H	M	M	M	L
Subnetting and VLSM by hand	H	M	M	M	H	H	H	H	M
Ethernet, MAC, switching	H	M	L	M	M	H	H	H	H
CRC, ARQ, sliding window	M	H	M	M	L	M	L	H	H
IPv4 header, fragmentation, MTU	H	H	M	M	H	H	H	H	H
ARP, DHCP, NAT	H	M	M	H	H	H	M	M	M
IPv6 and NDP	M	H	M	M	M	H	M	H	M
TCP handshake, states, TIME_WAIT	H	H	H	H	H	H	H	M	M
TCP congestion control	M	H	H	H	H	M	M	M	M
DNS, HTTP, TLS, QUIC	M	L	H	H	H	L	L	L	L
Routing algorithms	H	M	H	M	M	H	H	H	M

Qualcomm and Broadcom lean towards protocol-stack internals (ARQ, windows, headers, IPv6 for cellular); Google and Microsoft towards TCP behaviour and application performance; Amazon towards MTU, TCP and DNS operations; and Juniper, Arista and Nokia towards layer-2 and layer-3 forwarding.

WORKED EXAMPLE

Mini-project: PacketScope, a teaching packet dissector and subnet toolkit (a weekend). Build a Python command-line tool with three sub-commands, using only the standard library for the core.

1. `pscope subnet 172.16.45.200/21` prints the network, broadcast, range and wildcard, and draws the 32-bit ruler of Section 3.4 in ASCII.
2. `pscope parse capture.pcap` reads the file with `struct` and decodes Ethernet, 802.1Q, IPv4, IPv6, TCP, UDP, ICMP, ARP and DNS; it flags TCP handshakes and computes the round-trip time of each flow from SYN to SYN-ACK. A stretch goal is to detect retransmissions and duplicate ACKs, reproducing Wireshark's "TCP Analysis Flags".
3. `pscope cwnd log.txt` reads `ss -ti` samples taken during the Lab 3.9b runs and plots cwnd against time for CUBIC and BBR.

Result: a repository that exercises bit manipulation (Cisco, Arista), protocol parsing (Qualcomm, Broadcom) and TCP performance (Google, Amazon). Résumé bullet template: "Built a Python packet dissector that decodes N protocols from raw bytes and reports per-flow RTT and retransmissions; validated against Wireshark on M captures."

1. Within 600 seconds of the moment the resolver cached the old record.↩
2. The UDP answer did not fit, so the server set the

truncation flag and the resolver retried over TCP.↩

3. DNS (the answer is cached) and, if the connection is kept alive, the TCP and TLS handshakes; ARP is also cached.↩
4. At step 2, from the DNS answer.↩

Problem Set

Q3.1 Name the OSI layers and give one protocol and one device for each.

ALL · TECH-1 · ■□□ · S

30-second answer: 7 Application (HTTP; gateway), 6 Presentation (TLS), 5 Session (RPC), 4 Transport (TCP; firewall, L4 load balancer), 3 Network (IP; router), 2 Data link (Ethernet; switch, AP), 1 Physical (cable, 802.3 PHY; hub, repeater). PDUs: data, segment, packet, frame, bits.

Follow-ups. Where is ARP (between 2 and 3, carried in Ethernet)? Which layers does TCP/IP merge (5 to 7, and 1 to 2)? **Mistakes.** Calling a switch a layer-3 device (only an L3 switch is); placing TLS in the TCP/IP model without saying it sits between transport and application. Details: §3.2 (p. 149).

Q3.2 Hub, switch, router: collision and broadcast domains?

CISCO · NOKIA · TECH-1 · ■□□ · S

30-second answer: a hub is one collision domain and one broadcast domain for all its ports; a switch gives one collision domain per port but keeps one broadcast domain (per VLAN); a router gives each interface its own collision and broadcast domain.

Approach. Collision domain: who can collide on a shared medium. Broadcast domain: who hears a broadcast. **Answer.** Hub repeats bits to all ports, so everything shares one wire. Switch forwards per port by MAC, so full-duplex ports never collide, but floods broadcasts everywhere in the VLAN. Router does not forward broadcasts, so it ends the domain. For the counting exercise (2 broadcast, 8 collision domains) see Figure 3.2 in §3.1 (p. 147). **Mistake.** Saying a switch splits broadcast domains; VLANs do, not the switch itself.

Q3.3 Which IP header fields change at each router hop, and which never change?

CISCO · AMAZON · JUNIPER · TECH-1 · ■□□ · S

30-second answer: at every hop the TTL falls by one and the header checksum is recomputed, and the Ethernet source and destination MAC are rewritten. Source and destination IP, protocol and the TCP or UDP ports never change, unless NAT rewrites them.

Answer. Constant end to end: IP source and destination, protocol, identification, data. Changing per hop: TTL, checksum, and the layer-2 header and FCS. Changing only when something special happens: fragment fields (a router fragments), DSCP and ECN (a router re-marks or signals congestion), addresses and ports (NAT). **Mistake.** Saying the IP address changes and the MAC does not; the correct statement is the reverse (§3.6 (p. 167)).

Q3.4 Why is the minimum Ethernet frame 64 bytes? Derive it from CSMA/CD.

QUALCOMM · BROADCOM · TECH-1 · ■■□ · S

30-second answer: a sender must still be transmitting when a collision signal returns from the far end, so frame time must be at least twice the one-way delay: $L \geq 2T_pR$. For 10 Mb/s and 2,500 m, $2T_p = 25$ µs gives 250 bits; the standard uses 512 bits (64 bytes, 51.2 µs) to allow for repeaters and jam signal.

Approach and answer. $T_p = 2{,}500 / (2 \times 10^8) = 12.5$ µs; $2T_p \times 10$ Mb/s = 250 bits; round up to 512 bits. Payload is padded to 46 bytes, so that 14 + 46 + 4 = 64. **Follow-ups.** What changes at 1 Gb/s (carrier extension; full duplex has no collisions)? **Mistake.** Dividing by T_p, not $2T_p$. See §3.3 (p. 152).

Q3.5 How does a switch learn MAC addresses? What happens when the CAM table is full?

CISCO · ARISTA · BROADCOM · TECH-1 · ■■□ · S

30-second answer: it records the source MAC of every frame against the arrival port, forwards by the destination MAC, and floods broadcast, multicast and unknown unicast. When the table is full it cannot learn, so unknown destinations are flooded and the switch behaves like a hub, which an attacker exploits with MAC flooding.

Answer. Entries age out (300 s on Cisco) and are refreshed by traffic, so moves are relearned; a frame whose destination is on its arrival port is filtered. A full table does not drop traffic; it degrades to flooding, so the attacker sees other hosts' frames. **Mitigation.** Port security with a MAC limit per port, sticky addresses, and storm control (Pillar 9). **Mistake.** Saying that a full table drops frames. See §3.3 (p. 152).

Q3.6 Compute the CRC for data 1101011011 with generator 10011.

QUALCOMM · NOKIA · OA · ■■□ · S

30-second answer: append four zeros (the generator's degree), divide modulo 2, and the remainder is 1110; the transmitted word is 11010110111110, and the receiver's division gives remainder 0.

Approach. The register after each XOR, and the check, are worked out in the CRC example of §3.3 (p. 152). **Check.** 11010110111110 divided by 10011 leaves 0000, as verified by running the same loop on the full word. **Mistake.** Forgetting to append the zeros first, or using subtraction with borrows instead of XOR.

Q3.7 With 3-bit sequence numbers, what are the maximum Go-Back-N and Selective Repeat windows?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · S

30-second answer: Go-Back-N $2^3 - 1 = 7$; Selective Repeat $2^{3-1} = 4$.

Reason. In GBN the receiver window is 1, so the sender window may use all but one of the numbers: if all 8 were outstanding, an ACK of 0 could mean "all 8 received" or "none received". In SR both windows have size W and must not overlap modulo 2^n, so $2W \le 2^n$. **Mistake.** Quoting 2^n for both.

Q3.8 Stop-and-Wait, 1 Mb/s, 1,000-bit frames, T_p = 20 ms: efficiency, and the window for 100%?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: T_t = 1 ms, a = 20, $\eta = 1/(1 + 2a) = 1/41 \approx 2.4\%$; a window of 41 frames gives 100%.

Follow-ups. Sequence bits: GBN needs 6, SR needs 7 (the full working is in §3.3 (p. 152)). If the frame error rate is p, Stop-and-Wait efficiency becomes $(1 - p)/(1 + 2a)$. **Mistake.** Using the round-trip time as T_p.

Q3.9 Cut-through versus store-and-forward: which is faster, which propagates errors, and why do data-centre switches offer cut-through?

ARISTA · BROADCOM · TECH-2 · ■■■ · T

30-second answer: cut-through is faster, because it starts forwarding after the destination MAC (latency of tens of nanoseconds, independent of frame size) whereas store-and-forward waits for the whole frame and its FCS (1,500 bytes cost 1.2 µs at 10 Gb/s per hop). The price is that cut-through forwards corrupted frames, which the receiver then drops. Data-centre switches use it because small latencies add up over several hops and matter to storage and HPC traffic.

Answer. Store-and-forward checks the FCS, can convert speeds and queue-shape, and never forwards a bad frame. Cut-through needs equal ingress and egress speed and an uncongested egress; otherwise the switch must buffer, and falls back. Many switches count FCS errors per port and fall back to store-and-forward above a threshold. **Mistake.** Saying cut-through checks the FCS.

Q3.10 Network, broadcast and usable range for 172.16.45.200/21, without a calculator.

CISCO · AMAZON · TECH-1 · ■□□ · S

30-second answer: block size 8 in the third octet, 45 lies in 40 to 47: network 172.16.40.0, broadcast 172.16.47.255, usable 172.16.40.1 to 172.16.47.254, 2,046 hosts.

Approach. Mask 255.255.248.0, block = 256 − 248 = 8; the full working and the wildcard (0.0.7.255) are in §3.4 (p. 158). **Mistake.** Writing 172.16.45.0 as the network address.

Q3.11 How many usable hosts on a /27 and a /31? Why is /31 allowed?

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: /27 has 5 host bits, $2^5 - 2 = 30$ usable. A /31 has 2 addresses and both are usable, because RFC 3021 allows a point-to-point link to dispense with the network and broadcast addresses.

Answer. A /32 holds one address and names a host. The saving matters: a /31 per link uses half the addresses of a /30. Not all equipment supports /31 on all interface types, so verify before deploying. **Mistake.** Saying that /31 has zero hosts.

Q3.12 VLSM plan from 192.168.10.0/24 for LANs of 100, 50 and 25 hosts and two WAN links.

CISCO · NOKIA · TECH-1 · ■■□ · S

30-second answer: /25 at .0, /26 at .128, /27 at .192, then /30 at .224 and /30 at .228, leaving .232/29 and .240/28 spare.

Approach. Largest first, each on a multiple of its block size. The usable range of each is in §3.4 (p. 158): .1 to .126, .129 to .190, .193 to .222, .225 and .226, .229 and .230. **Mistake.** Starting with the small subnets, which leaves no aligned room for the /25.

Q3.13 Summarise 10.1.4.0/24 to 10.1.7.0/24. Can 10.1.5.0 to 10.1.8.0 become one /22?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: 10.1.4.0/22. No: the third octets 5 to 8 share only four bits, so one summary would be 10.1.0.0/20, which over-claims 12 other /24s; use 10.1.5.0/24, 10.1.6.0/23 and 10.1.8.0/24.

Approach. Binary alignment: the block must start at a multiple of its size, and contain only the required networks. 4 to 7 is a four-block range starting at a multiple of 4; 5 to 8 is not. See §3.4 (p. 158).

Q3.14 Write `in_subnet(ip, cidr)` using only bitwise operations.

CISCO · GOOGLE · ARISTA · TECH-1 · ■■□ · S

30-second answer: build the mask by shifting, and compare the two addresses under the mask: `(ip & mask) == (net & mask)`. Guard the prefix 0 case, because shifting a 32-bit value by 32 is undefined in C.

```c
#include <stdint.h>
#include <stdio.h>
int parse(const char *s, uint32_t *o) {
  unsigned a, b, c, d;
  if (sscanf(s, "%u.%u.%u.%u",
             &a, &b, &c, &d) != 4
      || (a|b|c|d) > 255) return -1;
  *o = a << 24 | b << 16 | c << 8 | d;
  return 0;
}
int in_subnet(uint32_t ip,
              uint32_t net, int p) {
  uint32_t m = p ? ~0u << (32 - p) : 0;
  return (ip & m) == (net & m);
}
```

Complexity. O(1). **Follow-ups.** Return the network and broadcast: `net & mask` and `net | ~mask`. Support IPv6 by comparing 16 bytes with a partial last byte. **Mistake.** Using signed integers (right shifts then sign-extend), or string comparison.

Q3.15 Fragment a 4,000-byte datagram for MTU 1,500: offsets and MF flags.

QUALCOMM · BROADCOM · JUNIPER · TECH-1 · ■■□ · S

30-second answer: three fragments with data 1,480, 1,480 and 1,020 bytes, Total Length 1,500, 1,500 and 1,040, offset field 0, 185 and 370 (in 8-byte units), MF 1, 1 and 0.

Approach. Data per fragment = 1,500 − 20 = 1,480, a multiple of 8. The table and the second-link variant (MTU 576) are in §3.5 (p. 163). **Mistake.** Writing 1,480 in the offset field.

Q3.16 MTU versus MSS: what is the MSS over PPPoE, and what does `ip tcp adjust-mss` do?

AMAZON · CISCO · TECH-1 · ■■□ · S

30-second answer: MTU is the largest IP packet on a link; MSS is the largest TCP payload, MTU − 40 for IPv4. PPPoE has MTU 1,492, so MSS = 1,452. `ip tcp adjust-mss 1452` makes the router rewrite the MSS option in transit SYNs, so that endpoints never send segments that exceed the link, which avoids the stalls of a broken PMTUD.

Answer. Each endpoint advertises an MSS from its own interface MTU and cannot know of a smaller link in the middle; clamping fixes that without relying on ICMP. IPv6 subtracts 60 (40 + 20). **Mistake.** Including the Ethernet header in the MTU.

Q3.17 Scenario: pages do not load, though ping and the TCP handshake work. What is happening, and how do you prove it?

AMAZON · MICROSOFT · CISCO · TECH-2 · ■■■ · T

30-second answer: a path-MTU black hole. Small packets pass, but a full-size packet with DF set is too big for a link in the path, and the ICMP "fragmentation needed" message is blocked, so the sender keeps resending the same large segment. Prove it with DF-set pings of decreasing size, and a capture that shows repeated retransmissions and no ICMP type 3 code 4.

Approach. (i) `ping -M do -s 1472 host` fails but `-s 1372` succeeds: bisect for the largest working size. (ii) Capture at the client: SYN, SYN-ACK and the request succeed; the response's large segments never arrive or arrive after retransmissions; no ICMP 3/4. (iii) Locate it with `tracepath` or by pinging each hop with DF. (iv) Fix: permit ICMP 3/4 at firewalls; clamp MSS at the tunnel or PPPoE edge; lower the interface MTU. **Mistakes.** Blaming DNS or the server; leaving ICMP blocked "for security". See Figure 3.14 and §3.5 (p. 163).

Q3.18 How does traceroute work on Linux and on Windows? Why `* * *`?

AMAZON · CISCO · TECH-1 · ■■□ · S

30-second answer: it sends probes with TTL 1, 2, 3 and so on, and each router that decrements the TTL to zero returns ICMP time exceeded, revealing itself. Linux sends UDP probes to ports from 33434 and the destination answers ICMP port unreachable; Windows `tracert` sends ICMP echo requests and the destination answers echo reply. `* * *` means no reply arrived in time.

Answer. Reasons for stars: the router rate-limits or suppresses ICMP, a firewall drops probes of this type (try `traceroute -I` or `-T -p 443`), the return path differs, or the destination ignores the probe. A star in the middle with replies afterwards is harmless. **Mistake.** Reading a star as an outage. See §3.5 (p. 163).

Q3.19 What is APIPA (169.254.0.0/16), and what does it tell you about DHCP?

MICROSOFT · CISCO · TECH-1 · ■□□ · S

30-second answer: a host that gets no DHCP reply gives itself a link-local 169.254.x.y address, after probing with ARP that it is unused. Seeing it means the host could not reach a DHCP server: check the cable and VLAN, the relay (`ip helper-address`), the pool (exhausted?) and the server.

Mistake. Treating it as a working configuration: such a host can talk only to others on its own link.

Q3.20 Explain DHCP DORA with ports and addresses. Why is Discover a broadcast?

CISCO · MICROSOFT · TECH-1 · ■□□ · S

30-second answer: Discover (client 0.0.0.0:68 to 255.255.255.255:67), Offer, Request (broadcast, names the chosen server and the address wanted), Acknowledgement (carries mask, gateway, DNS, lease). Discover is a broadcast because the client has no address and does not know any server.

Answer. Offer and Ack may be unicast to the client MAC or broadcast, depending on the broadcast flag. The Request is broadcast too, so that losing servers learn that their offer was declined. Renewal: unicast at T1 (50%), broadcast at T2 (87.5%). The relay case is in §3.6 (p. 167).

Q3.21 DHCP server on another subnet: how does it work, and what is giaddr?

CISCO · JUNIPER · TECH-2 · ■■□ · S

30-second answer: a relay agent on the client's router (`ip helper-address`) turns the broadcast into a unicast to the server and fills giaddr with the address of the interface the request arrived on. The server uses giaddr to select the pool for that subnet and replies to giaddr; the relay delivers the reply to the client.

Answer. Addresses: Discover 0.0.0.0 to 255.255.255.255; relayed copy relay-IP to server-IP, UDP 67 to 67. Option 82 can add circuit identifiers. **Mistake.** Forgetting that without giaddr the server cannot know which subnet to allocate from. See Figure 3.17.

Q3.22 What is gratuitous ARP, and where is it used?

CISCO · ARISTA · TECH-2 · ■■□ · T

30-second answer: an ARP message in which a host announces its own address (sender and target IP equal). It detects duplicate addresses, refreshes neighbours' caches after a change, and is sent by a new active router in first-hop redundancy protocols (HSRP, VRRP) or by a failed-over server so that switches and hosts relearn the virtual address.

Mistake. Thinking it requests anything; no reply is expected. If one arrives, the address is a duplicate. Security: it is also the tool of ARP spoofing (Pillar 9).

Q3.23 Walk a packet from PC-A to a web server through a PAT router. Give MAC, IP and port at each hop.

CISCO · AMAZON · GOOGLE · TECH-1 · ■■□ · S

30-second answer: PC-A ARPs for its gateway; the frame goes AA:AA to 11:11 with 192.168.1.10:51000 to 198.51.100.80:80. R1 applies PAT, so the second leg is 12:12 to 21:21 with 203.0.113.2:51000 to the same destination; the third is 22:22 to BB:BB with the same IPs. MACs change at every link, IPs only at the NAT.

Answer. The complete walk, the NAT table row and the return path are in §3.6 (p. 167). **Follow-ups.** Server on the same subnet: no gateway, ARP for the server. Two hosts with the same source port: R1 assigns another global port. **Mistake.** Reversing the roles of MAC and IP.

Q3.24 Scenario: two hosts behind different NATs need peer-to-peer VoIP. Why does it fail, and how do STUN, TURN and ICE help?

QUALCOMM · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: each NAT drops unsolicited inbound packets, since it has no mapping for them, and each host knows only its private address. STUN lets a host learn its public mapping from a server; both sides exchange these and send simultaneously (hole punching), which works for most NATs. TURN relays media through a server when punching fails (symmetric NATs). ICE gathers all candidates, tests the pairs and picks the best.

Answer. Candidate types: host (private), server-reflexive (from STUN), relayed (TURN). A signalling channel (SIP, WebRTC) carries the candidates. Carrier-grade NAT often forces TURN. IPv6 removes NAT but not firewalls. **Mistake.** Forgetting that TURN costs bandwidth, which is why it is the last resort.

Q3.25 Proxy ARP: what is it, and what can it hide?

CISCO · TECH-2 · ■■□ · T

30-second answer: a router answers ARP requests for addresses it can route to, with its own MAC, so a host that believes a remote address is local still reaches it. It hides wrong masks and missing default gateways, inflates ARP tables, and breaks when the router fails; it is usually disabled (`no ip proxy-arp`).

Q3.26 Compress 2001:0DB8:0000:0000:0000:FF00:0042:8329

CISCO · NOKIA · TECH-1 · ■□□ · S

30-second answer: 2001:db8::ff00:42:8329.

Rules. Drop leading zeros in each group; replace the longest run of all-zero groups by `::`, once only; on a tie, the leftmost run. Hence `2001:db8:0:0:1:0:0:1` becomes `2001:db8::1:0:0:1`. **Mistake.** Using `::` twice.

Q3.27 Derive the EUI-64 link-local address from MAC 00:1A:2B:3C:4D:5E.

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: fe80::21a:2bff:fe3c:4d5e. Insert FF:FE in the middle of the MAC and flip bit 7 (U/L) of the first byte, so 00 becomes 02.

Steps. 00:1A:2B | FF:FE | 3C:4D:5E gives 001A:2BFF:FE3C:4D5E; flipping gives 021A:2BFF:FE3C:4D5E; prefix fe80::/64. **Mistake.** Flipping the last bit, or not flipping at all. See §3.7 (p. 171).

Q3.28 How does IPv6 replace ARP? What is a solicited-node multicast address?

QUALCOMM · NOKIA · JUNIPER · TECH-1 · ■■□ · S

30-second answer: by NDP, using ICMPv6 neighbour solicitation (135) and neighbour advertisement (136). The solicitation goes to the solicited-node multicast address, ff02::1:ff plus the last 24 bits of the target address, so that only the few hosts sharing those bits are interrupted.

Example. For fe80::21a:2bff:fe3c:4d5e the group is ff02::1:ff3c:4d5e, and the Ethernet multicast MAC is 33:33:ff:3c:4d:5e. The reply is unicast and carries the MAC. The same mechanism runs duplicate address detection. **Mistake.** Saying that IPv6 broadcasts.

Q3.29 SLAAC versus DHCPv6. What do the RA M and O flags mean?

CISCO · QUALCOMM · TECH-2 · ■■□ · T

30-second answer: in SLAAC the host builds its address from the advertised /64 prefix and its own interface identifier, needing no server. M = 1 tells the host to use stateful DHCPv6 for the address; O = 1 tells it to use stateless DHCPv6 for other settings such as DNS, while still using SLAAC for the address.

Answer. The prefix option has its own A flag, which permits SLAAC. RDNSS in the RA can also carry DNS, making DHCPv6 unnecessary. DHCPv6 gives no default gateway; only the RA does. **Mistake.** Expecting DHCPv6 to provide the gateway.

Q3.30 Why don't IPv6 routers fragment? What breaks if ICMPv6 is blocked?

AMAZON · GOOGLE · TECH-2 · ■■■ · T

30-second answer: fragmenting costs router time, so only the source may fragment, using an extension header, and it learns the path MTU from ICMPv6 "packet too big" (type 2). If ICMPv6 is blocked, PMTUD fails with the black hole of Q3.17, and neighbour discovery, router advertisements and duplicate address detection stop, so the link does not work at all.

Answer. Every link must support 1,280 bytes, which is the safe size to use when PMTUD is uncertain. Firewalls must permit types 1 to 4, 128 and 129 (echo) as needed, and 133 to 137 on the local link. **Mistake.** Copying an IPv4 rule that drops all ICMP.

Q3.31 TCP versus UDP: three applications of each, with reasons.

ALL · TECH-1 · ■□□ · S

30-second answer: TCP for correctness and ordering: web (HTTP/1.1 and 2), email, file transfer, SSH. UDP when late data is useless or the application recovers itself: DNS, voice and video, online games, QUIC.

Answer. TCP: connection set-up, reliable ordered stream, flow and congestion control. UDP: no state, no delay, multicast and broadcast, and the ability to build a custom protocol (QUIC reimplements reliability, streams and congestion control in user space). The comparison table is in §3.9 (p. 176). **Mistake.** Saying that UDP is "unreliable so it is bad".

Q3.32 Draw the three-way handshake with ISNs 1000 and 5000.

ALL · TECH-1 · ■□□ · S

30-second answer: SYN seq 1000; SYN,ACK seq 5000 ack 1001; ACK seq 1001 ack 5001. States: client SYN_SENT then ESTABLISHED; server LISTEN, SYN_RCVD then ESTABLISHED.

Follow-ups. After 500 bytes from the client: seq 1001, ack from the server 1501; a FIN takes the number 1501 and is acknowledged with 1502. Both SYN and FIN consume one number; a pure ACK consumes none. The diagram is in Figure 3.24.

Q3.33 Why is the handshake three-way and not two-way?

GOOGLE · MICROSOFT · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: each side must prove it received the other's initial sequence number. With two messages the server cannot tell whether its SYN,ACK arrived, and cannot distinguish a new SYN from a delayed duplicate of an old one, so it would hold state for connections nobody wants. The third segment confirms both.

Answer. Four messages are unnecessary, since the server's ACK and SYN travel together. Random ISNs complete the defence against old duplicates and blind injection. **Mistake.** Saying that "it saves a message"; the point is mutual confirmation.

Q3.34 Explain TIME_WAIT: why 2 MSL, which side enters it, and what happens on a busy proxy?

GOOGLE · AMAZON · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: the side that closes first (the active closer) waits 2 MSL (60 s on Linux) so that it can re-acknowledge a retransmitted FIN if its last ACK was lost, and so that stray segments of the old connection die before the same 4-tuple is reused. A proxy that opens many short connections to one backend accumulates TIME_WAIT sockets and can exhaust ephemeral ports.

Answer. With about 28,000 ephemeral ports and 60 s, one source IP can make about 470 new connections per second to one destination. Remedies: persistent connections and pooling, more source IPs or ports, `tcp_tw_reuse` for outgoing connections (needs timestamps), and letting the client close first. Do not shorten 2 MSL or use the removed `tcp_tw_recycle`, which broke NAT users. **Mistake.** Saying that the client always has TIME_WAIT. See §3.9 (p. 176).

Q3.35 A server shows thousands of CLOSE_WAIT sockets. Whose bug is it?

MICROSOFT · AMAZON · TECH-1 · ■■□ · S

30-second answer: the server application's. CLOSE_WAIT means that the peer sent a FIN, the kernel acknowledged it, and the application has not called `close()` on its socket; so the program leaks sockets, usually on an error path or a missing close of a pooled connection.

Approach. `ss -tanp state close-wait` to find the process; check the code for paths that skip `close()`, such as exceptions, timeouts and abandoned connections; look at file-descriptor use. No kernel or network tuning fixes it, and the leak ends when the process exits. **Mistake.** Confusing it with TIME_WAIT.

Q3.36 Flow control versus congestion control: which window is which, and what is the effective send window?

QUALCOMM · GOOGLE · AMAZON · TECH-1 · ■■□ · S

30-second answer: flow control protects the receiver's buffer and uses the advertised window rwnd, carried in every segment. Congestion control protects the network and uses cwnd, which the sender maintains from loss or delay signals. The sender may have min(rwnd, cwnd) bytes in flight.

Answer. rwnd changes with the application's reads; cwnd changes with ACKs and losses. The binding window is the smaller. **Mistake.** Believing that cwnd is sent in a header; it is local to the sender. See §3.9 (p. 176).

Q3.37 Trace cwnd: initial 1, ssthresh 16, loss at cwnd 20. Tahoe, and Reno on three duplicate ACKs.

QUALCOMM · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: slow start 1, 2, 4, 8, 16, then +1 per RTT 17, 18, 19, 20. At the loss ssthresh = 10. Tahoe sets cwnd = 1 and runs slow start (1, 2, 4, 8, 10, 11, …); Reno on three duplicate ACKs sets cwnd = 10 (13 while in recovery) and continues 11, 12, ….

Follow-ups. A timeout in Reno gives cwnd = 1, like Tahoe. NewReno handles several losses in one window. The plot is in Figure 3.29.

Q3.38 CUBIC versus BBR: which signal does each use, and why can BBR v1 be unfair to CUBIC?

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■■ · T

30-second answer: CUBIC reacts to loss and grows its window as a cubic function of the time since the last loss. BBR builds a model from the measured bottleneck bandwidth and minimum RTT, paces at that rate and keeps about one BDP in flight, so it does not need loss. BBR v1 ignores loss, so against CUBIC in a shallow buffer it keeps sending while CUBIC backs off, and takes more than its share; v2 and v3 respond to loss and ECN.

Answer. BBR keeps queues short and tolerates random loss; CUBIC fills the buffer, which causes bufferbloat but is simple and robust. In deep buffers BBR v1 can instead lose share, because its inflight cap is about 2 × BDP. **Mistake.** Saying that BBR "is better": it depends on loss pattern and buffer depth. See §3.9 (p. 176).

Q3.39 Maximum throughput for rwnd 64 KB and RTT 100 ms. How do you fill a 1 Gb/s, 100 ms path?

GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: throughput ≤ window / RTT = 65,535 × 8 / 0.1 = 5.24 Mb/s. The BDP of a 1 Gb/s, 100 ms path is 12.5 MB, so use window scaling with shift 8 (16.8 MB) and receive and send buffers of that size.

Answer. The shift is 9 for RTT 190 ms (BDP 23.75 MB; Mumbai to Virginia). Also: large enough `tcp_rmem` and `tcp_wmem` maxima, a congestion algorithm that survives a little loss at this rate (CUBIC, BBR), or several parallel streams. **Mistake.** Raising the link speed and expecting more throughput. See §3.9 (p. 176).

Q3.40 Explain fast retransmit and SACK. Why three duplicate ACKs, not one?

QUALCOMM · GOOGLE · TECH-1 · ■■□ · S

30-second answer: when three duplicate ACKs arrive, the sender resends the missing segment immediately instead of waiting for the RTO. SACK lets the receiver list the blocks it already holds, so that the sender repairs several losses per round trip and resends only what is missing. One or two duplicates can result from mere reordering, so three avoids spurious retransmissions at a small cost in delay.

Answer. On small windows there may never be three duplicates, and then only the timeout helps (tail loss probes and RACK address this). See Figure 3.27.

Q3.41 Nagle and delayed ACK: why do interactive applications see 40 to 200 ms stalls, and how are they fixed?

GOOGLE · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: Nagle holds a small segment while earlier data is unacknowledged; the receiver delays its ACK for up to 40 to 200 ms hoping to piggyback it; each waits for the other. The stall appears in write-write-read patterns. Fix by setting `TCP_NODELAY`, by sending the message in one write (`writev`), or by `TCP_QUICKACK` on the receiver.

Answer. Nagle helps bulk and telnet-style traffic by reducing small packets, so do not disable it blindly: batch writes first. **Mistake.** Blaming the network.

Q3.42 What is a SYN flood? How do SYN cookies work?

CISCO · AMAZON · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: an attacker sends many SYNs, usually with forged sources, filling the server's half-open queue so that real clients are refused. With SYN cookies the server keeps no state: it encodes a hash of the 4-tuple, a secret and a timestamp (plus an MSS code) in its initial sequence number, and rebuilds the connection only when a valid ACK returns that number plus one.

Answer. Cost: some SYN options such as window scale may be lost unless encoded in the timestamp. Other defences: bigger backlog, shorter SYN-ACK retries, rate limiting and upstream filtering. See §3.9 (p. 176).

Q3.43 Coding: simulate a sliding-window sender. Given a stream of ACKs and timeouts, output which segments are retransmitted.

GOOGLE · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: keep `base` (oldest unacknowledged) and `next` (next to send). A cumulative ACK for k advances `base` to k and releases new segments up to `base + N`; a repeated ACK of `base` three times, or a timer expiry, retransmits segment `base`.

```c
#include <stdio.h>
/* ev[i] >= 0: cumulative ACK k (segments < k received); ev[i] == -1: timer expiry */
void run(int total, int N, const int *ev, int n) {
    int base = 0, next = 0, dup = 0;
    while (next < base + N && next < total) printf("send %d\n", next++);
    for (int i = 0; i < n; i++) {
        if (ev[i] == -1) {
/* RTO: resend the oldest segment */
            if (base < next) printf("retransmit %d (timeout)\n", base);
        } else if (ev[i] > base) {
/* new data acknowledged */
            base = ev[i]; dup = 0;
            while (next < base + N && next < total) printf("send %d\n", next++);
        } else if (ev[i] == base && base < next && ++dup == 3) {
            printf("retransmit %d (fast)\n", base);
        }
    }
}
```

Complexity. O(total + events). **Follow-ups.** Go-Back-N would resend `base .. next - 1` on timeout; Selective Repeat needs a per-segment ACK flag; the timer restarts whenever `base` advances. **Mistakes.** Counting duplicates across new ACKs (reset `dup` when `base` moves), and sending beyond `base + N`.

Q3.44 When does a TCP stack send RST?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: when a segment arrives for which no connection exists, which includes a SYN to a closed port; when an application aborts (closing with unread data, or a zero linger time); when a firewall or IPS injects one to kill a flow; and when a segment violates the connection state, such as data after the peer has restarted.

Answer. An RST needs no acknowledgement and the connection is dropped at once. UDP has the analogue ICMP port unreachable. A SYN to a filtered port gets no answer, while a closed port gets RST, which helps in diagnosis. **Mistake.** Mixing RST with FIN, which closes gracefully.

Q3.45 What happens when you type google.com and press Enter? Start at the keyboard.

ALL · TECH-1 · ■□□ · S

30-second answer: the browser checks caches and HSTS; DNS resolves the name (stub resolver, recursive resolver, root, `com.`, `google.com.`); the host ARPs for its gateway; TCP (or QUIC) connects to port 443; TLS 1.3 agrees keys and validates the certificate; HTTP GET goes out through routers that decrement the TTL and forward by longest-prefix match, with PAT at the home router; Google's nearest front end replies; the reply returns through the NAT table and the browser renders and reuses the connection.

Answer. The eleven-step scaffold, the 5- and 15-minute versions, the timeline and the answers to the usual interruptions are in §3.11 (p. 190). **Mistake.** Starting with the network and forgetting HSTS, DNS caching or the NAT return path.

Q3.46 Recursive versus iterative DNS. What do TTL and negative caching do? When does DNS use TCP?

AMAZON · GOOGLE · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: the client asks its resolver recursively ("find it for me"); the resolver asks the root, TLD and authoritative servers iteratively, following referrals. The TTL says how long a cache may keep a record; negative caching keeps "no such name" for a time from the SOA. DNS uses TCP when an answer is truncated (too large for UDP) and for zone transfers.

Answer. Hence a change takes effect after the old TTL: lower it before a migration. Negative caching saves repeated queries but delays a newly created name. DNS over TLS and HTTPS also use TCP. **Mistake.** Saying that DNS is UDP only. See §3.10 (p. 187).

Q3.47 CNAME versus A versus ALIAS. Why can't you put a CNAME at a zone apex?

AMAZON · MICROSOFT · TECH-2 · ■■□ · T

30-second answer: A maps a name to an address; CNAME makes a name an alias of another name, and the resolver restarts the lookup on the target; ALIAS (or ANAME) is a provider feature that flattens a target into A records at query time. A CNAME may not coexist with other records at its name, and the apex must hold SOA and NS, so a CNAME there is illegal.

Answer. Use ALIAS for `example.com` pointing at a CDN or load balancer, and CNAME for `www`. **Mistake.** A CNAME chain that is long, which adds lookups.

Q3.48 TLS 1.2 versus 1.3: how many round trips does each need, and what is SNI?

GOOGLE · MICROSOFT · TECH-2 · ■■□ · S

30-second answer: TLS 1.2 needs two round trips after the TCP handshake; TLS 1.3 needs one, because the client sends its key share in the first message; a resumed 1.3 session can send data in 0-RTT, with replay risk. SNI is the extension in the ClientHello that names the host wanted, so that one IP address can serve many certificates.

Answer. Time to first byte over TCP: 1 RTT (TCP) + 2 (TLS 1.2) or + 1 (TLS 1.3), before the request. TLS 1.3 also removes weak ciphers and encrypts the certificate. The SNI is visible unless Encrypted Client Hello is used. Details in Pillar 9; see also §3.10 (p. 187).

Q3.49 HTTP/2 versus HTTP/3: what is head-of-line blocking, and how does QUIC remove it?

GOOGLE · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: head-of-line blocking is the delay of data that has arrived because earlier data is missing. HTTP/2 multiplexes streams on one TCP connection, so one lost segment stalls all streams behind it. QUIC runs over UDP and has independent streams: loss on one stream delays only that stream.

Answer. QUIC also integrates TLS 1.3 (one handshake round trip, 0-RTT on resumption), uses connection IDs that survive address changes, and runs congestion control in user space. Costs: UDP is blocked or throttled on some networks (browsers fall back to TCP), and CPU use is higher. See Figure 3.32.

Q3.50 Scenario: a website is slow only for users in one city. Give a step-by-step plan.

AMAZON · GOOGLE · TECH-2 · ■■■ · T

30-second answer: narrow by layer and by hop. Reproduce from that city, then check in order: DNS (is the user sent to a distant region?), the anycast or routing path (traceroute and BGP), packet loss and RTT along the path, MTU problems, and the CDN or origin for that region.

Approach. (i) Compare measurements from that city and from a good one: DNS answer and TTL (`dig` through the local resolver and `@8.8.8.8`; geo-routing or ECS misconfiguration), time to first byte split into DNS, TCP, TLS and server (`curl -w`). (ii) Path: `mtr` both ways; look for loss starting at one hop, an unexpected detour, or a changed BGP route or anycast catchment. (iii) Loss and MTU: DF-set pings, retransmission rate in a capture; an ISP peering problem looks like loss at specific hours. (iv) Edge: is the local CDN node unhealthy, over capacity or serving from cache misses? (v) Mitigate: shift the traffic (weighted DNS, withdraw the anycast prefix at that site), then open a ticket with the ISP with evidence. **Mistake.** Restarting servers before locating the problem.

L3.1 Long form: a six-department campus from 10.20.0.0/22

CISCO · NOKIA · TECH-2 · ■■■ · T

30-second answer: sort the needs, give each the smallest block that fits, and place the blocks largest first on aligned boundaries. Six LANs of 450, 250, 120, 60, 28 and 12 hosts need /23, /24, /25, /26, /27 and /28, which use 1,008 of the 1,024 addresses.

Department	Hosts	Prefix	Network	Usable range	Broadcast
Computer science	450	/23	10.20.0.0	10.20.0.1 to 10.20.1.254	10.20.1.255
Electronics	250	/24	10.20.2.0	10.20.2.1 to 10.20.2.254	10.20.2.255
Library	120	/25	10.20.3.0	10.20.3.1 to 10.20.3.126	10.20.3.127
Administration	60	/26	10.20.3.128	10.20.3.129 to 10.20.3.190	10.20.3.191
Laboratories	28	/27	10.20.3.192	10.20.3.193 to 10.20.3.222	10.20.3.223
Guest Wi-Fi	12	/28	10.20.3.224	10.20.3.225 to 10.20.3.238	10.20.3.239

Answer. The spare 10.20.3.240/28 holds four /30 router links (or eight /31s). The whole /22 can be advertised as one summary. **Check.** 512 + 256 + 128 + 64 + 32 + 16 = 1,008. **Mistake.** Giving 450 hosts a /24, or leaving no room for growth; in practice add 20 to 30% headroom before choosing prefixes.

L3.2 Long form: decode a captured SYN frame, byte by byte

QUALCOMM · BROADCOM · TECH-2 · ■■□ · T

30-second answer: split the 54 bytes into 14 (Ethernet), 20 (IPv4) and 20 (TCP) and read each field in turn; it is a TCP SYN from 192.168.0.104:49822 to 192.168.0.1:80, ISN 1000, TTL 64, DF set.

```
5254 0012 3502 5254 0012 3456 0800
4500 0028 1c46 4000 4006 9cd0 c0a8 0068
c0a8 0001
c29e 0050 0000 03e8 0000 0000 5002 faf0
6c61 0000
```

Ethernet. Destination 52:54:00:12:35:02 (the gateway), source 52:54:00:12:34:56, type 0x0800 (IPv4). **IPv4.** `45`: version 4, IHL 5; total length 0x28 = 40 (20 + 20, no data); ID 0x1c46; `4000`: DF; TTL 0x40 = 64; protocol 6; checksum 0x9cd0 (the sum of all ten words including it folds to 0xFFFF); addresses c0a80068 = 192.168.0.104 and c0a80001 = 192.168.0.1. **TCP.** Source port 0xc29e = 49822; destination 0x0050 = 80; sequence 0x3e8 = 1000; ack 0; `50`: offset 5 (20 bytes); `02`: SYN only; window 0xfaf0 = 64,240; checksum 0x6c61 over the pseudo-header, header and data; urgent 0. **Mistake.** Reading the flags byte as the offset byte: `5002` is offset-and-reserved `50`, flags `02`.

L3.3 Long form: fragments across two links, MTU 1,500 then 576

QUALCOMM · BROADCOM · JUNIPER · TECH-2 · ■■■ · T

30-second answer: a 4,000-byte datagram becomes three fragments on the 1,500 link, and each is fragmented again on the 576 link, giving eight fragments; the offsets stay relative to the original datagram, and only the final piece has MF = 0.

Approach. On the 576 link data per fragment = 576 − 20 = 556, rounded down to a multiple of 8: 552 bytes (69 units). The first-stage fragments carry 1,480, 1,480 and 1,020 bytes at offsets 0, 185 and 370.

First-stage fragment	Second-stage data	Offsets	MF
1 (offset 0, 1,480 B)	552, 552, 376	0, 69, 138	1, 1, 1
2 (offset 185, 1,480 B)	552, 552, 376	185, 254, 323	1, 1, 1
3 (offset 370, 1,020 B, MF 0)	552, 468	370, 439	1, 0

Answer. The 376-byte pieces keep MF = 1 because more of the datagram follows. The data adds up: (552 + 552 + 376) × 2 + 552 + 468 = 2,960 + 1,020 = 3,980. **Mistake.** Restarting the offsets at 0 for each fragment, or clearing MF on the last piece of a non-final fragment.

L3.4 Long form: cwnd over 25 round trips with a 3-duplicate-ACK loss and a timeout

QUALCOMM · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: slow start to ssthresh 32, linear growth to 36, a fast-recovery cut to 18, linear growth to 22, then a timeout that sets ssthresh to 11 and cwnd to 1, and slow start back up to 11 and linear growth.

- RTT 1 to 6: 1, 2, 4, 8, 16, 32 (slow start up to ssthresh = 32).
- RTT 7 to 10: 33, 34, 35, 36 (congestion avoidance); three duplicate ACKs arrive in RTT 10, so ssthresh = 18.
- RTT 11 to 15: 18, 19, 20, 21, 22 (fast recovery has ended; congestion avoidance); a timeout occurs in RTT 15, so ssthresh = 11 and cwnd = 1.
- RTT 16 to 20: 1, 2, 4, 8, 11 (slow start, capped at ssthresh).
- RTT 21 to 25: 12, 13, 14, 15, 16 (congestion avoidance).

Approach. Reno halves on the duplicate-ACK loss (36 / 2 = 18) and resumes there; the timeout at 22 gives ssthresh = 11 and cwnd = 1, and slow start is capped at ssthresh (8 doubles to 16, which is cut to 11). **Mistake.** Resetting to 1 on the duplicate-ACK loss, or continuing slow start beyond ssthresh.

L3.5 Subnetting sprint: twenty drills (target 45 seconds each)

CISCO · AMAZON · JUNIPER · ARISTA · NOKIA · OA · ■□□ · T

30-second answer: for each address, find the block size in the interesting octet, take the multiple below, and subtract one from the next multiple for the broadcast. Answers are listed for self-marking.

Address/prefix	Network	Broadcast	Usable
10.20.30.77/26	10.20.30.64	10.20.30.127	62
192.168.5.130/25	192.168.5.128	192.168.5.255	126
172.16.200.9/20	172.16.192.0	172.16.207.255	4,094
10.0.0.200/27	10.0.0.192	10.0.0.223	30
172.31.255.1/18	172.31.192.0	172.31.255.255	16,382
192.168.100.77/28	192.168.100.64	192.168.100.79	14
10.8.130.5/21	10.8.128.0	10.8.135.255	2,046
172.20.15.200/22	172.20.12.0	172.20.15.255	1,022
203.0.113.77/29	203.0.113.72	203.0.113.79	6
192.168.7.250/30	192.168.7.248	192.168.7.251	2
10.255.3.4/13	10.248.0.0	10.255.255.255	524,286
100.64.77.9/10	100.64.0.0	100.127.255.255	4,194,302
172.17.33.33/19	172.17.32.0	172.17.63.255	8,190
192.0.2.129/26	192.0.2.128	192.0.2.191	62
10.4.5.6/9	10.0.0.0	10.127.255.255	8,388,606
198.51.100.99/27	198.51.100.96	198.51.100.127	30
172.16.31.254/21	172.16.24.0	172.16.31.255	2,046
10.10.10.10/12	10.0.0.0	10.15.255.255	1,048,574
192.168.1.1/23	192.168.0.0	192.168.1.255	510
10.1.2.3/16	10.1.0.0	10.1.255.255	65,534

Mistakes. Using the wrong octet for the block (a /13 works in the second octet), and forgetting that /30 leaves two hosts.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) MAC changes at every hop; IP does not, except at NAT. (2) Block size = 256 − mask octet; hosts = $2^h - 2$, except /31 (2) and /32 (1). (3) Fragment offset counts 8-byte units; MSS = MTU − 40. (4) TTL and header checksum change at every hop. (5) SYN and FIN each use one sequence number; a pure ACK uses none. (6) TIME_WAIT is on the active closer; CLOSE_WAIT is an application bug. (7) Effective window = min(rwnd, cwnd); throughput ≤ window / RTT; BDP = rate × RTT. (8) Reno: 3 duplicate ACKs give ssthresh = cwnd / 2; a timeout gives cwnd = 1. (9) CUBIC and Reno react to loss; BBR models bandwidth and RTT. (10) DNS: recursive from the client, iterative from the resolver; TTL decides when a change is seen; TCP for truncation and zone transfers.

Three figures to redraw from memory. The three-way handshake and teardown ladders with sequence numbers (Figure 3.24, Figure 3.25); the cwnd sawtooth with ssthresh (Figure 3.29); the packet-walk strip with the L2 row changing and the L3 row constant (Figure 3.19).

MODELS, DEVICES, PDUS

- 7 Application, 6 Presentation, 5 Session, 4 Transport (segment), 3 Network (packet), 2 Data link (frame), 1 Physical (bits). Mnemonic: All People Seem To Need Data Processing.
- Hub: L1, one collision and one broadcast domain. Switch: L2, collision domain per port. Router: L3, a broadcast domain per interface. Firewall and load balancer: L4 to L7.
- Demultiplexing keys: EtherType, IP protocol, port. Say in an interview: “each layer’s header tells its peer what to do.”

LAYER 2

- Ethernet II: dst 6, src 6, type 2, payload 46 to 1,500, FCS 4 (frame 64 to 1,518; +4 with an 802.1Q tag). Preamble 7 + SFD 1 and gap 12 are outside the frame.
- MAC: OUI 3 + device 3 bytes; I/G = low bit of the first byte (1 = group); U/L = next bit.
- Switch: learn source, forward or flood on destination, age after 300 s. Cut-through starts after the destination MAC; store-and-forward checks the FCS.
- Min frame: $L \geq 2T_pR$. CRC: append r zeros, divide mod 2, remainder replaces them. Hamming: syndrome = position of the error.
- ARQ: SW efficiency $1/(1 + 2a)$, $a = T_p/T_t$. GBN window $\leq 2^n - 1$; SR window $\leq 2^{n-1}$.

IPV4 AND SUBNETTING

- Header 20 to 60 bytes (IHL × 4); fields: ver, IHL, DSCP/ECN, length, ID, flags (DF, MF), offset, TTL, protocol, checksum, src, dst.
- Block-size method: network = largest multiple of the block not above the address; broadcast = next network − 1.
- Private: 10/8, 172.16/12, 192.168/16. Special: 127/8, 169.254/16, 100.64/10, 224/4, 0.0.0.0, 255.255.255.255.
- VLSM: largest first, each on a multiple of its block. Summary: common leading bits of aligned, contiguous blocks.

Prefix	Mask	Block	Addresses	Usable
/24	255.255.255.0	1	256	254
/25	255.255.255.128	128	128	126
/26	255.255.255.192	64	64	62
/27	255.255.255.224	32	32	30
/28	255.255.255.240	16	16	14
/29	255.255.255.248	8	8	6
/30	255.255.255.252	4	4	2
/31	255.255.255.254	2	2	2 (RFC 3021)
/32	255.255.255.255	1	1	1

ARP, DHCP, NAT

- ARP: broadcast request (opcode 1), unicast reply (2). Gratuitous ARP: duplicate check and cache refresh. Proxy ARP: router answers for others.
- DHCP: Discover, Offer, Request, Ack; UDP 68 to 67; T1 50%, T2 87.5%; relay sets giaddr; options 1, 3, 6, 51, 54, 82.
- NAT: static 1:1; dynamic pool; PAT many:1 by port. Inside local, inside global, outside local, outside global. Port forwarding for servers; STUN, TURN and ICE for peer-to-peer.

IPV6

- 128 bits; header 40 bytes; no checksum, no router fragmentation; minimum MTU 1,280. Compress: drop leading zeros, one `::` on the longest run.

- 2000::/3 global, fc00::/7 unique local, fe80::/10 link-local, ff00::/8 multicast, ::1 loopback, 2001:db8::/32 documentation.
- EUI-64: insert FF:FE, flip bit 7. Solicited-node: ff02::1:ff + last 24 bits; MAC 33:33 + last 32 bits.
- NDP (ICMPv6): RS 133, RA 134, NS 135, NA 136, Redirect 137; PTB is type 2. RA flags: M (stateful DHCPv6), O (other options).

TCP AND UDP

- UDP header 8 bytes: ports, length, checksum. TCP header 20 to 60 bytes: ports, seq, ack, offset, flags CWR ECE URG ACK PSH RST SYN FIN, window, checksum, urgent pointer, options (MSS, window scale, SACK, timestamps).
- Handshake: SYN, SYN-ACK, ACK. Teardown: FIN, ACK, FIN, ACK; active closer waits 2 MSL (60 s on Linux). RTO = SRTT + 4 RTTVAR, minimum 200 ms on Linux (1 s in RFC 6298), doubled on each timeout.
- Fast retransmit after 3 duplicate ACKs. Slow start doubles per RTT up to ssthresh, then +1 MSS per RTT. Initial window 10 MSS.
- Window scale shift s multiplies 65,535 by 2^s. Mathis: rate ≈ (MSS / RTT) × 1.22 / √p.

State	Who	Meaning
LISTEN	server	waiting for a SYN
SYN_SENT, SYN_RCVD	client, server	handshake in progress
ESTABLISHED	both	data flows
FIN_WAIT_1, FIN_WAIT_2	active closer	FIN sent; ACK received
CLOSE_WAIT, LAST_ACK	passive closer	FIN received; waiting for the application to close
CLOSING	both closing	simultaneous close
TIME_WAIT	active closer	2 MSL before CLOSED

PORTS, PROTOCOL NUMBERS, ETHERTYPES

Port	Service	Port	Service
20/21	FTP data/control	123	NTP (UDP)
22	SSH	143	IMAP
23	Telnet	161/162	SNMP
25	SMTP	179	BGP
53	DNS (UDP/TCP)	389	LDAP
67/68	DHCP server/client	443	HTTPS, QUIC
69	TFTP	514	syslog
80	HTTP	520	RIP
110	POP3	3389	RDP

IP protocol numbers: 1 ICMP, 2 IGMP, 6 TCP, 17 UDP, 47 GRE, 50 ESP, 51 AH, 58 ICMPv6, 89 OSPF. EtherTypes: 0x0800 IPv4, 0x0806 ARP, 0x86DD IPv6, 0x8100 802.1Q. ICMP: 0 echo reply, 3 unreachable (code 4 fragmentation needed), 5 redirect, 8 echo request, 11 time exceeded.

DNS, HTTP, TLS

- Records: A, AAAA, CNAME, NS, MX, PTR, TXT, SOA, SRV, CAA. UDP 53; TCP for truncation and zone transfer; DoT 853; DoH 443.
- Status classes: 2xx success, 3xx redirect (301, 302, 304), 4xx client (401, 403, 404, 429), 5xx server (500, 502, 503).
- TLS 1.3: 1 RTT; TLS 1.2: 2 RTT; SNI names the host. HTTP/2: streams over TCP, blocked by one loss. HTTP/3: QUIC over UDP 443, per-stream recovery.

FORMULAS

Transmission L / R. Propagation d / v (v ≈ 2×10^8 m/s). Total delay = processing + queueing + transmission + propagation. Throughput ≤ W / RTT. BDP = R × RTT. Hosts = $2^h - 2$. Subnets = 2^b. Frame efficiency = payload / (payload + headers + 20).

FLASHCARDS

Q	A	Q	A
Layer of a router; of a switch?	3; 2	Compress 2001:0db8:0000:0000:0000:0000:0000:0001	2001:db8::1
PDU names, layer 4 down to 1	Segment, packet, frame, bits	EUI-64 steps	Insert FF:FE; flip bit 7
Fields that change per hop	TTL, header checksum, MAC addresses	Solicited-node multicast	ff02::1:ff + last 24 bits
Minimum and maximum Ethernet frame	64 and 1,518 bytes (1,522 tagged)	RA flags M and O	Stateful DHCPv6; other options by DHCPv6
Why 64 bytes?	Sender must still transmit when a collision returns: $L \geq 2T_pR$	ICMPv6 NDP types	133 to 137
Switch, unknown destination	Flood out of all ports except the arrival port	UDP header size; TCP minimum	8; 20 bytes
CRC remainder bits for generator 10011	4	What does a SYN consume?	One sequence number
Stop-and-Wait efficiency	1 / (1 + 2a)	Which side has TIME_WAIT?	The active closer
GBN and SR window, n-bit numbers	$2^n - 1$; 2^{n-1}	Many CLOSE_WAIT sockets mean	Application does not call close()
/27: block, hosts	32; 30	Effective window	min(rwnd, cwnd)
/30: hosts; /31: hosts	2; 2 (RFC 3021)	Slow start growth	Doubles each RTT
/22 mask	255.255.252.0	Reno on 3 duplicate ACKs	ssthresh = cwnd / 2; cwnd = ssthresh (+3 inflation)
Wildcard of /21	0.0.7.255	Reno on timeout	cwnd = 1
Private IPv4 ranges	10/8, 172.16/12, 192.168/16	CUBIC reacts to	Loss; cubic growth about W_{max}
APIPA range	169.254.0.0/16	BBR models	Bottleneck bandwidth and minimum RTT
CGNAT range	100.64.0.0/10	64 KB window, 100 ms RTT	5.24 Mb/s
Fragment offset unit	8 bytes	BDP of 1 Gb/s, 100 ms	12.5 MB
MSS on Ethernet; on PPPoE	1,460; 1,452	Why Nagle plus delayed ACK stalls	Each waits for the other; use TCP_NODELAY
ICMP for PMTUD	Type 3 code 4	SYN cookie holds	Hash of 4-tuple, secret and time in the ISN
Traceroute probes, Linux; Windows	UDP from port 33434; ICMP echo	TLS 1.3 handshake	1 RTT
ARP request destination MAC	ff:ff:ff:ff:ff:ff	DNS over TCP when	Truncation, zone transfer

DHCP ports	67 server, 68 client	CNAME at the apex	Not allowed (SOA and NS there)
DHCP T1; T2	50%; 87.5% of the lease	HTTP/3 transport	QUIC over UDP 443
Role of giaddr	Identifies the client subnet to the server	First step of the google.com walk	Browser caches and HSTS, then DNS
PAT distinguishes hosts by	Source port		

PART I · PILLAR 4

4

Routing and Switching, CCNA to CCNP ENCOR

A packet crosses a campus, an ISP and the Internet; we learn who decides each hop, and why.

4.1 Inside the box: planes and the CLI	SHOULD
4.2 Switching, VLANs, trunking, inter-VLAN routing	MUST
4.3 Spanning Tree and EtherChannel	MUST
4.4 The routing table, static and default routes	MUST
4.5 OSPF	MUST
4.6 EIGRP	OPTIONAL
4.7 BGP	MUST
4.8 First-hop redundancy	SHOULD
4.9 Route control: prefix lists, route maps, redistribution, VRF-lite	SHOULD
4.10 MPLS and L3VPN	SHOULD
4.11 Quality of Service	SHOULD
4.12 IP multicast	OPTIONAL
4.13 IP services and access lists on IOS	MUST
4.14 Tunnels and enterprise architecture	OPTIONAL
4.15 Mini-project: Campus-to-Internet	OPTIONAL

Every topology and lab in this chapter belongs to one fictional organisation, **Campus Net**: a university network with a sales-and-engineering office floor (VLANs 10 and 20), two border routers, BR1 and BR2, and two upstream providers, ISP-A (AS 100) and ISP-B (AS 200). Subnetting, NAT theory, the longest-prefix-match algorithm, Dijkstra, access-list theory and the internals of the forwarding hardware are taught in their home chapters, and are cited here by one sentence and a page reference. This chapter teaches what is configured on the devices, what the protocols do on the wire, and how each is verified. The CCNA topics are mixed with the CCNP ENCOR topics; the tags tell you which to read first.

4.1 Inside the box: planes and the CLI SHOULD · ■□□

A router or switch performs three jobs that run at three different speeds. The *management plane* (the CLI, SSH, SNMP, NETCONF) is how humans and tools configure it. The *control plane* (OSPF, BGP, spanning tree, ARP) runs on the CPU and builds tables. The *data plane* (the forwarding hardware) uses those tables to move transit packets at line rate, without involving the CPU. The tables are the routing information base (RIB) and the forwarding information base (FIB); how the FIB is held in TCAM is taught in §5.2. Here it is enough to know that a packet *addressed to the router* (an OSPF hello, an SSH session) is punted to the CPU, whereas a *transit* packet never leaves the fast path.

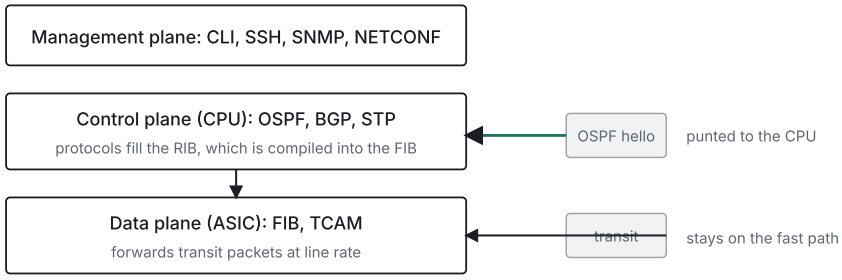


Figure 4.1 Three planes. A packet for the router (accent arrow) is punted to the control plane; a transit packet never leaves the data plane.

The Cisco IOS command line has modes that are visible in the prompt: `Router>` (user), `Router#` (privileged), `Router(config)#` (global configuration) and `Router(config-if)#` or `Router(config-router)#` (a sub-mode). `enable` and `configure terminal` go up a level, `exit` goes back one and `end` returns to privileged mode. `show running-config` displays the active configuration in RAM, whereas `show startup-config` displays the copy in NVRAM that survives a reload; `copy running-config startup-config` saves. Prefix a privileged command with `do` to run it from a configuration mode, and use `?` and the Tab key freely, because abbreviations (`sh ip int br`) are accepted when unambiguous.

WORKED EXAMPLE

The first fifteen minutes on a new Campus Net router.

1. Name the device and protect privileged mode with a hashed secret, because `enable password` is stored readably.
2. Create a local user and enable SSH, which needs a hostname, a domain name and an RSA key.
3. Restrict the virtual terminal lines to SSH, address the interfaces, add a banner and save.

```
hostname BR1
ip domain-name campus.example
enable secret S3cr3t!
username admin privilege 15 secret Adm1n!
crypto key generate rsa modulus 2048
ip ssh version 2
line vty 0 4
 login local
 transport input ssh
interface GigabitEthernet0/1
 ip address 192.0.2.2 255.255.255.252
 no shutdown
banner motd # Authorised use only #
end
copy running-config startup-config
```

```
BR1# show ip interface brief
Interface           IP-Address     OK? Method Status  Protocol
GigabitEthernet0/1  192.0.2.2      YES manual up      up
BR1# show ip ssh
SSH Enabled - version 2.0
```

Result: Status is the Layer 1 state and Protocol the Layer 2 state; “up/up” is the only healthy pair.

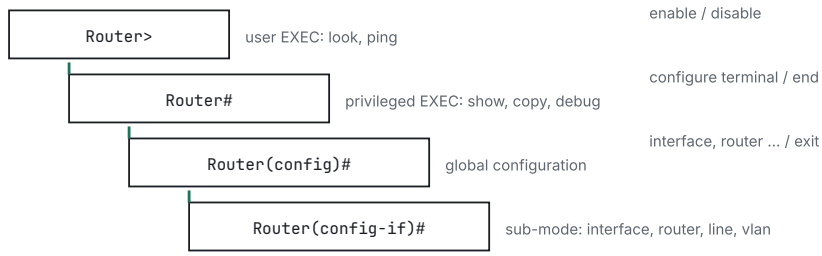
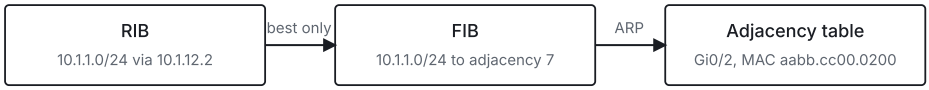


Figure 4.2 The CLI modes. The prompt names the mode: > user, # privileged, (config...)# a configuration mode.

A device boots in a fixed order: the ROM monitor runs a power-on self-test, loads the operating-system image from flash, and then reads the saved configuration from NVRAM; if none exists it offers the setup dialogue. The *configuration register* (`show version` prints it, normally 0x2102) controls this. Setting it to 0x2142 makes the router ignore the saved configuration at the next boot, which is the basis of password recovery: boot with 0x2142, copy the startup configuration into running configuration, change the secret, restore 0x2102 and save. Physical access is therefore equivalent to administrative access, which is why console ports are kept in locked rooms.

Packet forwarding has also evolved in three generations. *Process switching* sends every packet to the CPU, which looks up the route and rebuilds the header; it is the slowest and is used today only for packets addressed to the router. *Fast switching* caches the result of the first packet of a flow. *Cisco Express Forwarding* (CEF) precomputes two tables, the FIB (a copy of the best routes, organised for longest-prefix lookup) and the *adjacency table* (the Layer 2 rewrite string for each next hop, including its MAC address learnt by ARP), and forwards every packet by a single lookup in each. On a switch or high-end router those tables sit in hardware. A missing adjacency, for instance because ARP failed, produces packet drops that `show ip cef` and `show adjacency` reveal even when the route looks correct.



Control plane builds the RIB; CEF derives the FIB and the Layer 2 rewrite; the data plane reads only the last two.

Figure 4.3 From routing table to forwarding. A transit packet needs one FIB lookup and one adjacency rewrite.

The same three planes exist on every vendor, but the commands differ. The table maps the five commands one uses most.

Junos and FRR — Junos has a candidate configuration: you edit it, run `commit`, and can `rollback 1`. IOS applies each line instantly. FRR's `vtysh` is IOS-like, and Arista EOS is almost identical to IOS.

Task	Cisco IOS-XE	Junos	Arista EOS	FRR (`vtysh`)
Interface summary	`show ip interface brief`	`show interfaces terse`	`show ip interface brief`	`show interface brief`
Routing table	`show ip route`	`show route`	`show ip route`	`show ip route`
OSPF neighbours	`show ip ospf neighbor`	`show ospf neighbor`	`show ip ospf neighbor`	`show ip ospf neighbor`
BGP summary	`show ip bgp summary`	`show bgp summary`	`show ip bgp summary`	`show bgp summary`
Save configuration	`copy run start`	`commit`	`write memory`	`write memory`

Configuration is managed by copying it. `copy running-config tftp:` (or `scp:`) saves it off the device; `copy tftp: flash:` loads a new image; `show version` reports the image, the uptime and the register; `show running-config | section ospf` filters output. On a production device, change windows are protected by `configure replace` or by a rollback timer, because a mistyped command over SSH can cut the session that is needed to correct it.

CDP (Cisco) and LLDP (the open standard, IEEE 802.1AB) advertise a device's name, port and platform to its neighbours. `show cdp neighbors detail` is the quickest way to learn what is on the other end of a cable. Both are best disabled on ports facing untrusted users.

> LAB
>
> `LAB 4.1 · 15 MIN` Packet Tracer: two routers and one switch, base configuration and SSH as in the worked example. Then run the multi-vendor task on Containerlab, with one Nokia SR Linux and one Arista cEOS node.
>
> ```
> BR1# show cdp neighbors detail | include Device|IP address
> Device ID: SW1
>   IP address: 192.168.99.2
> ```
>
> **What you should notice:** CDP reveals the neighbour's address without a routing protocol. Fill in the equivalence table above for the other two vendors; the *concepts* (interface, route, neighbour) are the same, and only the commands differ.

> CHECK YOURSELF
>
> 1. What does it mean when `show ip interface brief` prints "up/down"? [1]
> 2. Why is `enable secret` preferred to `enable password`? [2]

4.2 Switching, VLANs, trunking and inter-VLAN routing

MUST · ■□□

BIG IDEA

A VLAN divides one physical switch into several broadcast domains, because the switch refuses to forward a frame to a port of a different VLAN; a router, or a Layer 3 switch, is therefore required to carry traffic between them.

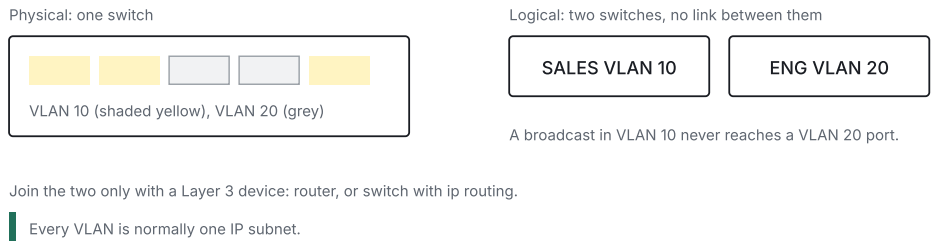


Figure 4.4 A VLAN is a logical switch. Ports of one colour behave as if they belonged to a separate device.

A Layer 2 switch forwards on destination MAC address. It *learns* by reading the source MAC of every arriving frame and recording the (MAC, port, VLAN) triple in its MAC address table, which entries expire after 300 seconds of silence by default. For a known destination the frame leaves one port; for a broadcast, a multicast (without snooping) or an *unknown unicast* the switch floods the frame out of every other port in the VLAN. Thus, without VLANs, a single switch is a single broadcast domain, and every host hears every ARP request.

```
SW1# show mac address-table dynamic vlan 10
Vlan    Mac Address       Type     Ports
  10    0050.7966.6801    DYNAMIC  Gi0/1
  10    0050.7966.6802    DYNAMIC  Gi0/24
```

The forwarding decision of a switch can be summarised in a table. It also explains the failure modes met later: a flood is cheap in a small VLAN, but ruinous in a loop.

Frame arriving in VLAN V	Destination	The switch
Unicast, MAC known	learnt on port P	forwards out of P only (filters if P is the arrival port)
Unicast, MAC unknown	not in table	floods out of all other ports in V (*unknown unicast flooding*)
Broadcast (ff:ff:ff:ff:ff:ff)	all hosts	floods out of all other ports in V
Multicast, no snooping	group	floods like a broadcast; with IGMP snooping only to joined ports
Source MAC new or moved	learning	records or updates the (MAC, port, VLAN) entry

Two consequences follow. First, a MAC address appearing on two ports in quick succession (“MAC flapping”) signals either a loop or a duplicated address. Secondly, the MAC table has finite size (a few thousand to hundreds of thousands of entries), so an attacker who sends frames

with random source addresses can fill it and force the switch to flood everything; port security limits the number of addresses per port (§9.5).

An *access port* belongs to one VLAN and sends and receives untagged frames. A *trunk port* carries several VLANs on one link, and marks each frame with an IEEE 802.1Q tag so that the far end knows its VLAN. The tag is four bytes inserted between the source MAC and the EtherType, so the frame check sequence is recomputed and the maximum frame grows from 1518 to 1522 bytes.

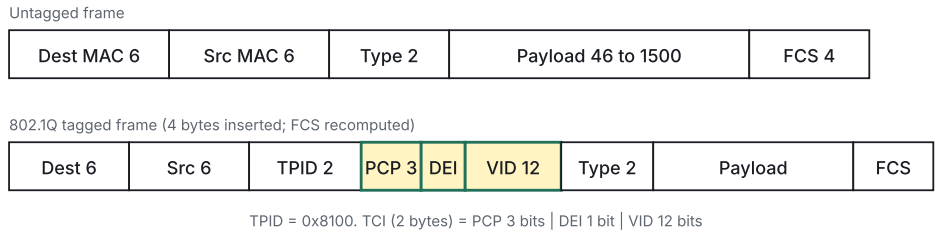


Figure 4.5 The 802.1Q tag. Twelve VID bits give 4096 values, of which 0 and 4095 are reserved, hence 4094 usable VLANs.

VLAN 1 is the default; 2 to 1001 are normal-range; 1002 to 1005 are reserved for legacy FDDI and Token Ring; 1006 to 4094 are extended. One VLAN on a trunk is the *native VLAN* (VLAN 1 by default), whose frames cross the trunk *untagged*. If the two ends disagree about it, frames leak between VLANs and CDP and spanning tree print a native-VLAN mismatch warning. Best practice is to set the native VLAN to an unused one (for example 99) on both ends and to prune the allowed list to the VLANs that need to cross.

WORKED EXAMPLE

Two VLANs, one trunk, router on a stick. Campus Net has SALES (VLAN 10, 192.168.10.0/24) and ENG (VLAN 20, 192.168.20.0/24). A SALES PC on SW1 and an ENG PC on SW2 must communicate. SW1 trunks to SW2 and to R1.

```
! SW1 and SW2
vlan 10
 name SALES
vlan 20
 name ENG
interface GigabitEthernet0/1
 switchport mode access
 switchport access vlan 10        ! PC on SW1 (SW2: access vlan 20)
interface GigabitEthernet0/24
 switchport mode trunk
 switchport trunk allowed vlan 10,20
 switchport trunk native vlan 99
! R1: one physical link, two sub-interfaces
interface GigabitEthernet0/0
 no shutdown
interface GigabitEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
interface GigabitEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
```

1. The SALES PC has no route to 192.168.20.0/24, so it ARPs for its gateway 192.168.10.1 and sends the packet to R1’s MAC (untagged on the access port).

2. SW1 tags the frame with VID 10 on the trunk to R1. R1 reads the tag and delivers it to sub-interface .10.
3. R1 looks up 192.168.20.0/24, a connected route out of .20, rewrites the Layer 2 header and sends the packet back on the same cable, now tagged VID 20.
4. SW1 forwards the VID 20 frame over the trunk to SW2, which removes the tag and delivers it on the ENG access port.

Result: the packet crosses the R1 link twice, which is why the design is called a hairpin and why that link limits inter-VLAN throughput.

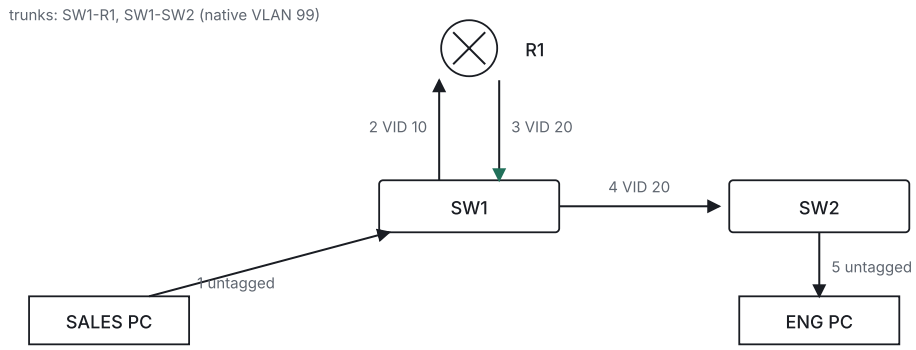


Figure 4.6 The router-on-a-stick hairpin. The packet enters R1 tagged 10 and leaves tagged 20 on the same link.

A *Layer 3 switch* removes the hairpin. Enabling `ip routing` and creating a *switched virtual interface* (SVI) per VLAN lets the switch route in hardware between its own VLANs: `interface Vlan10` with `ip address 192.168.10.1 255.255.255.0` replaces the sub-interface, and hosts keep the same gateway address. SVIs scale better than sub-interfaces, because the forwarding is done by the switching ASIC rather than by a single router link. A *routed port* (`no switchport`) is a Layer 3 port with no VLAN at all.

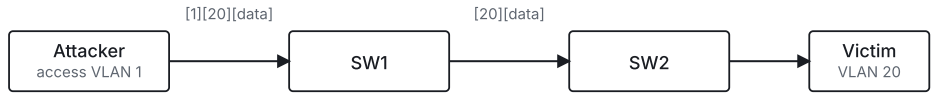
```
SW1(config)# ip routing
SW1(config)# interface Vlan10
SW1(config-if)# ip address 192.168.10.1 255.255.255.0
SW1# show ip route connected
C  192.168.10.0/24 is directly connected, Vlan10
```

Voice VLAN. An IP phone and the PC behind it share one access port. The port is given a second, *voice* VLAN (`switchport voice vlan 150`); CDP or LLDP-MED tells the phone which VLAN to tag, while the PC's frames stay untagged in the access VLAN. The port behaves as a trunk with just two VLANs, and the voice traffic can then be placed in a separate IP subnet and a priority queue (§4.11 (p. 258)).

```
interface GigabitEthernet0/5
 switchport mode access
 switchport access vlan 10
 switchport voice vlan 150
 spanning-tree portfast
 spanning-tree bpduguard enable
```

Two Layer 2 mechanisms deserve suspicion. The *Dynamic Trunking Protocol* (DTP) lets two ports negotiate a trunk; the outcome is in the table (a port set to `mode trunk` or `desirable` initiates, `auto` only agrees). An attacker who sends DTP frames to a port left on `dynamic auto`

can turn it into a trunk and see every VLAN. The *VLAN Trunking Protocol* (VTP) distributes the VLAN database from servers to clients, and a switch that joins with a higher revision number and an empty database overwrites the VLANs of the whole domain (the "VTP bomb"). The remedy is to configure ports explicitly (`switchport mode access` or `trunk`, plus `switchport nonegotiate`), to use VTP transparent or off, and to shut down unused ports. Port security, DHCP snooping and VLAN-hopping defences are taught in §9.5.



SW1 strips the outer tag (1) because VLAN 1 is the native VLAN of the trunk and is sent untagged, leaving tag 20 for SW2.
Defence: native VLAN unused (not 1) on every trunk; never put hosts in the native VLAN.

Figure 4.7 VLAN hopping by double tagging. The attack is one-way (the reply cannot return) and works only when the attacker's access VLAN equals the trunk's native VLAN. Further defences are in §9.5.

Local Peer	access	trunk	dynamic auto	dynamic desirable
access	access	mismatch	access	access
trunk	mismatch	trunk	trunk	trunk
dynamic auto	access	trunk	access	trunk
dynamic desirable	access	trunk	trunk	trunk

TRAP

Claim: two ports on `dynamic auto` will form a trunk. *Why it is wrong:* `auto` only responds to a negotiating peer and never initiates, so two `auto` ports both wait and settle on access mode. *Correct statement:* at least one end must be `trunk` or `desirable`; better, configure both ends statically.

WORKED EXAMPLE

Converting the design to an L3 switch. Campus Net upgrades the trunk-to-R1 design: SW1 becomes the gateway, and R1 is kept only for the border.

1. Enable routing in the switch: `ip routing`.
2. Create `interface Vlan10` and `interface Vlan20` with the addresses that R1's sub-interfaces had (192.168.10.1 and 192.168.20.1). Hosts need no change.
3. Remove the R1 sub-interfaces. Make SW1's uplink a routed port with `no switchport` and a /30 to R1, and exchange routes with OSPF or a static route.
4. Confirm: the VLAN must exist and have at least one active port in the VLAN, or the SVI stays down.

```
SW1# show ip interface brief | include Vlan
Vlan10   192.168.10.1  YES manual up   up
Vlan20   192.168.20.1  YES manual up   up
```

Result: inter-VLAN traffic never leaves SW1, and its capacity is the switching ASIC's, not one 1 Gb/s link's.

INTERVIEW LENS

CISCO · JUNIPER · ARISTA

Expected: the difference between access and trunk, what the native VLAN is, and the symptom of a mismatch; why 4094; ROAS against SVI. A strong answer volunteers the security point (DTP, native VLAN, VTP) and the hairpin bottleneck. Follow-ups: what is on the wire for a frame from VLAN 10 to VLAN 20; what happens when the trunk's allowed list omits a VLAN.

LAB

`LAB 4.2 · 15 MIN` Build Figure 4.6 in Packet Tracer, then break it twice.

```
SW1(config-if)# switchport trunk native vlan 1      ! on one end only
SW1(config-if)# switchport trunk allowed vlan remove 20
```

```
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on Gi0/24 (1),
with SW2 Gi0/24 (99).
SALES-PC> ping 192.168.20.10
Request timed out.   <-- VLAN 20 no longer crosses the trunk
```

What you should notice: the native mismatch is only a log message and traffic may still flow; the pruned VLAN silently fails. `show interfaces trunk` shows the third table ("VLANs in spanning tree forwarding state and not pruned") shrinking.

CHECK YOURSELF

1. Why does a trunk frame grow to 1522 bytes? [3]
2. A host on VLAN 10 can ping its gateway but not a host in VLAN 20 on another switch. Name two causes. [4]

4.3 Spanning Tree and EtherChannel MUST · ■■□

BIG IDEA

Ethernet frames carry no hop count, so a loop in the Layer 2 topology circulates a broadcast forever; the Spanning Tree Protocol (STP) therefore elects one root switch and blocks the minimum number of ports needed to leave a loop-free tree.

Redundant switch links are desirable, yet a loop produces three failures at once: a *broadcast storm* (frames multiply and fill every link), *MAC table instability* (the same source appears on alternating ports) and duplicate frame delivery. Switches exchange *bridge protocol data units* (BPDUs) every two seconds to agree on a tree. The agreement uses four steps in this order:

1. **Root bridge.** The switch with the lowest *bridge ID* (BID) wins. The BID is the priority followed by the MAC address.
2. **Root port**, one on every non-root switch: the port with the lowest *root path cost*; ties go to the lowest *sender BID*, then the lowest *sender port ID*.
3. **Designated port**, one on every segment: the port of the switch advertising the lowest root path cost on that segment (ties again by sender BID, then port ID). Every port of the root bridge is designated.
4. **Blocked** (alternate) ports: all remaining ports.



Figure 4.8 The bridge ID. The VLAN number occupies the low 12 bits, which is why the configurable priority must be a multiple of 4096.

A BPDU carries the fields needed for the election: the *root bridge ID* the sender believes in, the *root path cost* from the sender, the *sender's own bridge ID* and the *sender port ID* (port priority, 128 by default, and port number), plus the timers (hello, forward delay, max age) set by the root. Only the root generates BPDUs; every other switch relays an updated copy through its designated ports, adding the cost of the link on which it was received. That is why a non-root switch's root path cost is, in effect, the sum of the costs of the links its BPDU crossed. When a switch stops hearing the root's BPDUs on its root port for max-age (20 s), it assumes the path is lost and starts a new election. A *topology change* (a port moving to or from forwarding) is signalled up to the root, which tells every switch to shorten its MAC-table ageing to 15 s, so stale entries do not blackhole traffic.

Link costs depend on speed. The classic (short) values are 10 Mb/s = 100, 100 Mb/s = 19, 1 Gb/s = 4 and 10 Gb/s = 2; the long values used by `spanning-tree pathcost method long` and by MST are 1 Gb/s = 20,000 and 10 Gb/s = 2,000. Because per-VLAN spanning tree (PVST+) runs an independent tree per VLAN, the BID includes the VLAN, and the default priority for VLAN 1 is displayed as 32769.

WORKED EXAMPLE

Root election on the Campus Net triangle. Three switches in VLAN 1, all at default priority 32768 (shown 32769), MACs ending 0001 (SW1), 0002 (SW2) and 0003 (SW3), joined pairwise by 1 Gb/s links (cost 4).

Part 1. Default priorities.

1. Priorities tie, so the lowest MAC wins: SW1 is root. All its ports are designated.
2. SW2 and SW3 each reach the root by their direct link at cost 4 (the path through the third switch costs 8), so each direct port is a root port.
3. On the SW2-SW3 segment both switches offer root path cost 4. The tie goes to the lower sender BID, SW2. SW2's port is designated; SW3's port is not a root or designated port, hence **alternate (blocking)**.

Part 2. Set SW3's priority to 4096 (BID 4097 for VLAN 1).

4. SW3 now has the lowest BID and becomes root. SW1 and SW2 each reach SW3 directly at cost 4.
5. On the SW1-SW2 segment both offer cost 4; SW1's BID is lower, so SW1 is designated and SW2's port on that segment blocks.

Part 3. Two parallel links SW1-SW2 (Gi0/1 and Gi0/2), SW1 root again.

6. SW2 has two candidate root ports with equal cost 4 and an identical sender BID. The tie goes to the lowest *sender port ID*: SW1's Gi0/1 (128.1) beats Gi0/2 (128.2). SW2's port that hears 128.2 becomes alternate.

Result: three outcomes, each decided by a different step of the election: lowest MAC, lowest sender BID, lowest sender port ID.

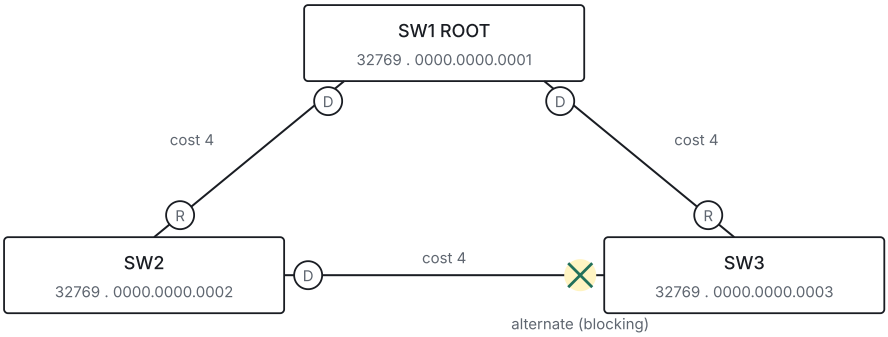


Figure 4.9 Part 1 of the election. R = root port, D = designated port, cross = blocked. Every port on the root is designated; SW3 blocks because SW2 has the lower BID.

TRAP

Claim: a root port is on the root switch and a designated port is the one that forwards toward it. *Why it is wrong:* the root bridge has no root port; the root port is on the *non-root* switch and faces the root. Designated ports are chosen per *segment*, not per switch. *Correct statement:* the root has only designated ports; each non-root switch has exactly one root port.

WORKED EXAMPLE

Load sharing with PVST+. The Campus Net triangle carries VLANs 10 and 20. Rather than let one tree decide, the designer makes SW2 root for VLAN 10 (`spanning-tree vlan 10 priority 4096`, BID 4106) and SW3 root for VLAN 20 (`spanning-tree vlan 20 priority 4096`, BID 4116). All links remain cost 4.

1. VLAN 10: root SW2. SW1 and SW3 use their direct links to SW2 as root ports. On the SW1-SW3 segment both offer cost 4; SW1's BID (32778, 0001) is lower than SW3's (32778, 0003), so SW1's port is designated and **SW3's port blocks**.
2. VLAN 20: root SW3. SW1 and SW2 use their links to SW3. On the SW1-SW2 segment SW1's BID is lower, so SW1 is designated and **SW2's port blocks**.
3. Result per link: SW1-SW2 forwards VLAN 10 and blocks VLAN 20 at SW2; SW1-SW3 forwards VLAN 20 and blocks VLAN 10 at SW3; SW2-SW3 forwards both.

```
SW3# show spanning-tree vlan 10 | include Gi0/1
Gi0/1   Altn BLK 4  128.1  P2p      <-- blocked for VLAN 10 only
SW3# show spanning-tree vlan 20 | include Gi0/1
Gi0/1   Desg FWD 4  128.1  P2p      <-- forwards VLAN 20
```

Result: every link carries traffic for at least one VLAN, which a single tree cannot do. The price is two trees to reason about, so the root of each VLAN should be aligned with the first-hop gateway of that VLAN.

Convergence. In 802.1D a port moves Blocking → Listening (15 s) → Learning (15 s) → Forwarding, and a topology change is detected only after the *max-age* timer (20 s) expires, so recovery takes up to 50 seconds. *Rapid STP* (802.1w) removes the timers. Its ports have three states (discarding, learning, forwarding) and four roles: root, designated, *alternate* (a backup path to the root through another switch) and *backup* (a second port on the same segment). When a link fails, the alternate port becomes the root port at once. On a new point-to-point link a *proposal/agreement* handshake lets a designated port go to forwarding after its neighbour blocks its other non-edge ports (the "sync"), which takes a fraction of a second. A neighbour is declared

dead after three missed hellos (6 s), not after max-age. An *edge port* (PortFast) goes straight to forwarding because it faces an end host.

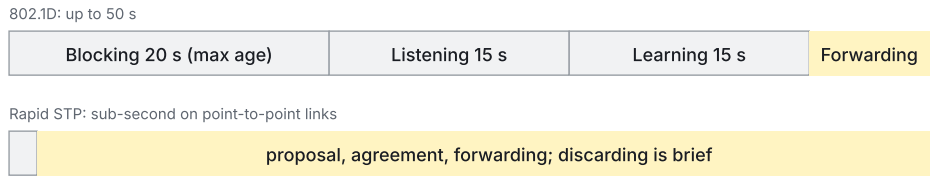


Figure 4.10 Convergence after a link failure, drawn to scale.

	802.1D	Rapid STP (802.1w)	PVST+ / Rapid PVST+	MST (802.1s)
Trees	one for all VLANs (CST)	one for all VLANs	one per VLAN	one per *instance*, each carrying many VLANs
Convergence	up to 50 s	under a second	802.1D / 802.1w speed	802.1w speed
Load sharing	none	none	by choosing a different root per VLAN	by mapping VLANs to instances
CPU and BPDU load	low	low	grows with VLAN count	low, scales to thousands of VLANs

MST groups VLANs into instances, so that 1000 VLANs can share two trees. Switches belong to the same *region* only if their region name, revision number and VLAN-to-instance mapping all match. On Cisco, `spanning-tree mode rapid-pvst` is the usual choice for a campus with up to a few hundred VLANs.

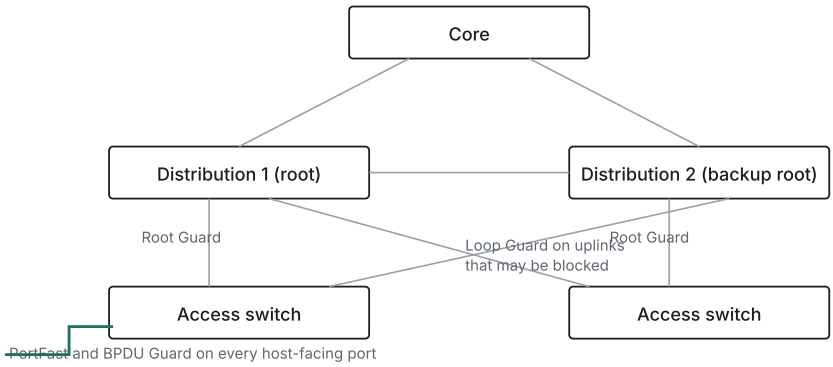


Figure 4.11 Where the guard features go in a campus: Root Guard downward from the distribution layer, Loop Guard on access uplinks, PortFast with BPDU Guard on edge ports.

Several safeguards complete the design. **PortFast** with **BPDU Guard** goes on access ports: PortFast skips the wait, and BPDU Guard puts the port in err-disabled state if a BPDU arrives, which stops a rogue switch. **Root Guard** goes on distribution downlinks to keep access switches from becoming root; a port that hears a superior BPDU moves to root-inconsistent. **Loop Guard** goes on non-designated uplinks, so that a port that suddenly stops receiving BPDUs (a unidirectional link) does not start forwarding. **UDLD** detects the unidirectional fibre itself.

```
spanning-tree mode rapid-pvst
spanning-tree vlan 1,10,20 priority 4096      ! root (use "root primary" to com-
pute it)
spanning-tree portfast default                ! all access ports become edge
ports
spanning-tree portfast bpduguard default
interface GigabitEthernet0/48                 ! distribution downlink
 spanning-tree guard root
interface GigabitEthernet0/1                  ! access uplink
 spanning-tree guard loop
```

```
SW3# show spanning-tree vlan 1
VLAN0001
  Spanning tree enabled protocol rstp
  Root ID    Priority    32769
             Address     0000.0000.0001
             Cost        4
             Port        1 (GigabitEthernet0/1)   <-- root port
Interface        Role Sts Cost      Prio.Nbr Type
Gi0/1            Root FWD 4         128.1    P2p
Gi0/2            Altn BLK 4         128.2    P2p   <-- alternate
```

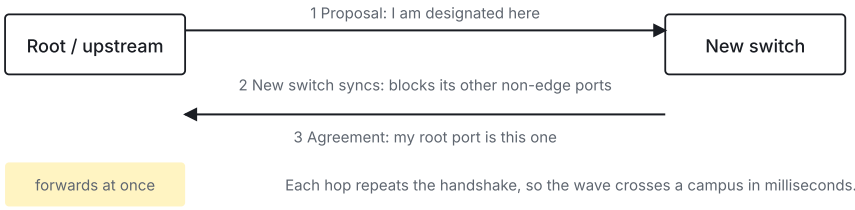


Figure 4.12 The Rapid STP proposal and agreement handshake on a point-to-point link.

EtherChannel bundles up to eight parallel links into one logical port-channel, which spanning tree treats as one link, so that no member is blocked and the bandwidth adds. Members must match in speed, duplex, VLAN list and trunk mode. Negotiation is by LACP (IEEE 802.1AX, modes `active` and `passive`), by Cisco's PAgP (`desirable`, `auto`) or statically with `on`. A bundle forms when at least one end initiates and both use the same protocol; `on` never negotiates, so it pairs only with `on`.

MST needs the instance map to be identical across the region. The mapping is configured once, and unmapped VLANs fall into instance 0 (the *internal spanning tree*, which also represents the region to the outside):

```
spanning-tree mode mst
spanning-tree mst configuration
 name CAMPUS
 revision 1
 instance 1 vlan 10,30
 instance 2 vlan 20,40
spanning-tree mst 1 priority 4096
spanning-tree mst 2 priority 8192
```

	active	passive	desirable	auto	on
active	yes	yes	no	no	no

passive	yes	no	no	no	no
desirable	no	no	yes	yes	no
auto	no	no	yes	no	no
on	no	no	no	no	yes

```
SW1(config)# interface range GigabitEthernet0/1-2
SW1(config-if-range)# channel-group 1 mode active
SW1# show etherchannel summary
Group  Port-channel  Protocol    Ports
1      Po1(SU)       LACP        Gi0/1(P)  Gi0/2(P)
```

The flags read: S = Layer 2, U = in use; P = bundled in the channel; `s` = suspended (the member's settings disagree), D = down. Traffic is spread over members by a hash of header fields (`port-channel load-balance src-dst-ip`), so one flow always uses one member, and a single flow never exceeds one link's speed. Multi-chassis versions (vPC, MLAG) that terminate a bundle on two switches are taught in §5.3.

A bundle is also a design tool: because the channel is one logical port, spanning tree sees one link and keeps it forwarding, and the bundle survives the failure of a member with no topology change. A *Layer 3 EtherChannel* (`no switchport` on the port-channel, then an IP address) carries routed traffic between switches without VLANs. Typical causes of a bundle that does not form or suspends members are tabulated.

Symptom in `show etherchannel summary`	Likely cause	Check
Members (`s`)	member settings differ from the bundle	`show run interface`, speed, duplex, trunk list
Members (`I`) stand-alone	partner is not running the same protocol, or silent	`show lacp neighbor`, partner mode
Port-channel (`SD`)	all members down	cabling, partner shutdown
Bundle up, one link saturated	hash puts many flows on one member	`port-channel load-balance`, `show etherchannel load-balance`
MAC flapping across members	one side `on`, other LACP	make both LACP

```
SW1(config)# port-channel load-balance src-dst-ip
SW1# test etherchannel load-balance interface Po1 ip 192.168.10.5 10.0.0.80
Would select Gi0/2 of Po1
```

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA
>
> STP is the most reliably asked Layer 2 topic. Expect a triangle or a square, and the request to name the root, root ports, designated ports and the blocked port. Show the four-step order aloud, because the examiner is checking the method more than the answer. Follow-ups: why 4096, why RSTP is fast, where each guard goes, and a "network freezes, MACs flap" scenario (a loop, found with `show spanning-tree` and the interface counters).

> LAB
>
> `LAB 4.3 · 20 MIN` Build the triangle, then fail the root port.

```
SW3(config)# spanning-tree mode rapid-pvst
SW3(config)# spanning-tree vlan 1 priority 4096
SW2(config)# interface GigabitEthernet0/1
SW2(config-if)# shutdown
```

```
SW3# show spanning-tree vlan 1 | include Root
  This bridge is the root
```

What you should notice: `show spanning-tree` on SW1 and SW2 now names SW3 as root. With the default 802.1D mode the alternate port takes 30 to 50 s to forward; with `rapid-pvst` pings recover in about a second. To see a storm, disable spanning tree on all three switches (`no spanning-tree vlan 1`) in simulation only.

CHECK YOURSELF

1. Why does the root bridge never have a root port? [5]
2. SW2's two links to the root carry cost 4 each. Which port blocks? [6]

4.4 The routing table, static and default routes MUST · ■□□

BIG IDEA

A router chooses among routes by three tests in a fixed order (the longest matching prefix, then the lowest administrative distance, then the lowest metric), because each test answers a different question: how specific, how trustworthy and how short.

The *routing table* holds one best route per prefix. When a packet arrives, the router finds the entry with the longest prefix that contains the destination address (the algorithm and its trie implementation are in §1.9 (p. 30)). Only if several *sources* offer the same prefix and mask does the *administrative distance* (AD) decide: it is a local trust rating from 0 to 255 assigned to each source of routes, and lower wins. Only if one protocol offers several paths to the same prefix does the *metric*, the protocol's own measure of path length, decide. Metrics from different protocols are never compared, since OSPF cost and RIP hop count measure different things.

Source	AD	Source	AD
Connected interface	0	IS-IS	115
Static route (next hop or interface)	1	RIP	120
EIGRP summary route	5	EIGRP external	170
eBGP	20	iBGP	200
EIGRP internal	90	DHCP-learned default	254
OSPF	110	Unusable	255

```
BR1# show ip route
Gateway of last resort is 192.0.2.1 to network 0.0.0.0
S*    0.0.0.0/0 [1/0] via 192.0.2.1
      10.0.0.0/8 is variably subnetted, 3 subnets, 2 masks
O        10.1.1.0/24 [110/20] via 10.1.12.2, 00:04:10, GigabitEthernet0/2
C        10.1.12.0/30 is directly connected, GigabitEthernet0/2
L        10.1.12.1/32 is directly connected, GigabitEthernet0/2
```

1. The line is up at Layer 1 but the protocol (Layer 2) is down, for example from a keepalive or encapsulation mismatch.↩
2. It stores a hash; `enable password` stores a readable (or weakly obfuscated) string.↩
3. The 4-byte 802.1Q tag is added to the maximum 1518-byte untagged frame.↩
4. VLAN 20 is missing from a trunk's allowed list or from SW2's VLAN database; or inter-VLAN routing (sub-interface or SVI) for VLAN 20 is down.↩
5. The root port is the best path *toward* the root; the root is already there.↩
6. The one connected to the root's higher sender port ID (for example 128.2 rather than 128.1).↩



Figure 4.13 One line of the routing table, token by token.

Read each line from the left. The code names the source (`C` connected, `L` local address of the router itself as a /32, `S` static, `O` OSPF, `B` BGP, `*` candidate default). Next come the prefix, then `[AD/metric]`, then the next hop, the age and the exit interface. The “gateway of last resort” is the default route.

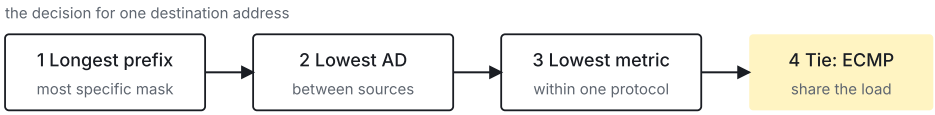


Figure 4.14 The route-selection order. Steps 2 and 3 matter only among candidates for the same prefix and mask.

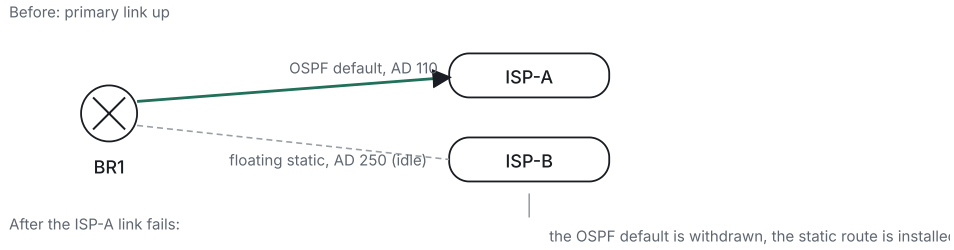


Figure 4.15 A floating static route as a standby. The route in the table changes from *OE2 to S* without any protocol exchange.

A destination that is covered by a *summary* static route can still be reached while the specific routes exist, and `Null0` prevents a loop: if a router advertises 10.0.0.0/8 but holds specifics for only part of it, a packet for an unused address would be sent to the default route and might return. The line `ip route 10.0.0.0 255.0.0.0 Null0` makes it drop instead; because the specifics are longer, they still win.

```
BR1(config)# ip route 10.0.0.0 255.0.0.0 Null0
BR1(config)# ipv6 route 2001:db8:9::/48 2001:db8:12::2
BR1(config)# ip route 172.16.0.0 255.255.0.0 10.1.12.2 10 name BACKUP-VIA-R2
BR1# show ip route 10.1.1.9
Routing entry for 10.1.1.0/24
  Known via "static", distance 1, metric 0
  Routing Descriptor Blocks:
  * 10.1.12.2
```

A *static route* is configured by hand. Pointing it at a next-hop address (`ip route 10.9.0.0 255.255.0.0 10.1.12.2`) needs a *recursive lookup* to find the exit interface; pointing it at an interface alone is acceptable only on point-to-point links, since on Ethernet the router would ARP

for every destination. A *default route* is the prefix 0.0.0.0/0, which matches everything with the shortest possible prefix, so it is used only when nothing else matches. A static route to `Null0` discards traffic, and is used to anchor a summary. The IPv6 forms are `ipv6 route 2001:db8:9::/48 2001:db8:12::2` and `ipv6 route ::/0 ...`. Equal-cost paths are installed together (ECMP, four by default on many IOS releases) and the load is spread per flow.

WORKED EXAMPLE

The routing-table puzzle. R1 holds four candidates: OSPF 10.1.0.0/16 [110/20], EIGRP 10.1.0.0/16 [90/3072], static 10.1.1.0/24 [1/0] and RIP 10.0.0.0/8 [120/2]. Where do three packets go?

1. Installed routes. The OSPF and EIGRP /16 entries have the same prefix and mask, so AD decides: EIGRP (90) is installed and OSPF (110) is held back. The static /24 and the RIP /8 have different masks, so both are installed.
2. 10.1.1.9 matches the /24, /16 and /8. The longest prefix is the /24, so it goes by the static route; AD is not consulted at all, because the candidates differ in mask.
3. 10.1.5.5 fails the /24 and matches the /16 (EIGRP) and the /8. The /16 wins.
4. 10.200.0.1 matches only the /8, so it goes to RIP.

Result: static /24, EIGRP /16, RIP /8; AD was used once, to choose between OSPF and EIGRP.

A *floating static route* is a static route given an AD higher than the dynamic protocol's, so it sits idle until the dynamic route disappears. For Campus Net's backup path to ISP-B, with the primary default learned by OSPF from the border at AD 110:

```
BR1(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.9 250
```

```
BR1# show ip route 0.0.0.0                  ! primary present
O*E2  0.0.0.0/0 [110/1] via 192.0.2.1
BR1# show ip route 0.0.0.0                  ! after the primary link fails
S*    0.0.0.0/0 [250/0] via 203.0.113.9     <-- floating route installed
```

When two or more routes of the same prefix, AD and metric exist, the router installs all of them (up to `maximum-paths`) and shares traffic by a per-flow hash, so that packets of one TCP connection stay in order. This is *equal-cost multipath* (ECMP). A static route can be given a lower or higher AD to engineer which is used: `ip route 172.16.0.0 255.255.0.0 10.1.12.2 10` is installed at AD 10, so it beats an OSPF route (110) for 172.16.0.0/16 but loses to a connected route. The same lever is used for protocols (`distance 105 ...`).

WORKED EXAMPLE

A routing table with eight entries and seven lookups. Table of R1 after convergence:

Prefix	Source	AD/metric
0.0.0.0/0	OSPF E2	110/1
10.0.0.0/8	static to Null0	1/0
10.1.0.0/16	OSPF	110/20
10.1.1.0/24	connected	0/0
10.1.1.0/25	static	1/0
10.1.2.0/24	EIGRP	90/3072

10.1.2.0/24	OSPF	110/30
192.168.0.0/16	eBGP	20/0

1. 10.1.1.10 matches the /24, /16, /8 and the /25 (0 to 127): the **/25 static** wins.
2. 10.1.1.200 is outside the /25, so the **connected /24** wins.
3. 10.1.2.5: the same /24 appears from EIGRP and OSPF; AD 90 beats 110, so **EIGRP**.
4. 10.1.9.9 matches only the /16 and /8: **OSPF /16**.
5. 10.50.0.1 matches only the /8: it goes to **Null0** (discarded), even though a default route exists, because the /8 is longer than the default.
6. 192.168.5.5: **eBGP**; and 8.8.8.8 matches only the default, so **OSPF E2 default**.

Result: specifics always beat defaults; AD arbitrated only for 10.1.2.0/24.

TRAP

Claim: a /16 connected route beats a /24 static route because connected has AD 0. *Why it is wrong:* AD is consulted only between routes for the same prefix and mask. *Correct statement:* the longest prefix is tested first, so a /24 static is used for traffic it matches whatever the AD of the shorter /16.

INTERVIEW LENS

CISCO · AMAZON · JUNIPER

Asked as “order the routing decision” or as a puzzle like the one above. State the order, then say that AD is evaluated once at install time and metric only inside one protocol. Follow-ups: why a floating static can be unreliable when the primary link stays up but the path is broken (use IP SLA tracking, §4.14 (p. 266)), and what a recursive lookup is.

LAB

`LAB 4.4 · 20 MIN` Packet Tracer: four routers in a square (R1-R2-R3-R4-R1), loopbacks 10.255.0.N/32, static routes only. Then add a floating route and test the failover.

```
R1(config)# ip route 10.255.0.3 255.255.255.255 10.1.12.2
R1(config)# ip route 10.255.0.3 255.255.255.255 10.1.14.4 200
R1(config)# interface GigabitEthernet0/1
R1(config-if)# shutdown
R1# traceroute 10.255.0.3
  1 10.1.14.4 4 msec
  2 10.1.34.3 8 msec                 <-- the path changed to the floating
route
```

What you should notice: before the shutdown the first hop is 10.1.12.2 (R2); afterwards `show ip route 10.255.0.3` shows `S [200/0]`. Repeat with `ipv6 route` and compare the table codes.

CHECK YOURSELF

1. A router learns 172.16.0.0/16 by eBGP and by OSPF. Which is installed? [1]
2. Why is `ip route 10.9.0.0 255.255.0.0 Gi0/1` unwise on an Ethernet link? [2]

4.5 OSPF MUST · ■■□

BIG IDEA

> OSPF is a link-state protocol: every router floods a description of its own links to all routers in its area, so that each builds the same map and independently computes shortest paths, which makes convergence fast and loop-free inside an area.

Each router describes itself in a *link-state advertisement* (LSA); the set of LSAs it holds is its *link-state database* (LSDB). Because every router in an area holds an identical LSDB, each runs the same shortest-path-first (SPF) computation, which is Dijkstra's algorithm (§1.11 (p. 34)), with itself as the root. The link metric is the *cost*, the reference bandwidth divided by the interface bandwidth. Routers become *neighbours* by exchanging Hello packets; neighbours that go on to synchronise databases are *adjacent*. OSPF runs directly over IP (protocol 89, TTL 1), sending to the multicast groups 224.0.0.5 (all OSPF routers) and 224.0.0.6 (DR and BDR only). Five packet types exist: Hello, Database Description (DBD, a catalogue of LSA headers), Link-State Request (LSR), Link-State Update (LSU, carrying full LSAs) and Link-State Acknowledgement (LSAck).

Hello and dead timers are 10 s and 40 s on broadcast and point-to-point links, and 30 s and 120 s on non-broadcast multi-access links. The *router ID* (RID), a 32-bit identifier, is chosen in this order: the `router-id` command, else the highest loopback address, else the highest active interface address.

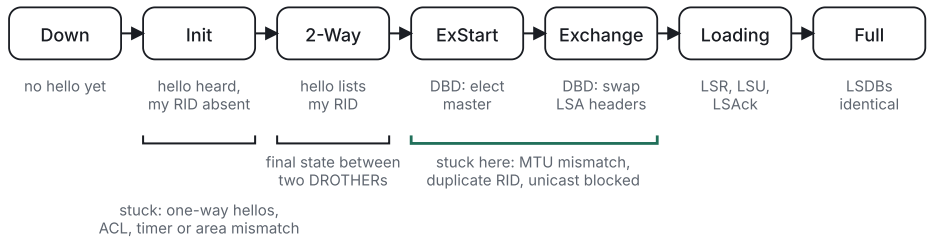


Figure 4.16 The OSPF neighbour state machine. A neighbour is adjacent only at Full; on a multi-access segment, 2-Way is the normal final state between two routers that are neither DR nor BDR.

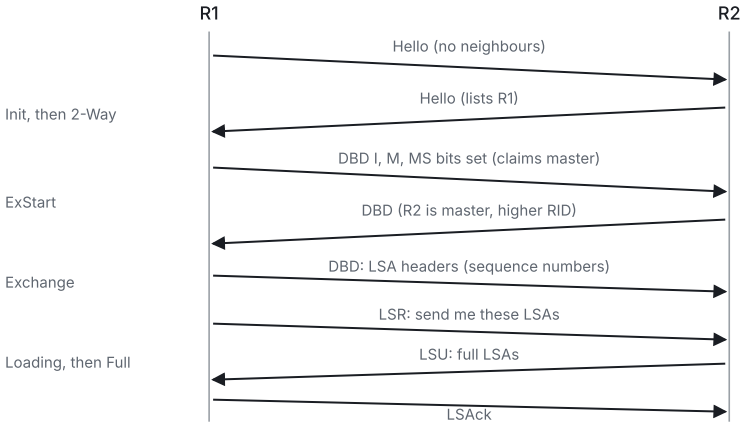


Figure 4.17 The packets of an OSPF adjacency, which Wireshark shows with the filter ospf. Bits in the DBD: I = initial, M = more, MS = master.

States, in order: *Down*; *Attempt* (non-broadcast networks only); *Init* (a hello has been heard, but it does not yet list our RID); *2-Way* (hellos are bidirectional; DR and BDR are elected here); *ExStart* (the two choose a master and initial sequence number); *Exchange* (DBDs list the LSA headers each holds); *Loading* (missing LSAs are fetched with LSR and delivered by LSU); *Full*.

IDs must differ. To reach Full, the interface MTU must also agree, since the MTU is carried in the DBD.

WORKED EXAMPLE

An adjacency stuck in EXSTART. BR1's Gi0/1 has MTU 1500 and the ISP-side router's has MTU 1400.

1. In the DBD, each side advertises its interface MTU. A router that receives a DBD advertising a *larger* MTU than its own interface can carry discards it.
2. Thus the 1400-byte side rejects BR1's DBDs. BR1 accepts the smaller ones and may reach Exchange, but its DBDs are never acknowledged, so it retransmits until the neighbour is declared down.
3. The symptom is a neighbour that cycles EXSTART/EXCHANGE and drops with "too many retransmissions". Hellos work, so the neighbour appears briefly healthy.

```
BR1# show ip ospf neighbor
Neighbor ID  Pri State        Dead Time  Address    Interface
2.2.2.2      1   EXSTART/DR   00:00:36   10.1.12.2  Gi0/1      <-- stuck
BR1# debug ip ospf adj
OSPF-1 ADJ Gi0/1: Nbr 2.2.2.2 has smaller interface MTU
```

Result: the fix is to make the MTUs equal (`ip mtu 1400` on both). `ip ospf mtu-ignore` removes the check but hides a real path-MTU problem, so treat it as a smell.

On a broadcast segment (Ethernet) with *n* routers, full adjacency among all would need n(n-1)/2 adjacencies and flood each LSA many times. OSPF therefore elects a *designated router* (DR) and a *backup* (BDR). Every other router (a *DROTHER*) becomes adjacent only with the DR and BDR: it sends updates to 224.0.0.6, and the DR re-floods them to 224.0.0.5. The election uses the interface *priority* (default 1; 0 means ineligible), then the highest RID. It is *non-preemptive*: a router joining later never displaces the current DR, whatever its priority. Point-to-point links need no election, and `ip ospf network point-to-point` on an Ethernet link between two routers avoids one.

WORKED EXAMPLE

DR election. R1 (priority 1, RID 1.1.1.1), R2 (1, 2.2.2.2), R3 (0), R4 (1, 4.4.4.4) boot together on one LAN.

1. R3 is excluded by priority 0. Among R1, R2 and R4 the priorities tie, so the highest RID wins: DR = R4, BDR = R2 (the next highest).
2. R5 joins with priority 255. Election is not preemptive, so nothing changes: R4 stays DR.
3. R4 fails. The BDR, R2, is promoted to DR, and a new BDR is elected among the rest, where R5's priority 255 beats R1's 1: BDR = R5.

Result: DR R4 then R2; BDR R2 then R5; R3 is never DR or BDR.

Network type	Default on	DR election	Hello / dead	Notes
Broadcast	Ethernet	yes	10 / 40	multicast hellos
Point-to-point	serial, tunnels	no	10 / 40	no DR; faster adjacency
Non-broadcast (NBMA)	Frame Relay, legacy	yes	30 / 120	neighbours configured by hand

Point-to-multipoint	configured	no	30 / 120	each neighbour a /32 route
Loopback	loopback	n/a	n/a	advertised as a /32 host route

Both ends of a link must agree on the *behaviour* implied by the type, for example both using or both skipping the DR election, although the hello timers can differ only if the dead timers are adjusted too (the timers are compared, not the type). A broadcast type on one end and point-to-point on the other forms an adjacency that does not work correctly, because one end expects the DR and the other does not.

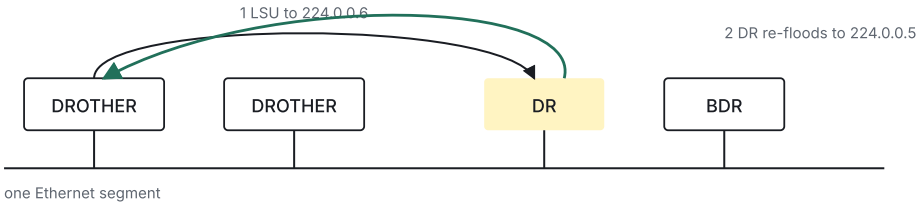


Figure 4.18 DR and BDR. A DROTHER sends to the DR and BDR group (224.0.0.6); the DR re-floods to all (224.0.0.5), so each LSA crosses the segment twice instead of n times.

The cost of an interface is the reference bandwidth divided by the interface bandwidth, with a minimum of 1. The default reference is 100 Mb/s, so Fast Ethernet is 1, and both Gigabit and 10 Gigabit Ethernet are also 1, because the quotient is truncated. The command `auto-cost reference-bandwidth 100000` (100 Gb/s) gives 1 Gb/s a cost of 100 and 10 Gb/s a cost of 10; it must be the same on every router in the domain.

WORKED EXAMPLE

An SPF calculation. Link costs: R1-R2 10, R1-R3 5, R3-R2 3, R2-R4 1, R3-R4 10. Which path does R1 use to reach R4, where a loopback 10.255.0.4/32 of cost 1 lives?

1. Candidate paths: R1-R2-R4 costs 10 + 1 = 11; R1-R3-R4 costs 5 + 10 = 15; R1-R3-R2-R4 costs 5 + 3 + 1 = 9.
2. Dijkstra settles R3 first (cost 5), then R2 via R3 (8, better than the direct 10), then R4 (9).
3. The loopback adds its own cost of 1.

```
R1# show ip route ospf
O   10.255.0.4/32 [110/10] via 10.1.13.3, 00:00:21, GigabitEthernet0/2
```

Result: next hop is R3; the metric is 9 + 1 = 10. The longer-looking path wins because its links are cheaper.

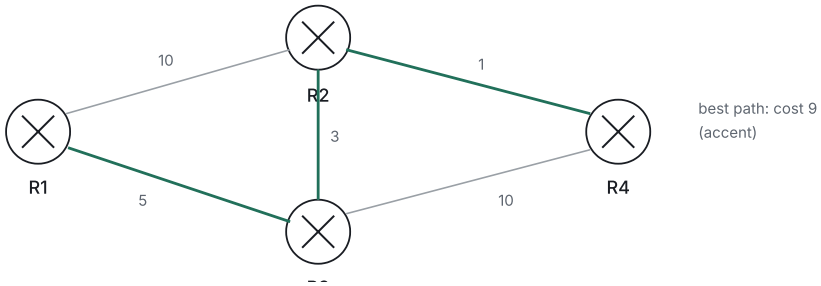


Figure 4.19 The shortest-path tree rooted at R1. The path R1-R3-R2-R4 costs 9; R1-R2-R4 would cost 11.

Large domains are divided into *areas* to limit the size of each LSDB and to confine the effect of flapping links. Area 0, the *backbone*, must connect every other area, because inter-area routes travel as distance vectors through it. A router with interfaces in two areas is an *area border router* (ABR); a router that injects routes from outside OSPF is an *autonomous system boundary router* (ASBR). If an area cannot physically touch area 0, a *virtual link* through a transit area patches the gap.

Type	Name	Originated by	Describes	Flooded
1	Router	every router	its own links and costs	within the area
2	Network	the DR	the routers on a multi-access segment	within the area
3	Summary	an ABR	a prefix from another area (with cost)	into adjacent areas
4	ASBR summary	an ABR	the *location* of an ASBR	into adjacent areas (not the ASBR's own)
5	AS external	an ASBR	a route from outside OSPF	whole domain except stub areas
7	NSSA external	an ASBR inside an NSSA	an external route	inside the NSSA; the ABR converts it to type 5

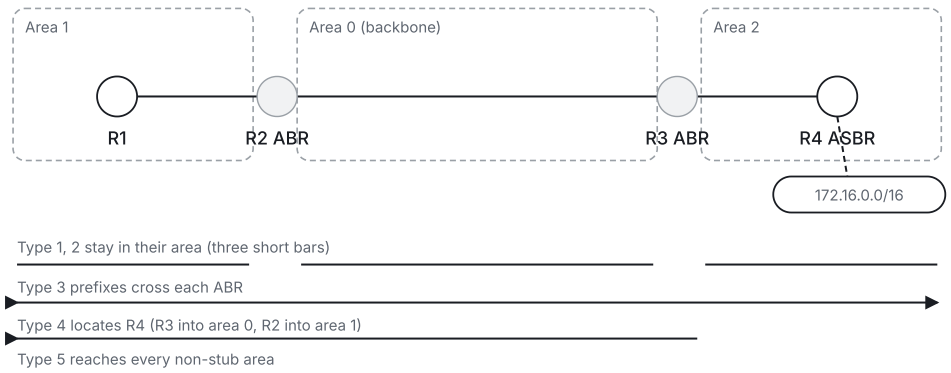


Figure 4.20 Flooding scope of the LSA types in a three-area network. R1 in area 1 holds type 3 LSAs for area 2's prefixes, type 4 for R4 and type 5 for 172.16.0.0/16.

The area types control which LSAs may enter, to shrink the LSDB of small routers. A *stub area* blocks types 4 and 5 and the ABR injects a default route as a type 3. A *totally stubby area* (Cisco: `stub no-summary`) also blocks types 3, leaving only intra-area LSAs and one default. A *not-so-stubby area* (NSSA) blocks types 4 and 5 but lets an ASBR inside the area originate type 7, which the ABR translates to type 5 on entering the backbone. A *totally NSSA* also blocks type 3, with a default from the ABR.

Area type	LSA 1, 2	LSA 3	LSA 4, 5	LSA 7	Default route
Normal	yes	yes	yes	no	none (optional)
Stub	yes	yes	no	no	type 3, automatic
Totally stubby	yes	default only	no	no	type 3, automatic

NSSA	yes	yes	no	yes	type 7, if configured
Totally NSSA	yes	default only	no	yes	type 3, automatic

WORKED EXAMPLE

Converting area 1 to totally stubby. In Figure 4.20, R1 holds type 1 (R1, R2), type 3 (prefixes of areas 0 and 2), type 4 (R4) and type 5 (172.16.0.0/16).

1. Configure `area 1 stub` on R1 and on the ABR R2. Hello packets carry a stub flag, so both must agree or the adjacency fails.
2. On R2 only, add `no-summary`. R2 stops flooding types 3, 4 and 5 into area 1 and injects a single type 3 default instead.

```
R2(config-router)# area 1 stub no-summary
R1# show ip route ospf
O*IA  0.0.0.0/0 [110/2] via 10.1.12.2, GigabitEthernet0/1
```

Result: R1's database shrinks to type 1 and 2 LSAs plus one default; traffic to everything else leaves through the ABR.

WORKED EXAMPLE

An LSA inventory before and after a stub conversion. Take Figure 4.20, with an extra internal prefix 10.2.0.0/24 in area 2 and the external prefix 172.16.0.0/16 injected by R4.

Router	Type 1	Type 2	Type 3	Type 4	Type 5
R1 (area 1), normal	1, 2	none	area 0 and 2 prefixes (via R2)	R4	172.16.0.0/16
R1 (area 1), stub	1, 2	none	same, plus default	none	none
R1 (area 1), totally stubby	1, 2	none	default only	none	none
R3 (ABR, areas 0 and 2)	its own LSAs per area	if multi-access	area-1 and area-2 prefixes	R4	172.16.0.0/16

1. In the normal state R1 must run SPF on a database that includes every inter-area prefix, so every flap in area 2 reaches R1.
2. As a stub, the type 4 and 5 LSAs vanish and R2 originates a default (type 3, cost 1 by default).
3. As totally stubby, a single default replaces all type 3 LSAs: the area survives any change elsewhere.

```
R2# show ip ospf database
            Router Link States (Area 1)
Link ID     ADV Router    Age   Seq#        Link count
1.1.1.1     1.1.1.1       412   0x80000007  2
2.2.2.2     2.2.2.2       403   0x80000009  2
            Summary Net Link States (Area 1)
0.0.0.0     2.2.2.2       380   0x80000001               <-- the default in-
jected into the stub
            Type-5 AS External Link States
172.16.0.0  4.4.4.4       951   0x80000003
```

Result: the database of the stub router shrinks from every prefix in the domain to its own area's LSAs plus one default; this is the mechanism behind the flapping-routes answer in Q4.26.

WORKED EXAMPLE

An NSSA with its own ASBR. Campus Net's engineering branch (area 2) has a small partner network attached through R5, which redistributes 172.20.0.0/16 into OSPF. A stub area cannot contain an ASBR, because type 5 LSAs are forbidden there, so area 2 is made an NSSA.

1. R5 (ASBR) originates the prefix as a **type 7** LSA, flooded only inside area 2.
2. The NSSA ABR (R3) translates it into a **type 5** LSA and floods that into area 0, so the rest of the domain sees an ordinary external route, `O E2`.
3. Inside area 2 the same route shows as `O N2`.

```
R3(config-router)# area 2 nssa
R5(config-router)# redistribute static subnets
R5# show ip route ospf | include 172.20
O N2  172.20.0.0/16 [110/20] via 10.2.0.1
R1# show ip route ospf | include 172.20
O E2  172.20.0.0/16 [110/20] via 10.1.12.2
```

Result: R1 never saw a type 7; the NSSA ABR is the only place it exists in translated form.

WORKED EXAMPLE

Summarising at the ABR. Area 3 holds 200 customer routes in 10.3.0.0/16. Without summarisation each of them becomes a type 3 LSA in area 0 and beyond.

```
R3(config-router)# area 3 range 10.3.0.0 255.255.0.0
R3(config-router)# area 3 range 10.3.0.0 255.255.0.0 cost 50
R3(config)# ip route 10.3.0.0 255.255.0.0 Null0
```

1. The ABR now advertises only 10.3.0.0/16 (type 3) into area 0, so a flap of one /24 inside area 3 stays in area 3.
2. The summary is kept as long as at least one component prefix is up; its cost is the highest of the components, unless a fixed cost is set.
3. The Null0 route stops a loop for addresses inside the /16 that have no specific route (the ABR discards instead of following its default route).

Result: backbone routers hold one LSA instead of 200, and area 3's instability is invisible to them.

Authentication of OSPF messages is configured per interface or per area. Modern IOS-XE supports SHA keychains (`ip ospf authentication key-chain`), and older MD5 uses `ip ospf message-digest-key 1 md5 SECRET`. Authentication failures keep the neighbour in Down or Init because the hellos are discarded. For IPv6, the equivalent is:

```
ipv6 unicast-routing
interface GigabitEthernet0/2
 ipv6 address 2001:db8:12::1/64
 ipv6 ospf 1 area 0
 ipv6 ospf authentication ipsec spi 256 sha1 <key>
ipv6 router ospf 1
 router-id 1.1.1.1
```

Routes carry a code that shows how they were learnt: `O` (intra-area), `O IA` (inter-area, from a type 3), `O E1` and `O E2` (external, type 5) and `O N1` / `O N2` (NSSA). A path's preference is `O`, then `O IA`, then `E1`, then `E2`, regardless of cost. An E2 route carries only the external cost, which is the default and is what is compared by all routers; an E1 route adds the internal cost to the ASBR, and is preferred when there are several ASBRs at different distances.

Three techniques contain flapping. *Summarisation* at the ABR (`area 3 range 10.3.0.0 255.255.0.0`) or at the ASBR (`summary-address`) hides detail so that one flapping /24 inside the range does not trigger SPF elsewhere. A *stub area* removes external LSAs. *Throttling* (`timers throttle spf 50 200 5000`) delays repeated SPF runs, and BFD detects failures in milliseconds so that timers can stay conservative. OSPFv3 uses the same states and algorithm for IPv6, but runs per link rather than per subnet, uses link-local addresses as next hops, sends to ff02::5 and ff02::6, and moves addresses out of the router and network LSAs into new LSA types (8, the link LSA, and 9, the intra-area-prefix LSA).

```
BR1(config)# router ospf 1
BR1(config-router)# router-id 1.1.1.1
BR1(config-router)# auto-cost reference-bandwidth 100000
BR1(config-router)# passive-interface default
BR1(config-router)# no passive-interface GigabitEthernet0/2
BR1(config)# interface GigabitEthernet0/2
BR1(config-if)# ip ospf 1 area 0
BR1(config-if)# ip ospf network point-to-point
```

Verification uses three commands: `show ip ospf neighbor` (state and DR role), `show ip ospf database` (the LSAs by type, and their age) and `show ip ospf interface` (cost, timers, network type, DR). The commonest adjacency faults are, in order: mismatched hello or dead timers, mismatched area, mismatched MTU, a passive interface, duplicate router IDs and a wrong authentication key.

Junos and FRR — Junos: `set protocols ospf area 0 interface ge-0/0/2.0`. FRR: `router ospf` then `network 10.1.12.0/30 area 0`, or `ip ospf area 0` on the interface.

TRAP

Claim: a router with priority 255 becomes DR as soon as it is configured. *Why it is wrong:* the DR election is not preemptive, so the current DR stays until it fails or OSPF is cleared. *Correct statement:* priority matters at election time; to force a change, clear the process (`clear ip ospf process`) after setting priorities. Similarly, 2-Way between two DROTHERs is correct and not a fault.

WORKED EXAMPLE

The six-fault adjacency challenge. A pre-built Campus Net lab has six independent faults; each appears as a neighbour that is missing or stuck. For each, the fastest diagnostic and the fix are below.

Symptom	Evidence	Cause	Fix
No neighbour, interface up	`show ip ospf interface` says hello 10, dead 40 on one side and 5, 20 on the other	hello/dead mismatch	match `ip ospf hello-interval` and `dead-interval`
No neighbour	`%OSPF-4-ERRRCV: mismatch area ID`	area mismatch	`ip ospf 1 area N` the same on both
Stuck in EXSTART	`debug ip ospf adj`: smaller interface MTU	MTU mismatch	equalise `ip mtu`
No neighbour, passive	`show ip protocols`: interface listed as		

	passive	passive interface	`no passive-interface`
Neighbour flaps in 2-Way/Down	log: duplicate router ID	two routers share a RID	set unique `router-id`, `clear ip ospf process`
Down or Init, hellos dropped	`debug ip ospf events`: mismatched authentication	wrong key or type	correct the key-chain

Result: in each case one `show` or `debug` line identifies the fault; always read the debug before changing configuration.

INTERVIEW LENS

CISCO · JUNIPER · NOKIA · AMAZON

The state list and the "stuck in EXSTART/INIT/2-WAY" question are asked at all of these. Strong answers give the cause for each (MTU, one-way hello or filter, normal for DROTHERs) and a command that proves it. The LSA-types table is a near certainty at Cisco and Juniper; learn who originates each and where it floods. The reported scenario "200 routes in one area flap and SPF runs everywhere" is solved by summarisation at the ABR, stub area and throttling.

LAB

`LAB 4.5 · 30 MIN` FRR Containerlab, three routers R1, R2, R3 in areas 1, 0, 2.

```
vtysh -c "show ip ospf database"
vtysh -c "configure terminal" -c "router ospf" -c "area 1 stub no-summary"
vtysh -c "show ip ospf database"
```

```
Summary Link States (Area 1)        <-- before: many; after: only 0.0.0.0
Type-5 AS External Link States      <-- absent in area 1 after conversion
```

What you should notice: the database of the stub router shrinks, and a default route appears. Then capture with Wireshark (`ospf` display filter) while you bring up a link, and match the DBD, LSR, LSU and LSAck packets to Figure 4.16.

CHECK YOURSELF

1. With the default reference bandwidth, what is the cost of a 1 Gb/s link, and why does it matter? [3]
2. Which LSA type does R1 in a totally stubby area still see besides types 1 and 2? [4]

1. eBGP (AD 20), because the prefix and mask are identical and 20 is below OSPF's 110.↩
2. The router treats 10.9.0.0/16 as directly connected and ARPs for every destination in it; this loads the segment and needs proxy ARP on the neighbour.↩
3. It is 1, because 100 / 1000 is rounded up to the

4.6 EIGRP OPTIONAL · ■■□

BIG IDEA

EIGRP is Cisco's advanced distance-vector protocol: it keeps a pre-verified loop-free backup path, the feasible successor, so that when the best path fails the router switches at once without asking its neighbours.

EIGRP (IP protocol 88, multicast 224.0.0.10) forms neighbours when the autonomous-system number, the K values, the subnet and any authentication match. It sends Hello, Update, Query, Reply and ACK packets, using its own reliable transport. The *DUAL* algorithm works on these quantities for each destination: the *reported distance* (RD) is the neighbour's own metric to the destination; the *feasible distance* (FD) is our best total metric; the *successor* is the neighbour on the best path. A neighbour is a *feasible successor* if its RD is below our FD, which is the *feasibility condition*. This guarantees the neighbour's path does not pass through us, because a path through us would have an RD at least as large as our FD, and so it is loop-free by construction.

With the default K values (K1 = K3 = 1) the classic metric is 256 × (10^7 / lowest bandwidth in kb/s + sum of delays in tens of microseconds), each part as an integer. Wide metrics (64-bit) replace this on named-mode configurations so that 10 Gb/s and faster links remain distinguishable.

WORKED EXAMPLE

An EIGRP metric and a feasibility test.

1. *Metric.* A path crosses a T1 (1544 kb/s, delay 20,000 µs) and a Gigabit link (delay 10 µs). Bandwidth term: 10^7 / 1544 = 6476. Delay term: (20,000 + 10) / 10 = 2001. Metric = 256 × (6476 + 2001) = 2,170,112.
2. *Feasibility.* Paths to 10.9.9.0/24 are: via R2, FD 3000 (RD 1000), the successor; via R3, RD 2500; via R4, RD 3500. R3's RD 2500 is below 3000, so R3 is a feasible successor. R4's RD 3500 is not, so it is not feasible (it might loop).
3. *Variance.* With `variance 2` EIGRP may also use paths whose FD is below 2 × 3000 = 6000. R3 qualifies if its FD is below 6000. R4 is never used, because unequal-cost load balancing still requires feasibility.

Result: 2,170,112; the feasible successor is R3; the used paths are R2 and R3.

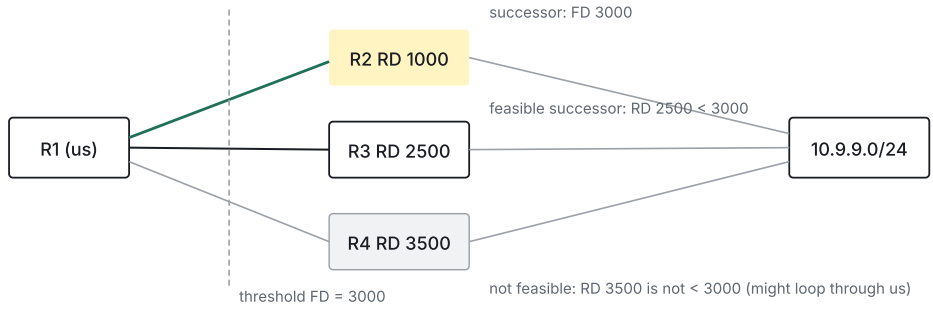


Figure 4.21 The DUAL feasibility condition. A neighbour is a safe backup only if it is closer to the destination than our own best distance.

If the successor fails and a feasible successor exists, it is promoted instantly and no packet is sent. If none exists, the router goes *active* for the route and sends *queries* to its neighbours, which may query theirs in turn. A route whose replies do not arrive in three minutes becomes *stuck in active*

minimum of 1; hence 1 Gb/s, 10 Gb/s and 100 Mb/s links all look equal and OSPF cannot prefer the faster one.↩

4. One type 3 LSA, the default route from the ABR.↩

(SIA) and the neighbour is reset. Two defences limit this: *summarisation* (a query for a prefix covered by a summary is answered at once) and *EIGRP stub* routers, which are not asked at all.

If no feasible successor exists, the router does not simply fail over; it queries. The query spreads outwards until a router either has an answer or is a *stub*. Thus the size of the *query domain* determines how long convergence takes and how likely SIA is, which is why large EIGRP networks use summarisation at distribution layers and stubs at branches.

```
R1# show ip eigrp topology 10.9.9.0/24
P 10.9.9.0/24, 1 successors, FD is 3000
        via 10.1.2.2 (3000/1000), GigabitEthernet0/1       <-- successor
        via 10.1.3.3 (5400/2500), GigabitEthernet0/2       <-- feasible suc-
cessor (2500 < 3000)
```

```
router eigrp CAMPUS
 address-family ipv4 unicast autonomous-system 100
  network 10.1.0.0 0.0.255.255
  af-interface default
   passive-interface
  exit-af-interface
  af-interface GigabitEthernet0/2
   no passive-interface
```

LAB

`LAB 4.6 · 25 MIN` Packet Tracer, five routers in a ring with a chord. Run EIGRP AS 100, inspect the topology table, then shut the successor link.

```
R1# show ip eigrp topology all-links
R1(config)# interface GigabitEthernet0/1
R1(config-if)# shutdown
R1# debug eigrp packets query
```

What you should notice: when a feasible successor exists the route flips with no debug output (no queries). Remove the chord so none exists, shut again, and queries appear. Make the far router an `eigrp stub` and the query disappears.

	OSPF	EIGRP
Algorithm	link-state, SPF	DUAL (distance-vector with loop-free backups)
Standard	open (RFC 2328)	Cisco-origin, informational RFC 7868
Metric	cost from bandwidth	bandwidth and delay (load, reliability optional)
Load sharing	equal-cost only	also unequal-cost (variance)
AD	110	90 internal, 170 external
Hierarchy	areas, strict	summarisation anywhere, no areas

4.7 BGP: the routing protocol of the Internet MUST · ■■■

BIG IDEA

> BGP is a path-vector protocol between autonomous systems that carries policy attributes with each route, and the best route to each prefix is chosen by an ordered list of tie-breakers; it is thus the tool with which operators choose where traffic goes, rather than simply find the shortest path.

An *autonomous system* (AS) is a set of networks under one administration with a single routing policy, identified by an AS number: 16-bit (1 to 65535, with 64512 to 65534 private) or 32-bit (private range 4,200,000,000 and above). BGP between routers of different ASes is *eBGP*; between routers of the same AS it is *iBGP*. BGP runs over TCP port 179, which provides reliable delivery and removes the need for BGP to retransmit. A route is advertised as *NLRI* (a prefix and length) together with *path attributes*, the most important of which, the AS_PATH, is the list of ASes the advertisement has crossed. A router rejects any route whose AS_PATH contains its own AS, which is the loop-prevention rule.

Messages: OPEN (version, AS number, hold time, BGP identifier, capabilities), UPDATE (withdrawn prefixes, path attributes, NLRI), KEEPALIVE, NOTIFICATION (an error; the session closes) and ROUTE-REFRESH. Cisco's default timers are keepalive 60 s and hold time 180 s, and the lower of the two peers' hold times is used.

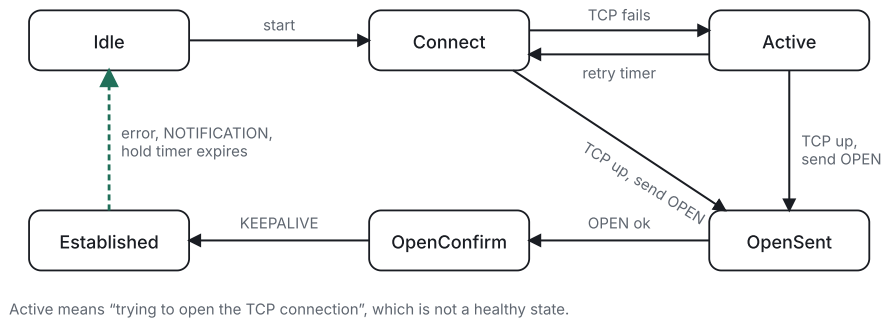


Figure 4.22 The BGP finite-state machine. Only Established exchanges UPDATEs; any error returns the session to Idle.

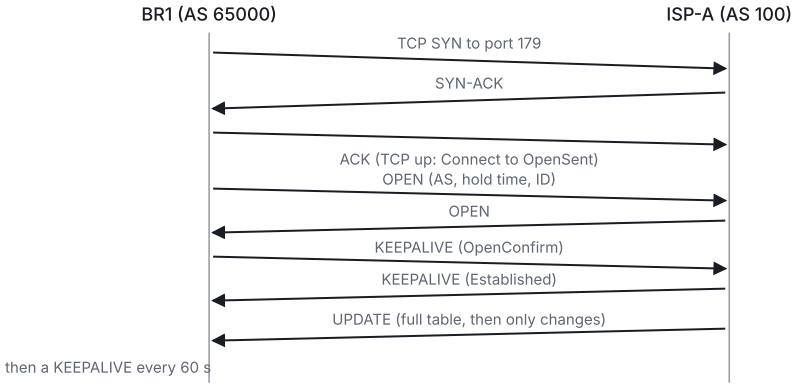


Figure 4.23 A BGP session. If both sides open a TCP connection at once, the one initiated by the router with the higher BGP identifier is kept.

Neighbours are configured explicitly (BGP does not discover them). An eBGP session between directly connected addresses needs only the address and the AS number. An iBGP session is best run between *loopbacks*, because a loopback stays up as long as any path exists, so `update-source Loopback0` tells BGP to originate the TCP session from that address. eBGP sessions

between loopbacks (to use several parallel links) need `ebgp-multihop 2`, since the default TTL of 1 would expire at the first router.

```
BR1(config)# router bgp 65000
BR1(config-router)# bgp router-id 10.255.0.1
BR1(config-router)# neighbor 192.0.2.1 remote-as 100           ! eBGP, direct
BR1(config-router)# neighbor 10.255.0.2 remote-as 65000        ! iBGP, loopback
BR1(config-router)# neighbor 10.255.0.2 update-source Loopback0
BR1# show ip bgp summary
BGP router identifier 10.255.0.1, local AS number 65000
Neighbor     V   AS MsgRcvd MsgSent TblVer InQ OutQ Up/Down  State/PfxRcd
192.0.2.1    4  100    1021     988     12   0    0 02:11:07        1500   <--
Established
10.255.0.2   4 65000      95      93     12   0    0 01:20:44           2
10.255.0.9   4 65000       0       0      0   0    0 never      Active   <--
trying
```

The states are: *Idle* (no resources allocated; waiting for a start event, and a persistent Idle often means no route to the neighbour or a refused session); *Connect* (a TCP connection is being attempted); *Active* (the attempt failed and the router is retrying and listening: not a good state); *OpenSent* (OPEN sent, awaiting the peer's); *OpenConfirm* (OPEN accepted, awaiting a KEEPALIVE); *Established*. In `show ip bgp summary` the last column shows the number of received prefixes when Established and the state name otherwise.

Reading the BGP table is a skill in its own right. The status codes at the left (`*` valid, `>` best, `i` learnt from iBGP, `r` RIB failure, `s` suppressed) and the columns are explained in the annotated output below.

```
BR1# show ip bgp
   Network          Next Hop        Metric LocPrf Weight Path
*> 0.0.0.0          192.0.2.1            0    200      0 100 i
*> 198.51.100.0/24  0.0.0.0              0         32768 i
*  203.0.113.0/24   192.0.2.5           10    100      0 200 300 i
*>i                 10.255.0.2           0    100      0 100 300 i
 |  |                  |                |     |       |   |    |
 |  |                  |                |     |       |   |    +-- origin: i
IGP, e EGP, ? incomplete
 |  |                  |                |     |       |   +------- AS_PATH,
nearest AS first
 |  |                  |                |     |       +----------- weight (32768
= locally originated)
 |  |                  |                |     +------------------- LOCAL_PREF
(iBGP only)
 |  |                  |                +------------------------- MED
 |  |                  +-------------------------------------------- next hop
(0.0.0.0 = local)
 |  +------------------------------------------------------------ > best, * valid,
i via iBGP
```

	eBGP	iBGP
Peers in	different ASes	the same AS
Administrative distance	20	200
TTL of packets	1 (use `ebgp-multihop` or loopbacks)	255
AS_PATH on advertisement	own AS prepended	unchanged

NEXT_HOP on advertisement	changed to the sender	**unchanged**
Re-advertise to a peer of the same type	yes	**no** (iBGP-learned routes are not passed to iBGP peers)

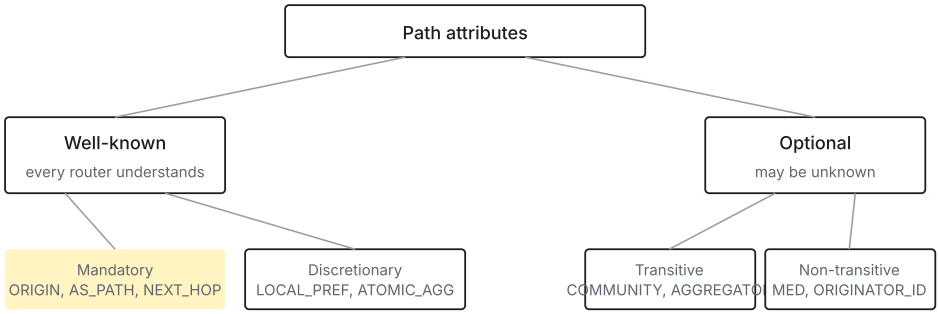


Figure 4.24 Taxonomy of path attributes. Well-known attributes must be recognised; an optional transitive one is passed on even if unknown, an optional non-transitive one is dropped.

The last row explains the iBGP *full mesh*. Because a route learnt from one iBGP peer is not passed to another, every iBGP router must peer with every other, which needs n(n-1)/2 sessions: 45 for ten routers. Two remedies exist. A *route reflector* (RR) is allowed to break the rule: it reflects routes from *clients* to other clients and non-clients, and from non-clients to clients only, while a route learnt by eBGP goes to everyone. To avoid loops the RR tags a reflected route with an ORIGINATOR_ID (the first router's ID) and prepends its CLUSTER_ID to a CLUSTER_LIST; a router discards a route with its own ID, and an RR discards a route listing its own cluster. *Confederations* split the AS into sub-ASes that behave like eBGP among themselves.

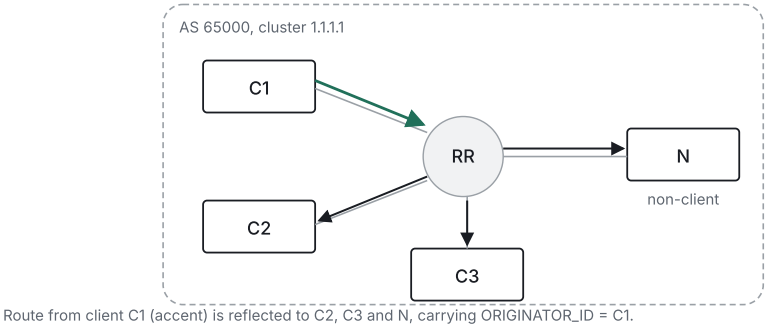


Figure 4.25 A route reflector. A client route goes to all peers; a route from the non-client N would go to clients only. The configuration is one line per client: `neighbor 10.255.0.11 route-reflector-client`.

WORKED EXAMPLE

Counting sessions. Ten routers in a full mesh need 10 × 9 / 2 = 45 iBGP sessions. With two route reflectors and eight clients, each client peers with both RRs (16 sessions) and the RRs peer with each other (1): 17 sessions.

Result: 45 against 17. The saving grows quadratically, which is why large networks use RRs.

Route reflection is configured on the reflector only; clients need no special configuration, and they cannot tell a reflector from a normal iBGP peer. The next hop is not changed by the RR, so an eBGP route reflected to a client still needs a reachable next hop.

WORKED EXAMPLE

A reflector trace. BR1 learns 203.0.113.0/24 from ISP-A and is a client of RR1 (router ID 10.255.0.100, cluster 10.255.0.100). RR1 also serves clients R3 and BR2.

1. BR1 sends the route to RR1 by iBGP with next hop BR1's loopback (`next-hop-self`).
2. RR1 reflects it to R3 and BR2, adding ORIGINATOR_ID = 10.255.0.1 and CLUSTER_LIST = {10.255.0.100}.
3. If a faulty configuration lets BR2 send the route back to RR1, RR1 sees its own cluster ID in the CLUSTER_LIST and discards it. If BR1 received its own route reflected back, it would see its own ORIGINATOR_ID and discard it.

```
R3# show ip bgp 203.0.113.0/24
  100 300
    10.255.0.1 (metric 20) from 10.255.0.100 (10.255.0.100)
      Origin IGP, metric 0, localpref 100, valid, internal, best
      Originator: 10.255.0.1, Cluster list: 10.255.0.100
```

Result: the two attributes give the loop protection that the AS_PATH cannot, since the AS_PATH does not change inside the AS.

A *confederation* is the other answer to the mesh. The AS is split into member sub-ASes (private numbers), full-meshed or reflected inside, with eBGP-like sessions between them that preserve LOCAL_PREF and next hop. The outside world sees only the confederation identifier. It is used rarely and mostly in legacy networks.

Path attributes come in four categories. *Well-known mandatory* attributes (ORIGIN, AS_PATH, NEXT_HOP) must be understood and sent with every route; *well-known discretionary* (LOCAL_PREF, ATOMIC_AGGREGATE) must be understood but need not be sent; *optional transitive* (COMMUNITY, AGGREGATOR) are passed on even if not understood; *optional non-transitive* (MED, ORIGINATOR_ID, CLUSTER_LIST) are dropped if not understood. **WEIGHT** is a Cisco-local value that is never sent to a peer.

Attribute	Scope and use	Winner
WEIGHT	one router; Cisco only; default 0 (32768 for local routes)	highest
LOCAL_PREF	whole AS; sent only in iBGP; default 100; steers *outbound* traffic	highest
AS_PATH	list of ASes crossed; length is a distance proxy	shortest
ORIGIN	IGP (i) < EGP (e) < incomplete (?)	lowest
MED	sent to a neighbouring AS to suggest an entry point; steers *inbound*	lowest
COMMUNITY	a 32-bit tag that carries policy requests	n/a

When a router holds several routes to a prefix, it selects *one* by this list; the first test that gives a unique winner stops the process. The Cisco order is shown. A short mnemonic is **"N WLLA OMNI"** for steps 0 to 8, then **"Old Routers Clear Neighbours"** for the final four.

Step	Test	Rule
0	**N**ext hop reachable	unreachable routes are ignored
1	**W**eight	highest
2	**L**ocal preference	highest
3	**L**ocally originated	prefer own (network, aggregate, redistribute)
4	**A**S_PATH length	shortest
5	**O**rigin	lowest (IGP, then EGP, then incomplete)
6	**M**ED	lowest, compared only between routes from the *same neighbouring AS*
7	**N**eighbour type	eBGP over iBGP
8	**I**GP metric to next hop	lowest
9	multipath check	install several if `maximum-paths` allows
10	**O**ldest	the oldest eBGP route, for stability
11	**R**outer ID	lowest
12	**C**luster-list length	shortest
13	**N**eighbour address	lowest

WORKED EXAMPLE

A path-selection race. BR1 (AS 65000) holds three paths to 203.0.113.0/24. (a) from ISP-A by eBGP: AS_PATH 100 300, LOCAL_PREF 100, MED 50. (b) from ISP-B by eBGP: AS_PATH 200 300, LOCAL_PREF 100, MED 10. (c) from the iBGP peer BR2: AS_PATH 100 300, LOCAL_PREF 150.

1. Weight is unset on all three, so no decision at step 1.
2. Step 2: local preference 150 beats 100. **(c) wins** and the process stops.

Now remove the local-preference setting on (c), so all are 100.

3. Steps 3 to 5 tie: no path is local, all three have AS_PATH length 2 and the same origin.
4. Step 6, MED. MED is compared only between routes from the same neighbouring AS. (a) and (c) both come from AS 100, but (b) comes from AS 200, so (b)'s lower MED is *not* compared against (a).
5. Step 7: eBGP is preferred to iBGP, so (c) is eliminated. (a) and (b) tie through step 8 (equal IGP cost to directly connected peers).
6. Step 10: the oldest eBGP path wins, so whichever was learnt first stays, which is unpredictable by design.

Result: (c) on LOCAL_PREF; without it, (a) or (b) by age. With `bgp always-compare-med`, step 6 would compare (a) and (b) too, and (b) would win with MED 10.

Other vendors — FRR follows nearly the same list (weight, local preference, ...). Junos has no weight; it compares route preference, then LOCAL_PREF, then AS_PATH, and the order of the later steps differs, so consult the vendor documentation.

WORKED EXAMPLE

Dual-homing Campus Net. BR1 connects to ISP-A (AS 100) and BR2 to ISP-B (AS 200); Campus Net (AS 65000 in this teaching topology) owns 198.51.100.0/24. The goal is for ISP-

A to be primary in both directions.

1. *Outbound*: local preference is the tool, since it is shared across the AS by iBGP. Set 200 on routes learnt from ISP-A.
2. *Inbound*: remote ASes decide, so we make the ISP-B path look longer by prepending our AS twice when advertising to ISP-B.

```
! BR1
ip route 198.51.100.0 255.255.255.0 Null0
ip prefix-list OUT seq 5 permit 198.51.100.0/24
route-map FROM-A permit 10
 set local-preference 200
router bgp 65000
 neighbor 192.0.2.1 remote-as 100
 neighbor 192.0.2.1 route-map FROM-A in
 neighbor 192.0.2.1 prefix-list OUT out
 neighbor 192.0.2.1 maximum-prefix 1000 80
 neighbor 10.255.0.2 remote-as 65000
 neighbor 10.255.0.2 update-source Loopback0
 neighbor 10.255.0.2 next-hop-self
 network 198.51.100.0 mask 255.255.255.0
! BR2
route-map TO-B permit 10
 set as-path prepend 65000 65000
router bgp 65000
 neighbor 192.0.2.5 remote-as 200
 neighbor 192.0.2.5 route-map TO-B out
```

```
BR1# show ip bgp 203.0.113.0/24
   Network          Next Hop     Metric LocPrf Weight Path
*> 203.0.113.0/24   192.0.2.1         0    200      0 100 300 i    <-- best
(>), via ISP-A
* i                 10.255.0.2        0    100      0 200 300 i
```

Result: Campus Net sends via ISP-A (LOCAL_PREF 200). Remote networks see paths “100 65000” and “200 65000 65000 65000” and prefer the shorter one via ISP-A.

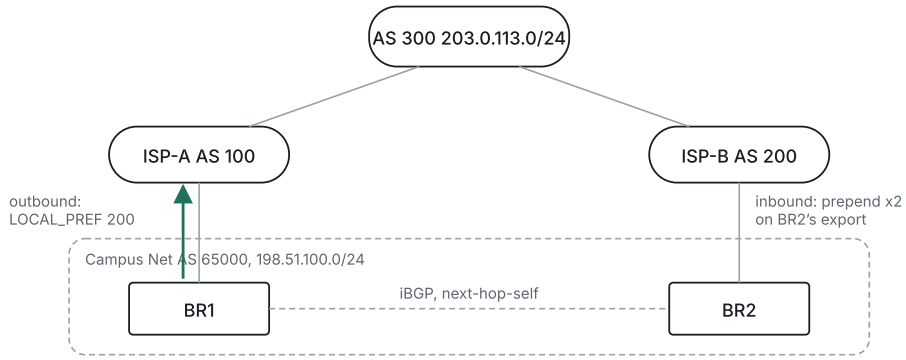


Figure 4.26 Dual-homed Campus Net. Outbound traffic is steered by LOCAL_PREF (inside our AS); inbound traffic by making the ISP-B path longer (outside our AS).

WORKED EXAMPLE

The iBGP next-hop problem. R3, an internal Campus Net router, is an iBGP client of BR1 and has no eBGP session. BR1 learns 203.0.113.0/24 with next hop 192.0.2.1 and passes it to R3 *unchanged*. R3’s IGP has no route to 192.0.2.1, so the next hop is unreachable.

```
R3# show ip bgp
   Network          Next Hop     Metric LocPrf Weight Path
 * i203.0.113.0/24  192.0.2.1          0    100      0 100 300 i   <-- no
">": not best, not in RIB
```

Result: add `neighbor 10.255.0.3 next-hop-self` on BR1, so the next hop becomes BR1's loopback, which the IGP knows. Alternatively advertise the ISP link into the IGP as a passive interface.

WORKED EXAMPLE

A five-path race. BR1 has five candidates for 203.0.113.0/24. None of them has a weight set. Step through the list.

Path	LOCAL_PREF	Local	AS_PATH	Origin	MED	Type	IGP metric to NH	Router ID
P1	100	no	100 300	i	20	eBGP	0	10.0.0.9
P2	100	no	100 300	i	5	eBGP	0	10.0.0.7
P3	100	no	200 400 500	i	0	eBGP	0	10.0.0.5
P4	100	no	100 300	i	5	iBGP	10	10.255.0.2
P5	90	no	100	e	0	eBGP	0	10.0.0.3

1. Step 0 and 1: all next hops reachable; weights tie.
2. Step 2: LOCAL_PREF 100 beats 90. **P5 is eliminated**, though it has the shortest AS_PATH: policy comes before distance.
3. Step 3: no local routes. Step 4: AS_PATH length: P1, P2 and P4 have 2, P3 has 3. **P3 is eliminated.**
4. Step 5: origin ties (all IGP). Step 6: MED, compared among paths from the same neighbouring AS (all three remaining come from AS 100): P2 and P4 have 5, P1 has 20. **P1 is eliminated.**
5. Step 7: eBGP over iBGP: **P4 is eliminated**; P2 remains.

Result: P2 wins at step 7. Had P2 and P4 both been eBGP, step 8 (IGP metric), then step 10 (age) and step 11 (router ID 10.0.0.7 against another) would have decided.

Communities and *aggregation* complete everyday practice. A community is set with a route map (`set community 65000:100 additive`) and sent to the neighbour only if `neighbor X send-community` is configured; the receiving AS then acts on it, for example by assigning local preference 80 to routes tagged 65000:80 (a *backup* community) or by blackholing a destination. `aggregate-address 198.51.100.0 255.255.255.0 summary-only` creates a summary (and suppresses the more-specific components); an aggregate that loses detail sets ATOMIC_AGGREGATE and may carry an AGGREGATOR attribute (and an AS_SET).

```
ip as-path access-list 10 permit ^100_            ! learnt from AS 100
ip as-path access-list 20 permit ^$               ! originated locally
ip community-list standard BACKUP permit 65000:80
route-map FROM-ISP permit 10
 match community BACKUP
 set local-preference 80
route-map FROM-ISP permit 20
router bgp 65000
 neighbor 192.0.2.5 route-map FROM-ISP in
 neighbor 192.0.2.5 send-community
```

Convergence is mostly a timer problem. With default timers a silent failure is noticed only after the hold time (180 s). `fast-external-fallover` (on by default) resets an eBGP session the moment its directly connected interface goes down, and *BFD* (`neighbor X fall-over bfd`) detects loss in a fraction of a second. After a policy change `clear ip bgp X soft in` applies the new inbound policy using a stored copy or a ROUTE-REFRESH, without resetting the session. Two further knobs are the advertisement interval (30 s for eBGP by default) and graceful restart.

Policy tools complete the picture. *Outbound* traffic is influenced by weight (one router) and local preference (the whole AS). *Inbound* traffic can only be influenced indirectly: AS-path prepending, MED (honoured only by the neighbouring AS), advertising more-specific prefixes (most of the Internet filters beyond /24), and communities that ask the provider to act. The well-known communities are `no-export` (do not advertise outside this AS) and `no-advertise` (do not advertise to any peer). Filtering uses prefix lists (§4.9 (p. 252)), AS-path access lists with regular expressions (`^$` means locally originated, `_65001_` any path through AS 65001) and route maps, with `maximum-prefix` as a safety valve, and `soft-reconfiguration inbound` or route refresh to re-apply a policy without resetting the session.

The `network` command in BGP advertises a prefix only if an *exact* match is present in the RIB (which is why the Null0 route above is configured), unlike the `network` command of OSPF, which selects interfaces. `maximum-paths 4` installs several equal paths, provided the weight, local preference, AS_PATH length, origin, MED, neighbouring AS and IGP cost all match; `bgp bestpath as-path multipath-relax` relaxes the requirement that the AS numbers be identical, which suits spine-leaf fabrics where each leaf has its own AS. Those designs (RFC 7938) use eBGP alone because of its policy control and its simple failure behaviour; see §5.4.

BGP is trusted by configuration, not by proof. In 2008 a Pakistani ISP's attempt to block YouTube leaked a more-specific prefix to the world, and most of the Internet followed the longer prefix; in 2021 a configuration change withdrew a large content provider's prefixes and took its DNS servers off the Internet with them. The defences are origin validation with RPKI (route origin authorisations decide whether a route is *valid*, *invalid* or *not found*), prefix and `maximum-prefix` limits, MD5 or TCP-AO session authentication, and the generalised TTL security mechanism (`ttl-security hops 1`).

A note on AS numbers: Campus Net uses the private AS 65000 in this book's examples. A private AS cannot be used on the Internet's global table, so a provider strips it (`neighbor X remove-private-as`) before advertising the customer's prefix. That suits a customer attached to one provider; a customer that is dual-homed, as in Figure 4.26, needs a public AS number, since otherwise the prefix would appear to originate from two different ASes and origin validation could not be applied.

TRAP

Claim: a lower MED always wins, so (b) with MED 10 beats (a) with MED 50. *Why it is wrong:* by default MED is compared only between routes from the same neighbouring AS, and these come from AS 200 and AS 100. *Correct statement:* the comparison is skipped unless `bgp always-compare-med` is configured. Note also that MED is lower-wins, whereas weight and local preference are higher-wins.

TRAP

Claim: "BGP neighbour Active" means the session is working. *Why it is wrong:* Active means the router is *trying* to open the TCP connection and has failed. *Correct statement:* a healthy session is Established; Active or Idle is a fault, usually reachability, ACL, a wrong AS number or a missing `update-source`.

INTERVIEW LENS

AMAZON · CISCO · JUNIPER · ARISTA · NOKIA

The best-path order is among the most frequently reported questions in this book; recite it with the mnemonic and then explain two of the steps unprompted (why MED is limited to one neighbour AS; why eBGP beats iBGP). Expect the dual-homing scenario (outbound by local preference, inbound by prepending or communities), the iBGP next-hop failure, the RR loop-prevention attributes and the full-mesh count. A coding variant asks you to implement the first eight steps; its solution is Q4.45.

LAB

`LAB 4.7 · 40 MIN` Containerlab with FRR: AS 65000 (BR1, BR2, R3), AS 100, AS 200 and AS 300. Configure eBGP, iBGP, `next-hop-self`, the two route maps above, then verify.

```
vtysh -c "show bgp ipv4 unicast 203.0.113.0/24"
vtysh -c "show bgp summary"
```

```
Paths: (2 available, best #1)
  100 300  from 192.0.2.1 (ISP-A)
    Origin IGP, localpref 200, valid, external, best    <-- best
```

What you should notice: shut the ISP-A link and the best path moves to ISP-B within the hold time (180 s, or at once with BFD or fast external fall-over). Capture TCP 179 in Wireshark and decode the AS_PATH and LOCAL_PREF in the UPDATE.

CHECK YOURSELF

1. A route is in the BGP table but absent from the routing table. Name the first thing you check. [1]
2. Why does a route reflector add a CLUSTER_LIST to a reflected route? [2]

4.8 First-hop redundancy: HSRP, VRRP and GLBP SHOULD · ■■□

BIG IDEA

A host has one default gateway address, so two routers share a *virtual* IP address and MAC address, and whichever is active answers for it; when it fails the other takes over and the hosts notice nothing.

1. The next hop: if it is unreachable in the IGP, the route is not installed (also check a better-AD route for the same prefix).↩
2. A route that returns to a reflector carrying its own cluster ID has looped and is discarded.↩

In *HSRP* (Cisco's Hot Standby Router Protocol) one router is *Active* and forwards for the virtual IP, another is *Standby*, and the rest *Listen*. Routers exchange hellos every 3 s (hold time 10 s) and the highest priority (default 100) becomes active. HSRP *preemption* is **off** by default, so a recovered higher-priority router does not take back the role unless `standby preempt` is configured. *VRRP* (RFC 5798) is the open standard, with a *master* and *backups*; its preemption is **on** by default, and a router that owns the virtual IP as its real address has priority 255. *GLBP* adds load balancing: one *active virtual gateway* hands out different virtual MACs, and up to four *active virtual forwarders* each carry part of the load.

	HSRP v1	HSRP v2	VRRP v2/v3	GLBP
Standard	Cisco	Cisco	IETF	Cisco
Transport and group address	UDP 1985, 224.0.0.2	UDP 1985, 224.0.0.102	IP protocol 112, 224.0.0.18	UDP 3222, 224.0.0.102
Virtual MAC	0000.0c07.acXX	0000.0c9f.fXXX	0000.5e00.01XX	0007.b400.XXYY
Default priority / preempt	100 / off	100 / off	100 / on	100 / off
Load sharing	per group	per group	per group	per host (several forwarders)

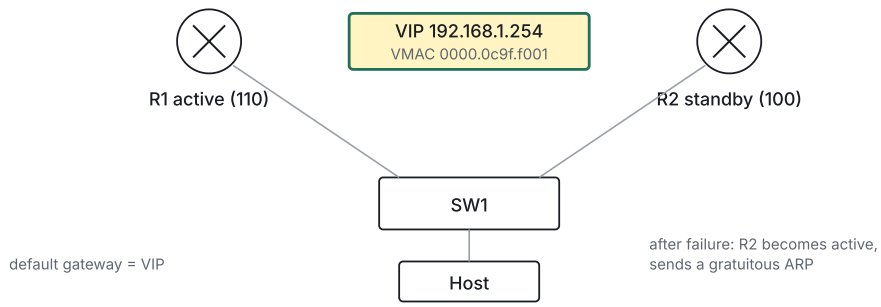


Figure 4.27 First-hop redundancy. Both routers know the VIP; only the active one answers ARP for it with the virtual MAC.

HSRP routers move through states: *Initial* (starting), *Learn* (waiting to learn the virtual IP from the active router), *Listen* (knows the virtual IP, hears hellos, is neither active nor standby), *Speak* (takes part in the election, sending hellos), *Standby* (next in line) and *Active*. The router that becomes active sends a gratuitous ARP and, since the virtual MAC is learnt by the switches from the HSRP hellos themselves, forwarding resumes at once. Failure of the active router is detected after the hold time (10 s by default), and a recoverable improvement is to shorten hello and hold (`standby 1 timers msec 200 msec 750`) or to use BFD.

WORKED EXAMPLE

HSRP with interface tracking. R1 has priority 110 and tracks its uplink Gi0/1 with a decrement of 20; R2 has priority 100. Both use `preempt`. The VIP is 192.168.1.254.

```
R1(config-if)# standby version 2
R1(config-if)# standby 1 ip 192.168.1.254
R1(config-if)# standby 1 priority 110
R1(config-if)# standby 1 preempt
R1(config-if)# standby 1 track GigabitEthernet0/1 20
```

1. Normal state: R1 is active with priority 110.

2. R1's uplink fails. The tracking decrement lowers R1 to 110 - 20 = 90, which is below R2's 100. Because R2 has `preempt`, it takes over.
3. The uplink returns. R1 climbs back to 110 and, with `preempt`, takes the role again.

```
R1# show standby brief
Interface   Grp  Pri P State   Active    Standby      Virtual IP
Gi0/0       1    90  P Standby 192.168.1.3 local      192.168.1.254   <--
after failure
```

Result: tracking changes priority only; the standby takes over only if it has `preempt`. A decrement smaller than the priority gap (here less than 10) would not cause a switch.

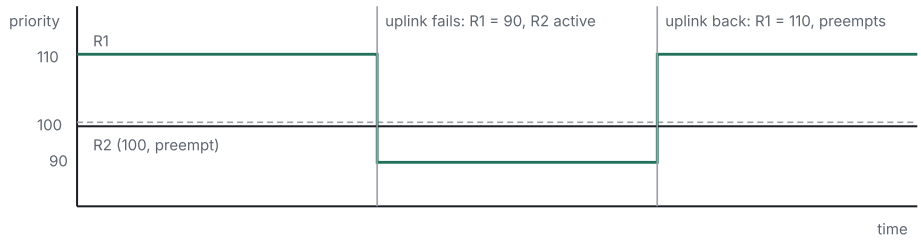


Figure 4.28 Priority against time for the tracking example. The roles swap when the lines cross, provided the lower-priority router has preempt enabled.

VRRP and *GLBP* are configured in the same way. The VRRP version on IOS-XE and on FRR/Linux (`vrrpd`) uses the group, priority and address; if the router's real interface address equals the virtual address it becomes the owner with priority 255. GLBP distributes different virtual MACs to different hosts, so both routers forward.

```
! VRRP (IOS-XE)                          ! GLBP
interface GigabitEthernet0/0             interface GigabitEthernet0/0
 vrrp 1 ip 192.168.1.254                  glbp 1 ip 192.168.1.254
 vrrp 1 priority 110                      glbp 1 priority 110
 vrrp 1 track 1 decrement 20              glbp 1 preempt
```

Align the Layer 2 and Layer 3 roles: make the HSRP active router also the spanning-tree root for the VLAN, so that traffic does not cross the inter-switch link to reach it. On failover the new active router sends a gratuitous ARP so that switches relearn the virtual MAC.

LAB

`LAB 4.8 · 20 MIN` Packet Tracer, two routers and a host, HSRPv2 with tracking as in the example. Start a continuous ping and shut R1's uplink.

```
C:\> ping -t 192.168.1.254
Request timed out.   <-- count the lost packets
R2# show standby brief
Gi0/0   1  100 P Active   local   192.168.1.2   192.168.1.254
```

What you should notice: with default timers up to ten seconds of loss when R1 is *disconnected*, but only a few pings when tracking lowers the priority and R2 has preempt, because the standby no longer waits for the hold timer. Convert to VRRP and compare the lost-ping counts.

CHECK YOURSELF

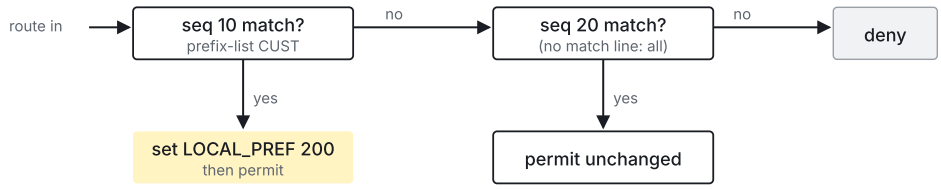
1. R1 loses its uplink but stays active. Name two causes. [1]

4.9 Route control: prefix lists, route maps, redistribution and VRF-lite SHOULD · ■■□

BIG IDEA

Policy tools decide which routes are accepted, which are modified and which packets are steered; redistribution joins routing domains but, without tagging, joins them into a loop.

A *prefix list* matches prefixes by address and length. `ip prefix-list X permit 10.0.0.0/8 ge 16 le 24` matches any prefix inside 10.0.0.0/8 whose length is between 16 and 24. Thus 10.0.0.0/8 does not match (length 8 is below 16), 10.1.0.0/16 and 10.1.1.0/24 match, and 10.1.1.0/25 does not (length 25 is above 24). With no `ge` or `le` only the exact prefix matches. Lists end with an implicit deny.



The first matching clause decides. If none matches, the implicit deny at the end rejects the route.
A deny clause with a matching condition removes the route; a permit clause without match lines matches everything.

Figure 4.29 How a route map is evaluated. Clauses are tried in sequence-number order.

A *route map* is an ordered list of numbered clauses, each with `match` conditions and `set` actions, and a `permit` or `deny`. The first clause that matches decides; if a permit clause has no match line it matches everything; at the end there is an implicit deny. Route maps are used for BGP policy, for redistribution and for policy-based routing.

```
ip prefix-list CUST seq 5 permit 10.0.0.0/8 ge 16 le 24
route-map SET-LP permit 10
 match ip address prefix-list CUST
 set local-preference 200
route-map SET-LP permit 20          ! no match: everything else passes unchanged
```

Redistribution imports routes of one protocol into another (`redistribute eigrp 100 subnets route-map X` under OSPF). The importing protocol needs a *seed metric*, since the foreign metric means nothing: OSPF assigns cost 20 by default (E2), EIGRP and RIP need explicit values. Done in one direction at one router it is safe. Done in both directions at two routers it is not: a route that enters OSPF from EIGRP at R1 can leave OSPF back into EIGRP at R2 with a better AD or metric, and so on, forming a loop. The fix is to *tag* routes as they are redistributed and to deny the tag in the reverse direction.

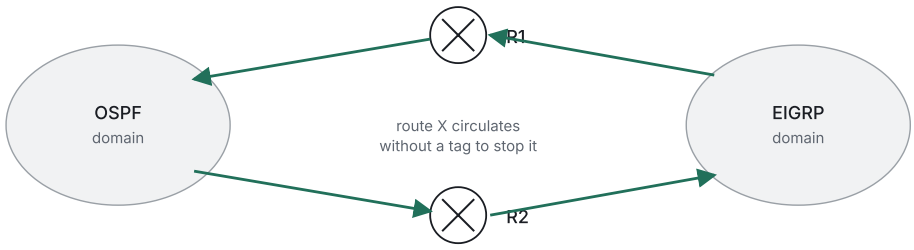


Figure 4.30 Two-way redistribution at two points. A route learnt from EIGRP re-enters EIGRP through OSPF at the other router.

```
route-map EIGRP-TO-OSPF deny 10
 match tag 110                      ! came from OSPF originally: do not send back
route-map EIGRP-TO-OSPF permit 20
 set tag 90
route-map OSPF-TO-EIGRP deny 10
 match tag 90
route-map OSPF-TO-EIGRP permit 20
 set tag 110
router ospf 1
 redistribute eigrp 100 subnets route-map EIGRP-TO-OSPF
router eigrp CAMPUS
 address-family ipv4 unicast autonomous-system 100
  topology base
   redistribute ospf 1 metric 100000 10 255 1 1500 route-map OSPF-TO-EIGRP
```

A seed metric is needed because the receiving protocol cannot interpret the sender's metric. The defaults differ and are a frequent source of surprise.

Redistributed into	Seed metric	Notes
OSPF	cost 20 (E2); 1 for BGP routes	`subnets` needed for classless prefixes on older IOS; `metric-type 1` for E1
EIGRP	none; route not installed without it	`metric bandwidth delay reliability load mtu` or `default-metric`
RIP	none (infinite)	`metric` or `default-metric`
BGP	metric copied (MED)	`redistribute ospf` brings only internal routes unless `match` is used

Policy-based routing (PBR) routes on something other than the destination: `ip policy route-map WEB-VIA-B` on an interface, with `match ip address` for the traffic and `set ip next-hop 192.0.2.5`. *VRF-lite* gives one router several independent routing tables, each with its own interfaces, so two customers can both use 10.0.0.0/24; commands accept a `vrf` keyword (`ping vrf A 10.0.0.1`).

> WORKED EXAMPLE
>
> **Policy-based routing.** Campus Net sends web traffic from SALES via ISP-B while everything else uses the default route toward ISP-A.

```
ip access-list extended WEB
 permit tcp 192.168.10.0 0.0.0.255 any eq 80
 permit tcp 192.168.10.0 0.0.0.255 any eq 443
route-map WEB-VIA-B permit 10
 match ip address WEB
 set ip next-hop 192.0.2.5
interface GigabitEthernet0/0.10
 ip policy route-map WEB-VIA-B
```

```
R1# show route-map WEB-VIA-B
route-map WEB-VIA-B, permit, sequence 10
  Policy routing matches: 412 packets, 307311 bytes
```

Result: matching packets leave via 192.0.2.5 regardless of the routing table; unmatched packets follow normal routing. PBR is stateless, so the return path may differ; add tracking so that it stops when the next hop is dead.

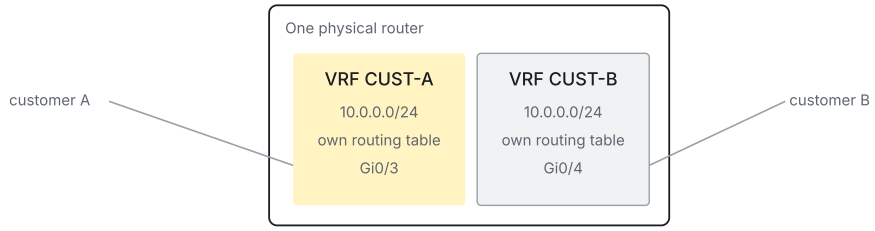


Figure 4.31 VRF-lite. The same prefix lives in two tables; a packet is looked up in the table of the interface it arrived on.

```
vrf definition CUST-A
 address-family ipv4
interface GigabitEthernet0/3
 vrf forwarding CUST-A
 ip address 10.0.0.1 255.255.255.0
R1# show ip route vrf CUST-A
```

TRAP

Claim: redistributing in both directions between two protocols is safe because AD stops loops. *Why it is wrong:* AD protects only routes for the same prefix learnt natively; a redistributed route is seen as a native one by the second protocol, with another AD. *Correct statement:* tag at the first redistribution and deny the tag at the second.

INTERVIEW LENS

CISCO · JUNIPER · AMAZON

Asked as “two-way redistribution loop and how do you stop it” and as “what does this route map do”. Show the sequence evaluation and the implicit deny, state that a permit clause with no match matches everything, and distinguish a deny clause in a route map (the route is not redistributed) from a deny in a prefix list.

LAB

`LAB 4.9 · 40 MIN` GNS3 with IOSv or FRR: three routers, OSPF on one side and EIGRP on the other, mutual redistribution at two routers. Create the loop first, then fix it with tags; finally build VRF-lite with an overlapping 10.0.0.0/24 in two VRFs.

```
R1# show ip route 10.9.9.0
Routing entry for 10.9.9.0/24
  Known via "ospf 1", distance 110, metric 20, type extern 2, tag 110
```

What you should notice: a tag on the route proves where it entered. Without the deny-tag clause, the router prefers the redistributed copy of its own route learned from the other router (an effect of AD); `traceroute` to the prefix bounces between the two redistribution points.

4.10 MPLS and L3VPN SHOULD · ■■■

BIG IDEA

MPLS forwards packets on short labels instead of IP addresses, which lets a provider carry many customers' overlapping address spaces across one core: inner labels identify the customer, outer labels identify the path.

A *label* is a 32-bit shim header inserted between the Layer 2 and IP headers (EtherType 0x8847): a 20-bit label, a 3-bit traffic class, a 1-bit bottom-of-stack flag and an 8-bit TTL. Labels are *pushed*, *swapped* and *popped*. The edge router (*PE*, provider edge) pushes labels; core routers (*P*) swap them using the *LFIB* and never look at the IP header; customer routers are *CE*s. Labels 0 to 15 are reserved: 0 (IPv4 explicit null), 1 (router alert), 2 (IPv6 explicit null), 3 (*implicit null*). Labels are distributed hop by hop by LDP (UDP 646 for discovery, TCP 646 for sessions), one per IGP prefix. With *penultimate-hop popping* (PHP) the router before the egress pops the transport label, because the egress PE advertises implicit null (3) for its own address: the egress then does a single lookup rather than two.



32 bits; S = 1 marks the last label of the stack, so a two-label VPN packet has S = 0 on the outer label.

Figure 4.32 The MPLS shim.

Each router keeps two label tables. The *LIB* (label information base) holds every label it has learnt from neighbours; the *LFIB* (label forwarding information base) holds the one chosen for each prefix, according to the IGP's best next hop. Label *swap* replaces the top label by the one the next hop advertised; the TTL field is decremented as in IP (and the ingress copies the IP TTL into it, unless `no mpls ip propagate-ttl` hides the core from `traceroute`). On the control plane, LDP (label distribution protocol) runs in a hop-by-hop fashion: each router assigns a local label for every prefix in its IGP and advertises it to its LDP neighbours, so that a *label-switched path* follows the IGP's shortest path.

```
P1# show mpls ldp neighbor
    Peer LDP Ident: 10.255.0.3:0; Local LDP Ident 10.255.0.2:0
        TCP connection: 10.255.0.3.646 - 10.255.0.2.43211
        State: Oper; Msgs sent/rcvd: 40/39
P1# show mpls forwarding-table
Local  Outgoing   Prefix           Bytes Label   Outgoing   Next Hop
Label  Label      or Tunnel Id     Switched      interface
24     35         10.255.0.5/32    0             Gi0/2      10.1.23.3     <--
swap 24 to 35
```

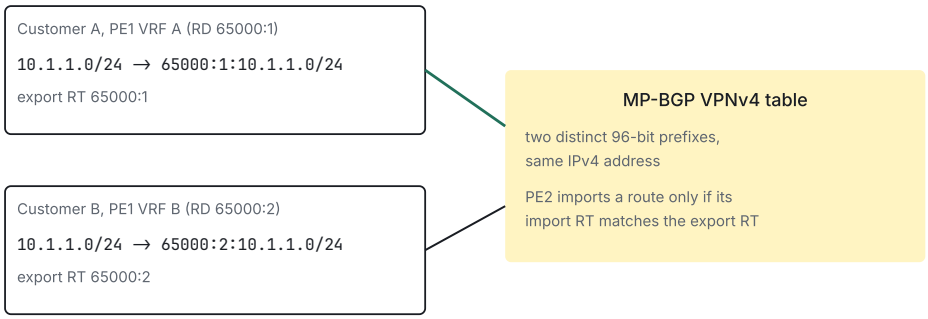


Figure 4.33 RD and RT. The RD only makes the prefixes distinct; the RT is the membership rule that decides which VRF receives each one.

An *L3VPN* (RFC 4364) uses three ideas. Each customer has a *VRF* on the PE. A *route distinguisher* (RD, 64 bits) is prefixed to each customer IPv4 route to make a 96-bit *VPNv4* prefix, so that two customers' 10.1.1.0/24 stay distinct in BGP. A *route target* (RT, a BGP extended community) controls which VRFs import which routes: export RTs are attached when a route is advertised, import RTs select it at the far PE. PEs exchange VPNv4 routes over MP-BGP; the transport path between them is given by LDP labels.

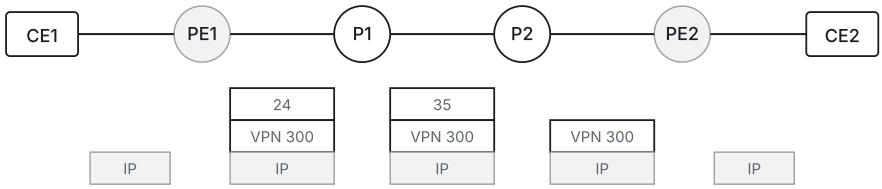
	RD	RT
Purpose	makes a prefix unique in the provider's BGP	decides which VRFs install the route
Where carried	prefix of the VPNv4 NLRI	extended community attribute
Typical value	65000:1	65000:1 (import and export)

WORKED EXAMPLE

An L3VPN packet walk. Customer A site 1 (CE1, 10.1.1.0/24) sends to site 2 (CE2, 10.2.2.0/24) through PE1, P1, P2, PE2. Customer B also uses 10.1.1.0/24 (RD and RT 65000:2).

1. CE1 to PE1: an ordinary IP packet. PE1 looks up the destination in VRF A, finds a VPNv4 route from PE2 with VPN label 300 and BGP next hop PE2's loopback.
2. PE1 pushes two labels: VPN label 300 (inner, bottom of stack) and the LDP label 24 for PE2's loopback (outer).
3. P1 swaps 24 for 35. P1 never reads label 300 or the IP header.
4. P2 is the penultimate hop. PE2 advertised implicit null, so P2 *pops* the transport label and forwards a packet carrying only label 300.
5. PE2 looks up 300, which belongs to VRF A, pops it, performs an IP lookup in VRF A and forwards the packet to CE2.

Result: customer A and B never meet, because each VPN label points to a VRF and each VRF has its own table.



Highlighted block: transport (LDP) label. Open block below it: VPN label. P2 pops the transport label (PHP).
Control plane: LDP between neighbours; MP-BGP VPNv4 between PE1 and PE2.

Figure 4.34 The label stack at each link of the L3VPN walk.

```
vrf definition CUST-A
 rd 65000:1
 address-family ipv4
  route-target both 65000:1
interface GigabitEthernet0/1
 vrf forwarding CUST-A
 ip address 10.1.1.1 255.255.255.0
router bgp 65000
 neighbor 10.255.0.5 remote-as 65000
 neighbor 10.255.0.5 update-source Loopback0
 address-family vpnv4
  neighbor 10.255.0.5 activate
  neighbor 10.255.0.5 send-community extended
```

Verification at the provider edge follows the control plane: first the VRF routing table at each PE, then the VPNv4 table, and finally the label path.

```
PE1# show ip route vrf CUST-A 10.2.2.0
B  10.2.2.0/24 [200/0] via 10.255.0.5, 00:12:30          <-- learnt by MP-BGP
from PE2
PE1# show bgp vpnv4 unicast all 10.2.2.0/24
  Route Distinguisher: 65000:1 (default for vrf CUST-A)
  Local: 10.255.0.5 (metric 20); label 300; Extended Community: RT:65000:1
PE1# traceroute vrf CUST-A 10.2.2.1
  1 10.1.12.2 [MPLS: Labels 24/300 Exp 0] 4 msec         <-- the two-label stack
```

Beyond L3VPN, MPLS traffic engineering and Segment Routing (SR-MPLS, SRv6) steer traffic along explicit paths by stacking labels or segment identifiers at the ingress; they matter mainly for service-provider roles and are outside the ENCOR core.

> **INTERVIEW LENS**
>
> JUNIPER · NOKIA · CISCO
>
> The L3VPN walk is the standard question. Answer with the two-label stack, who pushes, swaps and pops, and why PHP exists. A good answer separates control plane (LDP for transport, MP-BGP for VPN labels) from data plane, and states in one line what RD and RT each do.

> **LAB**
>
> `LAB 4.10 · 40 MIN` FRR Containerlab (Linux MPLS) or GNS3: four-router core, two customers with overlapping 10.1.1.0/24.

```
vtysh -c "show mpls ldp binding"
vtysh -c "show bgp ipv4 vpn"
ip -f mpls route
```

What you should notice: the VPNv4 table shows the same prefix twice under different RDs; the `ip -f mpls route` output shows `swap` at P routers and `pop` at the penultimate hop.

4.11 Quality of Service SHOULD · ■■□

BIG IDEA

When a link is congested, QoS decides who waits and who is dropped, so it is applied where congestion happens, using markings made once at the edge of the network.

Voice needs a one-way delay of no more than about 150 ms, jitter of at most 30 ms and loss below 1%; bulk transfers tolerate none of these constraints being met so strictly, since TCP retransmits. Three models exist: best effort (no distinction), IntServ (reserve per flow with RSVP, which does not scale) and *DiffServ*, in which packets carry a marking and each router applies a *per-hop behaviour* (PHB) by class. DiffServ is the model in use.

The marking is the *DSCP* (differentiated services code point), the top six bits of the IP ToS byte; the bottom two are ECN. Layer 2 has its own 3-bit field, CoS (802.1p), in the 802.1Q tag. Marking is trusted only inside a *trust boundary*, typically at the access switch where an IP phone is recognised; everything else is re-marked.

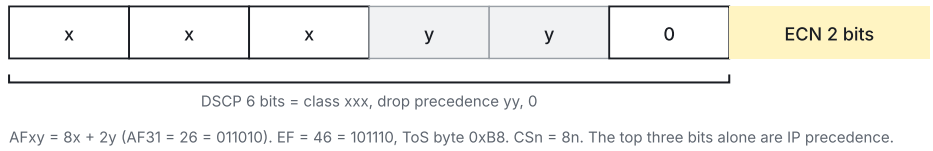


Figure 4.35 The ToS byte. Assured Forwarding classes AF1 to AF4 each have three drop precedences.

PHB	DSCP	Typical use
EF	46	voice
AF41 / AF31 / AF21 / AF11	34 / 26 / 18 / 10	video, critical data, transactional, bulk
CS6 / CS3	48 / 24	network control, signalling
CS1	8	scavenger (below best effort)
default	0	best effort

1. The tracking decrement is too small to drop below the standby's priority, or R2 lacks `preempt` (or the track object is not applied).↩

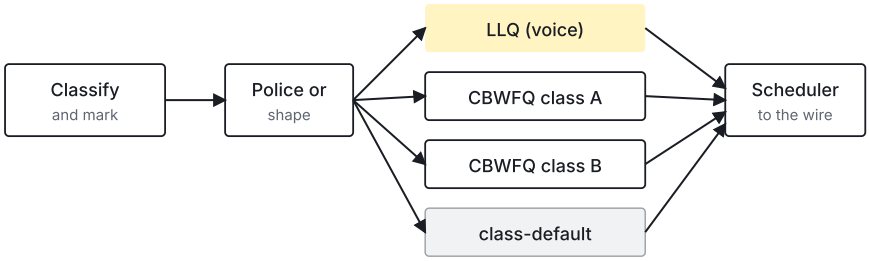


Figure 4.36 The QoS pipeline. Marking happens once at the trust boundary; queuing happens at every congested egress.

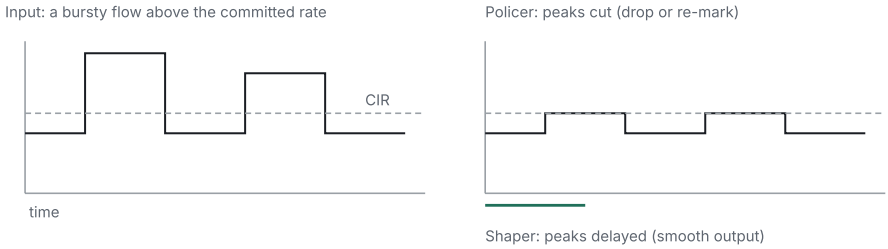


Figure 4.37 Policing against shaping for the same input (left). The policer clips the peaks at the committed rate; the shaper would instead hold the excess and send it later, so total volume is preserved but delay grows.

Queuing then orders the packets. *CBWFQ* gives each class a guaranteed share of bandwidth; *LLQ* adds a strict-priority queue for voice, which has an implicit policer so that it cannot starve the other classes. *WRED* drops packets early and randomly as a queue fills, with a rising probability between a minimum and a maximum threshold, and with higher drop probability for higher drop-precedence markings; it prevents *global synchronisation*, in which tail drop makes all TCP flows back off together. *Policing* and *shaping* both use a token bucket (fills at the committed rate, CIR; capacity is the burst size Bc): a policer discards or re-marks packets that find no tokens, while a shaper delays them in a buffer. Shaping thus adds delay but loses less; policing adds none but TCP sees loss and its window oscillates. A shaper's time interval is Tc = Bc / CIR.

A token bucket works as follows. Tokens, each worth one bit, arrive at the committed rate (CIR) and accumulate up to a limit Bc (a second bucket with Be allows an excess burst). A packet of *n* bits may be sent only if *n* tokens are available; they are then removed. A policer tests the bucket on arrival and acts on the packet at once (transmit, re-mark or drop, giving conform, exceed and violate actions). A shaper keeps the packet in a queue until enough tokens exist. The *time interval* Tc = Bc / CIR is the granularity of the shaper: a smaller Bc means smoother output but more timer work.

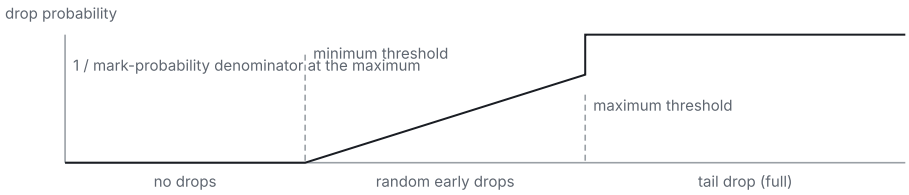


Figure 4.38 The WRED curve. Higher drop-precedence classes (AF13 against AF11) use lower thresholds, so they are dropped first.

WORKED EXAMPLE

Three QoS calculations.

1. *DSCP decode.* ToS 0xB8 = 1011 1000. The top six bits are 101110 = 46 = EF. AF31 has class 3 and drop precedence 1: 8 × 3 + 2 × 1 = 26.
2. *Shaper.* CIR 10 Mb/s with Bc 100,000 bits gives Tc = 100,000 / 10,000,000 = 10 ms, so up to 100 kb may be sent each 10 ms.
3. *Branch WAN policy.* On a 10 Mb/s link: voice 10% priority, video 30%, business-critical 25%, scavenger 1%, the rest default. The reserved sum is 66%, below the default 75% reservation limit, so the policy is accepted.

```
class-map match-all VOICE
 match dscp ef
! VIDEO (af41), CRIT (af31) and SCAV (cs1) are defined in the same way
policy-map WAN-CHILD
 class VOICE
  priority percent 10
 class VIDEO
  bandwidth percent 30
 class CRIT
  bandwidth percent 25
 class SCAV
  bandwidth percent 1
 class class-default
  fair-queue
policy-map WAN-PARENT
 class class-default
  shape average 10000000
  service-policy WAN-CHILD
interface GigabitEthernet0/1
 service-policy output WAN-PARENT
```

Result: the parent shaper makes the Ethernet port behave as a 10 Mb/s link, so the child queues engage; `show policy-map interface Gi0/1` shows per-class packets and drops.

TRAP

Claim: a policer and a shaper are two names for rate limiting. *Why it is wrong:* the policer has no queue, so excess traffic is dropped or re-marked at once; the shaper queues it. *Correct statement:* shape on egress toward a slower service, police to enforce a contract, and never police voice without understanding the consequences.

INTERVIEW LENS

CISCO · BROADCOM · NOKIA

Policing against shaping and the DSCP decode are near-certain. Give the unit conversions without hesitation (ToS byte against DSCP, AFxy = 8x + 2y). Broadcom and Nokia candidates also draw the queue pipeline and say where the buffers and the drop decisions are in hardware.

LAB

`LAB 4.11 · 40 MIN` GNS3 with IOSv and two Linux hosts. Apply the WAN-PARENT policy above to a 10 Mb/s link and mark traffic with iperf3.

```
host$ iperf3 -c 10.0.0.2 -u -b 8M -S 0xB8      ! EF-marked UDP, 8 Mb/s
host$ iperf3 -c 10.0.0.2 -b 50M                ! unmarked TCP
R1# show policy-map interface GigabitEthernet0/1
  Class-map: VOICE (match-all)    priority: 10% (1000 kbps)    drops 4120 <--
exceeds LLQ
```

> **What you should notice:** the EF stream above 1 Mb/s is dropped by the implicit policer of LLQ; the TCP flow takes the rest. Try the same on Linux with `tc qdisc add dev eth0 root tbf rate 10mbit burst 32kbit latency 50ms` and compare the shaper with a policer (`tc filter ... police`).

4.12 IP multicast OPTIONAL · ■■■

> BIG IDEA
>
> Multicast sends one copy of a stream towards many receivers and duplicates it only where paths diverge; receivers announce interest with IGMP, and routers build distribution trees with PIM, accepting packets only from the direction of the source.

Unicast, broadcast and multicast differ in how many copies the network carries. A source sending a stream to ten receivers by unicast sends ten copies over its first link; by broadcast it would reach everyone, wanted or not; by multicast it sends one copy, and the network replicates it only at branch points. Hence video distribution, market-data feeds and routing protocols themselves use multicast.

IPv4 multicast addresses are 224.0.0.0/4. Within it, 224.0.0.0/24 is link-local (never routed; 224.0.0.5 and .6 are OSPF, .10 EIGRP, .13 PIM), 232.0.0.0/8 is source-specific multicast (SSM) and 239.0.0.0/8 is administratively scoped, the private multicast range. A group address maps to a MAC address by placing its low 23 bits after the prefix 01:00:5E; since a group has 28 variable bits, 32 groups share one MAC.

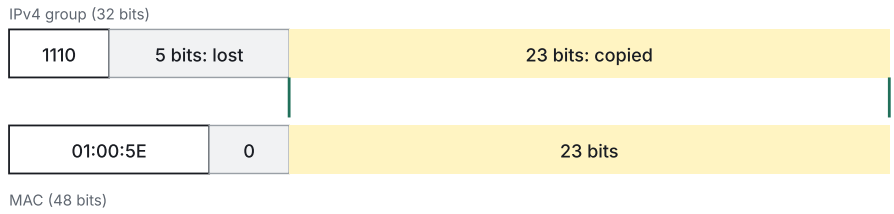


Figure 4.39 Multicast IP-to-MAC mapping. The five lost bits are why 32 groups collide.

> WORKED EXAMPLE
>
> **MAC mapping and ambiguity.** Map 239.1.1.1, and find another group with the same MAC.
>
> 1. The low 23 bits of 239.1.1.1 are the last three octets with the top bit of the second octet dropped: 1 = 0000001, then 1, then 1. The MAC is 01:00:5E:01:01:01.
> 2. 224.129.1.1 has second octet 129 = 1000 0001. Dropping the top bit gives 0000001 = 1, so its MAC is also 01:00:5E:01:01:01.
>
> **Result: 239.1.1.1 and 224.129.1.1 share a MAC; hosts filter again at IP, and 32 groups overlap by this rule.**

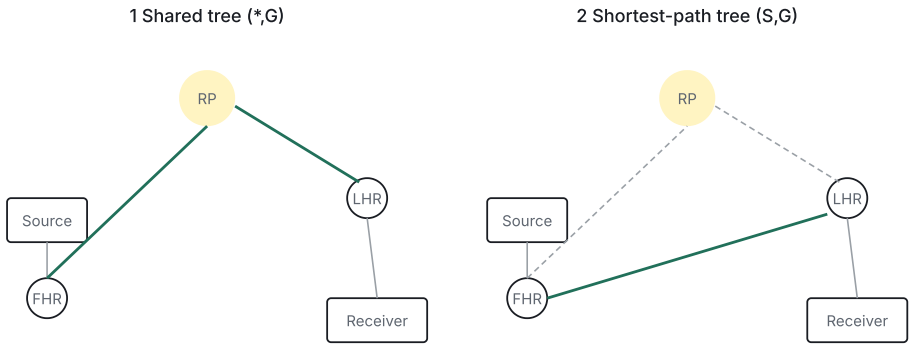


Figure 4.40 PIM sparse mode: shared tree, then switchover to the shortest-path tree. FHR = first-hop router.

Mechanism	Purpose	Notes
IGMPv2	host to router group membership	report to the group; leave to 224.0.0.2; querier election
IGMPv3	adds source filtering	reports to 224.0.0.22; required for SSM
IGMP snooping	switch limits flooding to joined ports	requires a querier in the VLAN
PIM-DM	flood and prune	small networks only
PIM-SM	RP and shared tree, then SPT	the common mode; RP by static, Auto-RP, BSR
PIM-SSM	(S,G) from the start, 232/8	no RP, needs IGMPv3
Bidir-PIM	(*,G) only, traffic both ways	many-to-many applications
MSDP	shares sources between RPs	inter-domain; mostly replaced by anycast RP

Hosts join a group with *IGMP*: v2 uses general queries (to 224.0.0.1), reports sent to the group address and Leave messages (to 224.0.0.2); v3 adds source filtering and is what SSM needs. A switch that does *IGMP snooping* forwards a group only to ports that joined. Between routers, *PIM* in *dense mode* floods and prunes. *PIM sparse mode* (PIM-SM, the standard) uses a *rendezvous point* (RP). A receiver's last-hop router sends a (*,G) join towards the RP, building a shared tree; the source's first-hop router registers with the RP; once traffic flows, the last-hop router switches to a source-rooted shortest-path tree (S,G) and the RP prunes itself off. The RP is found by static configuration, Auto-RP or BSR. Every multicast packet is subject to the *RPF check*: it is accepted only if it arrives on the interface that unicast routing would use to reach the source, which prevents loops. If unicast routing is asymmetric, the RPF interface may differ from the interface the stream uses, and the stream is dropped until a static multicast route corrects it. Bidirectional PIM suits many-to-many traffic, and MSDP connects RPs between domains.

```
R1(config)# ip multicast-routing
R1(config)# ip pim rp-address 10.255.0.1
R1(config-if)# ip pim sparse-mode
R1# show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:03:12/00:02:48, RP 10.255.0.1, flags: S
(10.2.2.5, 239.1.1.1), 00:01:07/00:02:53, flags: T    <-- T: SPT switchover
```

LAB

> `LAB 4.12 · 35 MIN` GNS3 with IOSv: PIM-SM with a static RP on the middle router, a source and a receiver on each side.
>
> ```
> R2# show ip igmp groups
> 239.1.1.1   GigabitEthernet0/1   00:02:10   00:02:49   10.2.2.9      <-- the
> receiver's join
> R2# show ip mroute 239.1.1.1 count
> Group: 239.1.1.1, Source count: 1, Packets forwarded: 5420, Packets received:
> 5420
> ```
>
> **What you should notice:** at first the flag is S (sparse, shared tree) and later T (SPT); break the reverse path with a static route to the source via the wrong interface and `show ip mroute count` reports RPF drops.

> CHECK YOURSELF
>
> 1. Why does a multicast packet fail the RPF check? [1]

4.13 IP services and access lists on IOS MUST · ■■□

> BIG IDEA
>
> A router supplies the services a network depends on (address translation, addressing, time, monitoring, logging) and filters packets with access lists, which are read from the top and stop at the first match.

Address translation. The theory of NAT and PAT is in §3.6 (p. 167); here is the IOS form. Interfaces are marked inside or outside, an ACL selects the inside addresses, and `overload` turns NAT into PAT by multiplexing on port numbers.

```
interface GigabitEthernet0/0.10
 ip nat inside
interface GigabitEthernet0/1
 ip nat outside
access-list 1 permit 192.168.10.0 0.0.0.255
ip nat inside source list 1 interface GigabitEthernet0/1 overload
ip nat inside source static 192.168.10.80 198.51.100.80      ! one-to-one for a
server
```

```
BR1# show ip nat translations
Pro Inside global        Inside local       Outside local      Outside global
tcp 192.0.2.2:1025       192.168.10.5:49152 203.0.113.10:443   203.0.113.10:443
```

DHCP. A router can be a server. A host broadcast does not cross a router, so a router on the client's subnet can *relay* it as a unicast to the real server with `ip helper-address`, filling in the `giaddr` field so that the server chooses the right pool. The helper also forwards some other UDP broadcasts (TFTP, DNS, TACACS, time, NetBIOS) by default.

```
ip dhcp excluded-address 192.168.10.1 192.168.10.20
ip dhcp pool SALES
 network 192.168.10.0 255.255.255.0
 default-router 192.168.10.1
 dns-server 10.0.0.53
interface GigabitEthernet0/0.20
 ip helper-address 10.0.0.5          ! relay to a central server
```

NTP keeps clocks aligned, which makes logs comparable. A server is given a *stratum*, its distance from a reference clock: stratum 1 has the clock attached, each hop adds one, and 16 means unsynchronised. NTP uses UDP 123. *Syslog* (UDP 514) sends log messages to a server, each with a severity from 0 to 7. *SNMP* (UDP 161 for polling, 162 for traps) lets a manager read and set device variables. Version 2c authenticates by a clear-text community string; version 3 adds authentication and encryption (`authPriv`) and should be used.

```
ntp server 10.0.0.5
ntp authentication-key 1 md5 N7pKey
ntp authenticate
ntp trusted-key 1
logging host 10.0.0.6
logging trap warnings
service timestamps log datetime msec
snmp-server group NMS v3 priv
snmp-server user nms NMS v3 auth sha AuthPass priv aes 128 PrivPass
```

```
BR1# show ntp status
Clock is synchronized, stratum 3, reference is 10.0.0.5
```

Level	Name	Level	Name
0	Emergency	4	Warning
1	Alert	5	Notice
2	Critical	6	Informational
3	Error	7	Debugging

Mnemonic — “Every Awesome Cisco Engineer Will Need Ice cream Daily”: Emergency, Alert, Critical, Error, Warning, Notice, Informational, Debugging. `logging trap warnings` sends levels 0 to 4.

Access control lists. The theory and placement of ACLs belong to §9.5; the IOS syntax is below. An ACL is a list of permit and deny lines examined top to bottom; the first match decides, and an invisible `deny ip any any` ends every list. Standard ACLs (1 to 99) match the source address only and are applied close to the destination; extended ACLs (100 to 199) match protocol, source, destination and ports and are applied close to the source. Named ACLs allow sequence numbers, so that lines can be inserted. Masks are *wildcards*: a 0 bit must match, a 1 bit is ignored, so the wildcard is the inverted subnet mask. IPv6 ACLs end with implicit permits for neighbour discovery before the final deny.

WORKED EXAMPLE

An ACL that protects a server. Only SALES (192.168.10.0/24) may reach the web server 10.0.0.80 on TCP 443; other networks must not reach it at all.

1. Place it as close as possible to the destination for a *single* protected host, as an outbound ACL on R1’s server-facing port; alternatively, place an extended ACL inbound on each user-facing interface so that denied packets are discarded at the first router.
2. Permit the allowed flow, deny everything else addressed to the server, and permit the rest.
3. Return traffic needs nothing here, since an outbound ACL on the server’s port inspects only packets *toward* the server. If the ACL were applied inbound on the server port (filtering replies), add `permit tcp host 10.0.0.80 eq 443 192.168.10.0 0.0.0.255 established` .

```
ip access-list extended TO-WEB
 10 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.80 eq 443
 20 deny   ip any host 10.0.0.80 log
 30 permit ip any any
interface GigabitEthernet0/2
 ip access-group TO-WEB out
```

```
R1# show ip access-lists TO-WEB
    10 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.80 eq 443 (153 matches)
    20 deny ip any host 10.0.0.80 log (7 matches)
```

Result: line 10 matches the permitted flow, line 20 catches everything else for the server, and line 30 stops the implicit deny from blocking unrelated traffic.

WORKED EXAMPLE

A wildcard mask. Match hosts 192.168.1.16 to 192.168.1.31. The block holds 16 addresses starting at a multiple of 16, so the mask is 16 - 1 = 15 in the last octet: `192.168.1.16 0.0.0.15`. Quick rule: wildcard = 255 minus the subnet-mask octet, so /24 is 0.0.0.255 and a single host is `0.0.0.0` (written `host x`).

TRAP

Claim: an ACL with only `deny` lines blocks those packets and passes the rest. *Why it is wrong:* every ACL ends in an implicit `deny any`, so a list of denies blocks everything. *Correct statement:* finish with `permit ip any any` when the intent is to filter selected traffic only.

INTERVIEW LENS

CISCO · AMAZON · MICROSOFT

Expected at Cisco as a configuration or “what is wrong with this ACL” exercise (order of lines, implicit deny, direction on the interface, wildcard arithmetic). Be able to write the NAT overload line, a DHCP relay and an SNMPv3 user from memory, and to name the verification command for each.

Address translation and filtering are most often debugged with a short routine, which is worth memorising.

Symptom	First check	Likely cause
Inside hosts cannot reach the Internet	`show ip nat translations`, `show ip nat statistics`	no `ip nat inside` or `outside` marking; ACL does not match the inside subnet
Some translations fail, then all	`show ip nat statistics` (misses, pool exhaustion)	too few ports or addresses; missing `overload`
Server unreachable from outside	`show ip nat translations`	no static entry, or ACL blocks the *inside local* address
DHCP clients get no address	`debug ip dhcp server packet`	missing `ip helper-address`; pool exhausted; wrong `giaddr` subnet
ACL seems ignored	`show ip access-lists`, `show ip interface Gi0/2 | include access list`	applied in the wrong direction or on the wrong interface; line order

Clock drifts, logs disagree	`show ntp associations`	NTP unreachable; wrong key; stratum too high

LAB

`LAB 4.13 · 45 MIN` Packet Tracer, integrated: PAT at BR1, a DHCP pool and relay, an NTP server, syslog and an SSH-only vty line, then TO-WEB.

```
BR1(config)# access-list 10 permit 10.0.0.0 0.0.0.255
BR1(config)# line vty 0 4
BR1(config-line)# access-class 10 in
```

What you should notice: `show ip nat translations` fills as hosts browse; a Telnet attempt from outside the admin subnet is refused; the match counters of TO-WEB rise on the permit line only for SALES.

CHECK YOURSELF

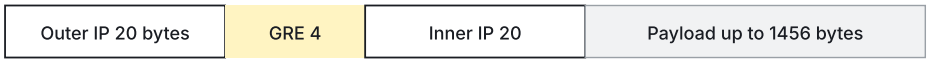
1. What does `ip nat inside source list 1 interface Gi0/1 overload` do? [2]
2. A standard ACL is applied near the source and blocks too much. Why? [3]

4.14 Tunnels and enterprise architecture OPTIONAL · ■■□

BIG IDEA

A tunnel wraps a packet inside another to cross a network that would not otherwise carry it, at the price of a header and a smaller usable MTU.

GRE (IP protocol 47) adds a 4-byte header and a new 20-byte IP header, 24 bytes in all, so a 1500-byte link leaves 1476 for the inner packet. GRE does not encrypt; IPsec does (§9.7), and GRE over IPsec is the standard way to carry routing protocols securely. DMVPN adds dynamic spoke-to-spoke tunnels.



Outer IP protocol = 47. Inner packet limit 1476 bytes on a 1500-byte link.

Figure 4.41 GRE encapsulation.

WORKED EXAMPLE

GRE with OSPF, and the recursive-routing fault. HQ-R (203.0.113.1) and Branch-R (198.51.100.1) are joined by a tunnel on 172.31.0.0/30, with OSPF running over it.

```
interface Tunnel0
 ip address 172.31.0.1 255.255.255.252
 tunnel source 203.0.113.1
 tunnel destination 198.51.100.1
 ip mtu 1476
 ip tcp adjust-mss 1436
router ospf 1
 network 172.31.0.0 0.0.0.3 area 0
```

1. The tunnel destination 198.51.100.1 must be reachable over the *underlay* (the real ISP path).
2. If OSPF learns 198.51.100.0/24 *through the tunnel itself*, the router tries to reach the tunnel endpoint through the tunnel. The tunnel then collapses and flaps:

```
%TUN-5-RECURDOWN: Tunnel0 temporarily disabled due to recursive routing
%LINEPROTO-5-UPDOWN: Line protocol on Interface Tunnel0, changed state to
down
```

Result: keep the underlay route to the tunnel destination more specific or of lower AD than anything learnt via the tunnel, for example by running OSPF over the tunnel in a separate process and filtering the underlay prefixes.

Failure detection beyond link state uses *BFD* (millisecond hellos) and *IP SLA* probes that feed a *track* object, for example a floating default route that is withdrawn when a probe to the ISP's gateway fails, not only when the link drops:

```
ip sla 1
 icmp-echo 192.0.2.1 source-interface GigabitEthernet0/1
 frequency 5
ip sla schedule 1 life forever start-time now
track 1 ip sla 1 reachability
ip route 0.0.0.0 0.0.0.0 192.0.2.1 track 1
```

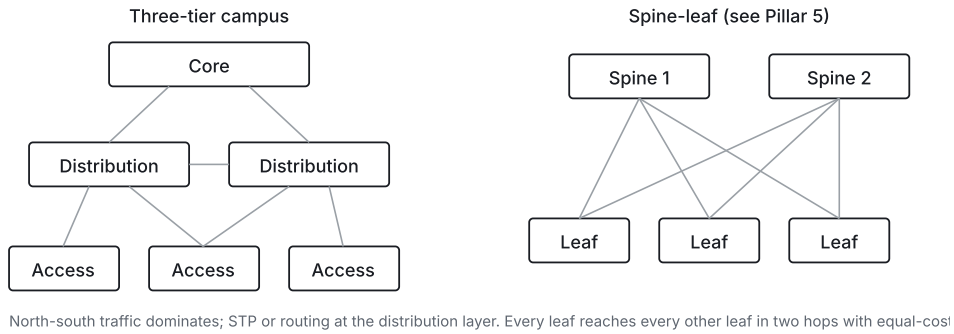


Figure 4.42 The two shapes. The campus hierarchy suits hosts that talk to services outside; the fabric suits servers that talk to each other.

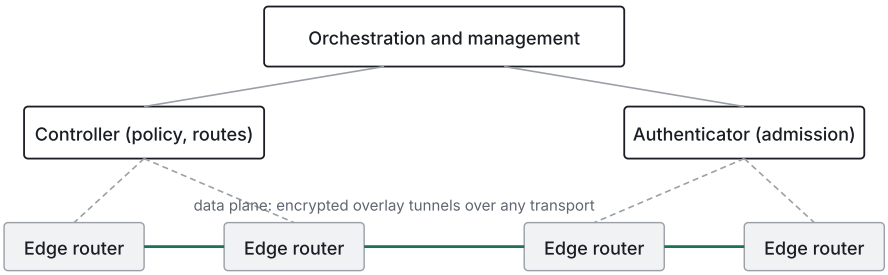


Figure 4.43 The SD-WAN idea: a central control plane (dashed) programs edge devices, which build the data-plane overlay (accent) between themselves.

Enterprise designs have two common shapes. A *three-tier campus* has access, distribution and core layers: access switches terminate hosts (PortFast, BPDU Guard), distribution pairs run the inter-VLAN routing and FHRP, and the core carries traffic between them. A *two-tier (collapsed-core)* design merges the last two; a *spine-leaf* fabric (see §5.1) replaces both in data centres. *SD-WAN* (for example Cisco's vManage, vSmart, vBond and edge routers) separates a central control plane and orchestration from the edge forwarding devices and builds overlay tunnels over any transport. *SD-Access* builds a campus *fabric* with a LISP control plane and a VXLAN data plane.

These overlays are taught in §5.5; the point here is that they rest on the routing and tunnelling in this chapter.

> CHECK YOURSELF
>
> 1. What MTU should the GRE tunnel use over a 1500-byte link, and why? [4]

4.15 Mini-project: Campus-to-Internet OPTIONAL · ■■■

> BIG IDEA
>
> A dual-homed enterprise built and verified as code shows every protocol of this chapter working together, and a test suite makes the result repeatable evidence of competence.

Topology (Containerlab; FRR plus Arista cEOS or Nokia SR Linux). Campus Net has two access switches (VLANs 10, 20 and a voice VLAN 99) with an LACP uplink to a collapsed-core pair running HSRP or VRRP, with the spanning-tree root aligned to the active gateway. A multi-area OSPF network has area 0 in the core and area 10 for a branch (totally stubby). Two border routers, BR1 and BR2, run eBGP to ISP-A (AS 100) and ISP-B (AS 200), which both peer with a content AS 300, and iBGP between themselves with `next-hop-self`.

Policy requirements.

- ISP-A is primary in both directions: local preference 200 inbound to Campus Net, AS-path prepend twice towards ISP-B.
- Advertise only 198.51.100.0/24, using a prefix list and `maximum-prefix`.
- Tag customer routes with communities and filter by community.
- PAT at the borders, LLQ for voice on the WAN, and a GRE tunnel to a branch carrying OSPF.

Deliverables.

1. A design document: the IP plan using VLSM (method in §3.4 (p. 158)) and a diagram in the style of this book.
2. `topology.clab.yml` and the start-up configurations.
3. A `verify.py` test suite (pytest) that asserts OSPF neighbours are Full, BGP sessions are Established, the best path is via ISP-A, traffic fails over to ISP-B within *N* seconds when the ISP-A link is cut, and DSCP EF is preserved end to end (Pillar 10 shows the Netmiko and `vtysh` calls).
4. A failure-injection runbook with five faults, each with symptom, diagnosis and fix, which feeds the troubleshooting method of Pillar 11.
5. A two-minute explanation of the project, and two résumé bullets.

Milestones. Week 1: L2 and the gateways. Week 2: OSPF and the branch. Week 3: BGP and the policy. Week 4: QoS, GRE, tests, runbook. **Stretch.** Replace the GRE tunnel by an MPLS L3VPN provider core, enable BFD, and measure the convergence improvement.

```
def test_best_path_via_isp_a(bgp):              # pytest fixture runs vtysh on
BR1
    best = bgp.best("203.0.113.0/24")
    assert best.as_path[0] == 100 and best.localpref == 200

def test_failover(lab, bgp):
    lab.link_down("br1-isp-a")
    assert bgp.wait_best("203.0.113.0/24", via_as=200, timeout=15)
```

1. It arrived on an interface that the unicast routing table would not use to reach the source. ↩
2. It translates addresses matched by ACL 1 to the address of Gi0/1, using port numbers to distinguish hosts (PAT). ↩
3. A standard ACL sees only the source address, so near the source it blocks that source's traffic to every destination. ↩
4. 1476, because the 24-byte GRE overhead (20 outer IP, 4 GRE) is subtracted. ↩

Problem Set

Each solution opens with the 30-second answer. Tags give the companies that report the topic, the round, the difficulty (■□□ to ■■■) and the evidence: S for a specific reported question, T for a reported topic with our wording.

Q4.1 Access vs trunk port. What is the native VLAN, and what happens on a mismatch?

CISCO · JUNIPER · TECH-1 · ■□□ · T

30-second answer: an access port carries one VLAN untagged; a trunk carries many VLANs with 802.1Q tags; the native VLAN is the one VLAN sent untagged on a trunk, and if the two ends disagree, frames cross from one VLAN into another.

Answer. A frame of VLAN 1 sent untagged by a switch whose native VLAN is 1 is read by a neighbour with native VLAN 99 as VLAN 99. Hence traffic leaks between VLANs and spanning tree on that VLAN may block the port as inconsistent. CDP prints a mismatch message. **Mistakes.** Treating the message as harmless; leaving the native VLAN as VLAN 1 (set an unused VLAN on both ends).

Q4.2 How many VLANs does 802.1Q support, and why 4094?

CISCO · ARISTA · TECH-1 · ■□□ · T

30-second answer: the VLAN ID is 12 bits, giving 4096 values; 0 (priority tag, no VLAN) and 4095 (reserved) are unusable, leaving 4094.

Answer. On Cisco, 1002 to 1005 are legacy and 1006 to 4094 need extended-range support. Beyond 4094 VLANs, VXLAN's 24-bit VNI is used (§5.5).

Q4.3 Explain inter-VLAN routing with ROAS and with SVIs. Which scales better?

CISCO · TECH-1 · ■■□ · T

30-second answer: router-on-a-stick uses one trunk and a sub-interface per VLAN on a router, with each packet crossing the link twice; an SVI is a virtual Layer 3 interface on a Layer 3 switch, routed in hardware; SVIs scale better.

Answer. ROAS is bounded by one link and the router's forwarding rate; it suits small branches. SVIs need `ip routing`, one `interface Vlan N` per VLAN, and the VLAN must exist and have an active port for the SVI to come up. The packet walk is in §4.2 (Figure 4.6 (p. 217)). **Follow-up.** Routed ports (`no switchport`) are used on links between Layer 3 switches.

Q4.4 Why is DTP a security risk? How do you harden trunk ports?

CISCO · TECH-1 · ■■□ · T

30-second answer: DTP lets a port become a trunk by negotiation, so an attacker who sends DTP frames can obtain a trunk and see all VLANs; configure every port statically and disable negotiation.

Answer. Access ports: `switchport mode access`, `switchport nonegotiate`, an unused native VLAN, `spanning-tree portfast` with BPDU guard, and shut unused ports into a parking VLAN. Trunks: `switchport mode trunk`, `nonegotiate`, an explicit allowed list, native VLAN not 1. Double-tagging and its defence are in §9.5.

Q4.5 Walk through the STP root election and port roles for the triangle.

CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · S

30-second answer: lowest bridge ID wins root (SW1, lowest MAC at equal priority); each other switch takes its lowest-cost port to the root as the root port; per segment the lowest-cost, then lowest-BID sender is designated; the remaining port on SW3 blocks.

Answer. The full three-part trace (default priorities, SW3 at 4096, parallel links) is the worked example of §4.3 (Figure 4.9 (p. 221)); learn the four-step order rather than the answer. **Mistake.** Giving the root bridge a root port.

Q4.6 Why must the STP priority be a multiple of 4096?

CISCO · TECH-1 · ■■□ · T

30-second answer: the 16-bit priority field is split into a 4-bit priority and a 12-bit VLAN ID (the extended system ID), so only the top four bits can be set.

Answer. The configurable values are 0, 4096, …, 61440. The VLAN ID is added to the configured value, hence VLAN 10 at default priority displays 32778.

Q4.7 802.1D vs RSTP: port states and roles. Why does RSTP converge in under a second?

CISCO · NOKIA · TECH-1 · ■■□ · T

30-second answer: 802.1D has five states (blocking, listening, learning, forwarding, disabled) and timers of up to 50 s; RSTP has three (discarding, learning, forwarding) and negotiates with proposal and agreement instead of waiting for timers.

Answer. Roles in RSTP are root, designated, alternate and backup. An alternate port is a precomputed path to the root and takes over immediately; a new link is handled by a sync handshake, and edge ports forward at once. Failure is detected after three missed hellos (6 s), or at once on loss of link. **Mistake.** Saying RSTP removes the need for the root election; it keeps it.

Q4.8 PortFast and BPDU Guard, Root Guard, Loop Guard: where does each go?

CISCO · ARISTA · TECH-1 · ■■□ · T

30-second answer: PortFast with BPDU Guard on access ports facing hosts; Root Guard on distribution ports facing access switches that must never be root; Loop Guard on non-designated uplinks.

Answer. BPDU Guard shuts an edge port if a BPDU arrives. Root Guard puts a port in root-inconsistent state if it hears a superior BPDU and recovers when they stop. Loop Guard stops a blocking port from forwarding when BPDUs vanish (a unidirectional link). UDLD detects the unidirectional fibre itself. **Mistake.** PortFast on a switch-facing port invites loops.

Q4.9 Scenario: the network freezes every few minutes, switch CPU is at 100% and MACs flap between ports. Diagnose and fix.

CISCO · AMAZON · TECH-2 · ■■■ · S

30-second answer: a Layer 2 loop or storm; find the loop port from the flapping MAC and the interface counters, shut it, then fix the cause (a missing or broken spanning tree) and add guards.

Approach. (i) `show processes cpu` confirms the load; (ii) `show mac address-table | include <mac>` repeated shows the same MAC on alternating ports; (iii) `show interfaces | include broadcasts` or `show spanning-tree detail | include occurr|from` shows the port with the highest broadcast rate and the topology-change source. **Fix.** Shut that port. Common causes: an unmanaged switch looped by a patch cable, STP disabled or a BPDU filter, an unexpected PortFast, an EtherChannel mismatch. Then enable BPDU Guard, `storm-control broadcast level 1`, Loop Guard, and errdisable auto-recovery.

Q4.10 LACP active/passive vs PAgP vs `on`: which combinations form a bundle?

CISCO · ARISTA · TECH-1 · ■■□ · T

30-second answer: LACP forms with active-active and active-passive, but not passive-passive; PAgP with desirable-desirable and desirable-auto; `on` only with `on`; the protocols do not mix.

Answer. See the matrix in §4.3. `on` does not negotiate, so a mismatch can form a one-sided bundle and a loop. Prefer LACP: it is the open standard and detects a failed partner.

Q4.11 An EtherChannel shows member ports as suspended (s). Give five causes.

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the member's settings disagree with the bundle or the partner: speed or duplex, trunk or VLAN configuration, channel mode or protocol, no LACP reply, and differing spanning-tree settings.

Answer. (i) Speed or duplex mismatch; (ii) trunk mode, allowed list or native VLAN differs between members; (iii) LACP partner silent, or both sides passive, or `on` against LACP; (iv) different LACP system or port priority limits (more than eight members); (v) different STP port cost or PortFast settings. Check with `show etherchannel summary` and `show etherchannel detail`. LACP suspends a member that hears no PDUs so that a loop cannot form.

Q4.12 How does MLAG or vPC avoid STP blocking? What is the peer link for?

ARISTA · CISCO · AMAZON · TECH-2 · ■■■ · T

30-second answer: two physical switches present one logical switch to the downstream device, which bundles its uplinks across both; with no loop in the logical topology nothing blocks; the peer link synchronises state and carries traffic when needed.

Answer. The peers share MAC and ARP tables over the peer link, and a keepalive on a separate path detects a split-brain. Orphan ports (single-attached) use the peer link if the other peer holds the path. Internals in §5.3.

Q4.13 What is administrative distance? Give the values for connected, static, eBGP, EIGRP, OSPF, RIP and iBGP.

CISCO · AMAZON · TECH-1 · ■□□ · S

30-second answer: AD is a local trust rating of a routing source, from 0 to 255, used to choose between sources of the same prefix; connected 0, static 1, eBGP 20, EIGRP 90, OSPF 110, RIP 120, iBGP 200.

Answer. Full table in §4.4. AD is not carried in any protocol; it is configurable per route or per protocol (`distance`).

Q4.14 Order the routing decision. Solve the routing-table puzzle.

CISCO · AMAZON · JUNIPER · TECH-1 · ■■□ · S

30-second answer: longest prefix first, then lowest AD, then lowest metric, then equal-cost sharing.

Answer. The puzzle is solved in §4.4: 10.1.1.9 goes to the static /24, 10.1.5.5 to the EIGRP /16 and 10.200.0.1 to the RIP /8. **Twist.** Add a static 10.1.0.0/16 [1/0]: it replaces the EIGRP route (AD 1 against 90) and 10.1.5.5 follows it.

Q4.15 What is a floating static route? Configure a backup default route.

CISCO · TECH-1 · ■■□ · T

30-second answer: a static route with an AD higher than the primary's, which enters the table only when the primary route disappears.

Answer. `ip route 0.0.0.0 0.0.0.0 203.0.113.9 250` as backup to an OSPF default (AD 110). **Mistake.** It reacts to the route vanishing, not to a broken path behind an up link; tie it to an IP SLA track object (§4.14).

Q4.16 Explain RIB, FIB and TCAM. How does a hardware switch do longest-prefix match at line rate?

BROADCOM · ARISTA · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the RIB holds all routes the protocols offer; the FIB is the best-route set compiled for forwarding; hardware stores it in TCAM, which matches all entries in parallel, ordered so the longest prefix wins.

Answer. Detail is in §5.2. A ternary CAM stores each prefix with a mask, so a lookup is a single cycle; entries are sorted by prefix length. Large tables use algorithmic LPM in SRAM (tries) because TCAM is costly and power-hungry. The algorithm is in §1.9 (p. 30).

Q4.17 List the OSPF neighbour states in order.

CISCO · JUNIPER · NOKIA · TECH-1 · ■□□ · S

30-second answer: Down, (Attempt), Init, 2-Way, ExStart, Exchange, Loading, Full.

Answer. Figure 4.16 shows the packet that causes each step: Hello (Init), a Hello listing our ID (2-Way), DBD master election (ExStart), DBD summaries (Exchange), LSR and LSU (Loading). Attempt applies only to non-broadcast networks.

Q4.18 Two routers are stuck in EXSTART. Why? And in INIT, or 2-WAY?

CISCO · AMAZON · JUNIPER · TECH-1 · ■■□ · S

30-second answer: EXSTART/EXCHANGE: MTU mismatch, or a duplicate router ID or blocked unicast; INIT: hellos are heard in one direction only; 2-WAY: normal between two DROTHERs, a fault otherwise.

Answer. Evidence: `show ip interface` for MTU, `debug ip ospf adj` ("smaller interface MTU"). INIT: an ACL or a multicast filter blocks one direction, or one side's hello lacks our ID. 2-Way on a point-to-point link points at a network-type mismatch.

Q4.19 List the parameters that must match for an OSPF adjacency.

CISCO · NOKIA · TECH-1 · ■■□ · T

30-second answer: area ID, subnet and mask, hello and dead timers, authentication, stub-area flags, MTU, compatible network type; router IDs must differ.

Q4.20 How is the DR elected? Is it pre-emptive? Why have a DR?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: highest interface priority, then highest router ID; priority 0 is ineligible; it is not pre-emptive; the DR limits adjacencies to n-1 and floods each LSA once.

Answer. Without a DR, a segment of n routers needs n(n-1)/2 adjacencies. Worked example in §4.5. Force a change by setting priorities and clearing the process.

Q4.21 Explain LSA types 1, 2, 3, 4, 5, 7: who generates each and its scope.

CISCO · JUNIPER · NOKIA · AMAZON · TECH-1 · ■■□ · S

30-second answer: 1 router (every router, area); 2 network (DR, area); 3 summary (ABR, other areas); 4 ASBR summary (ABR, other areas); 5 external (ASBR, domain-wide except stubs); 7 NSSA external (ASBR in NSSA, converted to 5 by the ABR).

Answer. Table and scope map in §4.5. Mention that type 3 is distance-vector-like (prefix and cost only), which is why every area must hang off area 0.

Q4.22 Stub vs totally stubby vs NSSA: which LSAs are blocked?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: stub blocks 4 and 5; totally stubby blocks 3, 4 and 5 (one default type 3); NSSA blocks 4 and 5 but allows 7; totally NSSA also blocks 3.

Answer. Use a stub area where no ASBR exists in the area and routers are small; an NSSA where a local ASBR must exist, for example a branch connected to a partner network.

Q4.23 Why must all areas connect to area 0? What is a virtual link?

CISCO · TECH-1 · ■■□ · T

30-second answer: inter-area routing passes through the backbone as prefix-and-cost summaries, so a non-backbone path could loop; a virtual link tunnels area 0 connectivity across a transit area.

Answer. `area 1 virtual-link <ABR RID>` on both ABRs. It is a repair, not a design; a discontiguous backbone is better fixed physically.

Q4.24 Why do 1 Gb/s and 10 Gb/s links have the same OSPF cost by default? How do you fix it?

CISCO · AMAZON · TECH-1 · ■■□ · T

30-second answer: cost = 100 Mb/s divided by bandwidth, with a minimum of 1, so both are 1; set `auto-cost reference-bandwidth` to a higher value on every router.

Answer. With 100000 (100 Gb/s), 1G costs 100 and 10G costs 10. Alternatively set `ip ospf cost` per link. **Mistake.** Changing it on only some routers, which produces inconsistent metrics.

Q4.25 E1 vs E2 external routes: which is preferred, and when does it matter?

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: E2 (default) keeps the external cost unchanged; E1 adds the internal cost to the ASBR; an E1 route is preferred over E2 for the same prefix, and E1 matters when several ASBRs advertise the same prefix.

Answer. With E2, the nearest exit is chosen only as a tie-break (the lowest internal cost to the ASBR, the forwarding address). With E1 total cost is compared, so the closer ASBR wins unless the external costs differ enough.

Q4.26 Scenario: 200 customer routes in area 3 flap every few minutes and SPF runs network-wide. How do you contain it?

CISCO · TECH-2 · ■■■ · S

30-second answer: summarise at the ABR so the flaps never leave area 3, make area 3 stub or totally stubby, throttle SPF and LSA generation, and fix the root cause.

Approach. Type 1 and 2 LSAs flood within the area, so only area 3's routers recompute; but each flapping prefix also generates a type 3 into area 0 and beyond. `area 3 range 10.3.0.0 255.255.0.0` hides detail: while any prefix inside is up, the summary stays. Then `timers throttle spf 50 200 5000`, `timers throttle lsa all 50 200 5000`, BFD for failure detection and interface dampening (`dampening` on the interface) on the unstable links. **Mistake.** Summarising a range that has holes, which attracts traffic for addresses that do not exist; add a Null0 route.

Q4.27 OSPF vs IS-IS: why do many providers and data centres prefer IS-IS?

JUNIPER · NOKIA · GOOGLE · TECH-2 · ■■■ · T

30-second answer: IS-IS runs directly over Layer 2 rather than IP, uses TLVs that extend easily (IPv4, IPv6, segment routing) with one protocol, and scales through a simple two-level hierarchy.

Answer. Differences: OSPF has areas that attach to a backbone through ABRs, IS-IS has areas per router and a contiguous Level 2 backbone; OSPFv2 and v3 are separate protocols, IS-IS carries both families. Both are link-state and use Dijkstra, so say which one you have configured and why.

Q4.28 Define successor, feasible successor, FD and RD. State the feasibility condition.

CISCO · TECH-1 · ■■□ · T

30-second answer: the successor is the next hop of the best path; FD is our best total metric; RD is the neighbour's own metric to the destination; a neighbour is a feasible successor if its RD is lower than our FD.

Answer. The condition guarantees the neighbour's path does not pass through us, so the backup is loop-free without computation. Example in §4.6.

Q4.29 Compute the EIGRP metric for a T1 plus GigE path.

CISCO · TECH-1 · ■■□ · S

30-second answer: 256 × (10^7 / 1544 + (20,000 + 10) / 10) = 256 × (6476 + 2001) = 2,170,112.

Answer. Slowest bandwidth in kb/s, sum of delays in tens of microseconds, integer division at each step, K1 = K3 = 1.

Q4.30 What is stuck-in-active, and how do stubs and summarisation prevent it?

CISCO · TECH-2 · ■■■ · T

30-second answer: a route with no feasible successor goes active and queries neighbours, who query theirs; if a reply does not return within three minutes the route is stuck in active and the neighbour is reset.

Answer. Stub routers are not queried at all; a summary lets a router answer a query for a covered prefix at once, so queries stop at the summarising boundary. Also: reliable links, and feasible successors by design.

Q4.31 OSPF vs EIGRP: five differences.

CISCO · MICROSOFT · TECH-1 · ■■□ · T

30-second answer: link-state against DUAL; open against Cisco-origin; cost against bandwidth-and-delay; equal-cost only against unequal-cost balancing; areas against summarisation anywhere.

Answer. A fifth point is the AD (110 against 90 and 170) and a sixth the packet transport (IP 89 against 88 with reliable delivery). The comparison table is in §4.6.

Q4.32 Why does BGP use TCP? Which port?

ALL · TECH-1 · ■□□ · T

30-second answer: BGP needs reliable, ordered delivery of large tables; TCP provides it so BGP does not need its own retransmission; the port is 179.

Answer. A BGP session is long-lived and incremental: after the initial table only changes are sent, which relies on a reliable stream. The drawback is that TCP failure detection is slow, so keepalives and BFD supplement it.

Q4.33 List the BGP states. What does Active mean?

CISCO · AMAZON · JUNIPER · TECH-1 · ■□□ · S

30-second answer: Idle, Connect, Active, OpenSent, OpenConfirm, Established; Active means the router is trying to open the TCP connection and has failed, not that the session is up.

Answer. Troubleshoot Idle and Active with reachability (`ping` from the update source), ACLs on TCP 179, the AS number, the update source and `ebgp-multihop`. `show ip bgp summary` prints the state in the last column.

Q4.34 eBGP vs iBGP: AD, TTL, next hop, AS_PATH.

AMAZON · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: eBGP AD 20, TTL 1, next hop set to self, own AS prepended; iBGP AD 200, TTL 255, next hop unchanged, AS_PATH unchanged, and no re-advertisement to other iBGP peers.

Answer. The unchanged next hop is why iBGP needs `next-hop-self` or an IGP route to the external link. Table in §4.7.

Q4.35 Recite the BGP best-path selection order.

CISCO · AMAZON · JUNIPER · ARISTA · NOKIA · TECH-1 · ■■□ · S

30-second answer: N WLLA OMNI: next hop reachable, weight (high), local preference (high), locally originated, AS_PATH (short), origin (low), MED (low, same neighbour AS), eBGP over iBGP, lowest IGP metric; then oldest, lowest router ID, shortest cluster list, lowest neighbour address.

Answer. Walk the table of §4.7 aloud, then volunteer two reasons: weight and local preference come first because they are local policy; MED is compared only for one neighbour AS because the values of different ASes are not comparable. This is the single most reported BGP question. **Mistake.** Placing AS_PATH above local preference.

Q4.36 Weight vs local preference vs MED: scope, direction and higher or lower wins.

AMAZON · CISCO · TECH-1 · ■■□ · T

30-second answer: weight is local to one Cisco router, steers outbound, higher wins; local preference is shared in the AS by iBGP, steers outbound, higher wins; MED is sent to a neighbouring AS, hints inbound, lower wins.

	Scope	Steers	Wins
Weight	one router	outbound	highest
Local preference	whole AS	outbound	highest
MED	neighbour AS	inbound	lowest

Q4.37 Why does iBGP need a full mesh? How do route reflectors and confederations solve it?

GOOGLE · ARISTA · JUNIPER · TECH-2 · ■■□ · S

30-second answer: an iBGP router does not pass iBGP-learned routes to other iBGP peers (the AS_PATH does not change inside the AS, so it cannot detect loops), so all must peer directly; a route reflector is permitted to reflect, and a confederation turns sub-ASes into eBGP.

Answer. Ten routers need 45 sessions; two RRs and eight clients need 17. Clients peer only with the RR, which reflects client routes to all and non-client routes to clients. A confederation splits the AS into private sub-ASes with eBGP-like rules between them and a single public AS outward. RRs are more common.

Q4.38 How does a route reflector prevent loops? What is a cluster?

ARISTA · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the RR adds ORIGINATOR_ID (the router that first injected the route) and prepends its CLUSTER_ID to CLUSTER_LIST; a router that sees its own ID, or an RR that sees its own cluster ID, discards the route.

Answer. A cluster is an RR with its clients; the cluster ID defaults to the RR's router ID. If two RRs serve one cluster, give them the same cluster ID so that they ignore each other's reflections; use distinct IDs for more redundancy at the price of more state.

Q4.39 An iBGP router has the route in its BGP table but not in its RIB. Why?

AMAZON · CISCO · TECH-1 · ■■□ · T

30-second answer: usually the BGP next hop is not reachable through the IGP, so the route is not valid for installation; else a better-AD route for the same prefix exists (RIB failure).

Answer. Check `show ip bgp <prefix>` (no `>`; "inaccessible"), `show ip route <next hop>`. Fix with `next-hop-self` on the border router or by carrying the link in the IGP. Also `show ip bgp rib-failure`.

Q4.40 Scenario: dual-homed to ISP-A and ISP-B. Make ISP-A primary for inbound and outbound traffic. Write the policy.

AMAZON · CISCO · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: outbound, set local preference 200 on routes from ISP-A; inbound, prepend our AS two or three times on advertisements to ISP-B (and tag with the provider's low-preference community if offered).

Answer. The configuration and the `show ip bgp` output are in the dual-homing example of §4.7 (Figure 4.26 (p. 240)). **Follow-ups.** Prepending is a hint, because a remote AS with higher local preference ignores it; communities to the provider are stronger. More-specific prefixes (/25s) attract traffic strongly but are filtered by many networks beyond /24. Verify from a looking glass.

Q4.41 How does BGP multipath work? Which conditions must match? What does `multipath-relax` do?

ARISTA · GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: `maximum-paths N` installs up to N paths that tie through the IGP-metric step; weight, local preference, AS_PATH length, origin, MED and the neighbouring AS must match; `as-path multipath-relax` drops the identical-AS requirement.

Answer. In spine-leaf fabrics each leaf may have its own AS, so equal paths have different AS numbers; relax allows ECMP across them as long as the path lengths are equal.

Q4.42 Why is eBGP used inside large data-centre fabrics instead of OSPF (RFC 7938)?

GOOGLE · MICROSOFT · ARISTA · AMAZON · TECH-2 · ■■■ · T

30-second answer: eBGP has per-neighbour policy, no flooding of the whole topology to every router, scales to very large fabrics, and its failures are easy to reason about; one session per link needs no IGP underneath.

Answer. Typical design: one AS per rack or per leaf pair, one AS for the spines, `allowas-in` or unique private ASNs, ECMP with multipath-relax, BFD for fast failure. Link-state flooding in a fabric of thousands of nodes produces churn that BGP's path-vector updates avoid. See §5.4.

Q4.43 What are BGP communities? Explain no-export and no-advertise.

JUNIPER · NOKIA · TECH-1 · ■■□ · T

30-second answer: a community is a 32-bit tag attached to a route to signal policy; `no-export` keeps a route inside the AS (or confederation); `no-advertise` keeps it from every peer.

Answer. Communities are optional transitive and are only sent if `send-community` is configured. Providers publish communities to set local preference or to blackhole.

Q4.44 Scenario: a customer announces a more-specific prefix of a major content provider. What happens globally? How do RPKI/ROV and max-prefix help?

AMAZON · GOOGLE · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: routers prefer the longest prefix, so traffic for that range is drawn to the hijacker; RPKI origin validation marks the route invalid and filters it; a maximum-prefix limit stops a flood of leaked routes.

Answer. Longest-prefix match, not BGP preference, decides, so the more-specific wins everywhere it is accepted. ROV compares the origin AS and maximum length against signed ROAs. Providers filter customers by prefix list and IRR data, and a max-prefix on each customer session. The 2008 YouTube incident is the textbook case.

Q4.45 Coding: given BGP paths with attributes, implement best-path selection (steps 1 to 8).

GOOGLE · ARISTA · AMAZON · TECH-2 · ■■■ · S

30-second answer: filter the candidate list step by step, keeping only the paths that are best on each attribute, and stop as soon as one remains.

Approach. Eight ordered filters, each a keep-the-best on a key. MED is compared only within one neighbouring AS unless always-compare is set.

```
from dataclasses import dataclass
ORIGIN = {"i": 0, "e": 1, "?": 2}

@dataclass
class Path:
    nh_ok: bool; weight: int; lp: int;
local: bool
    as_path: list; origin: str; med: int
    ebgp: bool; igp: int; peer_as: int;
rid: str

def keep(c, key, best=min):
    v = best(key(p) for p in c)
    return [p for p in c if key(p) == v]

def med_step(c, always):
    if always: return keep(c, lambda p:
p.med)
    low = {}
    for p in c: low[p.peer_as] =
min(low.get(p.peer_as, p.med), p.med)
    return [p for p in c if p.med ==
low[p.peer_as]]

def best_path(paths, always_med=False):
    c = [p for p in paths if p.nh_ok]
    steps = [lambda c: keep(c, lambda p:
p.weight, max),
             lambda c: keep(c, lambda p:
p.lp, max),
             lambda c: keep(c, lambda p:
p.local, max),
             lambda c: keep(c, lambda p:
len(p.as_path)),
             lambda c: keep(c, lambda p:
ORIGIN[p.origin]),
             lambda c: med_step(c,
always_med),
             lambda c: keep(c, lambda p:
p.ebgp, max),
             lambda c: keep(c, lambda p:
p.igp)]
    for s in steps:
        if len(c) <= 1: break
        c = s(c)
    rid = lambda p: tuple(map(int,
p.rid.split(".")))
    return min(c, key=rid) if c else
None
```

Complexity. O(8n) time, O(n) space. **Mistakes.** Comparing MED across neighbour ASes; treating higher MED as better; forgetting step 0 (unreacha-

ble next hop). Real implementations also add the age and cluster-list tie-breaks.

Q4.46 HSRP vs VRRP: which is the open standard? Default preempt?

CISCO · TECH-1 · ■□□ · T

30-second answer: VRRP is the IETF standard; HSRP is Cisco's; HSRP preempt is off by default and VRRP preempt is on.

Answer. Other differences: groups addresses (224.0.0.2 or .102 against 224.0.0.18), virtual MACs and protocol (UDP 1985 against IP 112), VRRP's owner priority 255. Table in §4.8.

Q4.47 With HSRP tracking, R1's uplink fails but R1 stays active. Why?

CISCO · TECH-1 · ■■□ · T

30-second answer: tracking only lowers R1's priority; R2 takes over only if R2 has `preempt` and the new priority is below R2's, so check preempt on R2, the size of the decrement, and the track object.

Answer. `show standby` displays the current priority and the tracked object's state. R1: 110 with decrement 20 becomes 90; with R2 at 100 and preempt, R2 becomes active. A decrement below the gap of 10 leaves R1 active.

Q4.48 Prefix list `10.0.0.0/8 ge 16 le 24`: which of four prefixes match?

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: 10.0.0.0/8 no, 10.1.0.0/16 yes, 10.1.1.0/24 yes, 10.1.1.0/25 no.

Answer. The prefix must lie inside 10.0.0.0/8 and its length must be between 16 and 24 inclusive.

Q4.49 Explain the loop from two-way redistribution at two points. How do tags fix it?

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: a route redistributed from protocol A into B at one router is seen as native B by the second router, which redistributes it back into A; tags mark routes at the first redistribution, and the reverse route map denies that tag.

Answer. Configuration in §4.9 (Figure 4.30 (p. 252)). Alternatives: filter by prefix list, or raise the AD of the redistributed routes.

Q4.50 What is an MPLS label stack? Explain PHP and why implicit null exists.

JUNIPER · NOKIA · CISCO · TECH-1 · ■■□ · T

30-second answer: a stack is a sequence of 32-bit shim headers, the top one used for forwarding; the penultimate router pops the top label so that the egress router does one lookup instead of two, and the egress signals this by advertising implicit null (3).

Answer. The bottom-of-stack bit marks the last label. Without PHP the egress router would pop the label and then perform an IP lookup, two lookups in a hot spot.

Q4.51 Walk a packet through an MPLS L3VPN. What do RD and RT do?

JUNIPER · NOKIA · CISCO · TECH-2 · ■■■ · T

30-second answer: the ingress PE looks up the customer's VRF and pushes a VPN label (inner) and a transport label (outer); P routers swap the transport label; the penultimate router pops it; the egress PE uses the VPN label to select the VRF and forwards by IP. RD makes prefixes unique; RT selects which VRFs import them.

Answer. Walk and figure in §4.10 (Figure 4.34 (p. 255)). Control plane: LDP for transport labels, MP-BGP VPNv4 for VPN labels.

Q4.52 Policing vs shaping: the effect on TCP flows.

CISCO · NOKIA · BROADCOM · TECH-1 · ■■□ · T

30-second answer: a policer drops or re-marks excess packets, so TCP sees loss and its window oscillates; a shaper queues them, so TCP sees extra delay but fewer losses.

Answer. Shape on egress towards a slower circuit; police on ingress to enforce a contract. A shaper adds jitter, so voice goes to a priority queue ahead of it.

Q4.53 Decode DSCP 46 and AF31. Why does voice use LLQ?

CISCO · QUALCOMM · NOKIA · TECH-1 · ■■□ · T

30-second answer: DSCP 46 is EF (101110, ToS 0xB8); AF31 is class 3, drop precedence 1: 8 × 3 + 2 × 1 = 26; voice needs low delay and jitter, which a strict-priority queue provides.

Answer. LLQ is CBWFQ plus a priority queue with an implicit policer to stop starvation. Voice targets: delay 150 ms, jitter 30 ms, loss 1%.

Q4.54 How does an IPv4 multicast address map to a MAC address? Why the 32:1 ambiguity?

CISCO · JUNIPER · BROADCOM · TECH-1 · ■■□ · T

30-second answer: the low 23 bits of the group address follow the prefix 01:00:5E; a group has 28 variable bits, so 5 are lost and 32 groups share a MAC.

Answer. 239.1.1.1 and 224.129.1.1 both map to 01:00:5E:01:01:01 (§4.12). The host filters again at IP level; the cost is a little CPU.

Q4.55 Explain the PIM-SM shared tree vs the shortest-path tree and the RPF check. What if unicast routing is asymmetric?

JUNIPER · NOKIA · TECH-2 · ■■■ · T

30-second answer: receivers first join a tree rooted at the RP, (*,G); the last-hop router then switches to a tree rooted at the source, (S,G), which is shorter; the RPF check accepts a packet only on the interface the unicast table would use to reach the source.

Answer. With asymmetric routing the interface that carries the stream differs from the RPF interface, so the stream is dropped. Fix with a static `ip mroute` , a multicast-specific routing table (MP-BGP), or by making the routing symmetric.

Q4.56 Long-form: PVST+ on a four-switch square with a diagonal, VLANs 10 and 20 with different roots.

CISCO · TECH-2 · ■■■ · T

30-second answer: each VLAN runs its own election; with the root at SW1 for VLAN 10 and at SW4 for VLAN 20, SW4 blocks both square links for VLAN 10 and SW2 and SW3 block their links to SW1 for VLAN 20.

Approach. SW1 to SW4 have MACs ending 0001 to 0004. Links: SW1-SW2, SW2-SW4, SW4-SW3, SW3-SW1 and the diagonal SW1-SW4, all 1 Gb/s (cost 4). VLAN 10 keeps the default priority; VLAN 20 has priority 4096 on SW4.

Answer. *VLAN 10* (root SW1, lowest MAC): SW2 and SW3 use their direct links to SW1; SW4 uses the diagonal (cost 4). On SW2-SW4 both offer cost 4, so the lower BID (SW2) is designated and SW4's port blocks; on SW3-SW4 likewise SW3 is designated and SW4 blocks. *VLAN 20* (root SW4): SW2 and SW3 use their links to SW4; SW1 uses the diagonal. On SW1-SW2 and SW1-SW3 SW1 has the lower BID, so it is designated and SW2 and SW3 block. Hence the diagonal forwards both VLANs, and each square link carries one VLAN only.

Mistakes. Using one tree for both VLANs; forgetting that the root has no root port.

Q4.57 Long-form: EIGRP topology table, feasible successors and variance.

CISCO · TECH-2 · ■■■ · T

30-second answer: the successor is the lowest-FD neighbour; every neighbour with RD below that FD is a feasible successor; `variance 2` uses those whose own FD is below twice the best FD.

Approach. R1 has three neighbours for 10.9.9.0/24: R2 (FD 3000, RD 1000), R3 (FD 5400, RD 2500), R4 (FD 4200, RD 3500).

Answer. Successor R2, FD 3000. Feasibility condition RD < 3000: R3 (2500) passes, R4 (3500) fails. Variance 2 gives a limit of 6000: R3's FD 5400 is inside it, so R2 and R3 carry traffic, with a share inversely proportional to the metrics (about 64% and 36%). R4 is never used, though its FD of 4200 is inside the limit, because it is not feasible. If R2 fails, R3 is promoted with no query; R4 would require a query.

```
R1# show ip eigrp topology 10.9.9.0/24
P 10.9.9.0/24, 1 successors, FD is 3000
        via 10.1.2.2 (3000/1000)
        via 10.1.3.3 (5400/2500)
```

Q4.58 Long-form: an HSRP failover timeline.

CISCO · TECH-2 · ■■■ · T

30-second answer: tracking lowers R1's priority at once; with preempt, R2 becomes active in under a second (or at the hold time if R1 simply dies), then R1 preempts back after recovery.

Answer. Times are for hello 3 s, hold 10 s, R1 priority 110 with a decrement of 20 on Gi0/1, R2 priority 100 with preempt.

Time	Event	R1	R2
0	steady state	110, Active	100, Standby
10 s	Gi0/1 fails	90	100, hears a lower priority hello
10 s + 1 hello	R2 preempts	Standby	Active, gratuitous ARP
40 s	Gi0/1 returns	110, preempts after delay	Standby

If R1 were powered off instead, R2 would take over only after the 10 s hold time (about 3 s with `timers msec`). Verification is `show standby brief`. **Mistake.** Forgetting that R2 also needs `preempt` when the cause is a priority change.

Q4.59 Long-form: label tables at every node of an MPLS L3VPN.

JUNIPER · NOKIA · TECH-2 · ■■■ · T

30-second answer: PE1 pushes (24, 300); P1 swaps 24 for 35; P2 pops 35 (PHP); PE2 maps 300 to VRF A and routes by IP.

Answer. Customer A: 10.2.2.0/24 behind PE2, loopback 10.255.0.5/32.

Node	Table	Entry	Action
PE1	VRF A FIB	10.2.2.0/24: VPN label 300, next hop 10.255.0.5	push 300, then LDP label 24
P1	LFIB	in 24, 10.255.0.5/32	swap to 35, out to P2
P2	LFIB	in 35, 10.255.0.5/32	pop (PE2 advertised implicit null), out to PE2
PE2	VPN LFIB	in 300	pop, look up in VRF A, forward to CE2

Customer B's identical 10.1.1.0/24 has a different RD and RT (65000:2) and a different VPN label, so it lands in VRF B. **Mistake.** Saying the P routers look at label 300; they never do.

Q4.60 Long-form: QoS policy for a 10 Mb/s branch WAN with a bandwidth check.

CISCO · NOKIA · TECH-2 · ■■■ · T

30-second answer: shape the interface to 10 Mb/s in a parent policy, and in the child give voice an LLQ of 10%, video 30%, critical data 25%, scavenger 1%, and fair queuing to the rest; the reserved sum is 66%, below the 75% limit.

Answer. The configuration is in §4.11. The check: 10% of 10 Mb/s is 1 Mb/s of strict priority, enough for about ten G.711 calls at roughly 87 kb/s each with overhead. Video 3 Mb/s, critical data 2.5 Mb/s, scavenger 100 kb/s; 3.4 Mb/s remains for default. The shaper makes the Ethernet port behave as a 10 Mb/s link so that the child queues engage. Verify with `show policy-map interface`, and mark with iperf3. **Mistake.** Applying the child policy to a 1 Gb/s port without the parent shaper: it never congests, so it never queues.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Routing decision: longest prefix, then AD, then metric. (2) STP: lowest bridge ID is root; root port = lowest path cost, then lowest sender BID, then lowest sender port ID; the root has no root port. (3) STP priority moves in steps of 4096; VLAN is added (32768 + 1 = 32769). (4) OSPF states: Down, Init, 2-Way, ExStart, Exchange, Loading, Full; stuck in ExStart means MTU. (5) OSPF DR: priority, then router ID; not pre-emptive; 224.0.0.5 and .6. (6) LSA 1 and 2 stay in the area, 3 and 4 cross ABRs, 5 is domain-wide, 7 is converted to 5. (7) BGP: TCP 179, AD 20 and 200, iBGP never re-advertises iBGP routes, next hop unchanged in iBGP. (8) Best path: **N WLLA OMNI**, then **Old Routers Clear Neighbours**. (9) HSRP preempt off, VRRP on; tracking needs preempt on the standby. (10) EF = 46, AFxy = 8x + 2y, multicast MAC = 01:00:5E + low 23 bits.

Three figures to redraw. The STP triangle with R, D and blocked ports (Figure 4.9); the OSPF state ladder with its stuck points (Figure 4.16); the dual-homed AS diagram with local preference outbound and prepending inbound (Figure 4.26).

Administrative distance

Source	AD	Source	AD
Connected	0	RIP	120
Static	1	EIGRP external	170
EIGRP summary	5	iBGP	200
eBGP	20	DHCP default	254
EIGRP internal	90	Unusable	255
OSPF	110	IS-IS	115

OSPF states, LSAs and areas

State	Meaning
Down / Init	no hello / hello heard, own ID absent
2-Way	bidirectional; DR election; final for DROTHERs
ExStart / Exchange	master elected / DBD headers swapped (MTU)
Loading / Full	LSR and LSU / databases equal

LSA	Origin	Scope
1 Router	each router	area
2 Network	DR	area
3 Summary	ABR	other areas
4 ASBR summary	ABR	other areas
5 External	ASBR	all but stub
7 NSSA	ASBR in NSSA	NSSA, to 5 at ABR

Stub: no 4, 5. Totally stubby: no 3, 4, 5. NSSA: no 4, 5; has 7. Timers 10 and 40 s (broadcast), 30 and 120 s (NBMA). Cost = reference / bandwidth, default reference 100 Mb/s. Route preference: O, O IA, E1, E2.

BGP best path (Cisco)

0 next hop reachable. 1 weight, high. 2 local preference, high (100). 3 locally originated. 4 AS_PATH, short. 5 origin, i < e < ?. 6 MED, low, same neighbour AS. 7 eBGP over iBGP. 8 IGP metric, low. 9 multipath. 10 oldest eBGP. 11 router ID, low. 12 cluster list, short. 13 neighbour address, low.

Attributes: well-known mandatory (ORIGIN, AS_PATH, NEXT_HOP); well-known discretionary (LOCAL_PREF, ATOMIC_AGGREGATE); optional transitive (COMMUNITY, AGGREGATOR); optional non-transitive (MED, ORIGINATOR_ID, CLUSTER_LIST). Keepalive 60 s, hold 180 s. Sessions: full mesh n(n-1)/2.

STP and EtherChannel

802.1D state	Time	RSTP state
Blocking	max age 20 s	Discarding
Listening	15 s	Discarding
Learning	15 s	Learning
Forwarding		Forwarding

Hello 2 s, forward delay 15 s, max age 20 s. Costs (short): 10M 100, 100M 19, 1G 4, 10G 2. Roles: root,

inks; Loop Guard on non-designated uplinks. LACP active or passive, PAgP desirable or auto; passive-passive and auto-auto never form. Flags: SU in use, P bundled, s suspended, D down.

Short notes by section

4.1 CLI. Three planes: management, control, data. Prompts show the mode. `copy run start` saves. Register 0x2142 skips the saved configuration (recovery). CEF = FIB + adjacency table. Say: "the control plane builds the tables; the data plane only reads them."

4.2 VLANs. Access untagged, trunk tagged; native untagged. Tag 4 bytes: TPID 0x8100, PCP 3, DEI 1, VID 12. ROAS hairpins; SVI routes in hardware and needs an active port. DTP: configure statically. VTP: transparent or off. Say: "a VLAN is a broadcast domain; routing is needed between them."

4.3 STP. Four-step election; costs 4 (1G), 2 (10G); Rapid PVST+ is the default choice; MST for many VLANs. BPDU Guard on edge, Root Guard down, Loop Guard on uplinks. EtherChannel: LACP, same speed, duplex, VLANs; one flow, one member. Say: "STP elects a root and blocks what is left."

4.4 Routing table. LPM, then AD, then metric. Floating static: AD above the primary. Null0 anchors summaries. Say: "the most specific route wins, whatever its source."

4.5 OSPF. Link-state, Dijkstra, protocol 89, 224.0.0.5 and .6. Adjacency checklist: area, subnet, timers, authentication, stub flag, MTU, unique RID. Summarise at ABRs; stub areas shrink the database. Say: "every router in an area holds the same database and runs its own SPF."

4.6 EIGRP. DUAL; FC: RD < FD; K1 = K3 = 1; variance only over feasible successors; stubs and summaries limit queries. Say: "a feasible successor is a loop-free backup chosen in advance."

4.7 BGP. Path vector over TCP 179. iBGP: next hop unchanged, no re-advertisement, full mesh or RR. LOCAL_PREF outbound; prepend, MED, communities inbound. `network` needs an exact RIB match. Say: "BGP chooses by policy first and distance second."

4.8 FHRP. Virtual IP and MAC; HSRP preempt off, VRRP on; track and preempt together; align with the STP root.

4.9 Policy. Prefix list `ge` / `le`; route-map first match, implicit deny; tag at redistribution, deny the tag back; PBR is per interface; VRF-lite = separate tables.

4.10 MPLS. 20 / 3 / 1 / 8; push, swap, pop; PHP; VPN label inner, transport label outer; RD unique, RT selects.

4.11 QoS. Mark at the edge; LLQ for voice; CBWFQ shares; WRED avoids synchronisation; policer drops, shaper delays; Tc = Bc / CIR.

4.12 Multicast. IGMP join; PIM-SM via RP, then SPT; RPF check; MAC 01:00:5E + 23 bits.

4.13 Services. NAT overload by ACL; relay with `ip helper-address`; NTP stratum; SNMPv3; syslog 0 to 7; ACL top-down, implicit deny, wildcard = inverse mask.

4.14 Tunnels. GRE 24 bytes, MTU 1476, recursive-routing fault; IP SLA with a tracked static route; SD-WAN separates control from edge.

Top show commands

Task	IOS command	What to look for
Interfaces	`show ip interface brief`	up/up
VLANs and trunks	`show vlan brief`, `show interfaces trunk`	allowed list, native VLAN
STP	`show spanning-tree vlan N`	root, role, state
EtherChannel	`show etherchannel summary`	SU, P, s
Routes	`show ip route`, `show ip cef N`	source, AD/metric, next hop
OSPF	`show ip ospf neighbor`, `... database`, `... interface`	state, LSAs, timers
BGP	`show ip bgp summary`, `show ip bgp N`	state/prefixes, >, attributes
FHRP	`show standby brief`	priority, state
MPLS	`show mpls forwarding-table`, `show bgp vpnv4 unicast all`	labels
QoS	`show policy-map interface`	per-class drops
Multicast	`show ip mroute`, `show`	

	`ip igmp groups`	flags, RPF
Services	`show ip nat translations`, `show ntp status`, `show ip access-lists`	matches, sync

Other facts

Topic	Facts
FHRP	HSRP 224.0.0.2/.102, UDP 1985, 3 s / 10 s; VRRP 224.0.0.18, proto 112; GLBP 224.0.0.102
MPLS	label 20, TC 3, S 1, TTL 8; labels 0 to 3 reserved; LDP 646; RD unique, RT selects
QoS	EF 46, AF11 10, AF21 18, AF31 26, AF41 34, CS1 8; voice 150 ms, 30 ms, 1%; Tc = Bc / CIR
Multicast	224/4, 232/8 SSM, 239/8 private; IGMPv2, v3; PIM-SM; RPF
Services	syslog 0 to 7: EACCEWNID; NTP stratum, UDP 123; SNMPv3 authPriv; GRE 24 bytes

Q	A
Access vs trunk?	One VLAN untagged; many VLANs tagged
Usable VLAN IDs?	4094 (12 bits, 0 and 4095 reserved)
802.1Q tag size?	4 bytes; frame 1522
`auto` + `auto` DTP?	Access
ROAS drawback?	One link carries each packet twice
STP root?	Lowest bridge ID
STP priority step?	4096
Root port?	Lowest root path cost, on non-root only
802.1D worst convergence?	50 s
RSTP port states?	Discarding, learning, forwarding
Where Root Guard?	Distribution downlinks
LACP passive + passive?	No bundle

Q	A
`(s)` in channel summary?	Suspended member
AD of OSPF, eBGP, iBGP?	110, 20, 200
Floating static?	AD above the primary route
/24 static vs /16 connected?	/24: longest prefix
OSPF protocol number?	89
Stuck in ExStart?	MTU mismatch
2-Way between DROTHERs?	Normal
Is the DR preemptive?	No
Default OSPF reference?	100 Mb/s: 1G = 10G = 1
LSA 3 originator?	ABR
LSA 5 blocked in?	Stub, NSSA
Totally stubby gets?	Type 1, 2 and one type 3 default
E1 vs E2?	E1 adds internal cost; E1 preferred
EIGRP FC?	RD < FD
EIGRP K defaults?	K1 = K3 = 1
EIGRP AD?	90, 170 external
BGP port?	TCP 179
BGP Active?	Trying to connect, not up
iBGP next hop?	Unchanged
Full mesh count?	n(n-1)/2
RR loop attributes?	ORIGINATOR_ID, CLUSTER_LIST
Highest wins?	Weight and local preference
MED compared?	Same neighbour AS only
Inbound TE tools?	Prepend, MED, more specifics, communities
`network` in BGP needs?	Exact RIB match
`no-export`?	Not outside the AS
HSRP default preempt?	Off
VRRP owner priority?	255
`ge 16 le 24`?	Length 16 to 24 inclusive
Redistribution loop fix?	Tag and deny the tag

PHP?	Penultimate router pops the label
RD vs RT?	Uniqueness vs import and export
DSCP EF?	46, ToS 0xB8
AF31?	26
Policer vs shaper?	Drops vs delays
Multicast MAC?	01:00:5E + low 23 bits
RPF check?	Accept only from the unicast path to the source
GRE overhead?	24 bytes; MTU 1476

::: {.chapter .p7} ::: opener PART I · PILLAR 7

7

Operating Systems and Socket Programming

Beneath every packet is a machine that schedules, locks, maps and interrupts; we learn to follow the packet through it.

1. 7.1 What an operating system does MUST
2. 7.2 Processes, fork and exec MUST
3. 7.3 Threads and context switching MUST
4. 7.4 CPU scheduling SHOULD
5. 7.5 Synchronisation MUST
6. 7.6 Deadlock, starvation, priority inversion MUST
7. 7.7 Memory management and virtual memory MUST
8. 7.8 Interrupts, system calls and DMA MUST
9. 7.9 Inter-process communication and signals SHOULD
10. 7.10 Sockets from first principles MUST
11. 7.11 Server concurrency: threads, select, poll, epoll MUST
12. 7.12 The Linux receive path: NIC to recv() MUST
13. 7.13 Firmware and embedded essentials SHOULD
14. 7.14 Debugging a slow server OPTIONAL :::

7.0 Why this pillar matters, and how to read it

Networking products are programs running on an operating system, and the questions asked about them are operating-system questions: how a process differs from a thread, why two threads corrupt a counter, what a lock is, what happens when memory runs out, and how a packet gets from the wire to `recv()`. Cisco, Juniper and Arista ask these for software roles; Qualcomm and Broadcom ask them again for firmware, with interrupts, DMA and `volatile` added.

The chapter has three movements. Sections 7.1–7.9 are the classical core: processes, threads, synchronisation, deadlock, memory, interrupts. Sections 7.10–7.11 teach the Berkeley sockets API and the server architectures built on it; this is the canonical home of the API in the book. Sections 7.12–7.14 follow a packet through the Linux kernel. Two boundaries keep the book free of repetition. C memory, pointers and byte order belong to Pillar 2 (p. 89), and we use them here without re-teaching them. The behaviour of the TCP protocol itself, its state machine and congestion control, belongs to Pillar 3 (p. 146); here we meet TCP only as the thing a socket call triggers.

Every program in this chapter was compiled with `gcc -Wall -Wextra` on 64-bit Linux and run, and every printed output is from that run. Compile threaded programs with `-pthread`. Exact

timings and counts vary between machines; where a number varies, the text gives its range and not a promise.

> **BIG IDEA**
>
> An operating system multiplexes one CPU, one memory and a set of devices among many programs, and it can do so safely only because the hardware gives it a privileged mode that programs do not have.

7.1 What an operating system does MUST · ■□□

An *operating system* is a resource manager and a protection boundary. As a manager it divides the CPU among runnable tasks (scheduling), gives each program its own view of memory (virtual memory), and shares disks and network cards through uniform abstractions (files, sockets). As a boundary it prevents a faulty or hostile program from reading another program's memory or taking over a device. The mechanism is a *mode bit* in the processor: in *user mode* the CPU refuses privileged instructions and accesses to kernel memory; in *kernel mode* it allows them. The *kernel* is the part of the operating system that runs in kernel mode.

Mode bit — one bit of CPU state. Think of a hostel warden's master key: residents (user mode) cannot copy it.

A program asks the kernel for service through a *system call*. The C library wrapper (`read()`, `send()`, `fork()`) loads the call number and arguments into registers and executes a special instruction (`syscall` on x86-64). The processor switches to kernel mode, jumps to a fixed entry point, switches to a kernel stack, and the kernel indexes a table of handlers by the call number. The result returns in a register, and a return instruction (`sysret`) restores user mode. Figure 7.1 shows the layers and the boundary.

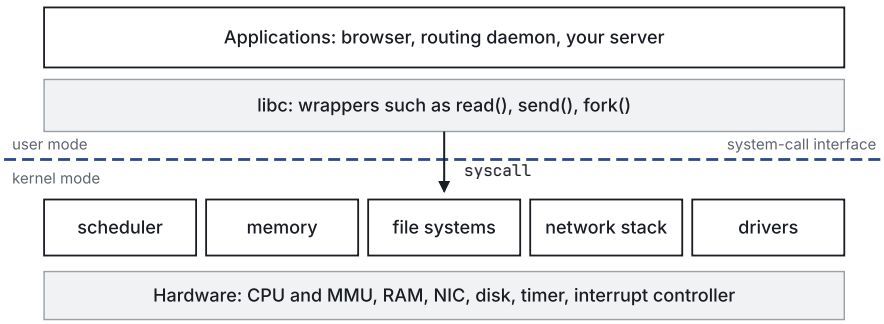


Figure 7.1 The user/kernel boundary. System calls cross it on request; interrupts cross it from below without a request.

> **LAB**
>
> `LAB 7.1 · 10 MIN` Count the system calls a trivial program makes.

```
$ strace -c ls /usr > /dev/null
% time     seconds  usecs/call     calls    errors syscall
 18.61    0.000190          27         7           read
 17.34    0.000177          35         5           mprotect
 11.75    0.000120          13         9           close
 10.19    0.000104          14         7           openat
  5.78    0.000059          29         2           getdents64
```

> **What you should notice:** `ls` is a few lines of logic, yet it makes dozens of calls: `openat` to open the directory, `getdents64` to read its entries, `write` to print (not shown), `mprotect` and `mmap` from the dynamic loader. Every line of the table is a mode switch.

> TRAP
>
> *Claim:* "A system call is just a function call." *Why it is wrong:* it changes privilege level, switches stacks, and with the speculative-execution mitigations in current kernels costs from about a hundred nanoseconds to over a microsecond, against a few nanoseconds for an ordinary call. *Correct statement:* a system call is a controlled, expensive entry into the kernel; high-performance I/O designs (batching, `epoll`, `io_uring`, kernel bypass) exist to make fewer of them.

> CHECK YOURSELF
>
> 1. Why can a user program not simply execute the instruction that disables interrupts? [1]
> 2. Name two things that move execution from user mode to kernel mode other than a system call. [2]

7.2 Processes, fork and exec MUST · ■■□

A *process* is a program in execution. The kernel represents it by a *process control block*, called `task_struct` in Linux, which records the identifier (PID), the scheduling state, the saved registers, a pointer to the memory map, the open files and the signal settings. Each process owns a private *virtual address space* (the memory model, segment by segment, is in Pillar 2 (p. 91)); Figure 7.2 shows how the control block points to it.

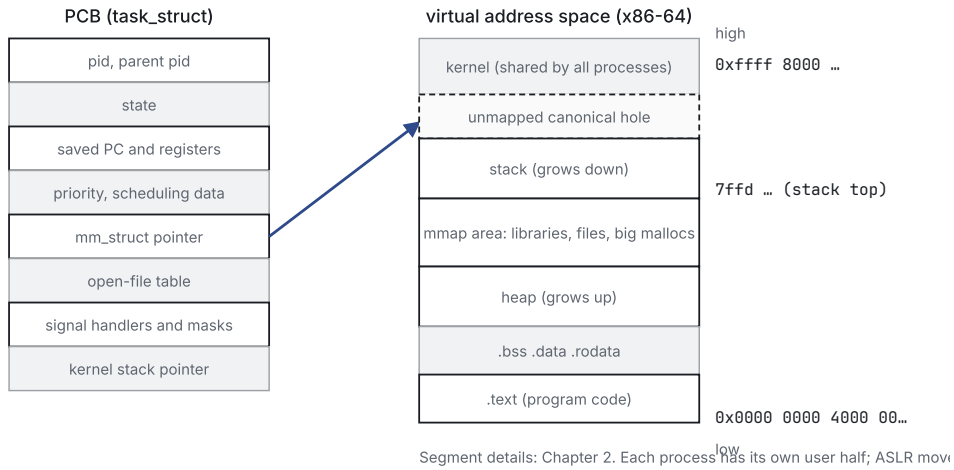


Figure 7.2 The PCB (left) and the address space it describes (right). The kernel half is the same in every process; the user half is private.

A process is always in one of five states, shown in Figure 7.3. It is *ready* when it could run but is waiting for a CPU, *running* when it holds one, and *waiting* (or blocked) when it needs an event such as disk or network data. Only the scheduler moves a task between ready and running; only an event moves it from waiting to ready.

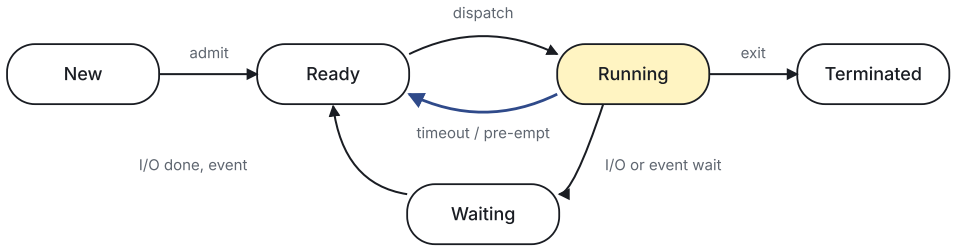


Figure 7.3 Process states. Only the accent-coloured transition, pre-emption, is forced on the running process.

Creating processes: fork, exec, wait

Linux creates a process by duplication. `fork()` returns twice: in the parent it returns the child's PID, and in the child it returns 0. The child receives a copy of the parent's address space, made cheap by *copy-on-write* (7.7 (p. 306)): both processes share the same physical pages, marked read-only, until one writes. `exec()` replaces the calling process's program with a new one while keeping its PID and open files. A shell runs a command as `fork`, then `exec` in the child, then `wait` in the parent. `wait()` returns the child's exit status and lets the kernel discard its control block.

Fork puzzles ask how many lines a program prints. The rule is that each `fork()` doubles the number of processes that execute the code after it. Hence a loop of three forks yields $2^3 = 8$ processes.

```
for (int i = 0; i < 3; i++)
    fork();
printf("hi\n");        /* printed by 8 processes */
```

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    for (int i = 0; i < 3; i++)
        fork();
    printf("hi\n");
    while (wait(NULL) > 0)      /* reap children so output is complete */
        ;
    return 0;
}
```

The output is eight lines (`./fork3 | wc -l` prints 8). The variant `fork() && fork() || fork()` is subtler, because `&&` and `||` short-circuit and the child sees a different return value from the parent. The parent gets a non-zero PID from both forks, so the `||` is skipped. The first child gets 0 from its fork, skips the `&&` branch and evaluates the third fork; the second child gets 0 from its own fork and does the same. Counting the survivors gives five processes (Figure 7.4).

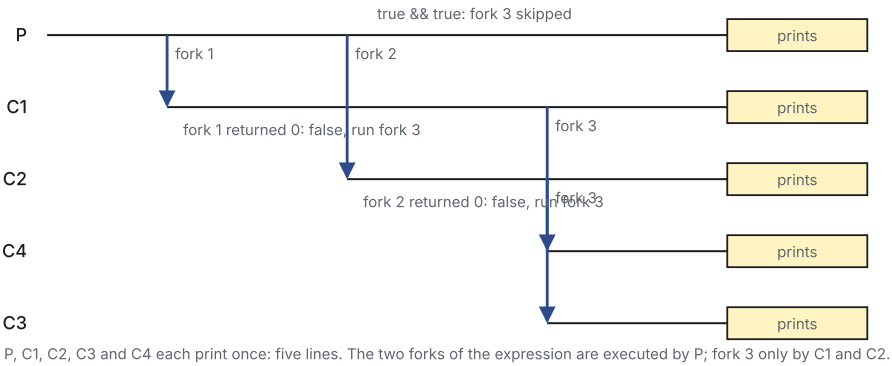


Figure 7.4 Process tree for fork() && fork() || fork(). Shaded: processes that continue after the expression; there are five.

> **TRAP**
>
> *Claim:* "`printf("hi")` before a `fork` prints once." *Why it is wrong:* when standard output is a pipe or file it is fully buffered, so the unflushed text sits in the process's memory and is copied into the child. Both processes flush it at exit. *Correct statement:* the text is printed twice; end the line with `\n` on a terminal, or call `fflush(stdout)` before forking.

Zombies and orphans

When a child exits, the kernel keeps its control block until the parent calls `wait()`, so the parent can still read the exit status. Between exit and `wait()` the child is a *zombie*: it uses no memory beyond the block, but it occupies a PID, and a parent that never waits eventually exhausts them. An *orphan* is the opposite case, a child whose parent has exited first. The kernel re-parents it to PID 1 (or to a designated subreaper), which calls `wait()` for it.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0)
        _exit(7);                       /* child exits at once */
    printf("child %d is a zombie for 5 s\n", (int)pid);
    fflush(stdout);
    sleep(5);                           /* run: ps -o pid,ppid,stat,cmd */
    int st;
    waitpid(pid, &st, 0);               /* reaping removes the zombie */
    printf("exit status %d\n", WEXITSTATUS(st));
    return 0;
}
```

> **LAB**
>
> `LAB 7.2 · 10 MIN` Create and reap a zombie. Run `./zombie &`, then in the same second:

```
$ ps -o pid,ppid,stat,cmd --ppid $(pgrep zombie)
  PID  PPID STAT CMD
 8935  8933 Z    [zombie] <defunct>
exit status 7
```

What you should notice: state `Z` and the label `<defunct>` mark the zombie. It cannot be killed, because it is already dead; only its parent's `wait()` (or the parent's death) removes it.

INTERVIEW LENS

CISCO · JUNIPER · QUALCOMM

"How many times does this print?" with `fork` in a loop, and "what is a zombie?", are reported in Cisco, Juniper and Qualcomm technical rounds. A strong answer states the doubling rule, draws the process tree, mentions stdout buffering if the output is redirected, and gives both the zombie definition and its cause (the parent did not wait). Follow-ups: how `exec` differs from `fork`, and what the shell does between them.

CHECK YOURSELF

1. How many processes exist after `fork(); fork(); fork();` executed once by one process? [3]
2. What does `fork()` return in the child? [4]
3. Does `exec()` create a new process? [5]

7.3 Threads and context switching MUST · ■■□

A *thread* is an execution context inside a process. All threads of a process share the code, the global data, the heap and the open files; each has its own stack, its own saved registers and its own *thread-local storage*, including `errno`. Because the address space is shared, creating a thread needs no new page tables and communicating between threads needs no kernel call: a write by one thread is visible to the others. The same sharing is the hazard.

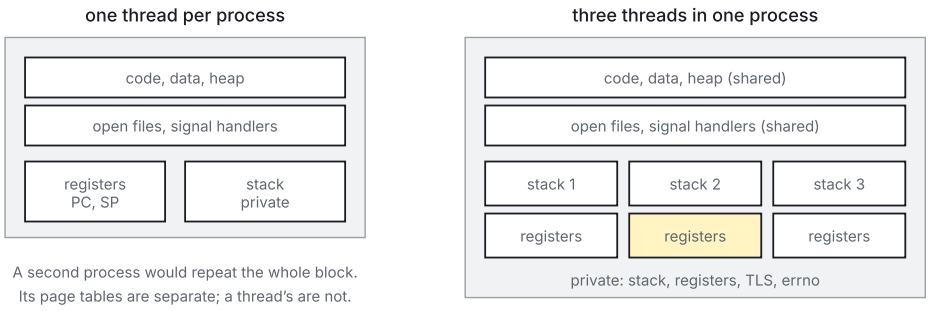


Figure 7.5 Process versus threads. Threads share everything except stack, registers and thread-local storage.

Linux implements a thread as a task that shares its parent's address space (the `clone()` system call with the sharing flags set), which is the *1:1* model: one kernel-scheduled task per user thread, so threads run in parallel on different cores. Runtimes such as Go multiplex many lightweight tasks onto few kernel threads (*M:N*), gaining cheap creation at the cost of a second scheduler.

BIG IDEA

Sharing memory makes threads cheap to create and to switch, and it is also why a bug in one thread can corrupt, or terminate, every other thread in the process.

The race condition

Two threads that each increment a shared counter one million times should produce two million. They do not, because `counter++` compiles to three steps (load, add, store), and the threads interleave between them. the table below shows how one update is lost.

Step	Thread A	Thread B	counter in memory
1	load counter (5)		5
2		load counter (5)	5
3	add 1 (6)		5
4		add 1 (6)	5
5	store 6		6
6		store 6	6

Both threads read 5 and both store 6, so one increment vanishes.

```
#include <pthread.h>
#include <stdio.h>

#define N 1000000
static volatile long counter;   /* volatile: stop the compiler hoisting it into
a register */
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

static void *plain(void *arg)
{
    (void)arg;
    for (int i = 0; i < N; i++)
        counter++;                      /* load, add, store: not atomic */
    return NULL;
}

static void *locked(void *arg)
{
    (void)arg;
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&m);
        counter++;
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

static void run(void *(*fn)(void *), const char *name)
{
    pthread_t a, b;
    counter = 0;
    pthread_create(&a, NULL, fn, NULL);
    pthread_create(&b, NULL, fn, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("%-8s counter = %ld (expected %d)\n", name, counter, 2 * N);
}

int main(void)
{
    run(plain, "plain");
    run(locked, "mutex");
    return 0;
}
```

```
$ gcc -O1 -pthread race.c -o race && ./race
plain    counter = 1092934 (expected 2000000)
mutex    counter = 2000000 (expected 2000000)
```

The unprotected total falls anywhere between about one and two million, and differs on each run. The `volatile` qualifier is present only to stop the optimiser from keeping the counter in a register, which would hide the race on some compilers; it does not fix it (7.13). The ThreadSanitizer (`gcc -fsanitize=thread -g`) reports it directly: `WARNING: ThreadSanitizer: data race`, with the two source lines involved.

What a context switch does

A *context switch* stops one task and starts another. The kernel (i) saves the running task's registers and program counter into its control block, (ii) lets the scheduler choose the next task, (iii) if the

next task belongs to a different process, loads its page-table base register, which invalidates or re-tags the TLB entries (7.7 (p. 306)), and (iv) restores the next task's registers. Step (iii) is the reason that switching between threads of the same process is cheaper than switching between processes: the address space does not change, so the cached translations and, to a lesser degree, the data caches stay useful. The direct cost of a switch is a few microseconds; the indirect cost, the cold caches afterwards, is often larger.

TRAP

Claim: "Threads have their own heaps" or "a thread switch flushes the TLB." *Why it is wrong:* the heap belongs to the process, and a switch inside one process keeps the same page tables. *Correct statement:* threads share the heap; only the stack, registers and thread-local storage are private, and a TLB flush happens only when the address space changes.

INTERVIEW LENS

CISCO · QUALCOMM · JUNIPER · ARISTA

"Process versus thread" is asked at nearly every networking employer, usually followed by "what happens if one thread crashes?". A strong answer gives the shared and private lists, states that a fault such as SIGSEGV is delivered to the process and ends all threads, and compares the switch cost with a reason (no address-space change). Follow-ups: thread safety and reentrancy (`strtok` keeps a hidden static pointer, so it is unsafe; `strtok_r` takes the state as an argument), and why a server prefers a pool of threads to one thread per request.

CHECK YOURSELF

1. Which segments are private to a thread? [6]
2. Why is the unprotected counter result different on each run? [7]

7.4 CPU scheduling SHOULD · ■■□

The *scheduler* chooses which ready task runs next, balancing throughput (work finished per second), latency (time to first response) and fairness. A *non-pre-emptive* algorithm lets a task run until it blocks or finishes; a *pre-emptive* one can interrupt it, normally on a timer tick. For one task we define the *turnaround time* TAT = completion − arrival and the *waiting time* WT = TAT − burst, where the burst is the CPU time needed.

We compare four algorithms on five processes: P1 arrives at 0 with burst 8, P2 at 1 with 4, P3 at 2 with 9, P4 at 3 with 5, P5 at 4 with 2.

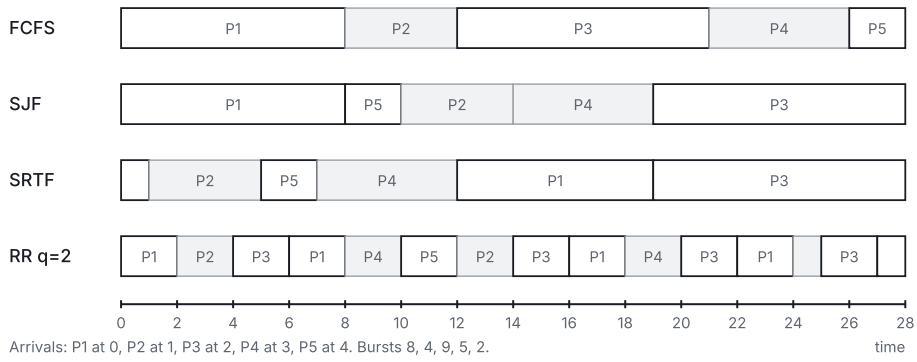


Figure 7.6 Gantt charts for the five processes. Shorter jobs first (SJF, SRTF) cut the average wait; round robin raises it here but bounds the response time.

WORKED EXAMPLE

Average waiting time under FCFS. First-come-first-served runs in arrival order, so P1 runs 0–8, P2 8–12, P3 12–21, P4 21–26, P5 26–28.

1. Waiting time = start − arrival for a task that runs without interruption: P1 0, P2 8 − 1 = 7, P3 12 − 2 = 10, P4 21 − 3 = 18, P5 26 − 4 = 22.
2. Sum = 57, average = 57/5.

Result: average waiting time 11.4, average turnaround 17.0. The short P5 waits behind the long P3, the *convoy effect*.

Algorithm	Pre-emptive?	Avg waiting	Avg turnaround	Property
FCFS	no	11.4	17.0	simple; convoy effect
SJF (shortest job first)	no	8.2	13.8	optimal waiting among non-pre-emptive; needs burst lengths; starves long jobs
SRTF (shortest re-maining time first)	yes	6.6	12.2	optimal average waiting overall; many switches
Round robin, q = 2	yes	13.0	18.6	fair; response bounded by (n − 1)q; q too small wastes time in switches

Round robin lets each task run for at most a *quantum* q and then moves it to the back of the queue. Its numbers above assume that a task arriving at the instant a quantum ends joins the queue before the pre-empted task; the opposite convention changes the order and the totals slightly, so a question should state it. *Priority scheduling* runs the highest-priority ready task; low-priority tasks may *starve*, and *ageing*, which slowly raises the priority of waiting tasks, cures it. A *multilevel feedback queue* (MLFQ) combines these ideas: new tasks enter the top queue with a short quantum, a task that uses its whole quantum is demoted, and a periodic boost lifts everything back so that nothing starves.

Linux does not use a fixed quantum. The Completely Fair Scheduler kept a *virtual runtime* for each task, advanced more slowly for tasks of higher weight, and always ran the task with the smallest value; two tasks with weights 2 : 1 therefore receive CPU in the ratio 2 : 1. From Linux 6.6 the pick logic is EEVDF (earliest eligible virtual deadline first), which keeps the weighted-fair accounting but also lets latency-sensitive tasks ask for shorter slices. The interview-level statement is that Linux gives each runnable task a weighted fair share and picks the one that has had the least.

TRAP

Claim: "SJF is the best algorithm." *Why it is wrong:* it is optimal for average waiting time only if burst lengths are known in advance, which they are not, and it starves long jobs when short ones keep arriving. *Correct statement:* real schedulers estimate the next burst from history (exponential averaging) and use ageing or feedback to prevent starvation.

CHECK YOURSELF

1. In FCFS, which of TAT and WT is larger for a task, and by how much? [8]
2. What does ageing prevent? [9]

1. It is a privileged instruction; in user mode the CPU raises a fault and the kernel decides what happens, normally terminating the program.↩
2. A hardware interrupt (timer, NIC) and an exception such as a page fault or a divide by zero.↩
3. Eight, counting the original: each fork doubles the count.↩
4. Zero. In the parent it returns the child's PID; on failure it returns -1 and no child exists.↩
5. No. It replaces the program image of the calling process; the PID and open descriptors survive.↩
6. The stack, the registers (including the program counter) and thread-local storage.↩
7. The interleaving depends on when the scheduler pre-empts each thread, which varies with load and timing; the lost updates vary with it.↩
8. TAT is larger, by exactly the burst time, since WT = TAT − burst.↩
9. Starvation of low-priority tasks, by raising a task's

7.5 Synchronisation MUST · ■■□

priority the longer it waits.↩

A *critical section* is a stretch of code that touches shared data and must not be executed by two threads at once. Enforcing this is *mutual exclusion*, and the tools differ in who may release them, whether a waiter sleeps, and where they may be used. We meet five: the mutex, the semaphore, the spinlock, the condition variable and the read-write lock.

A *mutex* (mutual-exclusion lock) has two states and an owner. `pthread_mutex_lock` takes it, sleeping if another thread holds it; `pthread_mutex_unlock` releases it, and only the owner may do so. Figure 7.7 shows the second thread being put to sleep and woken. The race in 7.3 (p. 290) disappears when the increment is placed between the two calls, at the price of slower code. On the four-core test machine, two threads making two million increments in total took about 120 ms under a contended mutex and about 20 ms with `__atomic_fetch_add`, an *atomic* instruction that performs the single-word update without ever sleeping; the unprotected loop was faster still, and wrong. Only the ordering of these figures should be trusted, not their values.

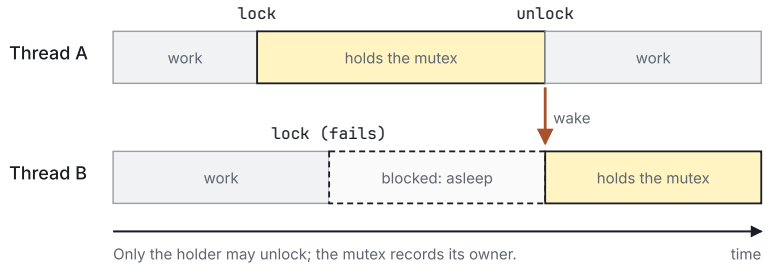


Figure 7.7 A mutex timeline. Thread B fails to lock, sleeps, and is woken when A unlocks.

A *semaphore* is a non-negative counter with two atomic operations: `wait` (also P or down) decrements it, sleeping while it is zero, and `post` (V or up) increments it and wakes a sleeper. It has no owner, so any thread may post. A *counting* semaphore initialised to N admits N threads at once, as a pool of N identical resources; a *binary* semaphore (N = 1) behaves like a lock but may be released by a different thread than the one that took it, which makes it a signalling device between threads.

Semaphore — a counter with two operations. A car park with a "spaces free: 5" board: `wait` takes a space, `post` returns one.

A *spinlock* does not sleep: a waiter loops on an atomic test until the lock is free. This is wasteful on a single core and fine for very short critical sections on several. It is the only lock usable where sleeping is forbidden, such as an interrupt handler (7.8 (p. 312)), because a sleeping task cannot be scheduled from interrupt context. A naive test-and-set loop writes to the lock's cache line on every attempt, so the line bounces between cores; the *test-and-test-and-set* refinement spins on an ordinary read, which hits the local cache, and attempts the atomic write only when the lock looks free.

```
#include <pthread.h>
#include <stdio.h>

#if defined(__x86_64__)
#define cpu_relax() __builtin_ia32_pause()   /* be kind to the sibling hyper-
thread */
#else
#define cpu_relax() ((void)0)
#endif

typedef struct { volatile char flag; } spinlock_t;

static void spin_lock(spinlock_t *l)
{
    for (;;) {
        /* test-and-set: atomically write 1 and return the old value */
        if (!__atomic_test_and_set(&l->flag, __ATOMIC_ACQUIRE))
            return;
        /* test-and-test-and-set: spin on a plain read, which stays in
           the local cache, and retry the atomic only when it looks free */
        while (__atomic_load_n(&l->flag, __ATOMIC_RELAXED))
            cpu_relax();
    }
}

static void spin_unlock(spinlock_t *l)
{
    __atomic_clear(&l->flag, __ATOMIC_RELEASE);
}

static spinlock_t lock;
static long counter;

static void *work(void *arg)
{
    (void)arg;
    for (int i = 0; i < 1000000; i++) {
        spin_lock(&lock);
        counter++;
        spin_unlock(&lock);
    }
    return NULL;
}

int main(void)
{
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, work, NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    printf("counter = %ld (expected 4000000)\n", counter);
    return 0;
}
```

The program prints `counter = 4000000 (expected 4000000)`. Note the acquire and release memory orders: acquire on lock and release on unlock ensure that the data protected by the lock is seen consistently by the next holder.

A *condition variable* lets a thread sleep until a predicate on shared data becomes true. It is always used with a mutex: `pthread_cond_wait(&c, &m)` atomically releases the mutex and sleeps, and re-acquires the mutex before returning. The waiter must recheck the predicate in a `while` loop,

not an `if`, for two reasons: a wake-up may be *spurious* (the standard permits it), and a wake-up may be *stolen* (another thread takes the item between the signal and the waiter's re-acquisition of the mutex).

> BIG IDEA
>
> A condition variable does not remember signals; the predicate on the shared data is the truth, and the loop around `cond_wait` is how a thread rechecks it.

Worked example: the bounded buffer

The *producer–consumer* problem asks for a fixed-size queue shared by threads that add items and threads that remove them: producers must wait when the queue is full, consumers when it is empty. The solution below uses one mutex and two condition variables. Three producers make 10,000 items between them, two consumers take them, and the program compares the sum put in with the sum taken out.

```
#include <pthread.h>
#include <stdio.h>

#define SLOTS 8
#define ITEMS 10000
#define PRODUCERS 3
#define CONSUMERS 2

static int buf[SLOTS];
static int head, tail, count;
static int produced, consumed;          /* items handed out / taken so far */
static long sum_in, sum_out;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
static pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

static void *producer(void *arg)
{
    (void)arg;
    for (;;) {
        pthread_mutex_lock(&m);
        if (produced == ITEMS) {        /* nothing left to make */
            pthread_mutex_unlock(&m);
            return NULL;
        }
        int item = ++produced;          /* claim item number under the lock */
        while (count == SLOTS)          /* while, not if */
            pthread_cond_wait(&not_full, &m);
        buf[tail] = item;
        tail = (tail + 1) % SLOTS;
        count++;
        sum_in += item;
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&m);
    }
}

static void *consumer(void *arg)
{
    (void)arg;
    for (;;) {
        pthread_mutex_lock(&m);
        while (count == 0 && consumed < ITEMS)
            pthread_cond_wait(&not_empty, &m);
        if (count == 0) {               /* all items consumed: finish */
            pthread_mutex_unlock(&m);
            return NULL;
        }
        int item = buf[head];
        head = (head + 1) % SLOTS;
        count--;
        consumed++;
        sum_out += item;
        pthread_cond_signal(&not_full);
        if (consumed == ITEMS)
            pthread_cond_broadcast(&not_empty);   /* release idle consumers */
        pthread_mutex_unlock(&m);
    }
}

int main(void)
{
```

```
    pthread_t p[PRODUCERS], c[CONSUMERS];    for (int i = 0; i < PRODUCERS;
i++) pthread_create(&p[i], NULL, producer, NULL);    for (int i = 0; i <
CONSUMERS; i++) pthread_create(&c[i], NULL, consumer, NULL);    for (int i = 0;
i < PRODUCERS; i++) pthread_join(p[i], NULL);    for (int i = 0; i < CONSUMERS;
i++) pthread_join(c[i], NULL);    printf("sum in = %ld, sum out = %ld, %s\n",
sum_in, sum_out,           sum_in == sum_out ? "match" : "MISMATCH");    return
0;}
```

```
$ gcc -Wall -Wextra -O2 -pthread bbuf.c -o bbuf && ./bbuf
sum in = 50005000, sum out = 50005000, match
```

Four details deserve attention. First, `count` is read and written only with the mutex held, so it is consistent. Second, a producer waits on `not_full` and a consumer on `not_empty`, so each signal wakes only a thread that can make progress. Third, the consumers must be released at the end; the `broadcast` when the last item is taken does this, and without it a consumer sleeping on an empty buffer would never return. Fourth, the same problem with semaphores uses `empty` (initially the number of slots) and `full` (initially 0), plus a mutex around the buffer indices:

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

#define SLOTS 4
#define ITEMS 1000

static int buf[SLOTS], in, out;
static sem_t empty, full;                  /* free slots, filled slots */
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static long sum;

static void *producer(void *arg)
{
    (void)arg;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);                  /* first: reserve a slot ... */
        pthread_mutex_lock(&m);            /* ... then take the lock. Swap them
and a  */
        buf[in] = i; in = (in + 1) % SLOTS;/* full buffer deadlocks: producer
holds m   */
        pthread_mutex_unlock(&m);          /* while sleeping, so the consumer
cannot    */
        sem_post(&full);                   /* take m to make room.
*/
    }
    return NULL;
}

static void *consumer(void *arg)
{
    (void)arg;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&m);
        sum += buf[out]; out = (out + 1) % SLOTS;
        pthread_mutex_unlock(&m);
        sem_post(&empty);
    }
    return NULL;
}

int main(void)
{
    pthread_t p, c;
    sem_init(&empty, 0, SLOTS);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL); pthread_join(c, NULL);
    printf("sum = %ld (expected %d)\n", sum, ITEMS * (ITEMS + 1) / 2);
    return 0;
}
```

The order of the two waits in the producer matters. If the producer took the mutex first and then waited on `empty`, a full buffer would leave it asleep holding the mutex; the consumer could then never take the mutex to remove an item and make room, and both would wait for ever. This is a deadlock (7.6 (p. 303)).

Choosing a primitive

	Mutex	Binary semaphore	Counting semaphore	Spinlock	Condition variable
Owner	yes, the locker	no	no	yes, by convention	not a lock
Waiter sleeps?	yes	yes	yes	no, spins	yes
Usable in an interrupt handler?	no	`post` only	`post` only	yes	no
Typical use	protect shared data	signal an event	limit a pool of N	very short sections, kernel	wait for a predicate
Linux kernel API	`mutex_lock`	`down` / `up`	`down` / `up`	`spin_lock_irqsave`	`wait_event`

A mutex and a binary semaphore are not interchangeable. Because a mutex has an owner, the system can lend the owner a higher priority when a more urgent thread waits (priority inheritance, 7.6 (p. 303)), can detect an unlock by the wrong thread, and can offer recursive and error-checking variants. A semaphore, lacking an owner, can be posted from an interrupt handler or from another thread, which is exactly what signalling needs.

Read-write locks

When many threads read and few write, `pthread_rwlock` lets any number of readers hold the lock together but gives a writer exclusive access. A reader-preferring implementation can *starve* a writer that waits behind a steady stream of readers; a writer-preferring one blocks new readers once a writer is waiting, trading reader latency for freedom from starvation.

> INTERVIEW LENS
>
> JUNIPER · CISCO · QUALCOMM · ARISTA
>
> “Implement a mutex or semaphore”, “mutex versus semaphore” and “write a multithreaded program with a lock” are reported Juniper and Cisco questions, and Qualcomm asks which lock is safe inside an interrupt handler. A strong answer gives the table above in words, states the ownership difference, writes the bounded buffer with a `while` around the wait, and explains why a spinlock is correct in an interrupt handler (nothing may sleep there) but wasteful in user code. Follow-ups: how to build a semaphore from a mutex and a condition variable (Q7.11), and what happens if `unlock` is called by a thread that does not own a mutex (undefined for a default mutex, an error for an error-checking one).

> TRAP
>
> *Claim:* “`volatile int flag` protects a critical section.” *Why it is wrong:* `volatile` stops the compiler from caching the variable but does not make a read-modify-write atomic and does not order other memory accesses. *Correct statement:* use a lock or an atomic operation with the right memory order; `volatile` is for device registers (7.13).

> CHECK YOURSELF
>
> 1. A consumer wakes from `cond_wait` and finds the queue empty. Why can this happen? [1]
> 2. Which of mutex and spinlock may be used in an interrupt handler? [2]
> 3. Can a thread unlock a mutex that another thread locked? [3]

7.6 Deadlock, starvation and priority inversion MUST · ■■□

A set of tasks is *deadlocked* when each waits for a resource that another task in the set holds, so none can proceed. Four conditions, the *Coffman conditions*, must hold together: (i) *mutual exclusion*, a resource is held by at most one task; (ii) *hold and wait*, a task holds one resource while requesting another; (iii) *no pre-emption*, a resource cannot be taken away; (iv) *circular wait*, a cycle of tasks each waits for the next. Hence preventing deadlock means breaking any one of them.

Condition	Prevention	Cost
Mutual exclusion	make the resource shareable (read-only data, lock-free structures)	not possible for a printer or a write
Hold and wait	request all resources at once, or release before requesting	low utilisation, possible starvation
No pre-emption	if a request fails, release everything held and retry (`pthread_mutex_trylock`)	restart cost; livelock if both retry in step
Circular wait	impose a global order on locks and always acquire in that order	the standard practical answer

A *resource-allocation graph* draws tasks as circles and resources as squares, with a request edge from a task to a resource and an assignment edge from a resource to a task. A cycle is the circular-wait condition made visible.

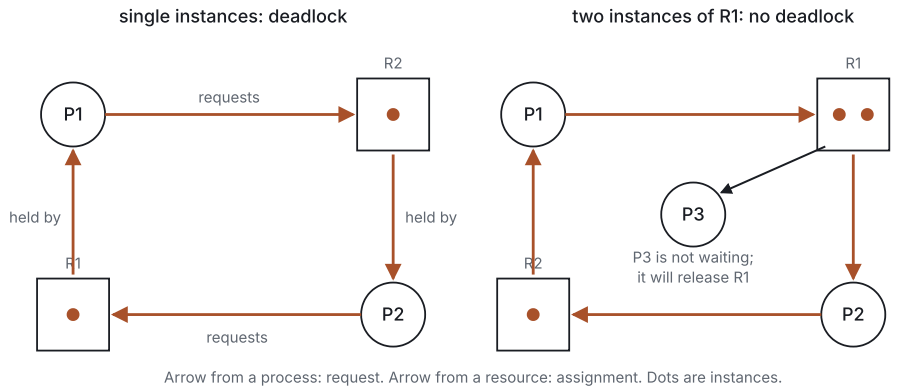


Figure 7.8 Resource-allocation graphs. Left: a cycle with single-instance resources is a deadlock. Right: a cycle with a multi-instance resource is not, since P3 can finish and release R1.

> **TRAP**
>
> *Claim:* "A cycle in the resource graph always means deadlock." *Why it is wrong:* with several instances of a resource, a task outside the cycle may release one and let the cycle dissolve. *Correct statement:* a cycle is necessary for deadlock; it is sufficient only when every resource has a single instance.

Worked example: lock ordering

Thread 1 locks A then B; thread 2 locks B then A. If each takes its first lock before either takes its second, both wait for ever. The program below does this deliberately, with a short sleep to make the bad interleaving certain.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER, B =
PTHREAD_MUTEX_INITIALIZER;

static void *t1(void *x) { (void)x;
    pthread_mutex_lock(&A); usleep(100000); pthread_mutex_lock(&B);   /* A then
B */
    puts("t1 done"); pthread_mutex_unlock(&B); pthread_mutex_unlock(&A); return
NULL; }

static void *t2(void *x) { (void)x;
#ifdef FIXED
    pthread_mutex_lock(&A); usleep(100000); pthread_mutex_lock(&B);   /* same
order */
#else
    pthread_mutex_lock(&B); usleep(100000); pthread_mutex_lock(&A);   /* B then
A: cycle */
#endif
    puts("t2 done"); pthread_mutex_unlock(&A); pthread_mutex_unlock(&B); return
NULL; }

int main(void)
{
    pthread_t a, b;
    pthread_create(&a, NULL, t1, NULL); pthread_create(&b, NULL, t2, NULL);
    pthread_join(a, NULL); pthread_join(b, NULL);
    return 0;
}
```

```
$ gcc -g -pthread dl.c -o dl && ./dl &
$ gdb -p $! -batch -ex "thread apply all bt 2"
Thread 3 ...  #1  __lll_lock_wait (futex=... <B>, private=0)
Thread 2 ...  #1  __lll_lock_wait (futex=... <A>, private=0)
Thread 1 ...  #0  __futex_abstimed_wait_common64 ... pthread_join
```

What you should notice: two threads are stuck inside `__lll_lock_wait`, one on lock `B` and one on lock `A`, while the main thread waits in `pthread_join`. Compiled with `-DFIXED`, the second thread takes A first, as the first does, and the program prints both “done” lines: a global lock order removes the cycle.

Worked example: the banker’s algorithm

Deadlock avoidance lets a system grant a request only if the resulting state is *safe*, meaning some order exists in which every task can obtain its maximum claim and finish. The banker’s algorithm tests this. Take five tasks and three resource types with totals A = 10, B = 5, C = 7. Each task has declared a maximum claim; the *need* is the maximum minus what it holds now.

Task	Allocation (A B C)	Max (A B C)	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1

P4	0 0 2	4 3 3	4 3 1

WORKED EXAMPLE

Is the state safe? Available = totals − the column sums of Allocation = (10, 5, 7) − (7, 2, 5) = (3, 3, 2).

1. Work = (3, 3, 2). P1's need (1, 2, 2) fits. Run it to completion and take back its allocation: Work = (3, 3, 2) + (2, 0, 0) = (5, 3, 2).
2. P3 needs (0, 1, 1), which fits. Work = (5, 3, 2) + (2, 1, 1) = (7, 4, 3).
3. P4 needs (4, 3, 1), which fits. Work = (7, 4, 3) + (0, 0, 2) = (7, 4, 5).
4. P0 needs (7, 4, 3), which fits. Work = (7, 4, 5) + (0, 1, 0) = (7, 5, 5).
5. P2 needs (6, 0, 0), which fits. Work = (7, 5, 5) + (3, 0, 2) = (10, 5, 7), the totals.

Result: the sequence ⟨P1, P3, P4, P0, P2⟩ finishes everyone, so the state is safe.

Request test. Suppose P1 now requests (1, 0, 2). It is within its need (1, 2, 2) and within Available (3, 3, 2). Pretend to grant it: Available becomes (2, 3, 0), P1 holds (3, 0, 2) and needs (0, 2, 0). P1's need fits in (2, 3, 0), so it can finish and return its holdings, giving (5, 3, 2), after which the sequence above completes. The new state is safe, so the request is granted. By contrast, a request by P4 for (3, 3, 0) leaves Available at (0, 0, 2) with every need larger than that in some column; no task can proceed, the state is unsafe, and the request must wait even though resources are free at this moment.

Detection and recovery are the alternative when avoidance is too costly: build the wait-for graph, look for a cycle, and break it by terminating or rolling back a task. General-purpose systems mostly ignore deadlock and leave lock discipline to programmers; databases detect and abort a victim transaction.

Starvation and priority inversion

Starvation is indefinite postponement of a task that is ready but never chosen: a long job under SJF, a low-priority task under a busy high-priority load, a writer behind endless readers. Unlike deadlock, the system as a whole keeps moving; ageing, fair queues and writer-preferring locks are the cures.

Priority inversion occurs when a high-priority task H waits for a lock held by a low-priority task L, and a medium-priority task M, needing neither, pre-empts L. H is then delayed by M although H has the higher priority. In 1997 this reset the Mars Pathfinder lander's computer repeatedly: a low-priority meteorological task held a shared bus lock, a medium task ran, and the high-priority bus-management task missed its deadline, triggering the watchdog. The remedy is *priority inheritance*: while L holds a lock that H wants, L runs at H's priority, so M cannot pre-empt it; the Pathfinder team enabled this option on the lock remotely.

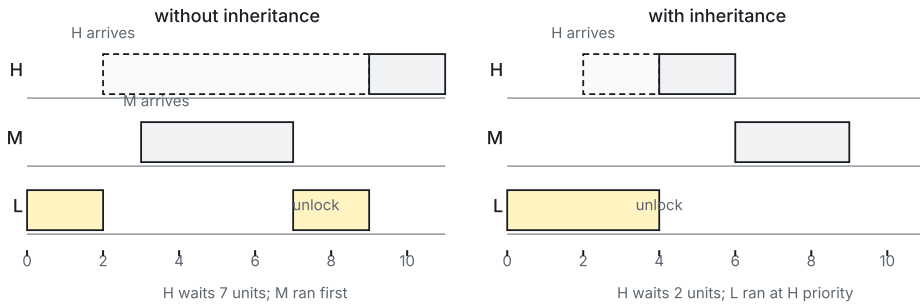


Figure 7.9 Priority inversion. Without inheritance, H waits for M's whole run. With inheritance, L finishes its critical section at H's priority and H resumes after two units.

> INTERVIEW LENS
>
> QUALCOMM · CISCO · JUNIPER
>
> "Name the four conditions for deadlock" and "run the banker's algorithm on this matrix" are reported in written tests and technical rounds; "explain priority inversion" is common for firmware and RTOS roles. A strong answer lists the conditions, gives a lock-ordering example, and for the banker's algorithm shows the Work vector at every step. Follow-ups: deadlock versus livelock (tasks keep changing state but make no progress) and why Linux's `PTHREAD_PRIO_INHERIT` mutexes exist.

> CHECK YOURSELF
>
> 1. Which Coffman condition does a global lock order break? [4]
> 2. Why is a safe state not the same as "no deadlock now"? [5]

7.7 Memory management and virtual memory MUST · ■■□

Each process addresses memory through *virtual addresses*, and a hardware unit, the *memory-management unit* (MMU), translates them to *physical addresses* using tables the kernel maintains. Memory is divided into fixed-size *pages* (4 KB on x86-64 and most ARM systems) in the virtual space and equal-sized *frames* in physical memory; a *page table* maps each virtual page number (VPN) to a physical frame number (PFN). Three benefits follow. Processes are isolated, since a process can name only the pages its table maps. Memory can be fragmented physically yet contiguous virtually. And the kernel can defer work: a page need not exist until it is first used.

> BIG IDEA
>
> Virtual memory is chiefly a translation-and-protection mechanism; using disk as spare memory is a secondary benefit, and a process that never touches a page never owns a frame for it.

Translating an address

A virtual address splits into a page number and an *offset* within the page. With 4 KB pages the offset takes the low 12 bits ($2^{12} = 4096$), and the translation replaces the page number by the frame number and keeps the offset.

> WORKED EXAMPLE

1. The wake-up was spurious, or another consumer took the item first; hence the `while` loop that rechecks the predicate. ↩
2. The spinlock. A mutex may sleep, which is not allowed in interrupt context. ↩
3. Not legitimately; a mutex has an owner. A semaphore has none, so any thread may post it. ↩
4. Circular wait: with a fixed order, no cycle of waiting tasks can form. ↩
5. Safety is a guarantee about the future, that some order lets everyone finish; an unsafe state may not be deadlocked yet but may become so. ↩

Translate 0x00403ABC in a 32-bit address space with 4 KB pages. The page table maps VPN 0x00403 to PFN 0x1A2.

1. Offset = the low 12 bits = 0xABC. VPN = the address shifted right by 12 = 0x00403.
2. Look up VPN 0x00403: PFN = 0x1A2.
3. Physical address = PFN × 4096 + offset = 0x1A2000 + 0xABC.

Result: physical address 0x1A2ABC.

A flat table for 2^{20} pages with 4-byte entries needs 4 MB for every process, even a tiny one. A *multi-level* table avoids this by allocating the lower levels only where addresses are in use. With a 10 + 10 + 12 split the top 10 bits index a first-level table whose entries point to second-level tables, and the next 10 bits index the second-level table; Figure 7.10 follows 0x00403ABC through it (first index 1, second index 3).

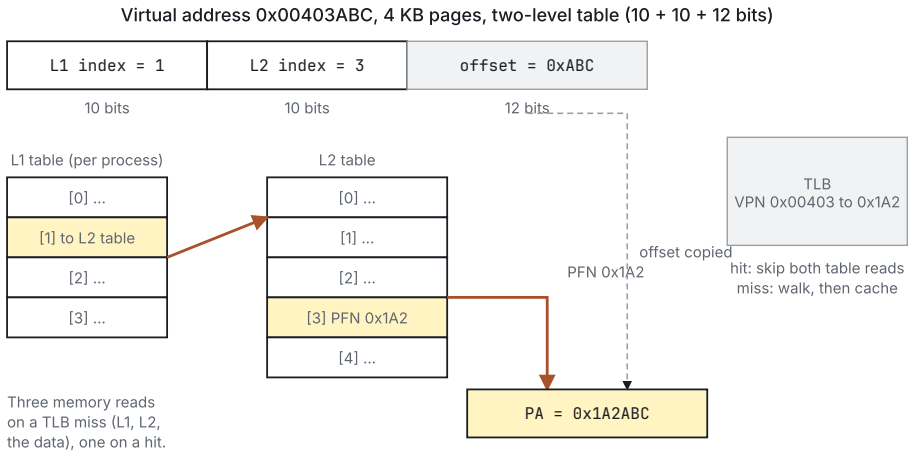


Figure 7.10 Two-level translation of 0x00403ABC. The TLB caches finished translations so that a hit skips both table reads.

WORKED EXAMPLE

Page-table memory for a process using 8 MB. Each second-level table (1024 entries × 4 bytes = 4 KB) maps 1024 pages = 4 MB.

1. 8 MB needs two second-level tables if the region is contiguous and aligned: 2 × 4 KB.
2. One first-level table: 4 KB.

Result: 12 KB of page tables, against 4 MB for the flat table.

On x86-64 with 48-bit virtual addresses, the same idea uses four levels of 9 bits each (512 eight-byte entries per table) plus the 12-bit offset: 4 × 9 + 12 = 48 bits.

The TLB and effective access time

Walking the table costs memory reads, so the MMU caches recent translations in the *translation lookaside buffer* (TLB), a small associative cache. A *hit* gives the frame at once; a *miss* triggers a table walk, after which the entry is cached. Because a process switch changes the page tables, the TLB entries are flushed or tagged with an address-space identifier, which is the hidden cost of switching processes (7.3 (p. 290)).

WORKED EXAMPLE

Effective access time (EAT). TLB lookup 1 ns, memory access 100 ns, TLB hit ratio 95 %, single-level table.

1. A hit costs the lookup and one memory access: 1 + 100 = 101 ns.
2. A miss costs the lookup, one table read and one data read: 1 + 100 + 100 = 201 ns.
3. EAT = 0.95 × 101 + 0.05 × 201 = 95.95 + 10.05.

Result: EAT = 106 ns, 6 % above the raw memory time. At a 50 % hit ratio it would be 151 ns.

Page faults

A *page fault* is the exception raised when the MMU cannot translate an address. The kernel's handler decides what kind it is. A *minor* fault finds the page already in memory or needing only a zeroed frame (first touch of `malloc`'d memory, a copy-on-write copy); it costs microseconds. A *major* fault must read the page from disk (a file mapping, or swap), costing milliseconds. An *invalid* fault is an access to an unmapped address or one that violates permissions, and the kernel delivers SIGSEGV.

LAB

`LAB 7.7 · 10 MIN` Watch first-touch page faults. The program reads `getrusage` before and after a 100 MB `malloc` and a `memset`.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>

static long faults(void)
{
    struct rusage r;
    getrusage(RUSAGE_SELF, &r);
    return r.ru_minflt;                    /* minor faults: no disk I/O
needed */
}

int main(void)
{
    size_t n = 100u << 20;                 /* 100 MB */
    long f0 = faults();
    char *p = malloc(n);                   /* reserves address space only */
    long f1 = faults();
    memset(p, 1, n);                       /* first touch of every page */
    long f2 = faults();
    printf("after malloc: +%ld faults\n", f1 - f0);
    printf("after touching 100 MB: +%ld faults (%zu pages of 4 KB)\n", f2 -
f1, n / 4096);
    printf("last byte = %d\n", p[n - 1]);    /* use the data so the optimiser
keeps it */
    free(p);
    return 0;
}
```

```
$ gcc -O1 pf.c -o pf && ./pf
after malloc: +3 faults
after touching 100 MB: +25600 faults (25600 pages of 4 KB)
```

What you should notice: `malloc` of 100 MB costs almost nothing; the 25,600 faults arrive only when the pages are first written, one per 4 KB page. With transparent huge pages enabled the count is lower, since one fault then maps 2 MB.

Copy-on-write, mmap and thrashing

After `fork()` the child's page table points at the parent's frames, and both sets of entries are marked read-only. The first write by either process raises a fault; the kernel copies that one frame, maps the copy writable for the writer, and resumes. Hence `fork` followed by `exec` copies almost nothing. `mmap` maps a file, or anonymous zeroed memory, into the address space, and pages are loaded on demand; `malloc` uses it for large blocks. When the total of the processes' actively used pages (their *working sets*) exceeds physical memory, pages are evicted just before they are needed again and the system spends its time faulting: this is *thrashing*.

Page replacement

When no frame is free, the kernel must evict a page. *FIFO* evicts the oldest-loaded page; *LRU* (least recently used) evicts the page unused for longest; *Optimal* evicts the page whose next use is furthest in the future, which needs knowledge of the future and serves as a bound. Real kernels approximate LRU with the *clock* algorithm: frames sit on a circle, each with a reference bit set by the hardware on use; a hand sweeps, clearing set bits and evicting the first frame whose bit is already clear.

Take the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 with three frames. The table shows the resident pages after each reference (oldest first for FIFO and LRU; slot order for Optimal), and F marks a fault.

Ref	FIFO		LRU		Optimal	
7	7	F	7	F	7	F
0	7 0	F	7 0	F	7 0	F
1	7 0 1	F	7 0 1	F	7 0 1	F
2	0 1 2	F	0 1 2	F	2 0 1	F
0	0 1 2		1 2 0		2 0 1	
3	1 2 3	F	2 0 3	F	2 0 3	F
0	2 3 0	F	2 3 0		2 0 3	
4	3 0 4	F	3 0 4	F	2 4 3	F
2	0 4 2	F	0 4 2	F	2 4 3	
3	4 2 3	F	4 2 3	F	2 4 3	
0	2 3 0	F	2 3 0	F	2 0 3	F
3	2 3 0		2 0 3		2 0 3	
2	2 3 0		0 3 2		2 0 3	
Faults		**10**		**9**		**7**

Bélády's anomaly is the surprising fact that for FIFO more frames can cause more faults. On the string 1 2 3 4 1 2 5 1 2 3 4 5, FIFO has 9 faults with three frames and 10 with four. LRU and Optimal are *stack algorithms*: the pages held with n frames are always a subset of those held with n + 1, so they cannot show the anomaly.

Stack, heap and a small allocator

The stack is fast to allocate because allocation is moving one register, the stack pointer, and freeing is moving it back; memory is reused immediately and is likely to be in cache. The heap must find a block of the right size among live and freed ones, which costs a search and risks fragmentation. *Internal fragmentation* is waste inside an allocated block (rounding up to 16 bytes); *external fragmentation* is free memory scattered in pieces too small to use. Structure padding and alignment, the other memory-layout question asked with these, are taught in Pillar 2 (p. 99).

Embedded targets often have no `malloc`, and Arista and Qualcomm ask candidates to design one. The usual answer is a *free list* over a fixed arena: each block begins with a header holding its size and a free flag; `malloc` takes the first block that fits (first fit) and splits off the remainder; `free` marks the block free and merges it with free neighbours (*coalescing*) so that fragments do not accumulate.

```
#include <stddef.h>
#include <stdio.h>

#define ARENA 4096
struct hdr { size_t size; int free; struct hdr *next; };   /* 24 bytes on LP64
*/

static _Alignas(16) unsigned char arena[ARENA];
static struct hdr *first;

static size_t round16(size_t n) { return (n + 15) & ~(size_t)15; }

void *a_malloc(size_t n)
{
    if (!first) {                               /* one big free block */
        first = (struct hdr *)arena;
        *first = (struct hdr){ ARENA - sizeof *first, 1, NULL };
    }
    n = round16(n);
    for (struct hdr *h = first; h; h = h->next) {
        if (!h->free || h->size < n) continue;  /* first fit */
        if (h->size >= n + sizeof *h + 16) {    /* split the remainder */
            struct hdr *r = (struct hdr *)((unsigned char *)(h + 1) + n);
            *r = (struct hdr){ h->size - n - sizeof *h, 1, h->next };
            h->size = n;
            h->next = r;
        }
        h->free = 0;
        return h + 1;
    }
    return NULL;
}

void a_free(void *p)
{
    if (!p) return;
    ((struct hdr *)p - 1)->free = 1;
    for (struct hdr *h = first; h && h->next; ) {   /* coalesce neighbours */
        if (h->free && h->next->free) {
            h->size += sizeof *h + h->next->size;
            h->next = h->next->next;
        } else h = h->next;
    }
}

int main(void)
{
    void *a = a_malloc(100), *b = a_malloc(200), *c = a_malloc(50);
    a_free(b); a_free(a);                       /* a and b merge into one hole
*/
    void *d = a_malloc(300);                    /* fits only in the merged hole
*/
    printf("a=%p d=%p same start: %s\n", a, d, a == d ? "yes" : "no");
    a_free(c); a_free(d);
    size_t total = 0; int blocks = 0;
    for (struct hdr *h = first; h; h = h->next) { total += h->size; blocks++; }
    printf("blocks=%d, free bytes=%zu\n", blocks, total);
    return 0;
}
```

```
$ gcc -Wall -Wextra alloc.c -o alloc && ./alloc
a=0x55afdad11038 d=0x55afdad11038 same start: yes
blocks=1, free bytes=4072
```

Freeing `a` and `b` creates one merged hole that the 300-byte request then fits; freeing everything coalesces the arena into a single block of 4096 − 24 bytes. A production allocator adds size classes, thread-local caches and a boundary tag at the block's end so that coalescing needs no list walk; the kernel's *slab allocator* keeps per-type caches of equal-sized objects, which is why `sk_buff` allocation is quick (7.12).

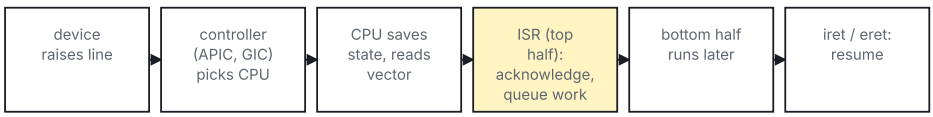
> **TRAP**
>
> *Claim:* "`malloc(100 MB)` gives me 100 MB of physical memory." *Why it is wrong:* it reserves virtual address space; frames are assigned at the first write of each page, as the lab shows, and the kernel may overcommit. *Correct statement:* `malloc` returns virtual memory, and physical frames arrive on first touch.

> **INTERVIEW LENS**
>
> CISCO · JUNIPER · QUALCOMM · ARISTA
>
> "What is virtual memory and why is it needed?" is asked almost everywhere, with "what happens on a TLB miss, and on a page fault?" as the follow-up. A strong answer names isolation and translation first, draws the VPN and offset split, and separates the TLB-miss path (hardware or kernel walks the table) from the page-fault path (the kernel loads or creates the page, or kills the process). Qualcomm and Arista add "design `malloc` for a system without an OS", which expects a header format, first-fit with splitting, and coalescing on free. Follow-ups: copy-on-write after `fork`, and why a TLB flush hurts.

> **CHECK YOURSELF**
>
> 1. How many bits of a 32-bit virtual address are the offset with 16 KB pages? [1]
> 2. Which page-replacement algorithm shows Bélády's anomaly? [2]
> 3. Why does `fork` followed by `exec` cost little? [3]

7.8 Interrupts, system calls and DMA MUST · ■■□

A device signals that it needs attention by raising an *interrupt*. The interrupt controller (the APIC on x86, the GIC on ARM) selects a CPU; the CPU finishes the current instruction, saves enough state to resume, switches to kernel mode and jumps through a *vector table* to the handler registered for that interrupt line, the *interrupt service routine* (ISR). When the ISR ends, a special return instruction (`iret` or `eret`) restores the interrupted code. *Exceptions* (page fault, divide by zero) use the same mechanism but arise from the running instruction, and a *system call* is a deliberate software-triggered entry.



Steps 1 to 4 and 6 are fast and run with the interrupt masked; step 5 is ordinary scheduled kernel work.

Figure 7.11 The life of an interrupt. Only the shaded step, the ISR, is time-critical; the heavy work is deferred.

An ISR runs while other interrupts may be masked, so it must be short. It acknowledges the device, grabs what cannot wait (reads a status register, notes the packet's buffer) and schedules the rest as a *bottom half*. Linux offers softirqs, tasklets and workqueues for this. The ISR itself, the *top half*, runs in *interrupt context*, where there is no process to put to sleep: it may not block, take a mutex, or call `kmalloc(GFP_KERNEL)`, which can sleep; it uses spinlocks and `GFP_ATOMIC`.

	Top half (ISR)	Softirq / tasklet	Workqueue
Context	interrupt	softirq (interrupt-like)	process (a kernel thread)
May sleep?	no	no	yes
May be interrupted by another interrupt?	by higher-priority ones only	yes	yes, and pre-empted by the scheduler
Example in a NIC driver	acknowledge, `napi_schedule()`	NAPI poll, protocol processing	reset after a hardware error

Direct memory access (DMA) lets a device read and write main memory without the CPU copying each word. The driver prepares a *descriptor ring*, a circular array of descriptors each holding the address and length of a buffer; it programs the NIC with the ring's base address. The NIC fills buffers and advances its position, the driver follows behind, cleans completed descriptors and hands fresh buffers back (Figure 7.12). On a CPU with caches the driver must also keep the cache consistent with the device's writes, which the kernel's DMA-mapping API arranges.

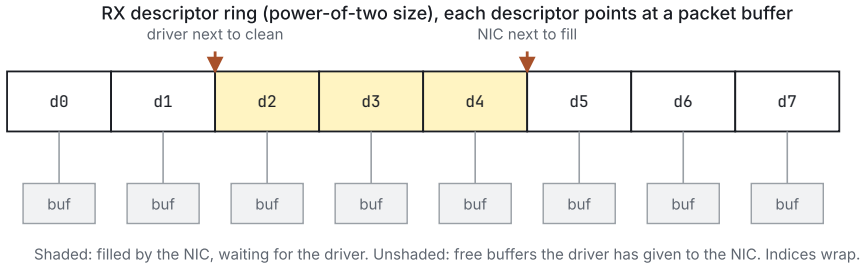


Figure 7.12 An RX descriptor ring. The NIC fills descriptors up to its pointer; the driver processes behind it and refills. Both indices wrap.

Why one interrupt per packet fails

A 10 Gb/s link carrying minimum-size frames moves 64 bytes plus 20 bytes of preamble and inter-frame gap per packet: 84 bytes = 672 bits, hence $10 \times 10^9 / 672 \approx 14.88$ million packets per second, one every 67 ns. An interrupt costs more than that, so a CPU taking one interrupt per packet would do nothing else; at the worst it would be *livelocked*, spending all its time entering and leaving ISRs. The cures are *interrupt coalescing* (the NIC raises one interrupt per batch or per timeout) and *NAPI* (7.12), which switches the device to polling under load.

	Interrupts	Polling
Latency at low load	best: reacts at once	up to one polling interval
CPU use at low load	none while idle	wasted: the poller runs regardless
Behaviour at high load	overhead per event; livelock risk	efficient: events are batched

Used by	NICs at low rate, keyboards	NAPI under load, DPDK, busy-polling drivers

LAB

`LAB 7.8 · 10 MIN` Look at the machine's interrupt and receive-processing counters.

```
$ head -4 /proc/interrupts
           CPU0       CPU1       CPU2       CPU3
 26:          0          0          2          0  IO-APIC   4-edge      ttyS0
 32:          0        972          0          0  PCI-MSIX-0000:00:01.0   3-
edge      virtio0-stats
$ head -1 /proc/net/softnet_stat
00039a92 00000000 00000019 ...
```

What you should notice: each row of `/proc/interrupts` is one interrupt line with a count per CPU; a multi-queue NIC shows one line per queue, each bound to a core, and the counts climb during an `iperf3` run. In `softnet_stat` the first three hexadecimal columns are packets processed, packets dropped, and `time_squeeze`, the number of times the receive poll ran out of budget with work left; here 0x39a92 packets, 0 drops, 0x19 = 25 squeezes.

TRAP

Claim: "An ISR can call a function that takes a mutex, provided the mutex is rarely contended." *Why it is wrong:* if the mutex is held, the ISR would sleep, but an interrupt handler has no task context to sleep in, and the kernel panics or deadlocks. *Correct statement:* in interrupt context use spinlocks (with interrupts disabled while the lock is held, to avoid self-deadlock) and atomic allocations.

INTERVIEW LENS

QUALCOMM · BROADCOM · CISCO · JUNIPER

"Describe what happens from the device signal to the end of the ISR" and "why can't you sleep in an ISR?" are reported firmware and driver questions, and "polling or interrupts?" was reported for a Cisco internship. A strong answer lists the steps in the figure, states the top-half rule, mentions deferral and DMA, and gives the polling trade-off with the packet-rate arithmetic above. Follow-ups: what NAPI is, and what `volatile` and memory barriers do for device registers (7.13).

CHECK YOURSELF

1. Why must an ISR be short? [4]
2. What does DMA save the CPU from? [5]

7.9 Inter-process communication and signals SHOULD · ■■□

Processes cannot touch each other's memory, so they communicate through the kernel or through memory the kernel maps into both. A *pipe* is a one-way byte channel between related processes; a *FIFO* (named pipe) is a pipe with a name in the file system, so unrelated processes may open it; a *message queue* preserves message boundaries; *shared memory* maps the same frames into two address spaces; and a *socket* (7.10) also works across machines.

Mechanism	Direction	Unrelated processes?	Kernel copies per message	Needs synchronisation?
Pipe	one way	no (shared ancestor)	2	no, the kernel blocks

FIFO	one way	yes	2	no
Message queue	any	yes	2	no
Shared memory	any	yes	0	yes, e.g. a process-shared mutex
Unix-domain socket	both	yes (same host)	2	no
Signal	notification only	yes	none, carries a number	n/a

Shared memory is fastest because after setup no kernel call is on the data path; its price is that the processes must synchronise themselves, as threads do.

The shell pipeline `ls | wc -l` shows how the pieces combine. The parent creates a pipe, forks twice, and each child uses `dup2` to make the pipe's end its standard output or input before `exec`.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)                          /* equivalent of: ls | wc -l */
{
    int fd[2];
    if (pipe(fd) < 0) { perror("pipe"); exit(1); }

    if (fork() == 0) {                  /* writer: ls */
        dup2(fd[1], STDOUT_FILENO);     /* stdout now feeds the pipe */
        close(fd[0]); close(fd[1]);     /* close the originals */
        execlp("ls", "ls", (char *)NULL);
        perror("execlp"); _exit(127);
    }
    if (fork() == 0) {                  /* reader: wc -l */
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]); close(fd[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp"); _exit(127);
    }
    close(fd[0]); close(fd[1]);         /* parent must close, or wc never sees
EOF */
    while (wait(NULL) > 0)
        ;
    return 0;
}
```

In each child, `dup2(fd[1], STDOUT_FILENO)` makes descriptor 1 refer to the pipe's write end, and the original two descriptors are closed. The parent must also close both ends: a pipe delivers end-of-file to the reader only when every write end is closed, so a forgotten copy in the parent leaves `wc` waiting for ever.

A *signal* is an asynchronous notification delivered to a process: SIGINT (Ctrl-C), SIGTERM (polite termination), SIGKILL (cannot be caught), SIGSEGV (invalid memory access), SIGCHLD (a child changed state; the standard place to reap zombies with `waitpid(-1, NULL, WNOHANG)`), and SIGPIPE (a write to a pipe or socket whose reader has gone). Because a handler can interrupt the program at any point, it may call only *async-signal-safe* functions: `write` is on the list, `printf` and `malloc` are not, since they may hold internal locks that the interrupted code already owns. The common practice is to set a flag in the handler and act on it in the main loop. A netw-

ork server normally ignores SIGPIPE (`signal(SIGPIPE, SIG_IGN)`, or `MSG_NOSIGNAL` on `send`) and handles the `EPIPE` error instead.

> **CHECK YOURSELF**
>
> 1. Which IPC mechanism avoids kernel copies of the data? [6]
> 2. Why can a signal handler not safely call `printf`? [7]

1. 14, since $2^{14} = 16384$; the remaining 18 bits are the page number.↩
2. FIFO. Stack algorithms such as LRU and Optimal cannot show it.↩
3. Copy-on-write: the pages are shared and not copied, and `exec` discards the child's address space almost at once.↩
4. Interrupts of equal or lower priority are held off while it runs, so a long ISR delays other devices and the scheduler's timer.↩
5. Copying every word between the device and memory; the CPU only sets up descriptors and handles the completion interrupt.↩
6. Shared memory, after the one-off setup; it needs explicit synchronisation.↩
7. `printf` takes an internal lock; if the signal interrupted code holding it, the handler would deadlock.↩